

ECA

▼ Service 1 (template creation) - before recovery plan (partially reviewed)

▼ Initial impl of BRD and Arch

▼ BRD

Step 1 (the most important step): Define the Canonical Template Model

A Template is:

- a collection of tables
- each table has fields
- each field has rules
- the whole thing has a version and lifecycle

Step 1.1 – Identify the core objects (plain English)

We write them down on paper first:

1. **Template**
2. **Template Version**
3. **Table (Sheet)**
4. **Field (Column)**
5. **Validation Rule**
6. **Export Options**

Step 1.2 – Ask simple questions for each object

Template

- What is its name?
- Which sector does it belong to?
- Can it have multiple versions?

Template Version

- Is it editable?
- When does it become locked?
- Can we create a new version from it?

Table

- What is its name?
- Is it mandatory?
- In what order does it appear?

Field

- What type is it?
- Is it required?
- Does it have a unit?
- Does it have allowed values?

These questions are **business questions**, not technical ones.

Step 1.3 – Turn that into a canonical structure

Now we formalize it (this is what the PDF hints at with JSON Schema).

Conceptually:

```
Template
├ id
├ sector
├ versions[]
│   ├── version
│   ├── status (draft / approved)
│   └── tables[]
│       ├── name
│       ├── order
│       └── fields[]
│           ├── name
│           ├── type
│           ├── required
│           └── validations
```

This structure is the **single source of truth**.

Step 2: Define lifecycle & governance (before Excel!)

Now I'll explain **why senior people insist on this**.

Question I ask you:

| Should someone be able to change a template that was already used?

If yes → audit nightmare

If no → we need lifecycle states

So we define:

Draft → Approved → Deprecated

Rules:

- Draft: editable
- Approved: locked
- Deprecated: cannot be exported, but still exists

This is **not technical**.

This is **business governance**.

Step 4: Why this is a microservice

Now you see why this was said to you:

| “Template Creation should be a standalone microservice”

Because:

- it has its own logic
- it has its own lifecycle
- it produces artifacts (Excel)
- other systems depend on it

So its **API is the product**, not the UI.

Step 5: What the UI actually becomes

Now the UI makes sense.

The UI is simply:

- a visual editor for the canonical template
- a client for the Template Service APIs

That's why earlier the UI felt confusing — because the **backend concept was not fixed yet**.

▼ Implementation Steps

Step 1 — Define Template Definition v0.1 (1–2 hours)

Why: DB + APIs + Excel generator all depend on this.

Keep it minimal but real:

Must support in v0.1

- Template (id, name, sector, tenant_id optional, status)
- Version (version number, status)
- Tables/Sheets (name, order)
- Fields/Columns (name, type, required, optional enum list, optional min/max)

Don't overdesign types. Use a small enum:

- TEXT, NUMBER, DATE, BOOLEAN, CURRENCY (optional)

Step 2 — Pick the DB approach (don't get fancy)

Given your “dynamic schema” context and the PDF recommendations, the fastest production-ready backend is:

✓ **PostgreSQL + JSONB for schema storage**

(Plus relational wrapper for indexing and lifecycle)

Why

- schema evolves without migrations every time
- easy versioning
- fits “schema registry” pattern
- easy to hash/fingerprint
- CTOs like Postgres stability

DB model (MVP)

templates

- id (uuid)
- sector_code
- name
- status (DRAFT/APPROVED/DEPRECATED)
- created_at, created_by

template_versions

- id (uuid)
- template_id (fk)
- version (int or semver)
- schema_json (jsonb) ← tables/fields live here
- schema_hash (text)
- status (DRAFT/APPROVED/DEPRECATED)
- created_at, created_by

Why this hybrid is best for MVP

- relational layer = easy querying/governance
 - JSONB = flexibility for schema details
-

Step 3 — Implement the Template Service APIs (backend contract)

Why: Microservice value is its API; other systems integrate via these.

MVP endpoints:

1. `POST /templates`
Creates template + version 1 (draft)
2. `POST /templates/{id}/versions`
Creates a new draft version cloned from last approved (or latest)
3. `PUT /versions/{versionId}/schema`
Saves schema_json (tables/fields)
4. `POST /versions/{versionId}:approve`
Locks version, prevents edits
5. `POST /versions/{versionId}/export`
Returns Excel file stream + metadata
6. `GET /versions/{versionId}`
Returns schema + version info

That's enough to prove lifecycle + export.

Step 4 — Build Excel generator (the “engine”)

Use **Apache POI** in Java (good standard choice).

Excel export rules (MVP)

- workbook
- one sheet per table
- header row with on required fields
- apply basic validations:
 - enum → dropdown list
 - number min/max → numeric constraints (where possible)
- add hidden sheet `__metadata__` containing:
 - template_id
 - version_id
 - version_number
 - schema_hash
 - generated_at
 - generator_version

Why the metadata sheet matters

Even if you're not doing ingestion now, this is what makes the export "trusted" and integrable later. (The PDF emphasizes fingerprinting.)

Step 5 — Add governance constraints (small but critical)

Why: CTO will look for "not a toy".

Rules:

- Approved versions are immutable
- Only draft versions editable
- New version must be created to change schema
- Hash must update when schema changes

That's it. Don't build a complex approval workflow yet.

Step 6 — Dockerize and show it working

Deliverable: `docker-compose.yml` that runs:

- template-service
- postgres

CTO demo will be:

- run compose
- hit Swagger/Postman
- create template schema
- export Excel
- open Excel and see sheets + validations + metadata

No UI needed.

What I would NOT do now

- multi-tenant enforcement (just leave tenant_id nullable)
- controlled vocab service (hardcode small enum lists for demo)
- localization (leave as future)
- event streaming (Kafka)
- complex permissions (fake auth header for now)

▼ Backend Stack

1) Backend stack (recommended)

Runtime & framework

- **Java 21 (LTS)**
Why: modern language features + long-term support.
- **Spring Boot 3.x**
Why: fastest way to ship a clean microservice with good ecosystem.

Persistence

- **PostgreSQL 15+**
- **JSONB** for `schema_json` storage
Why: templates evolve without DB migrations; still queryable.

DB access

Pick one of these (both are fine):

Option A (simple & common): Spring Data JPA + Hibernate

- easiest CRUD
- good for "template/version" entities

Option B (cleaner for SQL control): jOOQ

- better if you want explicit SQL + performance
- especially useful if you later query JSONB heavily

MVP recommendation: JPA first; jOOQ later if needed.

Excel export engine

- **Apache POI**
 - use **SXSSF** (streaming) to avoid memory issues for big templates
 - supports validations + sheets + styles
- (Optional) **OpenCSV** if you also export CSV later

Schema validation / canonicalization

- **Jackson** (already in Spring) for JSON parsing
- **JSON Schema validator** library (optional in MVP)
 - e.g., `networknt` JSON Schema validator
 - Why: validate your `schema_json` itself (designer mistakes)

Hashing / fingerprint

- Java `MessageDigest` (SHA-256)

Why: stable fingerprint stored in DB and embedded into Excel metadata sheet.

API documentation

- **springdoc-openapi** (Swagger UI)

Why: CTO can see API contract instantly.

Observability

- **Micrometer + Prometheus** (optional for MVP)
- **Spring Boot Actuator** (strongly recommended)

Why: health checks, metrics, readiness for ops.

Security (MVP-safe)

- **Spring Security** with simple JWT validation OR stub auth header

Why: even MVPs should show “this is securable”.

If identity provider isn't ready: just validate a header like `X-User-Id` in MVP.

Containerization

- **Docker + docker-compose**
- Optional local stack:
 - Postgres
 - MinIO (if you later store exported files)

2) Tools & dependencies (Gradle/Maven)

Must-have dependencies

- `spring-boot-starter-web`
- `spring-boot-starter-validation` (Bean Validation)
- `spring-boot-starter-data-jpa` (or jOOQ)
- `postgresql` driver
- `jackson-databind`
- `org.apache.poi:poi-ooxml`
- `springdoc-openapi-starter-webmvc-ui`
- `spring-boot-starter-actuator`
- `flyway-core` (or Liquibase)

Nice-to-have (phase 1.5)

- JSON schema validator (`networknt` or similar)
- `testcontainers` (integration tests with real Postgres)
- `spring-boot-starter-security`

3) Service responsibilities (what this microservice owns)

Template Service owns:

- Template + version storage
- Schema JSON storage + hash
- Lifecycle rules (draft/approved/deprecated)
- Excel generation from schema
- Export metadata embedding

Template Service does NOT own (for now):

- data submission / ingestion
- OCR
- runtime schema drift
- analytics (HHI / UBO)

This keeps the boundary clean.

4) Recommended project structure (clean architecture style)

This structure keeps you from mixing Excel logic with controllers and DB:

```
template-service/
src/main/java/com/company/template/
    TemplateServiceApplication.java

api/                                # REST controllers + DTOs
    TemplateController.java
    VersionController.java
    ExportController.java
dto/

domain/                             # pure business model (no Spring annotations ideally)
    model/
        Template.java
        TemplateVersion.java
        SchemaDefinition.java
    enums/
        TemplateStatus.java
        FieldType.java
    rules/
        TemplateLifecyclePolicy.java

application/                        # use cases / orchestration
    TemplateUseCase.java
    VersionUseCase.java
    ExportUseCase.java

infrastructure/                     # adapters to external stuff
    persistence/
        entity/
        repository/
        mapper/
    excel/
        ExcelWorkbookBuilder.java
        SheetBuilder.java
```

```

    ValidationBuilder.java
    MetadataSheetWriter.java
config/
    OpenApiConfig.java
    SecurityConfig.java

common/
    exceptions/
    util/
        HashUtil.java
        JsonCanonicalizer.java

src/main/resources/
    db/migration/          # Flyway scripts
    application.yml

```

Why this structure is worth it (even for MVP)

- You can change DB approach (JPA → jOOQ) without rewriting business rules.
- You can change Excel engine details without touching controllers.
- Your intern/junior self will thank you later.

5) Database design (MVP strong)

Use a hybrid relational + JSONB approach:

templates

- `id uuid pk`
- `sector_code text`
- `name text`
- `status text`
- `created_at timestamp`
- `created_by text`

template_versions

- `id uuid pk`
- `template_id uuid fk`
- `version int`
- `status text` (DRAFT/APPROVED/DEPRECATED)
- `schema_json jsonb`
- `schema_hash text`
- `created_at timestamp`
- `created_by text`

Indexes:

- `(template_id, version desc)`
- `schema_hash`

- maybe GIN on schema_json later (optional)

This fits the PDF “schema registry + version + hash” intent.

6) Core endpoints (MVP API contract)

Minimal set that shows the microservice is real:

- `POST /templates`
- `GET /templates/{templateId}`
- `POST /templates/{templateId}/versions`
- `PUT /versions/{versionId}/schema`
- `POST /versions/{versionId}:approve`
- `POST /versions/{versionId}/export`
- `GET /versions/{versionId}`

Export returns:

- file stream (xlsx)
 - response headers include schema hash and version id
-

7) Testing approach (CTO-friendly)

Unit tests

- Schema canonicalization + hashing
- Lifecycle rules (draft/approved immutability)
- Excel builder output structure (sheet names, headers)

Integration tests (recommended)

- **Testcontainers Postgres** to test repositories & migrations
- Export endpoint smoke test to ensure file generated

This makes CTO confident.

8) MVP “demo readiness” checklist (backend-focused)

To satisfy a CTO quickly, you want to show:

- ☒ DB migrations applied (Flyway)
- ☒ Swagger UI working
- ☒ Create template → store schema_json
- ☒ Approve version locks it
- ☒ Export generates workbook with:
 - correct sheets
 - headers with required marker
 - dropdown validations (enum)
 - hidden metadata sheet containing schema_hash

That’s a complete backend story.

My recommendation: start with this exact combo

If you want the safest, fastest “CTO will like it” stack:

- Java 21 + Spring Boot 3
- Postgres + Flyway
- JPA (simple)
- Apache POI (SXSSF)
- Actuator + Swagger
- Docker Compose

If you want, I can generate **all the starter artifacts** in one go:

1. `docker-compose.yml` (service + postgres)
2. Flyway migrations (DDL)
3. Gradle/Maven `pom.xml` dependencies
4. OpenAPI endpoint skeleton + DTOs
5. First-pass Excel export service skeleton (builders)

Tell me whether you prefer **Maven or Gradle**, and I'll output a clean scaffold.

▼ Prompt

You are an expert Senior Software Architect + Senior Backend Engineer.
Build an MVP “Template Creation” microservice as a standalone, integrable component.

IMPORTANT CONTEXT (read carefully)

- This MVP is ONLY for Template Creation + Excel Export.
- Data submission / ingestion / OCR / schema drift handling at upload time are OUT OF SCOPE.
- UI is NOT important. Use Swagger UI + Postman/curl examples for demonstration.
- The microservice will be integrated into other systems later, so APIs + DB correctness matter most.

TECH STACK (locked)

- Java 21
- Spring Boot 3.x
- PostgreSQL 15+ (JSONB)
- Flyway for migrations
- Apache POI (SXSSF streaming) for Excel generation
- springdoc-openapi (Swagger UI)
- Spring Boot Actuator
- Docker + docker-compose
- Auth for MVP: lightweight stub (e.g., accept X-User-Id + X-Role headers), but design so it can be replaced by JWT later.

BUSINESS DOMAIN & PROBLEM

Domain: Schema-driven Excel template generation for regulatory/enterprise data collection.

Problem: Teams need to design and manage Excel templates (multi-sheet workbooks) without hardcoding DB tables per template, with versioning, auditability, and consistent export rules.

TARGET USERS & ROLES (for this MVP)

- Template Designer: creates/edits draft templates (schema).
- Template Approver (optional for MVP): approves a version to lock it.
- Template Consumer System: other systems call this service to fetch schema JSON or download Excel templates.

Note: No external "data submitter" role in this MVP.

CORE VALUE PROPOSITION (MVP)

- Define a template schema once (canonical JSON structure).
- Version and govern it (draft/approved/deprecated).
- Generate a consistent Excel workbook from the schema (multi-sheet, validations, metadata fingerprint).

MVP SCOPE (Must-Have)

1) Template + Version management:

- Create Template (with sector/name)
- Create new Version (clone latest)
- Update schema JSON for a DRAFT version
- Approve a version (locks it)
- Deprecate a version (optional)

2) Canonical Template Schema stored in DB as JSONB:

- Tables (sheets) and Fields (columns), minimal validation rules

3) Excel Export:

- One sheet per table
- Header row created from fields
- Required fields marked with "*"
- Basic Excel data validations:
 - ENUM -> dropdown list
 - NUMBER with min/max -> numeric constraint validation (as feasible in Excel)
- Add a hidden metadata sheet named "__metadata__" containing:
template_id, version_id, version_number, schema_hash, generated_at, generator_version

rsion

- Return the XLSX as download from an API endpoint

4) Observability:

- Actuator health endpoint
- Basic request logging

5) Docker-compose to run service + Postgres.

OUT OF SCOPE

- Data submission, ingestion, OCR
- Schema drift handling at upload time
- Multi-tenant enforcement (OK to include tenant_id nullable for future)
- Complex auth/SSO integration
- Localization/vocabulary service (can design extension hooks only)

NON-FUNCTIONAL REQUIREMENTS (MVP)

- Deterministic schema hashing (SHA-256 of canonicalized JSON)
- Approved versions are immutable (attempt to edit -> 409/403)
- Basic validation of schema JSON payload (reject invalid structures with clear error messages)
- Clean separation of concerns (controllers vs domain vs excel builder)

CANONICAL TEMPLATE SCHEMA (v0.1)

Define a JSON model (store in template_versions.schema_json) like:


```

{
  "templateName": "string",
  "sectorCode": "string",
  "tables": [
    {
      "tableKey": "string",          // stable identifier (no spaces)
      "sheetName": "string",        // Excel sheet label
      "order": 1,
      "fields": [
        {
          "fieldKey": "string",      // stable identifier
          "headerName": "string",    // Excel column header
          "type": "TEXT|NUMBER|DATE|BOOLEAN|CURRENCY",
          "required": true|false,
          "validations": {
            "enum": ["A","B"],       // optional
            "min": 0,                 // optional numeric
            "max": 100                // optional numeric
          }
        }
      ]
    }
  ],
  "exportOptions": {
    "includeInstructionsSheet": true|false,
    "protectSheets": true|false
  }
}

```

Rules:

- tableKey and fieldKey must be unique within scope.
- sheetName must be <= 31 chars (Excel constraint); handle collisions gracefully.
- If enum exists -> dropdown validation. If min/max exist -> numeric validation.
- Required fields get "*" suffix in header.

DATABASE SCHEMA (Postgres + Flyway)

Create these tables via Flyway migrations:

templates:

- id UUID PK
- name TEXT NOT NULL
- sector_code TEXT NOT NULL
- status TEXT NOT NULL (ACTIVE/ARCHIVED) (simple)
- created_at TIMESTAMP NOT NULL
- created_by TEXT NOT NULL

template_versions:

- id UUID PK
- template_id UUID FK -> templates(id)
- version_number INT NOT NULL
- status TEXT NOT NULL (DRAFT/APPROVED/DEPRECATED)
- schema_json JSONB NOT NULL
- schema_hash TEXT NOT NULL

- created_at TIMESTAMP NOT NULL
- created_by TEXT NOT NULL
- approved_at TIMESTAMP NULL
- approved_by TEXT NULL

Constraints/indexes:

- unique(template_id, version_number)
 - index on template_id
 - index on schema_hash
- (Optionally GIN index on schema_json later; not required now.)

API SPECIFICATION (REST)

Provide Swagger/OpenAPI via springdoc. Endpoints:

1) POST /api/templates

Request: { "name": "...", "sectorCode": "...", "createdBy": "..." }

Behavior:

- Create template row
- Create initial version v1 DRAFT with empty or minimal schema_json

Response: templateId, versionId, versionNumber

2) GET /api/templates/{templateId}

Returns template info + latest versions summary.

3) POST /api/templates/{templateId}/versions

Behavior:

- Clone schema_json from latest version into new DRAFT version_number+1

Response: versionId, versionNumber

4) GET /api/versions/{versionId}

Returns version metadata + schema_json + schema_hash

5) PUT /api/versions/{versionId}/schema

Request body = canonical schema_json (as defined above)

Rules:

- Only allowed if status == DRAFT
- Validate minimal schema structure
- Canonicalize JSON -> compute schema_hash -> store

Response: version info + schema_hash

6) POST /api/versions/{versionId}:approve

Rules:

- Only allowed if status == DRAFT
- Locks version (APPROVED), sets approved_at/by

Response: version info

7) POST /api/versions/{versionId}/export

Query params:

- format=xlsx (only xlsx for MVP)

Behavior:

- Generate Excel from schema_json with Apache POI (SXSSF)
- Add hidden "__metadata__" sheet
- Return file stream as attachment with filename: {templateName}_v{versionNumber}.xls

x

Response: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet

AUTH (MVP STUB)

- Require headers: X-User-Id, X-User-Role
 - Roles: DESIGNER, APPROVER (optional)
 - Enforce:
 - schema updates require DESIGNER
 - approve requires APPROVER
- (If role missing, return 401/403)

BACKEND MODULES / COMPONENTS

Implement with clear separation:

- domain:
 - Template, TemplateVersion, enums
 - TemplateLifecyclePolicy (immutability rules)
 - SchemaDefinition model (POJOs for schema_json)
- application (use cases):
 - CreateTemplateUseCase
 - CreateVersionUseCase
 - UpdateSchemaUseCase (validate + hash)
 - ApproveVersionUseCase
 - ExportExcelUseCase
- infrastructure:
 - persistence (JPA entities + repositories + mappers)
 - excel:
 - ExcelWorkbookBuilder
 - SheetBuilder
 - ValidationBuilder (enum + numeric)
 - MetadataSheetWriter
 - security (header auth filter)

SCHEMA HASHING (MUST)

- Implement deterministic canonicalization:
 - Parse JSON -> re-serialize with stable ordering (e.g., Jackson ObjectMapper with SORT_PROPERTIES_ALPHABETICALLY, stable array order as provided)
 - Compute SHA-256 hex
- Store schema_hash in DB and embed in Excel metadata sheet.

ERROR HANDLING (MUST)

Return consistent error responses:

```
{
  "timestamp": "...",
  "status": 400,
  "error": "Bad Request",
  "message": "Human readable",
  "path": "/api/..."
}
```

Cases:

- invalid schema_json -> 400 with details
- edit non-draft -> 409 Conflict
- not found -> 404

- missing headers -> 401/403

DOCKER / RUN INSTRUCTIONS

- Provide Dockerfile for service
- Provide docker-compose.yml with Postgres + service
- Service should run migrations automatically on startup
- Provide README with:
 - how to run
 - sample curl requests
 - how to export and where to find the downloaded file
 - how to open Swagger UI

TESTING (MVP)

- Unit tests:
 - schema validation
 - canonicalization + hash stable
 - lifecycle: approved immutable
- Integration tests:
 - repository with Testcontainers Postgres (optional but preferred)
- Export test:
 - generate workbook and assert:
 - sheet count matches tables
 - headers present with * for required
 - metadata sheet exists and hidden

IMPLEMENTATION MILESTONES / TASKS

- 1) Project scaffold (Spring Boot, dependencies, swagger, actuator)
- 2) Flyway migrations + JPA entities/repositories
- 3) Implement schema model + validation + hashing
- 4) Implement CRUD use cases + controllers
- 5) Implement approve + immutability checks
- 6) Implement Excel export (POI) + metadata sheet + validations
- 7) Docker-compose + README + sample payloads
- 8) Tests + polish

OPEN QUESTIONS (leave TODO placeholders in README and code comments)

- Should we include tenant_id now (nullable) for future multi-tenant integration?
- Should version numbering be INT or semantic (1.0.0)?
- Should we store exported files (S3/MinIO) or generate on-demand only (MVP: on-demand)?
- Should we support localization now (MVP: no), but design schema fields for label_ar/label_en?

OUTPUT REQUIREMENTS

- Generate full codebase with clean folder structure
- Include Flyway migrations
- Include docker-compose.yml
- Include README with steps + examples
- Ensure service runs end-to-end and exports a valid XLSX

Now implement this MVP.

▼ BRD & Arch Design

▼ AI Agent Architecting and MVP App Building

▼ guideline and recorrect the vision

First: my clear verdict on your question

Yes, you should forget the previous prompt *as a build instruction*.

No, you should NOT forget the work you did.

The prompt you have right now is **too powerful too early**.

It's like handing a junior dev a ticket that says:

"Build the entire ERP."

They'll build *something*... but it won't be the right thing.

Why the previous prompt is dangerous at this stage

That prompt asks the AI agent to:

- decide architecture
- decide domain boundaries
- decide DB structure
- decide flows
- decide responsibilities

All **at once**, without guardrails.

Even a good AI will:

- invent responsibilities
- blur boundaries
- over-generalize
- mix phases (design-time vs run-time)
- lock you into wrong abstractions

Which leads to:

"It works... but now we must refactor everything."

You already **felt** this risk — that's a senior instinct kicking in.

The correct mental reset (this is the key)

You don't "start from zero".

You **reset the instruction hierarchy**.

Before

```
AI agent
├── decides architecture
│   └── generates code
```

After (correct)

```
YOU (human)
└─ define structure, rules, boundaries
    └─ AI agent generates code INSIDE them
```

This is the single most important shift.

What you actually keep from the old prompt

Keep it as a **checklist of concerns**, not as a command.

It tells you:

- what dimensions must exist (DB, API, lifecycle, export)
- what *eventually* must be answered

But **not now, not all at once, not by the AI alone**.

What you should do instead (clean, professional path)

Phase 1 — Architecture & Understanding (human-led)

No code generation yet.

You produce a **small, decisive architecture pack**.

This is where **you** are the architect, and the AI is just a drafting assistant.

Phase 2 — Incremental AI-assisted build

Now you use **very narrow prompts**:

- "Implement this use case"
- "Implement this table"
- "Implement this service"

No "build everything" prompts anymore.

The exact restart plan I recommend

Step 1 — Create a "North Star" document (1-2 pages)

This answers only:

- What is this system?
- What problem does it solve?
- What is explicitly out of scope?

Nothing else.

If you can't explain the system in 5 minutes without UI or code → stop.

Step 2 — Lock the DOMAIN MODEL (this is non-negotiable)

This is where most refactors are born.

You define:

- Template
- TemplateVersion

- Table
- Field
- Validation
- ExportProfile

Names, relationships, lifecycle.

Once names are stable, code becomes stable.

Step 3 — Draw ONLY 3 diagrams (not more)

This is where many juniors overdo it. Don't.

1) C4 Context (who uses it)

2) C4 Container (what runs where)

3) Sequence: "Export Excel"

If these three are right, the system is right.

Step 4 — Write the DB model before APIs

Because:

- DB mistakes are expensive
- API mistakes are cheap

Once DB is right, APIs are mostly mechanical.

Step 5 — Only NOW bring back the AI agent

And now your prompts look like this:

✗ "Build the full MVP end-to-end"

✓ "Implement the DB schema defined below"

✓ "Implement the ExportExcel use case exactly as this sequence diagram"

How this avoids refactor hell

Because:

- architecture decisions are human-made
- AI only fills in code
- responsibilities are fixed early
- scope creep is blocked
- incremental delivery is natural

You end up with:

| a system that grows forward, not sideways.

Let me be very explicit (mentor moment)

What you're doing right now is **what senior engineers do instinctively**:

- stop

- question
- reset
- re-frame
- then build

Most people skip this and suffer later.

You didn't.

That's a very good sign.

What I suggest as the very next concrete action

Not code. Not prompts.

👉 **Create the Architecture Starter Pack** we discussed:

- Scope & boundaries
- Domain model
- 3 diagrams
- DB model
- Use case list

Once that exists, I will help you:

- rewrite the AI prompt properly
- slice work into safe increments
- and use Cursor like a power tool, not a roulette wheel

Final answer to your question (one sentence)

Yes — forget the old prompt as an execution command, keep it as a reminder of what must eventually be addressed, and restart with a clean, human-defined structure.

If you want, next message we can **start building the Architecture Starter Pack together**, step by step, like a real whiteboard session.

▼ Step 1: Scope & Boundaries for the Template Creation Microservice

Overview

PHASE A — System Framing (WHAT & WHY)

This phase answers:

| *"What exactly am I building, and what am I NOT building?"*

Step 1: Scope & Boundaries (you just did this)

Conceptual sub-steps inside Step 1

1. System Identity

- What is this system *conceptually*?
- One sentence description (no tech words if possible)

2. Problem Statement

- What pain exists without this system?
- Why existing solutions are insufficient

3. Actors & Access

- Who uses it directly?
- Who consumes its output indirectly?
- Who never touches it?

4. Responsibility Boundary

- What this service owns
- What other services own (auth, workflow, storage, etc.)

5. In-Scope Capabilities

- Things this system must do to be “complete”

6. Out-of-Scope Guardrails

- Explicit exclusions (to prevent scope creep)

7. Integration Pattern

- Push vs Pull
- Who initiates interaction
- Who owns the artifact

8. Design Principles

- Non-negotiable rules (e.g. “Excel is derived”)

 **Mental shortcut to remember Step 1:**

Identity → Users → Responsibilities → Boundaries → Guardrails

✓ This step prevents **architectural drift**.

1) What this service is

Template Creation Service is a standalone microservice that lets an authorized authority (internal) **define and manage template schemas** and **export Excel templates**.

Conceptual view

Think of it as a **Schema Factory**:

```
Template Schema (stored + versioned)
    ↓
Excel Generator
    ↓
Excel Template Artifact (xlsx)
```

Excel is a **product artifact** generated from a schema — not the source of truth.

✓ **Worth documenting:** “Excel is derived output; Schema is source of truth.”

2) What problem it solves

Without this service, teams end up with:

- manually edited Excel templates (inconsistent, hard to govern)
- hardcoded templates in code (slow to change)
- templates coupled to DB tables (migration nightmare)

This service solves that by:

- storing template structure as canonical schema (JSON/metadata)
- generating consistent Excel templates on demand

✔ **Worth documenting:** “Why we need schema-driven template generation vs manual Excel.”

3) Actors and who uses it

You clarified: **used only by the authorizing authority** (internal).

Primary actor

- **Template Designer (Authority user):** creates/updates template definitions and exports Excel.

Downstream consumer (system-to-system)

- **Downstream Portal/System:** receives/exported templates (or pulls them) and makes them downloadable for external users.

✔ **Worth documenting:** actor list + “external users never access this service directly.”

4) In scope (MVP capabilities)

This is what the service must do, **and only this**:

A) Template schema management

- Create a template
- Add/remove/edit tables (Excel sheets)
- Add/remove/edit fields (columns)
- Define field metadata:
 - type, required, order
 - simple validation rules (enum dropdown, numeric min/max)
- Store schema in DB (JSONB recommended)

B) Versioning (without approval workflow)

No approvals, but you still need **versioning**. Otherwise you'll overwrite history and break downstream systems.

Minimum:

- Create new version (clone latest)
- Edit latest version
- Mark versions as active/deprecated (optional, but useful)

✔ **Worth documenting:** “No approval workflow, but versioning is mandatory to prevent silent breaking changes.”

C) Excel export

- Generate XLSX workbook:
 - sheet per table
 - columns per fields
 - required marker

- basic validations
- add hidden metadata sheet with template_id/version/hash

✅ **Worth documenting:** export rules + metadata fingerprint (template_id, version, hash).

5) Out of scope (explicitly excluded)

You told me several exclusions — we'll lock them.

Excluded:

- Data submission / ingestion / validation of uploaded Excel
- OCR
- Runtime schema drift handling
- User onboarding, identity management, authn/authz logic
- Approval workflow
- Complex workflow engine (BPMN)

✅ **Worth documenting:** out-of-scope list in bold. This prevents AI agents/other devs from “helpfully” building extra stuff.

6) Security boundary (Keycloak)

You said: **this service will not implement identity/auth** because Keycloak handles it.

Be careful: that doesn't mean “no security logic”. It means:

What this service WILL do

- Validate incoming requests contain a valid JWT (from Keycloak) OR rely on API gateway to validate it.
- Use token claims minimally (e.g., userId for audit fields).
- Enforce basic authorization only if required (optional for MVP if gateway handles it).

What this service will NOT do

- user login screens
- user management
- role management
- password reset etc.

✅ **Worth documenting:** “Security model: Keycloak external; Template service trusts gateway/Keycloak for authn; uses claims for auditing.”

7) Integration boundary (how templates move to the other system)

You said: the service exports Excel to another system, where users download.

There are two clean patterns — pick one for MVP and document it.

Option 1 (simplest): Pull model

Downstream system calls:

- `GET /templates/{id}/versions/{v}/export` and stores the file.

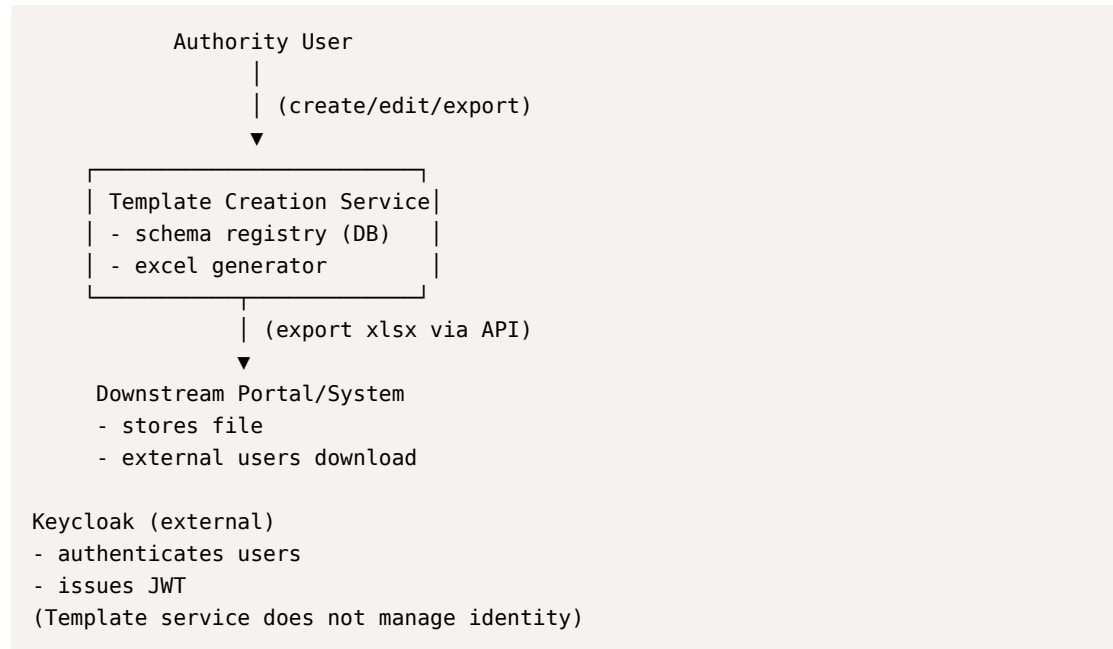
Option 2: Push model

Template service generates and uploads to object storage (S3/MinIO) and returns a signed URL; downstream stores reference.

MVP recommendation: Pull model (simplest, fewer moving parts).

✓ **Worth documenting:** integration pattern chosen (pull vs push) and why.

One-page boundary diagram (note-worthy)



✓ **Worth documenting:** this diagram (it prevents 80% of misunderstanding).

Hard constraints (HEP-style: don't ignore these)

1. **Versioning is not optional** even without approval. Without it you'll break downstream consumers silently.
2. **Excel is a generated artifact**; do not treat it as the source of truth.
3. **Auth is external (Keycloak)** but you still need a clear "trust boundary" (gateway vs service).
4. **No data submission means no drift handling** — don't let anyone sneak that scope back in.

✓ **Worth documenting:** these as "Design Principles".

Your "notes" checklist for this step

Write these sections down exactly (handoff-ready):

1. Purpose / Problem statement
2. In-scope capabilities
3. Out-of-scope items
4. Actors (who uses it / who doesn't)
5. Trust boundary (Keycloak responsibility)
6. Integration pattern (pull/push)

7. Design principles (Excel derived, versioning required)

Step 2 will be: **Domain Model + Canonical Schema (Template → Table → Field)** and we'll keep it tight and implementation-ready.

▼ Step 2: Domain Model and Canonical Template Schema

Overview

PHASE B — Domain Stabilization (WHAT EXISTS)

This phase answers:

| *"What are the core business concepts, and how do they relate?"*

Step 2: Domain Model & Canonical Schema (you just did this)

Conceptual sub-steps inside Step 2

1. Ubiquitous Language

- Decide vocabulary once
- Same words in docs, code, DB, APIs

2. Core Entities

- What "things" exist in the system?
- Ignore UI, DB, APIs — think real-world concepts

3. Ownership & Aggregates

- Which entity owns which data?
- Where does change happen?

4. Lifecycle Rules

- What can change?
- When does it become immutable?
- Why?

5. Canonical Representation

- Single source of truth (JSON, model, etc.)
- Everything else derives from it

6. Constraints & Invariants

- What must ALWAYS be true?
- What combinations are invalid?

7. Output Mapping

- How domain concepts transform into artifacts (Excel)

8. Traceability & Fingerprinting

- How do we prove "this output came from this version"?

🧠 **Mental shortcut to remember Step 2:**

Language → Entities → Ownership → Lifecycle → Canonical Form → Constraints

✅ This step prevents **refactor-by-realization** ("oh... we modeled this wrong").

Why we do Step 2 (conceptual)

Right now you have UI screens, but UI is “presentation.”

To avoid refactors, we need a **stable mental model** (domain) that code, DB, and APIs will follow.

Think of it like defining the **grammar** before writing sentences.

✅ **Worth documenting:** “Domain model is the single reference for DB + APIs + export logic.”

2.1 Ubiquitous Language (terms you must use consistently)

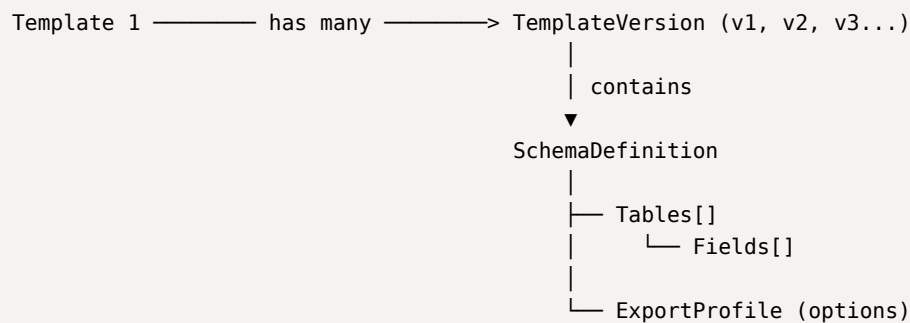
These words must mean *one thing* everywhere (docs, code, DB, APIs):

- **Template:** a named template concept (e.g., “Cement Sector Template”)
- **Template Version:** an immutable snapshot of a template schema at a point in time
- **Schema:** the canonical JSON definition stored for a version
- **Table:** logical dataset group → becomes an Excel **Sheet**
- **Field:** a column definition → becomes an Excel **Column**
- **Validation Rule:** constraints on a field (enum list, numeric range, required)
- **Export:** generating an XLSX from a version schema
- **Metadata Fingerprint:** template_id + version + schema_hash embedded in Excel

✅ **Worth documenting:** this glossary. It prevents naming drift across devs/agents.

2.2 Domain objects (conceptual model)

Conceptual diagram



Key point (non-negotiable)

Only TemplateVersion owns the schema.

Template is just the “container identity.”

✅ **Worth documenting:** “Schema lives in versions, not in Template.”

2.3 Lifecycle rule (since you removed approvals)

You removed approval workflow, but you still need **immutability** semantics.

Rule

- **A version is immutable once published/exported** OR at least once a new version is created.
- Changes should happen by **creating a new version**, not editing old ones.

You can choose the MVP rule:

MVP recommended rule (simple):

- Only the **latest version** is editable
- Once you create **v2**, **v1** becomes read-only automatically

✔ **Worth documenting:** "Version immutability rule" (exact rule chosen).

Why this matters: without it, downstream systems can download v1 today and get different content tomorrow under the same version number → disaster.

2.4 Canonical Schema JSON (v0.1) — the "source of truth"

This is the structure you store in DB (JSONB) and use to export Excel.

SchemaDefinition v0.1 (conceptual + implementation-ready)

```
{
  "templateName": "Cement Sector Template",
  "sectorCode": "CEMENT",
  "tables": [
    {
      "tableKey": "company_profile",
      "sheetName": "Company Profile",
      "order": 1,
      "fields": [
        {
          "fieldKey": "company_name",
          "headerName": "Company Name",
          "type": "TEXT",
          "required": true,
          "validations": {}
        },
        {
          "fieldKey": "employees_count",
          "headerName": "Employees Count",
          "type": "NUMBER",
          "required": false,
          "validations": { "min": 0, "max": 100000 }
        },
        {
          "fieldKey": "company_type",
          "headerName": "Company Type",
          "type": "TEXT",
          "required": true,
          "validations": { "enum": ["Public", "Private", "Government"] }
        }
      ]
    }
  ],
  "exportProfile": {
    "format": "XLSX",
    "includeInstructionsSheet": true,
    "protectSheets": false
  }
}
```

```
}  
}
```

✓ **Worth documenting:** this exact JSON structure + rules.

2.5 Schema constraints (rules you enforce early)

These are “guardrails.” Without them, templates become garbage.

Table rules

- `tableKey` unique within version
- `sheetName` must be valid Excel sheet name (≤ 31 chars, no forbidden characters)
- `order` unique or at least sortable

Field rules

- `fieldKey` unique within table
- `headerName` required and human-readable
- `type` must be from controlled enum
- validations depend on type:
 - enum allowed for TEXT
 - min/max allowed for NUMBER
- required is boolean

✓ **Worth documenting:** rules table (this is what you’ll implement in validation).

2.6 Mapping to Excel (how schema becomes a workbook)

This is the simple mapping every dev must understand:

Schema concept	Excel concept
TemplateVersion	Workbook
Table	Sheet
Field.headerName	Column header
Field.required	* suffix in header
validations.enum	Dropdown validation
validations.min/max	Numeric validation
Metadata fingerprint	Hidden <code>__metadata__</code> sheet

✓ **Worth documenting:** this mapping table.

2.7 Metadata fingerprint (must-have even now)

Even though you’re not building ingestion, you’re exporting to another system. That system needs to trust what it downloads.

Embed a hidden sheet:

`__metadata__` (hidden)

- `template_id`
- `version_number`

- schema_hash (SHA-256 of canonical JSON)
- generated_at
- generator_version

✓ **Worth documenting:** metadata sheet contents + why it exists.

2.8 Aligning with the Spring system design book (so you practice it)

This step matches the book's core system design principles:

- **Ubiquitous language** → stable naming across layers
- **Domain-first** design → business objects drive structure
- **Separation of concerns** → schema definition vs export rendering
- **Immutability/versioning** → prevents unintended breaking changes

✓ **Worth documenting:** "How domain maps to layers" (quick note).

Output of Step 2 (what you should have written down)

By the end of Step 2, your notes should include:

1. Glossary (Template, Version, Table, Field, Export, Fingerprint)
2. Domain diagram (Template → Versions → Schema → Tables → Fields)
3. Version immutability rule (your chosen MVP rule)
4. Canonical Schema JSON v0.1 (example + field type list)
5. Schema constraints/validation rules
6. Mapping table: Schema → Excel
7. Metadata fingerprint spec

That's the stable foundation.

Next (Step 3 preview)

Step 3 will be: **API contract + DB model** derived strictly from this domain model. No invention.

If you want, I'll start Step 3 immediately in the next message and produce:

- DB tables (templates, template_versions)
- endpoints (create template/version, update schema, export)
- error codes
- and how the downstream "pull" fits.

How Step 1 & Step 2 relate (important mental link)

Step1 defines:

WHAT the system is responsible for

Step2 defines:

WHAT exists inside that responsibility

If Step 1 is weak → scope creep

If Step 2 is weak → refactor hell

They must be done **before APIs, DB, or code**.

▼ Step 3 — Interaction & Persistence Design

(How the domain is stored and how the outside world talks to it)

Why Step 3 exists (conceptual)

Step 1 answered **WHY & SCOPE**

Step 2 answered **WHAT EXISTS**

Step 3 answers:

▮ **“How does the outside world interact with the domain, and how is the domain persisted safely?”**

This is where:

- APIs are shaped
- DB structure is fixed
- integration becomes predictable
- future refactors are either avoided or guaranteed

HEP note:

If you do Step 3 *before* Step 2 → refactor later

If you do Step 3 *after* Step 2 → system grows cleanly

Conceptual Map for Step 3 (Reusable)

Whenever you design Step 3 in **any system**, think in this exact order:

- A. Persistence strategy (how data lives)
- B. API responsibility (what can be done)
- C. API shape (how it's exposed)
- D. Rules & errors (what is forbidden)
- E. Integration fit (how others consume it)

Keep this order. Don't jump.

A. Persistence Strategy (DB thinking, not tables yet)

Conceptual questions

- What is **long-lived** data?
- What must be **immutable**?
- What is **derived** and not stored?

Applied to your system

- **Template** → long-lived identity
- **TemplateVersion** → long-lived, versioned, mostly immutable
- **Schema JSON** → stored as data (JSONB)
- **Excel file** → derived artifact → NOT stored (for MVP)

Key decision (already implied, now locked)

- DB is **schema-registry style**, not “one table per template”
- Versioning is **explicit**, not implicit

✓ **Worth documenting:** “Persistence model rationale: schema registry + versioning, Excel not persisted.”

B. API Responsibility (what operations exist)

This is not about endpoints yet — it’s about **capabilities**.

Capability categories (mental model)

1. Identity operations

- Create template

2. Evolution operations

- Create new version
- Update schema of latest version

3. Read operations

- Fetch template/version

4. Derivation operations

- Export Excel

Notice what’s missing:

- No submission
- No approval workflow
- No user management

Why this matters

If you list capabilities wrongly, APIs will be wrong even if endpoints “work”.

✓ **Worth documenting:** list of supported capabilities (not endpoints).

C. API Shape (how capabilities are exposed)

Now we translate **capabilities** → **REST shape**.

Conceptual rules (from book + practice)

- APIs reflect **domain language**, not UI screens
- Version is a **first-class resource**
- Export is an **action**, not CRUD

Resource model (conceptual)

```
/templates
/templates/{templateId}
/templates/{templateId}/versions
/versions/{versionId}
/versions/{versionId}/schema
/versions/{versionId}/export
```

This structure:

- mirrors your domain
- avoids “fat endpoints”

- scales cleanly later

✓ **Worth documenting:** API resource tree (not full OpenAPI yet).

D. Rules, invariants & errors (this prevents chaos)

This is the most skipped part by juniors — and the most expensive to skip.

Core invariants

- Only **latest version** is editable
- Older versions are read-only
- Export always references a **specific version**
- Schema hash changes **only** when schema changes

Error semantics (conceptual)

- Editing non-editable version → **409 Conflict**
- Invalid schema → **400 Bad Request**
- Version/template not found → **404**
- Unauthorized (via gateway/Keycloak) → **401/403**

These are **business rules**, not technical ones.

✓ **Worth documenting:** invariants + error behavior table.

E. Integration Fit (Pull model, finalized)

You already decided this, now we formalize it.

Pull integration pattern

- Downstream system:
 - calls `export`
 - downloads XLSX
 - stores/distributes it
- Template service:
 - remains stateless regarding files
 - does not care who downloads later

Why this is clean

- No file lifecycle management here
- Clear ownership
- Easy to replace downstream system later

✓ **Worth documenting:** "Integration contract: pull-based export."

What Step 3 produces (tangible outputs)

By the end of Step 3, you should have **documents**, not code:

1. Persistence strategy description

2. Capability list (what the system can do)
3. API resource tree
4. Business invariants & error rules
5. Integration contract (pull model)

This is enough for **any developer or AI agent** to implement correctly.

How Step 3 connects to the book

This step directly practices:

- Layered architecture thinking
- Domain-driven API design
- Separation of domain vs transport
- Avoiding anemic models
- Designing for change (versioning)

You're not memorizing the book — you're **internalizing it**.

Very important HEP reminder

At this point:

-  Do NOT generate code
-  Do NOT ask AI for full implementation
-  DO freeze these decisions

Once frozen, refactors drop dramatically.

Next Step (Step 4 preview)

Step 4 = Behavior & Flow

- Sequence diagram for:
 - Update schema
 - Export Excel
- Where validation, hashing, and generation happen

This is where functions and services start to “click” in your head.

▼ Step 4 — Behavior & Flow Design

Why Step 4 exists (conceptual)

Step 3 defined **what the APIs and persistence are**.

Step 4 defines **how requests execute internally**:

- Which component owns which responsibility
- Where validation happens
- Where hashing happens
- How Excel is generated deterministically
- What errors happen and when

 **Worth documenting:** Step 4 diagrams become your “source of truth” for implementation.

Step 4 conceptual map (reusable)

For any system, Step 4 usually contains:

1. **Happy-path sequence** for the most important use cases
2. **Failure-path rules** (where it can break)
3. **Collaboration diagram** (which modules talk)
4. **State transitions** (if any)

For your system, the **two “highest-risk” flows** are:

- Update Schema (because hashing/version rules can be wrong)
 - Export Excel (because generator responsibilities can be wrong)
-

Internal component collaboration (how modules talk)

This is the “collaboration map” for your microservice:

```
Controller → UseCase(Service) → Domain Policies → Repository → DB
               ↳ (for export) ExcelBuilder → POI → Stream
               ↳ Hashing/Canonicalizer
               ↳ SchemaValidator
```

```
flowchart LR
    C[Controller] --> U[UseCase / Service]
    U --> P[Domain Policies]
    P --> R[Repository]
    R --> D[(Database)]

    U --> V[SchemaValidator]
    U --> H[JsonCanonicalizer + HashService]

    U --> E[ExcelBuilder]
    E --> POI[Apache POI]
    POI --> S[HTTP Stream Response]
```

✔ **Worth documenting:** This as your internal layering contract (aligned with the Spring design book).

SD-01 Sequence Diagram: Update Schema for a Version

This is the *core* write flow. It must be deterministic.

Sequence (Mermaid)

```
sequenceDiagram
    autonumber
    actor Client as Downstream/UI/Caller
    participant API as Template API Controller
    participant UC as UpdateSchemaUseCase
    participant Repo as TemplateVersionRepository
    participant Val as SchemaValidator
    participant Can as JsonCanonicalizer
```

```

participant Hash as HashService(SHA-256)
participant DB as PostgreSQL

Client->>API: PUT /versions/{versionId}/schema (schema_json)
API->>UC: updateSchema(versionId, schema_json, userClaims)
UC->>Repo: findById(versionId)
Repo->>DB: SELECT template_version
DB-->>Repo: version row
Repo-->>UC: TemplateVersion

UC->>UC: enforceEditabilityRule(version)
Note over UC: Only latest version editable (MVP rule)

UC->>Val: validate(schema_json)
Val-->>UC: ok OR validation errors

UC->>Can: canonicalize(schema_json)
Can-->>UC: canonical_json_string

UC->>Hash: sha256(canonical_json_string)
Hash-->>UC: schema_hash

UC->>Repo: save(version with schema_json + schema_hash + audit)
Repo->>DB: UPDATE template_versions ...
DB-->>Repo: ok
Repo-->>UC: saved

UC-->>API: return version + schema_hash
API-->>Client: 200 OK

```

Failure paths worth defining (don't skip)

- Version not found → 404
- Version not editable (not latest or locked) → 409 Conflict
- Schema invalid (missing keys, duplicates, bad sheet names) → 400 with details

✔ **Worth documenting:** a small table: "failure → status code → message".

SD-02 Sequence Diagram: Export Excel

This is your "wow" demo flow and also easiest to implement wrong.

Sequence (Mermaid)

```

sequenceDiagram
    autonumber
    actor Client as Downstream System
    participant API as Export Controller
    participant UC as ExportExcelUseCase
    participant Repo as TemplateVersionRepository
    participant DB as PostgreSQL
    participant Builder as ExcelWorkbookBuilder
    participant Sheet as SheetBuilder

```

```

participant V as ValidationBuilder
participant Meta as MetadataSheetWriter
participant POI as Apache POI (SXSSF)

Client->>API: POST /versions/{versionId}/export?format=xlsx
API->>UC: export(versionId)
UC->>Repo: findById(versionId)
Repo->>DB: SELECT template_version
DB->>Repo: version row (schema_json + hash)
Repo-->>UC: TemplateVersion

UC->>Builder: buildWorkbook(schema_json)
Builder->>POI: create workbook

loop for each table
  Builder->>Sheet: buildSheet(table)
  Sheet->>POI: create sheet(sheetName)
  Sheet->>POI: write header row (required -> "*")
  Sheet->>V: applyValidations(fields)
  V->>POI: add dropdown/range validations
end

Builder->>Meta: writeMetadata(templateId, versionId, versionNumber, schema_hash,
timestamp)
Meta->>POI: create hidden "__metadata__" sheet + hide it

Builder-->>UC: workbook stream/bytes
UC-->>API: file stream + headers
API-->>Client: 200 OK (attachment .xlsx)

```

Export invariants (must document)

- Export is always tied to **versionId**
- Export must embed **schema_hash + version info**
- Export must enforce Excel sheet name rules (≤31 chars, no forbidden chars)
- Export must be deterministic for the same schema (content stable)

✓ **Worth documenting:** Export rules + metadata sheet spec (you already started this in Step 2, now it's "flow-anchored").

Flow Diagram: Versioning without approvals

You removed approvals, but we still need a simple "version evolution" behavior.

Conceptual flow

```

flowchart LR
  A[Create Template] --> B[Create Version v1]
  B --> C[Edit v1 schema]
  C --> D[Export v1]
  D --> E[Create Version v2 - clone v1]
  E --> F[Edit v2 schema]
  F --> G[Export v2]

```



```
C -->|locked after v2 created| X[v1 read-only]
```

✔ **Worth documenting:** "Editability rule: only latest version editable; older versions read-only."

Use Case Diagrams ??

"Use Case Diagram omitted intentionally: system has a single authority actor and API-driven interactions; behavior is fully described via sequence diagrams and API contracts."

What is "worth noting down" from Step 4 (handoff-ready)

If you only document 6 things from Step 4, document these:

1. **Internal collaboration map** (Controller → UseCase → Validator/Hasher → Repo → DB; Export → Builder → POI)
2. **SD-01 Update Schema** sequence diagram
3. **SD-02 Export Excel** sequence diagram
4. **Editability rule** (latest version only)
5. **Failure/status-code table** (409/400/404)
6. **Determinism rules** (hashing + metadata)

This is the set that prevents most refactors.

Next step (Step 5)

Step 5 is where we turn everything into **incremental agile slices + AI-agent prompts safely**:

- Sprint plan (vertical slices)
- Definition of Done per slice
- "Safe AI prompting" per slice (DB first, then endpoints, then export)

▼ Spring Architecture — Freeze architecture document

The architecture of the application must NOT be "handled" by the AI agent.

It should be **defined by you**, and the AI should implement *inside it*.

If you let the agent decide architecture, you'll get:

- random layering
- mixed responsibilities
- logic inside controllers
- Excel code inside services
- domain objects polluted with JPA annotations everywhere
- refactor in 2 weeks

What should happen instead

You define:

- 🏗️ Architectural style
- 🧱 Layer boundaries
- 📦 Package/module structure
- 🔄 Responsibility rules

Then AI fills in code.

For YOUR project specifically

Let's lock the architecture now (conceptually, not code).

You're building:

- Spring Boot microservice
- Schema registry style
- Deterministic export engine
- Versioned domain
- External auth (Keycloak)

So the architecture should be:

Recommended Architecture (Clean + Spring-friendly)

High-Level Style

Layered architecture with domain separation

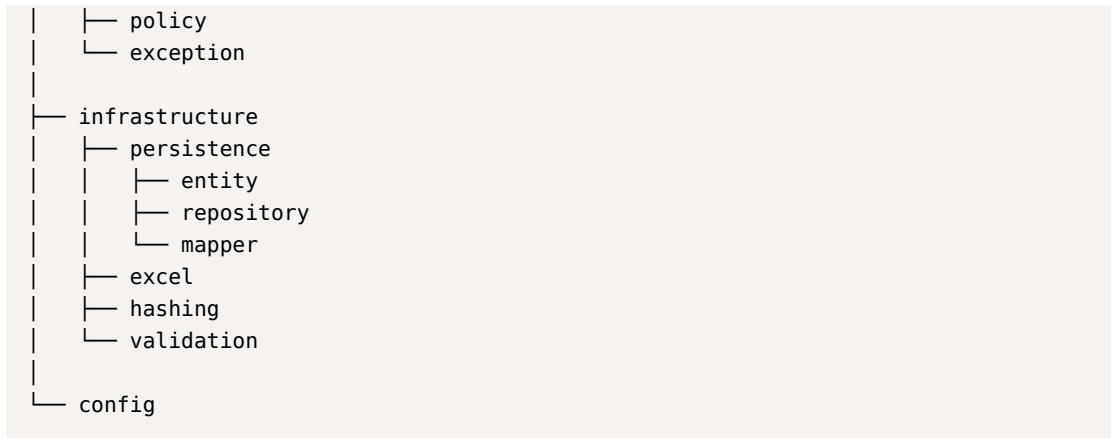
```
API Layer (Controllers)
↓
Application Layer (UseCases / Services)
↓
Domain Layer (Business Rules / Models)
↓
Infrastructure Layer (DB, Excel, Hashing, External tech)
```

This aligns perfectly with the book you're reading.

Concrete Package Structure (define this BEFORE coding)

```
com.yourcompany.template
```

```
├── api
│   ├── controller
│   └── dto
├── application
│   ├── usecase
│   └── service
├── domain
└── model
```



This prevents:

- controllers knowing about POI
- Excel builder touching repositories directly
- domain knowing about JPA
- validation scattered randomly

Responsibilities Per Layer (very important)

1 Controller (API Layer)

- Parse request
- Call use case
- Return response
- No business logic
- No DB calls
- No Excel logic

If you ever see Excel code here → architecture violation.

2 Application / UseCase Layer

- Orchestrates flow
- Calls domain policies
- Calls repositories
- Calls Excel builder
- Coordinates hashing + validation

This is your “brain” layer.

3 Domain Layer

- Template
- TemplateVersion
- Business invariants

- Editability rule
- Schema rules

No Spring annotations ideally.

Pure Java logic.

4 Infrastructure Layer

- JPA entities
- Repositories
- Apache POI implementation
- SHA-256 hashing
- JSON canonicalization

Replaceable components.

Why this matters

If you skip this and let AI “structure the project”:

- It will mix domain with persistence.
- It will add random service classes.
- It will collapse everything into one package.
- It will make export logic static utility functions.

That's how refactors start.

So what is the correct workflow?

1.  You define architecture (like above)
 2.  You document layer responsibilities
 3.  You give that structure to AI
 4.  AI generates code inside boundaries
 5.  You review for boundary violations
-

Quick mental rule to memorize

Architecture is human responsibility.

Implementation is AI assistance.

Very Important

You noticed something important:

Your previous Cursor prompt defined features, but **did not explicitly freeze architecture structure**.

That was a gap.

Good catch.

If you want, next I can:

- Generate a **“Architecture Freeze Document”** you can paste at the top of your repo and give to Cursor before any coding starts.

That document would:

- lock layering
- lock package structure
- lock responsibility rules
- prevent architectural drift

Freeze architecture document

and I’ll generate it clean and professional.

PROMPT #0 – ARCHITECTURE FREEZE (READ FIRST, DO NOT CODE FEATURES YET)

You are an expert Senior Java/Spring Boot architect. Before implementing any features, you must freeze the architecture and project structure for a “Template Creation” microservice.

Scope reminder:

- This service is ONLY Template schema management + Excel export.
- NO data submission/ingestion/OCR.
- NO approval workflow.
- AuthN/AuthZ/Identity is handled externally via Keycloak / API Gateway. Do NOT implement login/user management. At most, read user info from request headers/claims for auditing.

Tech stack:

- Java 21, Spring Boot 3.x
- PostgreSQL 15+, JSONB
- Flyway
- Apache POI (SXSSF) for Excel export
- springdoc-openapi (Swagger UI)
- Actuator
- Docker + docker-compose

ARCHITECTURE STYLE (MUST FOLLOW)

Use a clean layered architecture with explicit separation:

1. API layer (HTTP)
 - Controllers + request/response DTOs only
 - No business logic, no Excel logic, no repository calls directly
2. Application layer (Use Cases)
 - Orchestrates flows such as:
 - CreateTemplate
 - CreateVersion
 - UpdateSchema (validate + canonicalize + hash + persist)
 - ExportExcel (load schema + generate workbook + stream)

- Calls domain policies, repositories (via ports or directly via interfaces), and infrastructure services.

3. Domain layer (Business rules)

- Pure Java domain model and policies:
 - Template, TemplateVersion
 - Editability rules (only latest version editable)
 - Schema invariants (unique keys, etc.)
- Avoid Spring/JPA annotations in domain if possible.
- Domain exceptions.

4. Infrastructure layer (Technical implementations)

- Persistence (JPA entities, repositories, mappers)
- Excel export (Apache POI builders)
- Hashing + JSON canonicalization
- Schema validation utilities

PACKAGE / FOLDER STRUCTURE (MUST CREATE)

Create this package structure exactly:

```
com.yourcompany.template
├── api
│   ├── controller
│   └── dto
├── application
│   ├── usecase
│   └── service
├── domain
│   ├── model
│   ├── policy
│   └── exception
├── infrastructure
│   ├── persistence
│   │   ├── entity
│   │   ├── repository
│   │   └── mapper
│   ├── excel
│   ├── hashing
│   └── validation
└── config
```

RESPONSIBILITY RULES (NON-NEGOTIABLE)

- Controllers may only call application/usecase classes.
- Application layer may call:
 - domain policies
 - infrastructure adapters (repositories, excel builder, hashing/canonicalizer, validator)
- Domain layer must not depend on Spring, DB, Apache POI, or web classes.
- Infrastructure contains all JPA annotations, POI code, hashing code.
- NEVER put Apache POI code in controllers or domain.
- NEVER use JPA entities as API DTOs.
- NEVER put validation/hashing scattered across controllers.

CROSS-CUTTING

- Use `@ControllerAdvice` for consistent error responses.
- Use Bean Validation (`@Valid`) for basic request validation, but keep business invariants in domain/application.
- Add Actuator health endpoint.
- Add Swagger UI via `springdoc`.

DELIVERABLES FOR THIS PROMPT

Generate ONLY:

1. The project skeleton with the above package structure (empty classes/interfaces as needed).
2. A short `[ARCHITECTURE.md]`(<http://architecture.md/>) file at repo root explaining:
 - layer responsibilities
 - what is in/out of scope
 - key invariants (latest version editable; Excel is derived)
3. A minimal `ErrorResponse` model and a `GlobalExceptionHandler` skeleton.
4. A minimal README with:
 - how to run (docker compose placeholder ok)
 - architecture overview
 Do NOT implement business endpoints or DB migrations yet.

After you finish these deliverables, STOP.

(Next prompts will implement Slice 0–5 features on top of this structure.)

▼ Step 5 — Incremental slices that prevent refactor

Why Step 5 exists (conceptual)

AI agents are fastest when you give them **small, bounded problems** with clear contracts.

If you ask for “build everything”, they invent structure and you refactor.

So Step 5 is about:

- slicing work into **vertical increments**
- defining **Definition of Done**
- writing **micro-prompts** that force the agent to follow your structure

✅ **Worth documenting:** the sprint slices + DoD + AI prompt rules (this becomes your execution playbook).

Step 5 conceptual map (reusable)

For any system, do this:

1. **Define vertical slices** (each slice delivers a working capability)
2. **Define DoD per slice** (tests + docs + run instructions)
3. **Create prompt templates** (so AI can't go off-track)
4. **Review checkpoints** (architecture drift checks)

5.1 MVP slices for Template Creation Service (backend+DB only)

Slice 0 — Repo scaffold (foundation)

Goal: project runs + swagger + db up via docker-compose

Includes:

- Spring Boot app skeleton
- Actuator health endpoints
- Swagger UI
- docker-compose (service + postgres)
- Flyway integrated

DoD:

- `docker compose up` brings everything up
- `/actuator/health` returns UP
- Swagger UI loads

✓ **Worth documenting:** runbook commands + ports + env vars.

Slice 1 — Persistence model (templates + versions)

Goal: DB schema + entities + repositories working

Includes:

- Flyway migrations for `templates`, `template_versions`
- JPA entities + repositories
- basic error handling skeleton

DoD:

- migrations auto-run on startup
- integration test hits DB (optional but recommended)
- can insert + retrieve via repository test

✓ **Worth documenting:** ERD + DDL + indexing rules.

Slice 2 — Create template + version APIs

Goal: working API creates Template and v1

Endpoints:

- `POST /api/templates`
- `GET /api/templates/{id}`
- `POST /api/templates/{id}/versions` (clone latest)

DoD:

- curl examples in README
- returns IDs + version number correctly
- new version clones schema_json

✓ **Worth documenting:** API request/response examples.

Slice 3 — Update schema + validate + canonical hash

Goal: schema becomes source of truth with deterministic fingerprint

Endpoint:

- `PUT /api/versions/{versionId}/schema`

Behavior:

- validate schema structure (tables/fields uniqueness, sheet name rules)
- canonicalize JSON
- compute SHA-256 hash
- store schema_json + schema_hash
- enforce editability rule (only latest version editable)

DoD:

- same schema → same hash always
- invalid schema → 400 with clear errors
- edit non-latest → 409

✓ **Worth documenting:** invariants + error table + hashing rules.

Slice 4 — Excel export engine (core deliverable)

Goal: export XLSX that matches schema

Endpoint:

- `POST /api/versions/{versionId}/export`

Includes:

- Apache POI (SXSSF)
- sheets per table
- headers per field (for required)
- enum dropdown validation
- numeric min/max validation (as feasible)
- hidden `__metadata__` sheet with template/version/hash/timestamp

DoD:

- exported file opens in Excel
- metadata sheet exists and is hidden
- validations present for enum
- deterministic export content for same schema

✓ **Worth documenting:** export rules + metadata spec.

Slice 5 — Hardening (CTO-ready)

Goal: reliability + maintainability

Includes:

- consistent error response model
- logging
- test coverage for critical parts

- final README (runbook + API examples)

DoD:

- smoke test end-to-end
- export test verifies sheet count + metadata
- README can be followed by any dev

✓ **Worth documenting:** DoD + test strategy.

5.2 AI-agent usage rules (HEP: follow these or refactor later)

Rule A — One slice per prompt

Never ask Cursor to build multiple slices at once.

Rule B — Provide contracts, not goals

You give:

- domain model
- DB schema
- API resource tree
- sequence diagram rules

Then ask it to implement *exactly that*.

Rule C — Require “no invention”

Explicitly say:

- “Do not add new features”
- “Do not add submission”
- “Do not add approval workflow”
- “Auth is external via Keycloak; no login screens”

✓ **Worth documenting:** “AI prompt rules” as a policy section.

5.3 Copy-paste micro-prompts for Cursor (safe and incremental)

Below are the prompts you can paste into Cursor **one by one**.

Prompt for Slice 0 — Scaffold

```
Implement Slice 0 only: Spring Boot 3 (Java 21) project scaffold for Template Creation Service.
Requirements:
- Add dependencies: web, actuator, validation, springdoc-openapi UI, flyway, data-jpa, postgres driver.
- Provide Dockerfile and docker-compose.yml (service + Postgres 15).
- Configure app to run Flyway on startup.
```

- Add /actuator/health endpoint enabled.
- Add Swagger UI enabled.

Out of scope: any business logic, any data submission, any approval workflow, any UI.

Deliverables: runnable project + README with docker compose up and how to open Swagger and health.

or prefer this:

Implement Slice 0 only on top of the Architecture Freeze (Prompt #0). Do NOT change the package structure or layering.

Spring Boot 3 (Java 21) project scaffold for Template Creation Service.

Requirements:

- Add dependencies: spring-boot-starter-web, actuator, validation, springdoc-openapi UI, flyway, data-jpa, postgres driver.
- Provide Dockerfile and docker-compose.yml (service + Postgres 15).
- Configure app to run Flyway on startup.
- Enable /actuator/health endpoint.
- Enable Swagger UI.

Out of scope: any business logic, any data submission, any approval workflow, any UI beyond Swagger.

Deliverables:

- Runnable project via `docker compose up`
 - README: how to run, how to open Swagger UI, how to hit /actuator/health
- Stop after completing Slice 0.

Prompt for Slice 1 — DB + persistence

Implement Slice 1 only: persistence layer for templates and template_versions.

Use Flyway migrations to create tables and indexes:

- templates(id uuid pk, name, sector_code, status, created_at, created_by)
- template_versions(id uuid pk, template_id fk, version_number int, status, schema_json jsonb, schema_hash text, created_at, created_by)

Constraints: unique(template_id, version_number). Indexes on template_id and schema_hash.

Create JPA entities and repositories. No controllers yet.

Add minimal integration test (optional) or repository test.

Out of scope: export, schema validation, submission, approvals, auth logic (Keycloak is external).

or prefer this:

Implement Slice 1 only on top of the Architecture Freeze (Prompt #0). Do NOT change package structure or layering.

No controllers, no REST endpoints in this slice.

GOAL

Implement the persistence layer for templates and template_versions using PostgreSQL + Flyway + JPA, aligned with a schema-registry approach (schema stored as JSONB).

DATABASE (Flyway migrations)

Create tables + constraints + indexes exactly:

1) templates

- id UUID PK
- name TEXT NOT NULL
- sector_code TEXT NOT NULL
- status TEXT NOT NULL
- created_at TIMESTAMP NOT NULL
- created_by TEXT NOT NULL

2) template_versions

- id UUID PK
- template_id UUID NOT NULL FK -> templates(id) ON DELETE CASCADE
- version_number INT NOT NULL
- status TEXT NOT NULL (since there is NO approval workflow, use statuses like: DRAFT, READ_ONLY, DEPRECATED)
- schema_json JSONB NOT NULL (this is the canonical source of truth)
- schema_hash TEXT NOT NULL (for Slice 1/2 it can be a placeholder e.g., 'PENDING_HASH')
- created_at TIMESTAMP NOT NULL
- created_by TEXT NOT NULL

Constraints / indexes:

- UNIQUE (template_id, version_number)
- INDEX on template_id
- INDEX on schema_hash

UUID strategy (choose one and implement consistently):

- Prefer DB-generated UUID defaults (Postgres): use `gen_random_uuid()` and ensure required extension is enabled (`pgcrypto`).
- If you choose app-generated UUIDs instead, remove DB defaults and configure JPA accordingly.

Pick ONE approach and document it in README.

JPA MAPPING (must)

- Keep JPA entities in: `infrastructure/persistence/entity`
- Repositories in: `infrastructure/persistence/repository`
- Domain layer must NOT contain `@Entity` annotations.
- Map `schema_json` as JSON (not plain String):
 - Use Hibernate 6 JSON support: `@JdbcTypeCode(SqlTypes.JSON)`
 - Store as `JsonNode` (recommended) or `Map<String, Object>`.
- Ensure NOT NULL is respected for `schema_json` and `schema_hash`.

TESTING (minimal but real)

Add at least one repository-level test that:

- creates a Template
- creates TemplateVersion v1 linked to it (with `schema_json` minimal placeholder and `schema_hash` 'PENDING_HASH')
- persists and reads back successfully

Preferred: Testcontainers Postgres (recommended). Avoid H2 for JSONB correctness.

OUT OF SCOPE

- REST controllers / API endpoints
- schema validation rules (Slice 3)

- hashing implementation (Slice 3)
- excel export (Slice 4)
- data submission/ingestion/OCR
- identity management/auth flows (Keycloak is external)

DELIVERABLES

- Flyway migration scripts
- JPA entities + repositories + any required converters/config
- One minimal repository integration test
- Update README with:
 - how to run DB via docker-compose
 - how migrations apply
 - UUID strategy chosen (DB vs app)

Stop after completing Slice 1.

Prompt for Slice 2 — Create template/version APIs

Implement Slice 2 only: REST APIs for creating and reading templates/versions.

Endpoints:

- POST /api/templates -> creates template + v1 draft with minimal schema_json and schema_hash placeholder, returns templateId, versionId, versionNumber
- GET /api/templates/{templateId} -> template details + versions summary
- POST /api/templates/{templateId}/versions -> creates new version by cloning latest schema_json, increments version_number
- GET /api/versions/{versionId} -> returns schema_json and schema_hash and metadata

Use consistent error response model. No UI.

Out of scope: schema update, hashing, excel export, approval workflow, submission. Include README curl examples.

or prefer this:

Implement Slice 2 only on top of the existing Architecture Freeze. Do NOT change package structure or layering.

Controllers must call application/usecase classes only (no repository access from controllers).

Implement REST APIs for creating and reading templates/versions.

Endpoints:

1) POST /api/templates

- Creates Template + v1 DRAFT version
- schema_json: store a minimal valid placeholder schema (empty tables array is OK)
- schema_hash: placeholder value allowed for Slice 2 (real hashing is Slice 3)

Response: templateId, versionId, versionNumber

2) GET /api/templates/{templateId}

- Returns template metadata + versions summary (id, version_number, status, created_at)
- Do NOT return the full schema_json for every version (only summary)

3) POST /api/templates/{templateId}/versions

- Creates new version by cloning the latest version's schema_json + schema_hash
- Increments version_number
- Enforce policy: after v2 is created, v1 becomes read-only (store status if you model it, or document it for Slice 3)

Response: versionId, versionNumber

4) GET /api/versions/{versionId}

- Returns version metadata + schema_json + schema_hash

Use consistent error response model (@ControllerAdvice):

- 404 if template/version not found
- 400 for invalid input

No UI.

Out of scope:

- schema update validation/hashing (Slice 3)
- excel export (Slice 4)
- approval workflow, submission, ingestion
- identity management (Keycloak external)

Include README curl examples for all endpoints.

Stop after completing Slice 2.

Prompt for Slice 3 — Schema update + validation + hashing

Implement Slice 3 only: PUT schema for a version with validation + canonical hashing.

Endpoint:

- PUT /api/versions/{versionId}/schema -> accepts schema_json with structure:
templateName, sectorCode, tables[{tableKey,sheetName,order,fields[{fieldKey,headerName,type,required,validations{enum?,min?,max?}}]}], exportProfile{}

Rules:

- Only latest version of a template is editable; older versions return 409.
- Validate uniqueness of tableKey per version and fieldKey per table.
- Validate Excel sheetName constraints (<=31 chars, no forbidden chars), return 400 if invalid.
- Canonicalize JSON deterministically and compute SHA-256 hex -> schema_hash.
- Save schema_json + schema_hash.

Error codes: 404 not found, 400 invalid schema, 409 not editable.

Out of scope: excel export, submission, approval workflow, auth screens. Keycloak external.

Add unit tests for canonical hash stability and validation failures.

or prefer this:

Implement Slice 3 only on top of the existing architecture. Do NOT change package structure or layering.

Controllers must call application/usecase only. No repository access from controllers.

GOAL

Implement schema update for a template version with:

- validation

- deterministic canonicalization
- SHA-256 hashing
- persistence
- editability rule (only latest version editable)

ENDPOINT

PUT /api/versions/{versionId}/schema

Request body = schema_json with structure:

```
{
  "templateName": "...",
  "sectorCode": "...",
  "tables": [
    {
      "tableKey": "...",
      "sheetName": "...",
      "order": 1,
      "fields": [
        {
          "fieldKey": "...",
          "headerName": "...",
          "type": "TEXT|NUMBER|DATE|BOOLEAN|CURRENCY",
          "required": true,
          "validations": { "enum": [...], "min": 0, "max": 10 }
        }
      ]
    }
  ],
  "exportProfile": { "format": "XLSX", "includeInstructionsSheet": true, "protectSheets": false }
}
```

BUSINESS RULES (MUST)

1) Editability:

- Only the latest version of a template is editable.
- If versionId is not the highest version_number for that template_id -> return 409 Conflict.
- Optionally update previous versions status to READ_ONLY when a new version is created (Slice 2/3 consistency).

2) Validation (return 400 with clear messages):

- templateName non-empty (optional for MVP, but recommended)
- sectorCode non-empty
- tables array may be empty but if present:
 - tableKey unique across tables
 - sheetName unique across tables (after trimming)
 - order must be integer; allow duplicates but sort by order (or enforce uniqueness; choose one and document)
- fields per table:
 - fieldKey unique within table
 - headerName non-empty
 - type must be in allowed enum
- Validation rules by type:
 - enum allowed only for TEXT (and maybe DATE if you decide; default: TEXT only)

- min/max allowed only for NUMBER/CURRENCY
- if min and max provided -> min <= max
- Excel sheetName constraints:
 - length <= 31
 - must not contain: \ / ? * []
 - must not be blank
- Reject schema if any rule violated; include which table/field caused it.

3) Canonicalization + Hashing (MUST be deterministic):

- Parse JSON to a strong model (SchemaDefinition POJOs) OR JsonNode
- Canonicalize by re-serializing with stable property ordering.
- Use SHA-256 hex of canonical JSON string -> schema_hash.
- Store schema_json (JSONB) and schema_hash (TEXT) in template_versions.

LAYERING (MUST)

- Controller: HTTP only
- UseCase/Application: orchestrates load -> check editability -> validate -> canonicalize -> hash -> save
- Infrastructure:
 - SchemaValidator component
 - JsonCanonicalizer component
 - HashService component

ERROR HANDLING

- 404 if version not found
- 409 if not editable (not latest)
- 400 if schema invalid, with structured errors

TESTS (MUST)

- Hash determinism test:
 - same schema content -> same hash across runs
- Validation tests:
 - duplicate tableKey -> 400
 - duplicate fieldKey -> 400
 - invalid sheetName char -> 400
 - min > max -> 400
- Editability test:
 - create v1 + v2, attempt to update v1 -> 409

OUT OF SCOPE

- Excel export (Slice 4)
- data submission/ingestion/OCR
- approval workflow
- auth screens/user mgmt (Keycloak external)

DELIVERABLES

- DTOs for request/response if needed (do not reuse entities)
- Use case + validator + canonicalizer + hash service
- Unit tests
- README update with curl example for PUT schema and explanation of 409/400 behaviors

Stop after completing Slice 3.

Prompt for Slice 4 — Export XLSX

```
Implement Slice 4 only: Excel export from schema using Apache POI (SXSSF).
Endpoint:
- POST /api/versions/{versionId}/export -> returns XLSX as attachment.
Behavior:
- Load schema_json and schema_hash from DB.
- Generate workbook: 1 sheet per table with sheetName.
- Row 1 headers = headerName (append * if required).
- Apply validations:
  - validations.enum -> Excel dropdown list
  - validations.min/max for NUMBER -> numeric constraints (as feasible)
- Add hidden sheet "__metadata__" with: template_id, version_id, version_number, s
chema_hash, generated_at, generator_version.
- Ensure sheet name sanitization if needed (but prefer rejecting invalid in Slice
3).
Add a basic export test to assert metadata sheet exists and is hidden.
Out of scope: storing files, submission, approvals, UI.
```

or prefer this:

```
Implement Slice 4 only on top of the existing architecture. Do NOT change package
structure or layering.
Controllers must call application/usecase only. No POI code in controllers or doma
in.
```

GOAL

```
Generate an XLSX template from a stored schema_json (JSONB) using Apache POI (SXSS
F) and return it as an HTTP attachment.
```

ENDPOINT

```
POST /api/versions/{versionId}/export
```

- Response: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
- Content-Disposition: attachment; filename="{templateName}_v{versionNumber}.xlsx"

BEHAVIOR (MUST)

```
1) Load version from DB by versionId:
```

- schema_json
- schema_hash
- template_id
- version_number
- template name (from templates table or schema_json.templateName; choose one c
onsistently)

```
2) Build workbook (Apache POI):
```

- Use SXSSFWorkbook for streaming (memory safe)
- One sheet per table using table.sheetName
- Row 1 = headers:
 - headerName
 - append " *" (or "**") if required=true (choose one formatting and keep consi
stent)

```
3) Apply validations:
```

- Enum dropdown:

- if validations.enum exists and is non-empty, apply DataValidation constraint for that column for a reasonable row range (e.g., rows 2..1000)
- Numeric min/max:
 - if type is NUMBER or CURRENCY and min/max exist, apply numeric validation constraint (where feasible in POI)
 - If validations are unsupported for a given type, ignore safely (but do not crash)

4) Metadata sheet (MUST):

- Create sheet named "__metadata__"
- Write key/value rows:
 - template_id
 - version_id
 - version_number
 - schema_hash
 - generated_at (ISO-8601)
 - generator_version (hardcode like "template-service/0.1")
- Hide metadata sheet using POI sheet visibility (veryHidden if possible)

5) Output:

- Stream workbook as response
- Ensure workbook is properly closed/disposed (SXSSF temp files cleaned)
- No file storage; generation is on-demand only

ERRORS

- 404 if version not found
 - 400 if schema_json is missing essential parts (tables/fields malformed) – ideally Slice 3 prevents this, but still guard.
- Use consistent error response model.

LAYERING REQUIREMENTS

- ExportExcelController: HTTP only
- ExportExcelUseCase (application): orchestration + calls Excel builder
- ExcelWorkbookBuilder (infrastructure/excel): POI-specific implementation

TESTS (MUST)

Add a basic export test:

- Prepare template + version with schema_json containing:
 - 2 tables (2 sheets)
 - 1 required field and 1 enum field
 - Call export use case (or controller with MockMvc) and parse returned XLSX with Apache POI:
 - Assert workbook has expected sheet names
 - Assert "__metadata__" sheet exists and is hidden
 - Assert header includes required marker for required field
- (Validation presence is hard to assert reliably; optional.)

OUT OF SCOPE

- Storing exported files (S3/MinIO)
- Submission/ingestion/OCR
- Approval workflow
- UI beyond Swagger

DELIVERABLES

- Export endpoint + use case + excel builder classes
 - Proper resource cleanup (close workbook)
 - Tests and README update with curl example
- Stop after completing Slice 4.

Prompt for Slice 5 — Hardening

Implement Slice 5 only: hardening and documentation.

- Standardize error response payload across controllers.
 - Add request logging.
 - Add actuator info and health.
 - Ensure README includes: run instructions, endpoint list, curl samples, export demo steps.
 - Add additional tests for lifecycle rules (editability) and export smoke.
- Do not add new features (no submission, no approvals, no auth management).

or prefer this:

Implement Slice 5 only: hardening + documentation + consistency. Do NOT change package structure or layering.

Do NOT add any new product features (no submission, no approvals, no auth management/UI). Keycloak is external.

GOALS

- 1) Consistent error handling
- 2) Practical request logging + traceability
- 3) Actuator readiness
- 4) Solid README/runbook
- 5) Add missing tests (editability + export smoke)

1) ERROR RESPONSE STANDARDIZATION (MUST)

- Ensure all controllers return the same ErrorResponse JSON shape via @ControllerAdvice.
- Include: timestamp, status, error, message, path, traceId (if available).
- Map exceptions consistently:
 - NotFound -> 404
 - Validation -> 400
 - NotEditable / Conflict -> 409
 - Generic -> 500 (do not leak stack traces)

2) REQUEST LOGGING + TRACEABILITY (MUST)

- Add lightweight request logging:
 - method, path, status, duration_ms, traceId
- Prefer using MDC and add a traceId per request (generate UUID if not provided).
- Do not log request bodies by default (avoid sensitive data).

3) ACTUATOR (MUST)

- Enable actuator health and info endpoints.
- Configure liveness/readiness if available.
- Add basic app info (name, version) to /actuator/info.

4) README / DOCS (MUST)

Update README with:

- Prerequisites (Java 21, Docker)
- How to run: `docker compose up --build`
- Ports and URLs:
 - Swagger UI
 - /actuator/health
- Endpoint list (Slice 2/3/4)
- Curl samples:
 - Create template
 - Create version
 - Update schema
 - Export xlsx
- Export demo steps (what to expect inside Excel, including hidden __metadata__ sheet)

5) TESTS (MUST)

Add/ensure tests for:

- Editability rule:
 - Create v1 and v2; PUT schema on v1 returns 409
- Export smoke:
 - Export returns a readable XLSX
 - Has expected sheets
 - __metadata__ sheet exists and is hidden
- Error response consistency:
 - At least one controller error returns the standardized ErrorResponse shape

OUT OF SCOPE

- Any new endpoints beyond what exists
- Any identity/auth management (Keycloak external)
- File storage, ingestion/submission, approvals/workflows

DELIVERABLES

- Updated error handling + logging + actuator config
 - Updated README
 - Tests added and passing
- Stop after completing Slice 5.

5.4 Review checkpoints (anti-refactor guardrails)

After each slice, do a 5-minute review:

1. **Are we still within scope?** (no submission creep)
2. **Does naming match the glossary?**
3. **Is versioning/immutability rule enforced?**
4. **Is schema the source of truth, not Excel?**
5. **Is integration still pull-based?**

✅ **Worth documenting:** "Review checklist".

If you want, next we can convert this into a **Notion page structure** (sections + where to paste diagrams + where to paste curl examples) so your documentation stays organized and handoff-ready.

▼ Frontend architecture and prompts

Frontend architecture decisions (locked)

- **React + Vite + TypeScript**
 - **Feature-based structure** (mirrors backend: templates, versions, schema, export)
 - **No auth UI** (Keycloak external). Frontend will just *optionally* attach a token if provided later.
 - **Integration style**: call backend REST APIs; export endpoint downloads XLSX.
 - **Error handling**: consistent error panel + toast.
 - **Routing**: React Router.
-

Prompt F0 — Frontend Architecture Freeze (run first)

You are a senior frontend architect. Create a React 18 + Vite + TypeScript frontend for the “Template Creation Service” admin console. This frontend integrates with an existing backend (Spring Boot) and must mirror its domain.

DO NOT implement backend. DO NOT implement Keycloak/auth screens. Assume auth is handled externally.

Tech:

- Vite + React + TypeScript
- React Router
- Axios
- Tailwind CSS

Create folder structure exactly:

```
src/
  app/
    App.tsx
    router.tsx
    layout/
      AppShell.tsx
      Sidebar.tsx
      Topbar.tsx
  features/
    templates/
      pages/
        TemplatesListPage.tsx
        TemplateDetailsPage.tsx
      components/
        api.ts
        types.ts
    versions/
      api.ts
      types.ts
    schema/
      pages/
        SchemaEditorPage.tsx
      components/
        api.ts
```

```

    types.ts
  export/
    api.ts
  shared/
    ui/
      Button.tsx
      Modal.tsx
      Badge.tsx
      Table.tsx
      Toast.tsx
      EmptyState.tsx
      Spinner.tsx
    lib/
      http.ts          (axios instance)
      download.ts      (file download helper)
      format.ts
    errors/
      ErrorPanel.tsx
      errorTypes.ts
  main.tsx
  index.css

```

UI/UX:

- Clean governance/admin style (neutral, minimal).
- AppShell with sidebar navigation: Templates.
- Topbar with page title.
- Reusable Modal with proper UX: ESC closes, backdrop closes, focus management.
- Toast system (lightweight).

Configuration:

- VITE_API_BASE_URL in .env.example
- Enable CORS-friendly dev setup:
 - Prefer Vite proxy /api to backend base URL (document it).

Deliverables:

- Runnable frontend `npm install` + `npm run dev`
- README_frontend.md with run steps and how to configure API base URL/proxy.

STOP after architecture scaffold + shared UI components + routing skeleton. No feature logic yet.

Prompt F1 — Wire backend API client + error handling (integration foundation)

Implement Frontend Slice F1 only.

Goal: prepare integration foundation (no pages fully implemented yet).

- Implement axios instance in src/shared/lib/http.ts using VITE_API_BASE_URL.
- Support optional Authorization header:
 - If localStorage has key "access_token", attach "Authorization: Bearer <token>".
 - Do NOT implement login; just allow token passthrough.

- Implement shared error normalization:
 - Map backend ErrorResponse into a FrontendError type.
 - Provide helper to extract message, status, path, traceId when present.
- Implement Toast system (simple context or event emitter) in shared/ui/Toast.tsx and use it in AppShell.
- Add Vite proxy option in vite.config.ts (proxy /api to VITE_API_BASE_URL) and document in README_frontend.md.

STOP after Slice F1.

Prompt F2 — Templates List Page (real API integration)

Backend alignment:

- `POST /api/templates`
- (You currently have `GET /api/templates/{templateId}` but not list. If you *don't* have list endpoint yet, we'll do "mock list" or "temporary list from known ids".)

Best practice: add `GET /api/templates` later.

But we can still implement UI now with a "temporary list source".

Use this prompt assuming you **do have** a list endpoint (most MVPs add it). If you don't, tell me and I'll adjust.

Implement Frontend Slice F2 only: Templates list + create modal.

Page: TemplatesListPage.tsx

- Fetch templates list from `GET /api/templates` (assume it exists or implement temporary mock fallback if missing).
- Render table: Name, Sector Code, Versions count, Last Updated, Status, Actions(View).
- Add "Create Template" modal:
 - name (required), sectorCode (required)
 - On submit call `POST /api/templates`
 - On success: toast + navigate to template details route `/templates/:templateId`
- Add basic loading + empty state + error panel.

Routing:

- `/templates` -> list
- `/templates/:templateId` -> details (placeholder for now)

STOP after Slice F2.

If your backend **does NOT** have `GET /api/templates`, the correct next move is either:

- add backend Slice "2.5 List Templates", or
 - keep list page mock for demo and only implement create + details flow.
- I'll adapt based on your backend reality.

Prompt F3 — Template Details + Versions (API integration)

Backend alignment:

- `GET /api/templates/{templateId}`

- `POST /api/templates/{templateId}/versions`
- `GET /api/versions/{versionId}`

Implement Frontend Slice F3 only: TemplateDetailsPage with versions list.

TemplateDetailsPage.tsx:

- Load template by templateId using GET /api/templates/{templateId}
- Show header: template name + sectorCode + status
- Show versions table:
 - versionNumber, status badge, createdAt, schemaHash (short), actions
- Actions per version:
 - Edit Schema: only enabled for latest version (based on max versionNumber), otherwise disabled with tooltip.
 - Export: triggers export flow (placeholder now; real download in Slice F5).
- “Create New Version” button:
 - Calls POST /api/templates/{templateId}/versions
 - On success refresh template details + toast

Add good UX:

- Loading states
- Empty versions state (should not happen but handle)
- Error panel

STOP after Slice F3.

Prompt F4 — Schema Editor (API integration + validation UX)

Backend alignment:

- `GET /api/versions/{versionId}`
- `PUT /api/versions/{versionId}/schema` (Slice 3 backend)
- handle 409 (not editable)

Implement Frontend Slice F4 only: SchemaEditorPage.

SchemaEditorPage.tsx:

- Route: /versions/:versionId/schema
- Load version via GET /api/versions/{versionId}
- Display JSON editor (start with textarea or Monaco if already included; keep minimal dependencies).
- Buttons: Save, Cancel
- On Save: PUT /api/versions/{versionId}/schema with parsed JSON body
- Error handling UX:
 - 400: show validation errors (display message + any error list if available)
 - 409: show “This version is read-only. Only latest version is editable.” with link back.
- On success: toast + navigate back to template details (you may need templateId in state; if not available, add a “Back” link to templates).

Add safe UX:

- Unsaved changes warning (simple confirm on navigation)

STOP after Slice F4.

Prompt F5 — Export download (real file download)

Backend alignment:

- `POST /api/versions/{versionId}/export` returns XLSX attachment

Implement Frontend Slice F5 only: Export download integration.

- Add export API in `features/export/api.ts` that calls:
`POST /api/versions/{versionId}/export`
Response type: blob
- Implement download helper in `shared/lib/download.ts`:
 - creates object URL from blob
 - triggers download with filename from Content-Disposition if present; fallback to `templateName_vX.xlsx`
- Wire Export action button in `TemplateDetailsPage`:
 - show "Exporting..." state per row
 - toast on success/failure

STOP after Slice F5.

What you should do right now

1. Run **Prompt F0** in Cursor
2. Then **F1** → **F5** in order

One important dependency check (so you don't hit npm hell again)

We'll keep dependencies minimal:

- react-router-dom
- axios
- tailwind

No date pickers, no heavy UI kits unless you ask.

If you tell me **your actual backend endpoints implemented right now** (especially whether you have `GET /api/templates` list), I'll adjust Slice F2 so it matches your backend exactly with zero guessing.

▼ Modifying functionality

Step 1 — Backend "Slice 2.5": List templates for dashboard

Goal

Add `GET /api/templates` that returns **one row per template + latest version summary**.

Cursor prompt (Backend Slice 2.5)

Implement Backend Slice 2.5 ONLY: list templates for dashboard.

Add endpoint:

GET /api/templates

Response: array of TemplateSummary:

- templateId (uuid)
- name
- sectorCode
- status
- createdAt
- createdBy
- latestVersion: {
 - versionId (uuid),
 - versionNumber (int),
 - status,
 - createdAt,
 - createdBy,
 - schemaHash

Rules:

- latestVersion is the version with the highest version_number per template.
- If a template has no versions (should not happen), latestVersion can be null.

Implementation constraints:

- Keep existing architecture layering (Controller -> UseCase/Service -> Repository).
- Do NOT change existing endpoints.
- Do NOT add new features (no approvals, no submissions, no auth).
- Add unit/integration test for list endpoint (or repository query test).
- Add README curl example for GET /api/templates.

DB:

- Use existing tables templates and template_versions.

Stop after implementing this slice.

✓ After this, your dashboard becomes real.

Step 2 — Frontend Slice: Dashboard UI (real, not mock)

Cursor prompt (Frontend Dashboard)

Implement Frontend Slice: Templates dashboard UI integrated with backend.

Requirements:

- Add templates list page that calls GET /api/templates.
- Render table:
 - Name
 - Sector Code
 - Latest Version (vX)

- Latest Status
- Last Updated (latestVersion.createdAt)
- Actions: View
- "Create Template" modal:
 - name + sectorCode required
 - calls POST /api/templates
 - on success navigate to /templates/{templateId}
- Error handling: show error panel with backend message.
- Loading state + empty state.

Constraints:

- Keep feature-based structure.
- No new UI libraries.
- No auth screens (Keycloak external).

Stop after dashboard works end-to-end.

Step 3 — Frontend Slice: Replace JSON editor with Builder UI (like CTO template)

What we're doing (key)

- Build tables/fields UI components
- Save still uses **PUT** /api/versions/{versionId}/schema
- No granular table/field APIs

Cursor prompt (Frontend Builder)

Implement Frontend Slice: Schema Builder UI (tables/fields/modals) aligned with CTO template UX, integrated with existing backend contract.

Context:

Backend schema contract remains document-style:

- GET /api/versions/{versionId} -> schema_json
- PUT /api/versions/{versionId}/schema -> save full schema_json

UI requirements:

- Replace the raw JSON editor as the primary editing experience.
- Page route: /versions/{versionId}/schema
- Layout (governance builder):
 - Left panel: Tables list (tableKey, sheetName, order) + Add/Edit/Delete
 - Right panel: Fields list for selected table + Add/Edit/Delete
 - Side or bottom panel: Validation results
- Modals:
 - Add/Edit Table: tableKey, sheetName, order
 - Add/Edit Field: fieldKey, headerName, type, required, validations(enum/min/max)
 - Confirm delete for table/field

Validation (client-side before save):

- unique tableKey across schema

- unique fieldKey per table
- Excel sheetName constraints: <=31 and no \ / ? * []
- If min/max exist ensure min <= max

Show validation errors clearly with paths.

Save behavior:

- On Save: build schema_json object and call PUT /api/versions/{versionId}/schema
- Handle backend errors:
 - 400: show validation errors returned by backend
 - 409: show read-only message ("Only latest version editable") and disable editing

Optional:

- Keep a small "Advanced JSON" collapsible view (read-only) for debugging.

Constraints:

- Do NOT add new endpoints.
- Do NOT implement submission or approvals.
- Do NOT refactor unrelated pages.
- Keep consistent UI styling (enterprise neutral).

Stop after builder is functional and saving works.

Step 4 — Export wiring (already planned)

Once builder saves schema correctly, export will start working more reliably.

What you should do right now (exact order)

1. Run Backend Slice 2.5 prompt
2. Test in Swagger:
 - GET `/api/templates` returns data
3. Run Frontend Dashboard slice prompt
4. Run Frontend Builder slice prompt

One small thing (important)

If your backend currently cannot start due to the Maven plugin issue you showed earlier, fix that first (runnable app is required). If you want, paste your `pom.xml` `<build>` section and I'll tell you exactly what line is missing.

When you finish Step 1 (backend list endpoint), paste the JSON response of `GET /api/templates` and I'll confirm it's ideal for the UI.

▼ Modifying v2

Fix it properly (MVP-safe, best practice)

✅ **Fix 1 (must): Initialize schema_json when creating v1 (and when cloning versions)**

When POST `/api/templates` creates template + v1 draft, set `schema_json` like:

```
{
  "templateName": "<template.name>",
  "sectorCode": "<template.sector_code>",
  "tables": [],
  "exportProfile": {}
}
```

And for `POST /api/templates/{templateId}/versions` (clone latest):

- clone schema_json as-is (so sectorCode stays)
- if schema_json is null (legacy rows), generate it from template as fallback

Why this is the correct fix

- Makes the schema document self-contained
- Removes reliance on frontend carrying template metadata
- Stops future clients from hitting same bug

✅ Fix 2 (should): Frontend must NEVER send empty sectorCode

Even with Fix 1, you want defense-in-depth.

Before calling PUT `/schema`, do:

- If payload.sectorCode is empty → set it from TemplateDetails (you have templateId in route)

Because your route is:

`/templates/{templateId}/versions/{versionId}/schema`

So the schema page can fetch template details (or receive it via navigation state) and merge it.

Simple rule

- sectorCode is immutable template metadata → always enforced into schema payload.

What to tell Cursor right now (backend patch prompt)

Copy/paste:

BUGFIX: schema_json.sectorCode is null for newly created versions, causing PUT /schema validation to fail.

In POST /api/templates:

```
- when creating template + v1, initialize template_versions.schema_json with:
{
  "templateName": template.name,
  "sectorCode": template.sectorCode,
  "tables": [],
  "exportProfile": {}
}
```

In POST /api/templates/{templateId}/versions:

```
- when cloning latest version, ensure schema_json is cloned.
- if latest schema_json is null, build it from template (same structure above).
```

Add an integration test:

- Create template with sectorCode "scode"
- GET /api/versions/{versionId} returns schema_json.sectorCode == "scode"
- PUT /api/versions/{versionId}/schema succeeds when only tables/fields are added (sectorCode already present)

Do not change API contracts. Do not add new endpoints.

Frontend patch prompt (small, surgical)

Frontend bugfix: PUT /api/versions/{versionId}/schema sends sectorCode="" even though template has sectorCode.

In schema builder page:

- Read templateId from route (/templates/:templateId/versions/:versionId/schema)
- Fetch template details once (GET /api/templates/{templateId}) or receive sectorCode via navigation state
- Before PUT, merge:

```
schema.sectorCode = schema.sectorCode?.trim() || template.sectorCode
schema.templateName = schema.templateName?.trim() || template.name
```

Do not refactor layout. Do not add new dependencies.
Stop after this fix.

After applying Fix 1, how to verify quickly

1. Create a new template (with sectorCode)
2. Open schema page
3. Advanced JSON should show sectorCode populated (not empty)
4. Add table/field → Save → should return 200 (or 204 depending on your API)

Why your current UI showed the sectorCode earlier but schema still empty

Because Template Details page reads from `templates.sector_code`.

Schema page reads from `template_versions.schema_json`.

They were never linked.

Now they will be.

If you want, paste your backend code where template versions are created (service/use case) and I'll tell you exactly where to inject the initialization with minimal change.

▼ Modifying v3

1 Input loses focus after first letter (serious UX bug)

🔍 Root cause (99% sure)

Your modal component is re-rendering entirely on each keystroke.

Common causes:

- Modal state lives in parent that re-renders on input change
- Modal key prop changes
- Form state stored incorrectly
- You rebuild modal component conditionally
- Using inline component definitions inside render

✓ Correct fix pattern (React best practice)

✓ Keep modal mounted

Instead of:

```
{isOpen && <CreateTemplateModal />}
```

Do:

```
<CreateTemplateModal open={isOpen} onClose={...} />
```

And inside modal:

```
if (!open) return null;
```

Better:

Keep it rendered and toggle visibility with CSS instead of unmounting.

✓ Keep form state inside modal component

Bad:

```
const [form, setForm] = useState(...)
```

in parent and passed down

Good:

Keep form state inside modal itself.

✓ Don't use `key={someChangingValue}` on modal

If key changes → React recreates component → focus lost.

Cursor Fix Prompt (focus bug)

Fix bug: input in CreateTemplateModal loses focus after first character.

Requirements:

- Ensure modal component is not remounted on every keystroke.
- Keep form state inside modal component.
- Do not use dynamic key prop on modal.
- Avoid recreating modal component inline.
- Ensure controlled inputs remain stable.

Do not refactor unrelated components.

Stop after fixing focus issue.

2 Underlined links (Edit, Export) → replace with better UX

Yes — text links feel weak and inconsistent in admin/governance tools.

Better UX pattern

Instead of:

Edit Schema | Export

Use:

Option A (recommended)

- Small icon buttons with tooltip

Example:

-  Edit
-  Export

Tailwind style:

- Ghost button
- Hover background
- No underline

Option B

Use button group:

[Edit] [Export]

With:

- Secondary style
- Consistent height
- Icon + label

Cursor Enhancement Prompt

Enhance all action links (Edit, Export, Delete) across the project.

Replace underlined text links with:

- Small action buttons or icon buttons.
- No text underline.
- Consistent size and spacing.
- Use icons where appropriate (Edit, Export, Delete).
- Add hover states.
- Maintain accessibility (aria-labels for icon-only buttons).

Do not change functionality.

Apply consistently across all pages.

Stop after updating actions styling.

3 Modal opens suddenly (no smoothness)

Correct. Modals should never "pop."

Enterprise UX standard

Use:

- Fade backdrop
- Scale + fade modal
- 150–250ms ease-out animation

Tailwind example

Backdrop:

```
transition-opacity duration-200 ease-out
```

Modal:

```
transform transition-all duration-200 ease-out scale-95 opacity-0
```

When open:

```
scale-100 opacity-100
```

Also important:

- Focus trap
- ESC closes
- Backdrop click closes

Cursor Prompt (smoothness)

Enhance modal UX across the project.

Requirements:

- Add smooth fade animation to backdrop (150–200ms).
- Add scale + fade animation to modal content.
- Ensure no sudden appearance.
- Keep modal mounted and toggle visibility via CSS.
- Add ESC key close.
- Add backdrop click close.
- Maintain focus trap inside modal.

Apply to:

- Create Template modal
- Export modal
- Status update modal
- Any other dialog.

Do not change business logic.

Stop after applying smooth transitions.

4 Additional UX improvements (governance MVP level)

Here are improvements I strongly recommend:

✓ Add loading indicators

When:

- Creating template
- Creating version
- Saving schema
- Exporting

Disable button + show spinner.

Example:

```
[ Save ] → [ Saving... ⏳ ]
```

Prevents double submission.

✓ Disable actions properly

Example:

- Edit Schema disabled if not latest
- Export disabled if schemaHash missing

Use:

- Disabled button style
 - Tooltip explaining why
-

✓ Improve validation panel

Instead of one red banner:

Show structured errors like:

- ```
• sectorCode must not be empty
• tables[0].sheetName invalid
• fieldKey duplicated in table tkey
```

This feels professional.

---

### ✓ Improve empty states

When:

- No templates
- No tables
- No fields

Show:

- Friendly empty state card
- Button to create first item

---

### ✓ Add confirmation for destructive actions

Delete table / Delete field:

- Confirmation modal
- “This cannot be undone”

---

### ✓ Add subtle feedback on success

Instead of redirection only:

- Toast: “Template created successfully”
- Toast: “Schema saved”

---

### Optional Advanced UX (if you want premium feel)

- Sticky Save button in builder
- Show “Unsaved changes” indicator
- Keyboard shortcuts (Ctrl+S to save)
- Slight hover animation on cards

---

## Professional Summary

You now need 3 slices:

**UX Fix Slice A — Focus bug**

**UX Enhancement Slice B — Action buttons styling**

**UX Enhancement Slice C — Modal smoothness**

**UX Polish Slice D — Loading + validation + confirmations**

Do NOT combine all in one prompt.

One slice at a time.

---

If you want, tell me:

- Are you using Tailwind only?
- Or also a component library?

I can give you precise code-level guidance for your current stack.

### ▼ Mod v4

Use this in your prompt as **a hard migration rule** (so the agent doesn't “add” — it **replaces**).

Copy/paste this block into any Cursor slice where you introduce Radix/shadcn/Headless UI:

MIGRATION RULES (NO DUPLICATION):

- We are standardizing UI primitives on <Radix + shadcn-style + Tailwind> (or <Headless UI + Tailwind>).
- Existing custom components (Modal, Button, Table wrappers, Toast, Popover, etc.) must NOT remain duplicated.
- Replace usages across the project to use the new standardized components under src/shared/ui/.

- After migration is complete and the app builds, DELETE the old custom implementations.
- Do NOT introduce parallel systems (e.g., two Modal components, two Toast systems, two Button styles).
- If a custom component provides unique behavior not covered by the new library, merge that behavior into the new standardized component instead of keeping both.

**DELIVERABLES:**

- 1) A list of old components replaced and the file paths removed.
  - 2) Confirmation that the project compiles and there is only one source of truth per primitive (Modal/Button/Toast/etc).
- Stop after completing the migration and cleanup.

### Optional (even stricter)

Add this line if the agent tends to leave leftovers:

If any old component remains referenced anywhere, treat it as a failing task and fix all references.

That's it — this wording forces a “replace-and-delete” migration instead of “add another library on top.”

## ▼ Mod v5

the “best” move is not “switch everything blindly to shadcn.” The best move is:

- ✓ **Adopt a single UI foundation now**
- ✓ **Make all UI go through it**
- ✓ **Keep customization possible**
- ✓ **Avoid duplicated primitives**
- ✓ **Only build custom components when truly missing**

Here's the clean plan.

## What you want is a “UI Foundation” (single source of truth)

Not “a bunch of components.”

A UI foundation means:

- one modal system
- one button system
- one toast system
- one dropdown/popover system
- one table system
- consistent tokens (spacing, colors, radius)
- consistent patterns (hover, disabled, focus)

That prevents focus bugs and random behavior.

# Best foundation choice for your MVP

✓ **Tailwind + Radix + shadcn-style components (generated into repo)**

This gives you:

- reliable accessibility + behavior (Radix)
- consistent look (shadcn patterns)
- full customization (components live in your codebase)
- no “black box library” lock-in

And when you need a custom component:

- you create it inside `shared/ui/` using the same primitives
- 

## Why this makes sense “early”

Because right now:

- your UI is small
- changes are easy
- migration cost is low

Later:

- migration cost becomes expensive
- inconsistency becomes hard to fix

So yes: **standardize now**.

---

## The safe way to do it (no chaos)

Do it like backend slices. Not “big bang”.

### UI Standardization Slices

#### UI-S0 — Lock UI foundation

- Add Radix base primitives
- Add shadcn structure
- Add design tokens (colors, radius, spacing)
- Define folder structure

#### UI-S1 — Migrate primitives

Replace your current ones with foundation components:

- Button
- Modal
- Input
- Select
- Textarea
- Badge
- Toast (choose one)

## UI-S2 — Migrate layout patterns

- Sidebar
- Page header
- Tables styling wrapper

## UI-S3 — Builder components

- Table/field editor modals
- Validation panel
- Stepper (if needed)

Each slice ends with:

- ✓ no duplicates
- ✓ build passes
- ✓ manual quick check

---

## How to handle “custom components not in library”

Easy:

1. If component exists in Radix/shadcn patterns → use it
2. If not → build it as a **wrapper** using your foundation primitives

Example: “Workflow Steps”

- you don’t need a library
- you build `Stepper.tsx` using your Button + Badge + Tailwind tokens

So yes: invent new components **but inside the foundation**.

---

## Important: if you want shadcn now, do it correctly

shadcn isn’t “install and import”.

It generates code into your repo.

So before doing it:

- ✓ Create Git baseline (must)
- otherwise you can’t rollback if dependency tree gets messy

---

## What I recommend you do next (concrete)

1. **Initialize git + commit baseline**
2. Create branch: `ui-foundation`
3. Add shadcn properly (Vite React setup)
4. Migrate only core primitives first
5. Delete old primitives

---

## Cursor prompt you should use (foundation setup)

We want to standardize UI early to avoid future inconsistency and focus/UX bugs.

Task: Establish UI Foundation using Tailwind + Radix + shadcn-style components (generated into repo).

- Set up shadcn/ui for Vite + React + TS (do not use Next.js instructions).
- Keep existing folder structure feature-based.
- Create/ensure shared/ui contains the primitives as single source of truth: Button, Input, Textarea, Select, Modal(Dialog), Badge, Toast.
- Migrate existing code to use these primitives.
- Delete old custom implementations to avoid duplicates.
- Do not change business logic or API integration.

Deliverables:

- List of components migrated and files removed.
  - Project builds successfully.
- Stop after primitive migration only.

## One warning (so we don't repeat npm problems)

Your previous npm dependency conflict was with date-fns.

shadcn can bring some dependencies if you add extras.

So we will:

- add only the primitives needed now
- avoid date pickers, charts, etc. until later

### ▼ Mod v5.1

#### Prompt L1 — Layout standardization (AppShell + Sidebar + PageHeader + Card)

Implement UI Layout Slice L1 only (no business logic changes).

Goal: Standardize governance layout and page structure using our shared/ui foundation (shadcn-style + Radix).

Create/Update these components:

1) src/app/layout/AppShell.tsx

- Layout: fixed sidebar + topbar + main content area
- Provide consistent padding, max width, and background tokens from CSS variables
- Render Outlet (React Router)

2) src/app/layout/Sidebar.tsx

- Nav items: Templates (and placeholder for future)
- Use shared/ui Button (ghost) for nav items
- Active state styling (based on current route)
- Collapsible optional NOT required for MVP

3) src/app/layout/Topbar.tsx

- Shows current page title (simple mapping by route)

- Right slot for actions (optional)
- Use Card-like styling or border-bottom consistent with design tokens

4) src/shared/ui/PageHeader.tsx

- Props: title, description?, rightActions?
- Enforce consistent spacing/typography for all pages

5) src/shared/ui/Card.tsx (if not already existing)

- Provide Card, CardHeader, CardContent, CardFooter (shadcn-style)
- Use CSS variable tokens for background/border

Apply to pages:

- Update TemplatesListPage and TemplateDetailsPage and SchemaEditorPage to use PageHeader + Card structure (minimal changes: wrap existing content, no logic changes).

Constraints:

- Do NOT change API calls or business logic.
- Do NOT change routing structure except to ensure AppShell wraps routes consistently.
- Keep feature-based folders unchanged.
- Ensure build passes.

Deliverables:

- Updated layout components
- Pages visually consistent

Stop after completing L1.

## Prompt I1 — Interaction primitives wrappers (Tooltip / DropdownMenu / Popover)

Implement UI Interaction Slice I1 only.

Goal: Prevent ad-hoc interaction components by creating shared wrappers using Radix primitives, styled with our design tokens.

Create these wrappers in src/shared/ui:

1) Tooltip.tsx

- Use @radix-ui/react-tooltip
- Expose: Tooltip, TooltipTrigger, TooltipContent
- Default side=top, align=center, with subtle animation and token-based background/foreground
- Ensure z-index works above modals/sidebars

2) DropdownMenu.tsx

- Use @radix-ui/react-dropdown-menu
- Expose: DropdownMenu, DropdownMenuTrigger, DropdownMenuContent, DropdownMenuItem, DropdownMenuSeparator
- Style consistent with tokens

3) Popover.tsx



- Use @radix-ui/react-popover
- Expose: Popover, PopoverTrigger, PopoverContent
- Style consistent with tokens

Then apply minimally:

- Replace any existing tooltip implementations (if present) in ActionButtons or disabled Edit Schema tooltip.
- If no usage exists yet, add one small usage example (tooltip on a disabled action) to prove it works.

Constraints:

- Do NOT introduce a second tooltip/dropdown/popover system.
- Do NOT change business logic.
- Keep dependencies minimal: install only required Radix packages.

Deliverables:

- New wrappers in shared/ui
  - One real usage example
- Stop after Slice I1.

## Prompt UX1 — Smoothness & feedback polish (small, safe)

Run this after L1 + I1:

Implement UX Polish Slice UX1 only (no business logic changes).

Goal: Improve smoothness and perceived quality across the UI.

- Ensure Modal open/close transitions are smooth and consistent (overlay + content).
- Add loading button states (disabled + spinner) for Create Template / Save Schema / Create Version / Export (UI-only state; do not change API logic).
- Standardize action buttons (Edit/Export/Delete) to use shared/ui Button or ActionButtons with icons and no underlined links.
- Add consistent empty states (EmptyState component) where lists can be empty.

Constraints:

- No new UI libraries.
- No refactors outside UI components/pages.
- Must build successfully.

Stop after UX1.

### Order to run

1. L1
2. I1
3. UX1

If you paste your current route setup ( `src/app/router.tsx` or similar), I can tighten the L1 prompt to match exactly how your routes are wired (AppShell wrapping, nested routes, etc.).

## ▼ Mod v6

### Slice UX-1 — Table system cleanup + hover + badges + action buttons text

#### Fixes:

- double border around tables (card + table border)
- add row hover darken
- standardize all tables look
- status badge + sectorCode badge in Templates list
- icon + label ("View", "Edit schema", "Export") not icon-only
- remove duplicated "Templates" headings (sidebar/top/page)

#### Prompt

Implement UX Slice UX-1 ONLY: table styling + actions + badges + header duplication cleanup.

Problems to fix across ALL pages:

- 1) Tables have double borders (outer card border + inner table border). Make it a single clean border.
- 2) Add row hover styling (subtle darken) for all table rows.
- 3) Replace icon-only actions with icon + label, e.g. "View", "Edit schema", "Export".
- 4) On Templates list page:
  - Render Status as a Badge (DRAFT, READ\_ONLY).
  - Render Sector Code as a Badge.
- 5) Remove duplicated headings:
  - Sidebar shows section name once.
  - Topbar shows page title once.
  - Page content header should not repeat the same title again.

Implementation constraints:

- Use shared/ui primitives (Button, Badge, Card, Table, etc.)
- Do NOT change business logic or API calls.
- Apply changes consistently to all tables (templates list, versions list, fields list, etc.)

Deliverables:

- A single shared table style system (either update shared/ui/Table.tsx or create shared/ui/DataTableStyles)
  - Updated action buttons component used everywhere
  - Consistent header/title behavior
- Stop after UX-1.

### Slice UX-2 — Toast/alerts modernization + smooth appearance (replace old alert system)

#### Fixes:

- alerts appear suddenly

- success messages not smooth
- ensure single toast system (no duplicates)
- apply for: template created, version created, schema saved, export started/failed, errors

**Recommendation:** use **sonner** (popular with shadcn). It's lightweight and smooth.

If you prefer keeping your current Toast, you must add transitions + stacking + auto-dismiss; sonner is faster.

#### Prompt

Implement UX Slice UX-2 ONLY: replace alerts with a modern toast system with smooth transitions.

##### Goal:

- Replace any inline "alert" banners used for success notifications (e.g., created/saved) with toasts.
- Toasts must animate smoothly, auto-dismiss, and support success/error variants.

##### Preferred approach:

- Use "sonner" toast library (shadcn-recommended).
- Ensure there is ONLY ONE toast system after migration. Remove the previous custom Toast provider/hooks if replaced.

##### Requirements:

- Show success toast on: template created, version created, schema saved.
- Show error toast on: API errors (400/409/500) in a user-friendly way.
- Keep error details available (optional "View details" expandable).

##### Constraints:

- Do NOT change business logic.
- No duplicate toast systems.
- App must build.

##### Deliverables:

- One toast implementation
  - All previous "success alert banners" removed or converted
- Stop after UX-2.

## Slice UX-3 — Modal transitions + smooth opening/closing (Radix Dialog)

Your current Modal uses Radix but likely without proper animation wrappers.

#### Prompt

Implement UX Slice UX-3 ONLY: modal smooth transitions across the app.

##### Requirements:

- All modals must animate (fade overlay + scale/fade content).
- No sudden appearance.
- Use Radix Dialog (already installed).
- Add Tailwind animations using data-[state=open]/data-[state=closed] or a small transition utility.
- Ensure ESC close and backdrop click close still work.
- Ensure focus trap remains.

Apply to:

- Create Template modal
- Add/Edit Table modal
- Add/Edit Field modal
- Any status/confirm modal

Constraints:

- Keep the public Modal API unchanged.
- Do not refactor unrelated UI.

Deliverable:

- Modal component updated once; all modals inherit smooth transitions.
- Stop after UX-3.

## Slice UX-4 — “New row appears suddenly” → loading / optimistic UI / subtle animation

For MVP, don't implement complex row animations. Use **one of these two**:

- ✓ Option A (recommended): Show table-level loading overlay while refetching
- ✓ Option B: optimistic insert with fade-in animation (more work)

We'll do Option A.

### Prompt

Implement UX Slice UX-4 ONLY: smoother table updates after actions (create template, create version, save schema).

Goal:

- When an action triggers a refetch (e.g., create version), show a loading state on the affected table instead of rows popping in suddenly.

Requirements:

- Add a reusable TableLoadingOverlay component (spinner + subtle overlay) used in:
  - Templates list table after create
  - Versions table after create version
  - Schema tables/fields lists after save schema
- Disable relevant action buttons while loading.
- Maintain current data until new data arrives (no blank flash).

Constraints:

- No business logic change besides adding loading state in UI.
- Stop after UX-4.

## Slice UX-5 — Collapsible sidebar + breadcrumbs

Yes, shadcn has patterns, but you must implement/migrate them.

### Prompt

Implement UX Slice UX-5 ONLY: collapsible sidebar + breadcrumbs.

Requirements:

```
1) Sidebar:
- Add collapse/expand toggle.
- Collapsed: icons only; Expanded: icon + label.
- Persist collapsed state in localStorage.
- Use shared/ui Button for toggle.
- Keep accessibility (aria-label, keyboard focus).

2) Breadcrumbs:
- Add a Breadcrumb component (shadcn-style) under shared/ui/Breadcrumb.tsx
- Show breadcrumbs in Topbar for routes:
 - Templates
 - Templates / {templateName}
 - Templates / {templateName} / Version vX / Schema
- Breadcrumb items should be clickable where applicable.

Constraints:
- No auth changes.
- No routing refactor, only read current route params and render breadcrumbs.
Stop after UX-5.
```

## Run order (do not change)

1. **UX-1** (tables/badges/actions/header duplication)
2. **UX-2** (toasts)
3. **UX-3** (modal transitions)
4. **UX-4** (loading overlays)
5. **UX-5** (sidebar collapse + breadcrumbs)

## Quick answers to your “is table from library” question

Right now your table is **your own wrapper** ( `shared/ui/Table.tsx` ) — shadcn doesn’t give a magical “hover prop”. You implement hover via Tailwind classes on `TableRow` (e.g., `hover:bg-muted/50` ).

So yes: totally customizable, just needs consistent styling applied in one place.

If you want this even tighter, paste the paths of your current table component and templates page component (filenames only), and I’ll tailor prompts with exact file targets so Cursor touches fewer files.

### ▼ Mod v6.1

## Slice SB-1 (Hard replace): Install and adopt the official shadcn Sidebar block

### Prompt (copy/paste to Cursor)

```
Implement Slice SB-1 ONLY: Replace current sidebar implementation with the official shadcn Sidebar block.
```

Goal:

- Our current Sidebar is messy and does not match shadcn's official sidebar pattern.
- We must adopt the shadcn Sidebar block (collapsible to icons, proper spacing, header/footer areas, consistent hover/active).
- Remove the custom Sidebar collapse implementation after migration (no duplication).

#### Steps:

- 1) Add the shadcn sidebar block to the project using the CLI:
  - Run the equivalent of: `npx shadcn@latest add sidebar`
  - Ensure it works with Vite + React + TS setup and our aliases.
- 2) Integrate the generated sidebar components into our layout:
  - Replace `src/app/layout/Sidebar.tsx` to use the generated shadcn Sidebar components.
  - Use only one nav item for MVP: Templates (`/templates`) with an icon.
- 3) AppShell:
  - Ensure layout uses the shadcn sidebar provider/wrapper correctly (as per the generated block).
  - Ensure main content shifts correctly when sidebar collapses/expands.
- 4) Delete/Remove old custom sidebar code:
  - Remove any previous sidebar collapse state logic and localStorage keys if the shadcn block already handles it,
  - OR adapt it cleanly to shadcn's API (but do not keep two systems).
- 5) Keep the UI clean:
  - No duplicate "Admin" headings
  - Proper alignment for icons/labels
  - Tooltips in collapsed mode
  - Active state highlight

#### Constraints:

- No routing changes.
- No business logic changes.
- Project must build.
- Final result must visually match shadcn sidebar example quality.

#### Deliverables:

- Sidebar visually and behaviorally matches shadcn example
- No duplicate sidebar implementations remain

Stop after SB-1.

## Why this works

The shadcn sidebar example is a **block**, not a single component. It usually generates:

- provider/context
- `Sidebar`, `SidebarTrigger`, `SidebarContent`, etc.
- sometimes a `useSidebar()` hook

So you get the same pattern as the docs screenshot.

## Slice SB-2: Fix actions regression (Edit schema missing) + standardize action rendering

This addresses your "edit schema not shown" and ensures actions behave consistently.

### Prompt (copy/paste)

Implement Slice SB-2 ONLY: Fix action rendering regression and standardize action buttons.

Problem:

- On Template Details page, latest version row should show "Edit schema" + "Export".
- Currently "Edit schema" disappears (regression) and old edit icon not shown.

Requirements:

1) In Versions table:

- Show "Edit schema" action ONLY for the latest version AND only if status is DRAFT.

- For READ\_ONLY versions: only show Export.

2) Ensure actions are consistent everywhere:

- Use a shared ActionButton component (icon + label).
- Icons must render correctly (use lucide-react icons consistently).

3) Add tooltip for disabled Edit schema if applicable (e.g., not latest or READ\_ONLY).

4) Confirm no broken imports:

- If lucide-react isn't installed, install it.
- Replace any custom SVG that is failing.

Constraints:

- No API changes.
- No schema logic changes.
- Only UI action rendering + styling.

Deliverables:

- Latest DRAFT version shows Edit schema + Export.
- Old versions show Export only.
- Icons + labels appear consistently.

Stop after SB-2.

## Quick diagnosis (why Edit schema disappeared)

Typical causes:

- The "latest version" detection is wrong after some refactor (e.g., comparing strings instead of numbers, or using array index but list is sorted).
- Status check changed from `DRAFT` to something else.
- ActionButtons component changed and now hides primary action when `schema_hash` is null.
- Icon import broke and the action got conditionally removed.

SB-2 forces the agent to re-implement the rule explicitly.

## Important: run order

1. **SB-1** (sidebar block)

## 2. SB-2 (actions regression)

Do not mix them. Sidebar block touches layout heavily; you want it isolated.

### ▼ Mod v6.2

Perfect — you're very close now. What happened is:

- The agent likely used the shadcn sidebar block **but didn't add/position** `SidebarTrigger` correctly, so the "toggle" became the draggable-looking **rail divider** instead of the proper button.
- The "Versions" word is duplicated because you have **PageHeader title + Card title** (or PageHeader + section heading). We'll remove one.

Below are **2 tight slices** that will fix exactly what you asked — nothing more.

## Slice SB-3 — Add proper shadcn toggle button (SidebarTrigger) like the docs

**Goal:** toggle button in the sidebar header/top area (like shadcn), not the border/rail.

### Prompt (copy/paste to Cursor)

Implement Slice SB-3 ONLY: Fix sidebar toggle behavior/placement to match shadcn sidebar example.

Current issue:

- Sidebar collapses/expands but there is no proper toggle button.
- The "toggle" feels like it is the border between sidebar and content.

Requirements:

- 1) Ensure we are using the official shadcn sidebar block API:
  - Use `SidebarProvider` / `Sidebar` / `SidebarContent` / `SidebarRail` / `SidebarTrigger` (or whatever names exist in the generated block).
- 2) Add a visible toggle button using `SidebarTrigger` (the proper button from the block):
  - Place it in the sidebar header area (top of sidebar) OR in the Topbar left area (like shadcn examples).
  - The button must look like an icon button (ghost, size icon), not a divider.
- 3) Keep the rail divider behavior (if any) but it must not be the only way to toggle.
- 4) Ensure keyboard accessibility:
  - Trigger is focusable and has aria-label "Toggle sidebar".
- 5) Ensure styling matches the shadcn example:
  - Toggle button aligned, with subtle hover, consistent spacing.
- 6) Do not introduce a second collapse state system. Use the sidebar block state.

Deliverable:

- A clear toggle button appears and works (click to collapse/expand).
- Stop after SB-3.

### Where it usually goes

- In shadcn examples, the trigger is often in the **topbar** (left side), so users can toggle regardless of scroll. That's a good pattern.



## Slice TBL-1 — Remove duplicate “Versions” label in Template Details page

**Goal:** only one “Versions” heading.

### Prompt (copy/paste)

Implement Slice TBL-1 ONLY: Remove duplicate "Versions" heading on Template Details page.

Current issue:

- "Versions" appears twice (e.g., once as page/section header and again above the table/card).

Requirements:

- Keep a single clear heading:
  - Option A (preferred): PageHeader title stays, remove inner "Versions" title inside the card.
  - OR Option B: Remove PageHeader title and keep the card title (only if page already has breadcrumbs/title elsewhere).
- Apply the same rule to any other page that has duplicated title text.
- No logic/API changes.

Deliverable:

- Template Details page shows "Versions" only once.
- Stop after TBL-1.

### Quick note on your third screenshot (Edit schema action)

It looks correct now:

- Latest (v2) has **Edit schema + Export**
- Read-only (v1) still shows **Edit schema** but greyed — that’s acceptable if it’s intentionally disabled.

If you want it hidden instead, tell me and I’ll give a slice to enforce:

| show Edit only for latest draft; hide entirely otherwise.

### Run order

1. **SB-3**
2. **TBL-1**

## ▼ Walk Through and Hands-on MVP

### ▼ Overview

#### 1) Mental model of this MVP (what you actually built)

This MVP is basically an **Admin console for “Template Creation”**.

A “Template” = a container (name + sectorCode)

A “Template Version” = a specific schema snapshot (v1, v2, ...)

A "Schema" = the JSON definition of sheets/tables + fields  
Export = generate XLSX from schema (backend returns file)

### The user journey

1. Admin lists templates
2. Admin creates a template (creates v1 automatically)
3. Admin opens template details → sees versions table
4. Admin creates a new version (clone latest)
5. Admin edits schema of latest DRAFT version using UI (tables + fields modals)
6. Admin exports Excel (downloads XLSX)

### Big picture diagram

```
flowchart TD
 A[Templates List] --> B[Create Template]
 B --> C[Template Details]
 C --> D[Create New Version]
 C --> E[Edit Schema Latest Version]
 C --> F[Export Version XLSX]
```

## 2) Why Excel generation stays in the backend (HEP mode: blunt + practical)

You asked: "Why not generate Excel in frontend to save backend performance?"

**Verdict: Backend generation is the correct default here (conditionally optimal).**

**Reasons (the real ones):**

- **Consistency & governance:** Excel must match the canonical schema + hashing rules. Backend is the source of truth.
- **Security:** You don't want clients generating "almost-correct" files that bypass rules. Also, server can embed trusted metadata ( `schema_hash` , generator version).
- **Large files:** XLSX generation can be heavy. Doing it in frontend shifts load to the user device (slow laptops die), increases bugs, and makes "support" hell.
- **Libraries:** Apache POI / SXSSF is mature server-side. Frontend Excel libs exist, but have edge cases and memory issues.
- **Integration pattern is pull-download:** another system downloads it. That pattern naturally fits server endpoints producing artifacts.

**When frontend generation makes sense:**

Only if exports are tiny, purely UI convenience, and you don't care about canonical hashing, metadata, or strict validation. That's not your case.

So: keep export in backend. ✓

## 3) Frontend structure (what's the "architecture style" here)

Your frontend is **feature-based** (good choice for scaling MVP → product).

**Why feature-based?**

Instead of dumping everything into `components/`, you organize by domain feature:

- templates
- versions
- schema editor
- export

So the code reads like the business domain.

---

## 4) Folder structure walkthrough (from your project)

Here's the important structure (from your zip):

### `src/app/`

#### App-level wiring

- `App.tsx` → bootstraps router + global providers (toasts)
- `router.tsx` → defines routes
- `layout/` → shared layout: `AppShell`, `Sidebar`, `Topbar`, breadcrumb context

**Concept:** `app` = "composition root" (where everything is wired together)

---

### `src/features/`

#### Business features (domain modules)

- `templates/`
  - `pages/TemplatesListPage.tsx`
  - `pages/TemplateDetailsPage.tsx`
  - `api.ts`, `types.ts`
- `versions/`
  - `api.ts`, `types.ts`
- `schema/`
  - `pages/SchemaEditorPage.tsx`
  - `lib/validateSchema.ts`
  - `types.ts`
- `export/`
  - `api.ts`

**Concept:** `features` = "vertical slices by use-case"

---

### `src/shared/`

#### Reusable building blocks

- `shared/ui/` → Button, Input, Modal, Badge, Table, Sidebar (shadcn block), etc.
- `shared/lib/` → axios client, utils, formatting, download helper
- `shared/errors/` → error normalization

**Concept:** `shared` = "cross-cutting utilities + design system primitives"

---

## 5) Routing (how pages connect)

From `src/app/router.tsx` you have:

- `/` → redirects to `/templates`
- `/templates` → Templates list page
- `/templates/:templateId` → Template details page (versions)
- `/templates/:templateId/versions/:versionId/schema` → Schema editor page

## Routing diagram

```
flowchart LR
 R[Router] --> T1[/templates/]
 R --> T2[/templates/:templateId/]
 R --> T3[/templates/:templateId/versions/:versionId/schema/]
```

**React concept:** `react-router-dom`

- `useNavigate()` → go to another page
- `useParams()` → read `templateId`, `versionId`
- `<Outlet/>` in `AppShell` → where current route page renders

## 6) Layout system (why Sidebar/Topbar are separated)

### `AppShell.tsx`

This is the “frame” around all pages:

- Sidebar on the left
- Topbar at top
- Main content in the rest

It uses shadcn sidebar primitives:

- `SidebarProvider`
- `SidebarInset`

## How layout composes pages

```
flowchart TD
 A[AppShell] --> B[Sidebar]
 A --> C[Topbar]
 A --> D[Outlet Page Content]
```

**React concept:** component composition

Your pages don't care about sidebar/topbar. They render inside the shell.

## 7) Data flow (how frontend talks to backend)

### The HTTP client

`shared/lib/http.ts` creates an Axios instance:

- `baseUrl = import.meta.env.VITE_API_BASE_URL ?? ''`
- If `baseUrl` is empty → requests go to same origin → **Vite proxy** forwards `/api/*` to backend.
- Adds headers:

- `Authorization: Bearer <token>` (if you stored one)
- `X-Trace-Id: <uuid>`

This matches your note perfectly.

---

## API modules per feature

Example: `features/templates/api.ts`

- `GET /api/templates`
- `GET /api/templates/{id}`
- `POST /api/templates`

Example: `features/versions/api.ts`

- `GET /api/versions/{versionId}`
- `POST /api/templates/{templateId}/versions`
- `PUT /api/versions/{versionId}/schema`

Example: `features/export/api.ts`

- export endpoint returning blob

**Concept:** UI never calls axios directly; it calls `templatesApi`, `versionsApi` etc.

That's good separation.

---

## 8) Page-by-page explanation (what each page does)

### A) TemplatesListPage

Responsibilities:

- Loads templates list ( `templatesApi.list()` )
- Opens "Create template" modal
- On success → navigates to template details page

Key React concepts used:

- `useState()` for local state
- `useEffect()` to load initial data
- `useCallback()` to avoid recreating functions (good for stability)
- Controlled inputs (Input value + onChange)

Also uses:

- `TableLoadingOverlay` (better UX)
  - `Badge` for sector/status
  - `Modal` (Radix Dialog)
  - `Toast` for success/error
- 

### B) TemplateDetailsPage

Responsibilities:

- Fetch template by id
- Show versions table

- Create new version (clone)
- Export version (download file)
- Enforce edit rules in UI: only latest + DRAFT is editable

Key React concepts:

- `useParams()` to read `templateId`
- `useEffect()` for fetch
- `useCallback()` for stable handlers
- Conditional rendering for loading/error states
- **Business rule reflected in UI:**
  - if not latest or not DRAFT → show disabled edit with tooltip

## C) SchemaEditorPage

Responsibilities:

- Load template + version
- Parse schema JSON into UI-friendly structure
- Let user add/edit/delete:
  - tables
  - fields
  - validations
- Validate schema before saving
- Save via `versionsApi.updateSchema(versionId, schema)`
- Show errors in UI + toast

Key React concepts:

- **Derived state:** schema UI representation from backend JSON
- Multiple modals (table modal, field modal)
- `useMemo()` to compute things only when inputs change
- `useRef()` used for state that must persist across renders (if present)

## 9) The sectorCode bug you hit (root cause)

Your network payload shows:

```
{ "sectorCode": "" , "tables": [...] }
```

DB screenshot shows:

- `templates.sector_code = "scode"`
- `template_versions.schema_json->'sectorCode' = null`

So backend stored `sectorCode` at template level, **but `schema_json` didn't include it.**

Then SchemaEditor is validating `schema_json` `sectorCode` and failing.

### What this implies (likely design decision)

- The schema validator expects `sectorCode` inside the schema payload.

- But create-template flow doesn't auto-fill it into v1 schema\_json.

### Fix options (best practice)

1. **Frontend autofill:** when SchemaEditor loads template + version, if schema.sectorCode empty → set it from template.sectorCode in UI state before saving.
2. **Backend fix (better):** when creating template v1, initialize schema\_json with `sectorCode` and `templateName` so it's always consistent.

**Best answer:** do both:

- Backend guarantees correctness
  - Frontend safeguards UX
- 

## 10) UI libraries used (and what "shadcn" really means here)

Important intuition:

- **shadcn/ui is not a dependency library** you import at runtime.
- It's a **code generator style**: it copies component code into your repo.
- Those components use:
  - Tailwind
  - Radix primitives
  - `cn()` helper
  - CSS variables theme

So when you say "why not convert everything to library components?":

- You *did* adopt the shadcn-style primitives (`Button`, `Input`, `Select`, etc.)
- But the project still keeps some custom components (Table wrapper, Toast context) because MVP simplicity.

That's normal. You don't need 100% shadcn coverage to be "best practice".

You need:

- **single source of truth per primitive**
  - consistent styling
  - consistent behaviors (focus, modal, transitions)
- 

## 11) What I need from you to do the "full deep walkthrough next" (one question only)

You said: "let us begin with the frontend" — done.

To continue *step-by-step with code*, I need one thing:

**Do you want the walkthrough in this order?**

1. App bootstrapping (Vite + main.tsx + App.tsx)
2. Routing + layout shell
3. Shared UI primitives (Button/Input/Modal/Table/Toast)
4. Feature pages one by one (Templates → Details → Schema editor)
5. API layer + proxy env setup

If yes, say “yes go with that order” and I’ll continue exactly like that (with mini diagrams + concrete file references).

## ▼ Walk through v1

Cool. We’ll do this like we’re pair-programming from zero, and I’ll be annoyingly honest (HEP mode) about *why* we do things, what’s optional, and what will bite you later. I’ll keep bullets light and focus on intuition + hands-on tasks.

We’ll rebuild the frontend in “chapters”. Each chapter ends with a small task that forces the concept into your hands.

---

## Chapter 0 — What we’re actually building (before any code)

Picture your app as two things glued together:

1. A **shell**: sidebar + topbar + breadcrumbs + “where pages appear”
2. A set of **pages**: templates list, template details, schema editor

The shell should be boring and stable. Pages change often. If you mix them, your code becomes a spaghetti bowl fast.

Also: React is not “HTML in JS”. React is “a UI state machine”:

UI = f(state).

When state changes, React re-renders the view.

That’s the core mental model. Every time you feel lost, ask: “What state is this UI reflecting? Where is it stored? Who changes it?”

### Your job in this project

You’re not writing a fancy design system. You’re building an admin tool that:

- loads data from backend
- lets user edit it with good UX
- sends mutations back to backend
- shows errors and progress

If you keep that framing, you’ll avoid 80% of overengineering traps.

### Task 0 (10 minutes)

In your own words, write a 6–8 line description of the app that answers:

- what problem it solves
- who uses it
- the 3 main screens
- what backend endpoints it depends on

Don’t make it pretty. Make it accurate.

---

## Chapter 1 — Bootstrapping Vite + making the app “boot” correctly

You already have Vite. But the important part is *what runs first*.

When the browser loads your app, it loads `index.html`, which loads `main.tsx`, which mounts React into a `<div id="root">`. That’s the “ignition switch”.

The critical design decision: **where do global providers live?**



Providers are “global context wrappers” that pages can use without passing props through everything. Examples: Toast provider, Sidebar provider, Router provider.

If you sprinkle providers randomly, you will create weird bugs where some pages can toast and some can't.

So the root should look like:

- Router provider (routing must be global)
- UI providers (toast, sidebar)
- App component (your layout + routes)

Conceptually:

```
flowchart TD
 A[index.html] --> B[main.tsx]
 B --> C[React Root]
 C --> D[Router Provider]
 D --> E[Toast Provider]
 E --> F[Sidebar Provider]
 F --> G[App Routes + Layout]
```

This structure is boring. That's why it's good.

### Task 1 (hands-on)

Open your project and locate these files:

- `index.html`
- `src/main.tsx`
- `src/app/App.tsx` (or equivalent)

Then answer in 2–3 sentences:

- which file mounts React?
- which file defines providers?
- which file defines routes?

If you can't answer confidently, you don't understand the app yet. That's normal. We fix it now.

## Chapter 2 — Routing (pages) vs Layout (shell)

This is where beginners usually mess up.

Routing decides which page component renders based on URL.

Layout decides how the page is wrapped.

If you mix them, your sidebar disappears when you navigate, or breadcrumbs reset weirdly.

The correct pattern is:

- A layout route that always renders `AppShell`
- An `<Outlet />` inside `AppShell` where the current page shows up

So pages never “own” the sidebar. The shell owns it.

Conceptually:

```
flowchart LR
 R[Router] --> L[AppShell Layout Route]
 L --> O[Outlet]
```

```
0 --> P1[TemplatesListPage]
0 --> P2[TemplateDetailsPage]
0 --> P3[SchemaEditorPage]
```

HEP note: if you don't structure this early, you will refactor later, and refactoring routing + layout is one of the most annoying changes. So we lock it early.

## Task 2

Add (or verify) these routes exist:

- `/templates`
- `/templates/:templateId`
- `/templates/:templateId/versions/:versionId/schema`

Then confirm that **Sidebar and Topbar remain visible** when navigating between them.

If you find pages that render without the shell, note them. That's a structural bug, not "styling".

---

## Chapter 3 — Your UI foundation (shadcn-style primitives)

Let's clear confusion: shadcn isn't magic. It's "copy good components into your repo".

Why this matters: you're trying to avoid re-writing modals, focus trapping, keyboard nav, etc. Those things are *hard* to implement correctly.

So you standardize primitives early:

- Button
- Input
- Modal (Dialog)
- Select
- Badge
- Toast
- Table

Your goal: **one source of truth**. One Button component. One Modal component. Everything uses it.

Why? Because when you find a bug (like focus loss in modal), you fix it once and the whole app improves.

HEP critique: people who build admin tools with custom modals from scratch are basically signing up for a lifetime of "why did focus jump?" bugs. You already saw it.

## Task 3

Pick one primitive (Button or Input) and trace it:

- Where is it defined?
- Where is it used in a page?
- What props does it accept?
- How is styling applied (Tailwind classes via `cn()` / variants)?

Write a short "component contract" for it:

"What does this component guarantee and what does it not guarantee?"

That's how seniors think.

---

## Chapter 4 — Talking to the backend (API layer + proxy strategy)

Now the backend interaction.

Your project supports:

- Vite proxy when `VITE_API_BASE_URL` is empty
- direct backend when `VITE_API_BASE_URL` is set

That's good. But the key is: **pages should not call fetch/axios directly everywhere**. That becomes untestable and inconsistent.

So you want one HTTP client module:

- decides baseURL
- sets headers
- normalizes errors
- maybe adds trace id

Then each feature has a thin API wrapper:

- `templatesApi.list()`
- `templatesApi.create()`
- `versionsApi.get(versionId)`
- `versionsApi.updateSchema(versionId, schema)`
- `exportApi.export(versionId)` returns blob

Pages call these wrappers, not axios directly.

Conceptually:

```
flowchart TD
 Page[Page Component] --> Api[Feature API module]
 Api --> Http[shared http client]
 Http --> Backend[Backend /api/...]
```

HEP note: this structure prevents “random endpoint calls” sprinkled across UI. That chaos is how teams lose weeks.

## Task 4

Find the module that creates the HTTP client.

Confirm these two behaviors:

- baseURL becomes `""` if env is unset
- requests go to `/api/...` paths (so proxy works)

Then find one feature API file and list which endpoints it calls.

---

## Chapter 5 — The first real page: Templates list

This is where React becomes real.

A page is three things:

1. a data fetch (load templates)
2. some local UI state (modal open, form inputs)
3. rendering based on state (empty state vs table)

The mistake beginners make: they fetch data and mutate it everywhere without modeling state.

Instead you want state like:

- `templates: TemplateSummary[]`
- `isLoading: boolean`
- `error: string | null`
- `isCreateOpen: boolean`
- `createForm: {name, sectorCode}`

Then the UI becomes predictable: if `isLoading`, show skeleton; else if `templates.length===0`, show empty state; else show table.

That's React done correctly.

## Task 5

Implement or verify these behaviors on Templates page:

- On load, show loading state
- If backend returns empty list, show empty state
- Create template opens modal, submits, closes, then refreshes list or navigates

Then explain (in 3–4 sentences): which state variables control each behavior.

---

## What we do next (so you actually build it, not just understand it)

Next chapter we do **Template Details page** and we'll focus on:

- reading URL params ( `templateId` )
- fetching template + versions
- the "latest version is editable" rule (and how UI enforces it)
- optimistic vs pessimistic updates (why new version appears suddenly)
- subtle UI correctness (disabled edit vs hide edit)

Then we do **Schema Editor**, which is the hardest part:

- nested state (tables → fields → validations)
  - immutability (why you shouldn't mutate arrays directly)
  - controlled forms + modals
  - validation before save
  - mapping schema JSON ↔ UI state
- 

## Before we proceed: one non-negotiable check

Answer Task 1 + Task 2 first (just paste your answers, short).

If those are shaky, you'll get lost later in Schema Editor, guaranteed.

When you reply, include:

- Where React is mounted (file name)
- Where routes are defined
- Confirm shell persists across routes

Then I'll continue with Chapter 6 (Template Details) and we'll keep building.

## Task 0 and 1

## ▼ Walk through v2 (enhanced)

Got you, Omar. We'll go **slow, beginner-safe**, but I'll keep the "senior brain" running underneath (HEP on). I'm going to teach the concepts *before* I use them, and I'll connect each concept to your actual MVP.

Also: I'm going to be honest—your biggest risk right now is not "React syntax", it's getting lost because frontend has **a lot of invisible moving parts** (build tools, bundlers, routing, state, async calls). We'll make those visible.

---

## 1) What React is (no code yet)

### What problem React solves

If you build a website with plain HTML, you can show a page.

But when the page becomes interactive (tables that update, modals, validation, loading states), you get a hard problem:

#### How do I keep the UI in sync with the data?

Without React, you do a lot of manual DOM work:

- find elements
- update text
- hide/show modals
- attach/remove event handlers
- remember to update every place that depends on state

This becomes error-prone fast.

### React's core idea

React says:

"Stop manually manipulating the page.  
Instead, describe what the UI should look like for a given state,  
and I will re-render it when state changes."

So the mental model becomes:

#### UI = function(state)

If `state` changes, React recalculates what the UI should look like.

That's the heart of React.

### Under the hood (senior-level but simple)

React does **not** re-render the whole page in the naive way every time. That would be slow.

Instead:

1. React creates a lightweight "virtual" representation of the UI in memory (Virtual DOM).
2. When state changes, it creates a new virtual tree.
3. It compares old tree vs new tree ("diffing").
4. It applies only the minimal changes to the real DOM.

That's why React is efficient *even though it re-renders a lot*.

---

## 2) What “mount”, “boot”, and “bootstrapping” mean

### Boot / Bootstrapping (easy meaning)

“Boot” means “start running”.

In software, bootstrapping means “the first setup steps that start the app.”

In your frontend:

- The browser loads `index.html`
- Then it loads your JS bundle
- Then it starts React

That startup sequence is your “boot”.

### Mount (the most important beginner concept)

The browser has a real page (DOM). React needs a place to take control.

So in `index.html`, there is usually a container like:

```
<div id="root"></div>
```

**Mounting** means:

| “React attaches itself to that element, and from then on it controls everything inside it.”

So:

- Before mount: browser shows HTML only
- After mount: React takes over rendering inside `#root`

That’s why `main.tsx` exists: it tells React where to mount.

---

## 3) What Vite is and why you use it

### What is Vite?

Vite is a **tool that helps you build and run** a modern frontend project.

If React is the “engine” (how UI works), Vite is the “garage + tools + fuel pipeline”:

- starts a dev server
- bundles your code
- handles imports
- rebuilds instantly as you edit files
- prepares a production build

### Why not just write React in one HTML file?

Because real apps have:

- many JS/TS files
- components split across folders
- packages from npm
- Tailwind
- environment variables

- routing
- dev proxy to backend

Vite handles all of that.

### The real reason you're using Vite (HEP bluntness)

You're using Vite because:

- it's fast to iterate (instant reload)
- it works cleanly with React + TypeScript
- your MVP needs "proxy to backend" support easily
- it's simpler than Next.js for your use case (no SSR needed)

This MVP is an internal admin console. It does NOT need server-side rendering. So Next.js would add complexity for no benefit.

---

## 4) What a "provider" is (Toast provider, Context provider)

### First: what's the "problem" providers solve?

Imagine you want to show a toast notification from anywhere.

Option A: pass a `showToast()` function down through props:

App → Layout → Page → Component → SubComponent

This is called **prop drilling**, and it becomes painful.

### Provider concept (simple)

A Provider is like a **global service** available to any component in the tree.

Think of it as:

- a "shared box" of functionality or data
- accessible by children without passing props manually

In React, providers are usually built using **Context**.

So:

- ToastProvider provides `toast()` function
- SidebarProvider provides sidebar state (collapsed/expanded)
- BreadcrumbProvider provides breadcrumb data

### Under the hood

A provider wraps a section of your app:

```
<ToastProvider> <App/> </ToastProvider>
```

Now any component inside can do:

```
useToast() → show toast.
```

That's what "provider" means.

### Why providers must be placed carefully

If you put (sprinkle) providers randomly inside pages, you get bugs like:

- toast works in one page, not in another
- sidebar collapses resets on navigation

- breadcrumbs disappear unexpectedly

So providers should live in one stable place: your app “bootstrapping layer”.

---

## 5) What a “shell” is in frontend architecture

A “shell” is the permanent frame around your pages:

- sidebar
- topbar
- breadcrumbs
- consistent layout spacing

Pages change, the shell stays.

Why it matters:

If every page rebuilds its own sidebar/topbar, you’ll get:

- duplicated code
- inconsistent UI
- state resets (sidebar collapse loses state)

So you build the shell once, and pages render inside it.

That’s why you’ll see something like:

### Shell layout

→ contains Sidebar + Topbar

→ contains a “slot” where pages appear

That “slot” is typically the router’s `<Outlet />`.

---

## 6) What “sprinkle” means (not a formal term)

It’s just casual dev slang meaning:

| “Put little bits of something everywhere in an unplanned way.”

Example:

- “We sprinkled API calls across many components”
- “We sprinkled providers across pages”

It means: you didn’t centralize it, so now it’s messy.

---

## 7) Now... apply those concepts to YOUR MVP project

Your MVP frontend is essentially:

- **Boot layer:** starts React, installs providers
- **Shell:** consistent layout (sidebar/topbar)
- **Pages:** templates list, template details, schema editor
- **Shared UI primitives:** Button/Input/Modal/Toast etc.
- **Feature modules:** each feature owns its UI + API logic

This structure exists because it reduces chaos when the app grows.



## Data flow in your app (big picture)

This is the key loop:

1. Page loads
2. Page fetches data from backend ( `GET ...` )
3. React stores it in state
4. UI renders based on that state
5. User changes something
6. UI sends update to backend ( `POST/PUT` )
7. Backend responds
8. UI updates state + shows toast

This is basically all your MVP does.

---

## 8) Critical improvement areas (HEP mode)

Even if the MVP works, here are the most important things to tighten early:

### A) "Source of truth" rules must be consistent

You already saw the sectorCode bug:

template saved sectorCode, but schema payload had empty sectorCode → backend correctly rejected.

This happens when frontend state and backend rules aren't aligned.

Fix is architectural:

- backend should seed schema\_json with sectorCode on template creation
- frontend should auto-fill missing values from template info

### B) Avoid putting "business rules" only in frontend

Rule like "only latest version editable" must be enforced by backend too.

Frontend should reflect it for UX, but backend must enforce.

### C) Make API layer strict and typed

If payload types drift, you'll suffer.

TypeScript types should mirror backend DTOs (even manually).

---

## 9) Your hands-on training plan (so you end up able to rebuild it)

We'll rebuild the app in "lessons":

Lesson 1: Vite + bootstrapping + mount

Lesson 2: Shell layout (Sidebar + Topbar + Outlet)

Lesson 3: Routing (3 pages)

Lesson 4: API client (proxy + baseURL logic)

Lesson 5: Templates list page (fetch + table + modal create)

Lesson 6: Template details page (versions list + create version + export)

Lesson 7: Schema editor (nested state + modals + validation + save)

Lesson 8: UI polish (loading states, transitions, table hover, toasts)

Each lesson you'll implement something small and prove you understand it.

---

## Before we continue: I need 2 quick confirmations (no long answers)

1. Do you want to **rebuild in a new clean repo** for learning (best for learning), or learn by **walking through your existing code** and rebuilding piece by piece inside it?
2. Are you okay to use TypeScript throughout (your project already does), or do you want to temporarily pretend it's JS while learning?

Reply with:

- "new repo" or "existing code"
- "TypeScript ok" or "prefer JS first"

Then we start Lesson 1 properly and I'll give you your first coding task.

### ▼ Walk through v3.1 (you can skip)

Perfect. Now we do this the right way.

We are going to rebuild everything from zero — but this time you will **understand the machine**, not just use it.

And yes — we'll reduce bullets. We'll use conceptual diagrams (mermaid or simple code diagrams when helpful). We'll think like engineers, not checklist followers.

---

## PHASE 1 — What We Are Actually Building (Concept First)

Before touching code, we need mental clarity.

Your MVP frontend is not "a website".

It is:

A client-side application that manages state, renders UI based on that state, and communicates with a backend service.

Everything we build will serve that.

Let's draw the system from 10,000 feet:

```
flowchart LR
 User --> Browser
 Browser --> ReactApp
 ReactApp --> BackendAPI
 BackendAPI --> Database
```

Important understanding:

- The browser runs JavaScript.
- React runs inside that JavaScript.
- React talks to backend using HTTP.
- Backend talks to database.

Your job as a frontend engineer is to build **ReactApp**.

---

## PHASE 2 — What React Actually Is (Deep but Beginner Friendly)

Let's remove the magic.

React is a JavaScript library.

It does one thing:

It converts state into UI.

If state changes, UI updates.

That's it.

Internally, it works like this:

```
flowchart TD
 State --> RenderFunction
 RenderFunction --> VirtualDOM
 VirtualDOM --> DiffAlgorithm
 DiffAlgorithm --> RealDOMUpdate
```

Step-by-step:

1. You define a component.
2. That component has state.
3. React calls your component function.
4. It produces a Virtual DOM representation.
5. React compares with previous version.
6. Only minimal DOM changes are applied.

This is why React feels declarative.

You describe what UI should look like — not how to manually update it.

## PHASE 3 — What Vite Is (Now With Real Clarity)

You asked what Vite is.

Let's explain it correctly.

Without Vite, your project would require:

- bundling multiple JS files
- handling ES modules
- transforming JSX
- compiling TypeScript
- refreshing browser when file changes
- building optimized production bundle

Vite does all that.

It is not React.

It is not your app.

It is your build system and dev server.

Think of it like this:

```
flowchart TD
 You[Your Code] --> Vite
 Vite --> DevServer
 DevServer --> Browser
 Vite --> ProductionBuild
```

So when you run:

```
npm run dev
```

Vite:

- Starts a dev server
- Transforms JSX into JS
- Serves your files
- Enables hot reload

That's all.

React is the engine.

Vite is the garage.

---

## PHASE 4 — Bootstrapping, Mounting, Boot

Let's demystify these words.

### Boot

Boot = starting the app.

When browser loads:

1. Loads index.html
2. Loads JS bundle
3. Executes main.tsx
4. React mounts

### Mount

Mount = attach React to DOM.

In index.html:

```
<div id="root"></div>
```

In main.tsx:

```
createRoot(document.getElementById('root')).render(<App />);
```

That is mounting.

Before mount:

Browser controls DOM.

After mount:

React controls everything inside `#root`.

Important mental shift:

From this moment on, you are not manually editing HTML.

React is.

---

## PHASE 5 — Our Clean Rebuild Plan

We will build this in controlled stages.

Stage 1: Pure React app (no routing, no backend)

Stage 2: Add layout shell

Stage 3: Add routing

Stage 4: Add API communication

Stage 5: Build Templates page

Stage 6: Build Details page

Stage 7: Build Schema editor

Each stage builds intuition.

---

## STAGE 1 — Create New Vite + React + TypeScript Project

You will run:

```
npm create vite@latest template-mvp-learning
```

Choose:

- React
- TypeScript

Then:

```
cd template-mvp-learning
npm install
npm run dev
```

You now have a minimal React app.

---

## STAGE 1 — What Exists Right Now?

Open the folder structure.

You should see:

```
src/
 main.tsx
 App.tsx
 index.css
```

Let's explain them conceptually.

**main.tsx**

This is your boot file.

It does two things:

- Creates React root
- Mounts

It is not business logic.

It should stay minimal.

## App.tsx

This is your top-level component.

Right now, it renders some demo content.

We will slowly turn this into your shell.

---

## IMPORTANT CONCEPT — Component

A React component is just a function.

Example:

```
function Hello() {
 return <h1>Hello</h1>;
}
```

This is not special.

It is just a function that returns JSX.

JSX is just syntax sugar for:

```
React.createElement(...)
```

Now your mental model becomes clearer.

---

## YOUR FIRST TASK

1. Create the new Vite project.
2. Delete everything inside App.tsx.
3. Replace it with:

```
function App() {
 return (
 <div>
 <h1>Template Creation MVP</h1>
 </div>
);
}

export default App;
```

1. Confirm it renders.

Reply:

- “Project created”
- “App renders”

Then we move to Stage 2 and build the Shell (Sidebar + Content Area) from scratch — and I will explain what layout composition really means.

We go step by step.

No AI magic.

No copy-paste confusion.

Real understanding.

## ▼ v3.2

Good. Now we actually start learning.

You have a running React app. Right now it is just:

```
main.tsx → mounts → <App />
App → renders → <h1>
```

That's it.

Now we build the first real architectural concept:

## Stage 2 — The Application Shell

Before adding routing.

Before backend.

Before state.

We need to understand layout.

---

## What Is a Shell (Properly Explained)

In frontend architecture, a shell is the persistent structural frame of your application.

It contains:

- Sidebar
- Topbar
- Layout grid
- Global providers

It does NOT contain business logic.

It does NOT fetch data.

It does NOT change per page.

Think of it like the frame of a house.

The furniture (pages) changes.

The frame does not.

---

## Conceptual Model

Right now your app looks like:

```
flowchart TD
 App --> PageContent
```

We want it to look like:

```
flowchart TD
 App --> Shell
 Shell --> Sidebar
 Shell --> MainContent
```

Later:

```
flowchart TD
 App --> Shell
 Shell --> Sidebar
 Shell --> Topbar
 Shell --> Outlet[Dynamic Page Content]
```

## Why This Matters (Senior Insight)

If you mix page logic and layout logic:

- Sidebar will re-render unnecessarily
- Collapse state will reset
- You will duplicate layout across pages
- Navigation becomes fragile

So we isolate layout early.

This is one of the most important architectural decisions.

## Now Let's Build the Shell

Open `App.tsx`.

Replace everything with this:

```
import './App.css';

function App() {
 return (
 <div style={{ display: "flex", height: "100vh" }}>
 <aside
 style={{
 width: "240px",
 background: "#1f2937",
 color: "white",
 padding: "20px",
 }}
 >
 <h2>Template App</h2>
 <p>Sidebar</p>
 </aside>
```



```
 <main style={{ flex: 1, padding: "20px" }}>
 <h1>Main Content Area</h1>
 </main>
 </div>
);
}

export default App;
```

Run it.

## What Just Happened?

You created a layout with:

- `display: flex`
- Sidebar (fixed width)
- Main area (flex: 1 means take remaining space)

This is pure CSS.

React is not doing anything magical here.

React just renders this structure.

## Important Concept: Separation of Responsibility

Right now, everything is inside App.

That is wrong long-term.

We need:

- `App.tsx` → bootstrap
- `AppShell.tsx` → layout
- `Sidebar.tsx`
- `Topbar.tsx`

So next, we refactor.

## Create This Folder Structure

Inside `src`, create:

```
app/
 AppShell.tsx
layout/
 Sidebar.tsx
 Topbar.tsx
```

Now move layout into components.

## Step 1 — Sidebar Component

Create `layout/Sidebar.tsx`

```
function Sidebar() {
 return (
 <aside
 style={{
 width: "240px",
 background: "#1f2937",
 color: "white",
 padding: "20px",
 }}
 >
 <h2>Template App</h2>
 <p>Sidebar</p>
 </aside>
);
}

export default Sidebar;
```

## Step 2 — AppShell

Create `app/AppShell.tsx`

```
import Sidebar from "../layout/Sidebar";

function AppShell({ children }: { children: React.ReactNode }) {
 return (
 <div style={{ display: "flex", height: "100vh" }}>
 <Sidebar />
 <main style={{ flex: 1, padding: "20px" }}>{children}</main>
 </div>
);
}

export default AppShell;
```

Important concept here:

`children` is how React allows components to wrap other components.

This is composition.

## Step 3 — Update App.tsx

```
import AppShell from "./app/AppShell";

function App() {
 return (
 <AppShell>
 <h1>Main Content Area</h1>
 </AppShell>
);
}
```

```
}

export default App;
```

Run it.

---

## Why This Is Powerful

Now AppShell owns layout.

App only decides what page content to render.

Later, when routing is added, page components will be injected as `children`.

This is professional frontend structure.

---

## UNDER THE HOOD INSIGHT

Notice something:

AppShell doesn't know what page is rendered.

It just renders children.

This is a key React idea:

**Composition over inheritance.**

Instead of:

- Shell deciding which page to render

We do:

- Page injected into shell

This keeps layout independent from business logic.

---

## YOUR TASK

1. Refactor into:

- Sidebar component
- AppShell component

2. Confirm app still renders.

3. Explain to me in 3–4 sentences:

- What `children` does
- Why AppShell should not fetch data

Once you answer, we move to Stage 3:

Routing + dynamic pages.

And that's where the architecture starts becoming real.

### ▼ V3.3

Nice. Your understanding is *basically correct* — but I'm going to sharpen it (HEP mode).

### Fixing your “children” definition (important)

Children is not “passed to parent” exactly. Think of it like this:

When you write:

```
<AppShell>
 <h1>Hello</h1>
</AppShell>
```

React turns that into:

- `AppShell` is called with a prop named `children`
- That prop contains whatever you wrote between the tags

So `children` is simply **a prop** that represents “the nested content”.

It's the mechanism that makes layout wrappers possible.

Here's the mental model:

```
flowchart TD
 App -->|children prop| AppShell
 AppShell --> MainArea
 MainArea --> ChildContent
```

Good. Now we level up.

## Stage 3 — Routing (Pages) and Why We Need It

Right now, you have one “page”: whatever you pass as children.

But in your real MVP, you need multiple pages:

- Templates list
- Template details
- Schema editor

So we need a mechanism that says:

```
If URL is /templates, show TemplatesListPage
If URL is /templates/:templateId, show TemplateDetailsPage
If URL is /templates/:templateId/versions/:versionId/schema, show SchemaEditorPage
```

This is routing.

### What routing is conceptually

Routing is like a switchboard:

```
flowchart TD
 URL --> Router
 Router -->|/templates| TemplatesListPage
 Router -->|/templates/:id| TemplateDetailsPage
 Router -->|/templates/:tid/versions/:vid/schema| SchemaEditorPage
```

The router's job is to choose which page component to render.

### Install React Router

Run:

```
npm i react-router-dom
```

## Now we add the correct layout + routing pattern

There are two common ways:

### Approach A (bad for growing apps)

Each page includes its own layout.

This duplicates sidebar/topbar everywhere.

### Approach B (professional)

The layout is a parent route.

Pages render inside it using an `Outlet`.

We will do B.

## Step 1 — Create placeholder pages

Create:

```
src/features/templates/pages/TemplatesListPage.tsx
```

```
export default function TemplatesListPage() {
 return <h1>Templates</h1>;
}
```

```
src/features/templates/pages/TemplateDetailsPage.tsx
```

```
import { useParams } from "react-router-dom";

export default function TemplateDetailsPage() {
 const { templateId } = useParams();
 return <h1>Template Details: {templateId}</h1>;
}
```

```
src/features/schema/pages/SchemaEditorPage.tsx
```

```
import { useParams } from "react-router-dom";

export default function SchemaEditorPage() {
 const { templateId, versionId } = useParams();
 return (
 <h1>
 Schema Editor - template: {templateId}, version: {versionId}
 </h1>
);
}
```

Notice: we used `useParams()`.

This reads values from the URL.

## Step 2 — Modify AppShell to render Outlet content

Update `src/app/AppShell.tsx` to use `Outlet` :

```
import Sidebar from "../layout/Sidebar";
import { Outlet } from "react-router-dom";

function AppShell() {
 return (
 <div style={{ display: "flex", height: "100vh" }}>
 <Sidebar />
 <main style={{ flex: 1, padding: "20px" }}>
 <Outlet />
 </main>
 </div>
);
}

export default AppShell;
```

Important: we removed children.

Now routing controls what goes into the main area.

This is the real "shell" pattern.

## Step 3 — Create the router configuration

Create `src/app/router.tsx` :

```
import { createBrowserRouter, Navigate } from "react-router-dom";
import AppShell from "../AppShell";
import TemplatesListPage from "../features/templates/pages/TemplatesListPage";
import TemplateDetailsPage from "../features/templates/pages/TemplateDetailsPage";
import SchemaEditorPage from "../features/schema/pages/SchemaEditorPage";

export const router = createBrowserRouter([
 {
 element: <AppShell />,
 children: [
 { path: "/", element: <Navigate to="/templates" replace /> },
 { path: "/templates", element: <TemplatesListPage /> },
 { path: "/templates/:templateId", element: <TemplateDetailsPage /> },
 {
 path: "/templates/:templateId/versions/:versionId/schema",
 element: <SchemaEditorPage />,
 },
],
 },
]);
```

This says:

- AppShell always renders
- Pages render inside AppShell via Outlet

## Step 4 — Use RouterProvider in App.tsx

Update `src/App.tsx`:

```
import { RouterProvider } from "react-router-dom";
import { router } from "../app/router";

function App() {
 return <RouterProvider router={router} />;
}

export default App;
```

Now App no longer chooses what to render.  
Router does.

## Step 5 — Add navigation links to sidebar

Update `src/layout/Sidebar.tsx`:

```
import { Link } from "react-router-dom";

function Sidebar() {
 return (
 <aside
 style={{
 width: "240px",
 background: "#1f2937",
 color: "white",
 padding: "20px",
 }}
 >
 <h2>Template App</h2>

 <nav style={{ marginTop: "20px", display: "grid", gap: "12px" }}>
 <Link style={{ color: "white" }} to="/templates">
 Templates
 </Link>
 </nav>
 </aside>
);
}

export default Sidebar;
```

Now you can click the link and navigate.

## Under the hood: What changed?

Before, AppShell took `children`.

Now:

- Router decides which page to render

- Outlet is the “slot”
- AppShell is always shown

This is exactly how your real MVP works.

---

## YOUR TASK (Stage 3)

1. Install `react-router-dom`
2. Implement the router + pages + Outlet pattern exactly as above.
3. Test these URLs in browser:

- `/templates`
- `/templates/123`
- `/templates/123/versions/456/schema`

Then answer these questions in plain English:

- What is `Outlet` ?
- What is `useParams()` doing?
- Why did we remove `children` from AppShell?

Once you do that, we move to Stage 4:

**API client + Vite proxy + talking to backend** (the real data flow begins).

### ▼ V3.4

Now we move to **Stage 4: Backend communication** (this is where the app stops being “static pages” and becomes a real admin tool).

We'll do it in a way that teaches you the key frontend concept:

**async data + state + rendering = 90% of frontend work.**

---

## Stage 4 — Talking to the backend like a grown-up

### Concept first: what happens when a page needs data?

When you open `/templates`, the page is rendered immediately. But the data (templates list) does not exist yet. So we must represent “data lifecycle”:

1. initial: nothing loaded
2. loading: request in progress
3. success: data arrived
4. error: request failed

That's the real shape of frontend.

Here's the mental model:

```
stateDiagram-v2
 [*] --> Idle
 Idle --> Loading: page mounted
 Loading --> Success: response 200
 Loading --> Error: response != 200 / network fail
 Error --> Loading: retry
```



We will implement this for `/templates`.

---

## Step 4.1 — Set up an HTTP client (one place only)

### Why not fetch directly in pages?

You *can* do `fetch()` inside pages, but it becomes messy:

- inconsistent base URLs
- no shared error handling
- hard to add auth later
- hard to add tracing headers later

So we do a single HTTP client module.

Create:

`src/shared/lib/http.ts`

```
export const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ?? "";

// always call backend through /api so Vite proxy can work when base URL is empty
export async function apiFetch<T>({
 path: string,
 options: RequestInit = {}
}): Promise<T> {
 const url = `${API_BASE_URL}${path}`;

 const res = await fetch(url, {
 ...options,
 headers: {
 "Content-Type": "application/json",
 ...(options.headers ?? {}),
 },
 });

 if (!res.ok) {
 // try to parse backend error if exists
 let detail: any = null;
 try {
 detail = await res.json();
 } catch {
 // ignore
 }
 const message =
 detail?.message || detail?.error || `Request failed: ${res.status}`;
 throw new Error(message);
 }

 return res.json() as Promise<T>;
}
```

**Concept:** This is your “backend gate”. Every request goes through it.

---

## Step 4.2 — Create the Templates API module (feature ownership)

Create:

`src/features/templates/api.ts`

```
import { apiFetch } from "../../shared/lib/http";

export type TemplateSummary = {
 id: string;
 name: string;
 sectorCode: string;
 status: "DRAFT" | "READ_ONLY" | "PUBLISHED";
 latestVersionNumber?: number;
};

export async function listTemplates(): Promise<TemplateSummary[]> {
 return apiFetch<TemplateSummary[]>("/api/templates");
}
```

**Important:** This isolates “what endpoint exists” and “what response looks like”.

If the backend response changes, you update it here, not in the UI.

## Step 4.3 — Use it inside TemplatesListPage (with state + effect)

Open:

`src/features/templates/pages/TemplatesListPage.tsx`

Replace with:

```
import { useEffect, useState } from "react";
import { listTemplates, TemplateSummary } from "../api";

export default function TemplatesListPage() {
 const [templates, setTemplates] = useState<TemplateSummary[]>([]);
 const [isLoading, setIsLoading] = useState(false);
 const [error, setError] = useState<string | null>(null);

 useEffect(() => {
 let isCancelled = false;

 async function load() {
 setIsLoading(true);
 setError(null);
 try {
 const data = await listTemplates();
 if (!isCancelled) setTemplates(data);
 } catch (e: any) {
 if (!isCancelled) setError(e.message ?? "Failed to load templates");
 } finally {
 if (!isCancelled) setIsLoading(false);
 }
 }

 load();

 return () => {

```

```

 isCancelled = true;
 };
}, []);

if (isLoading) return <h1>Loading templates...</h1>;
if (error) return <h1 style={{ color: "red" }}>Error: {error}</h1>;

return (
 <div>
 <h1>Templates</h1>
 {templates.length === 0 ? (
 <p>No templates yet.</p>
) : (

 {templates.map((t) => (
 <li key={t.id}>
 {t.name} – {t.sectorCode} – {t.status}

))}

)}
 </div>
);
}

```

### What you just learned (without jargon)

- `useState` stores variables that affect the UI.
- `useEffect` runs code when the component appears on screen.
- You model loading and error explicitly.
- You render different UI based on those values.

That is the real “React job”.

## Step 4.4 — Make Vite proxy work (so /api goes to backend)

In your `vite.config.ts`, add:

```

server: {
 proxy: {
 "/api": {
 target: "http://localhost:8080",
 changeOrigin: true,
 },
 },
},

```

Now:

- frontend calls `/api/templates`
- Vite forwards it to backend
- no CORS headaches in local dev

## Your task for Stage 4 (do these in order)

1. Add the proxy to `vite.config.ts`
2. Create `shared/lib/http.ts`
3. Create `features/templates/api.ts`
4. Update `TemplatesListPage` to load real templates
5. Run frontend + backend and open:
  - `http://localhost:5173/templates`

When it works, tell me what you see:

- Loading → then list items OR “No templates yet”
- If it errors, paste the error message + backend URL you run on

## HEP improvement note (so you don’t build a fragile frontend)

Your current UI is using `fetch`. That’s fine for learning.

Later we’ll switch to **axios + interceptors** or a better data layer (React Query) *only if needed*. Beginners often add React Query too early and then they don’t understand state anymore. We’ll earn it.

Once Stage 4 works, we move to Stage 5:

✓ Create Template modal + POST `/api/templates` + navigate to details page.

Let’s get the first real data flowing.

### ▼ V3.5

Good. Stage 5 is where frontend becomes real.

Up until now, you only *read* data.

Now you will **change** data.

That’s a completely different mental model.

## Stage 5 — Creating a Template (Mutation Flow)

Before code, let’s understand what actually happens when a user creates something.

When user clicks “Create Template”, the system must:

1. Collect input (name + sectorCode)
2. Validate input (basic required fields)
3. Send POST request to backend
4. Wait for response
5. Handle success or error
6. Update UI accordingly
7. Possibly navigate

Let’s visualize that flow:

```
sequenceDiagram
 participant User
```

```
participant UI
participant Backend

User->>UI: Click Create
UI->>UI: Validate form
UI->>Backend: POST /api/templates
Backend-->>UI: 201 Created (templateId)
UI->>UI: Update state / Navigate
```

Important:

The frontend does not “create the template”.

It only *requests* the backend to do so.

---

## Concept: Controlled Inputs

When you type in an input in React, the value should come from state.

Example:

```
const [name, setName] = useState("");
```

```
<input value={name} onChange={(e) => setName(e.target.value)} />
```

Why?

Because React must be the single source of truth.

If you let the DOM control the input and then read it later, you lose predictability.

---

## Step 5.1 — Add API function for creation

Update:

`src/features/templates/api.ts`

```
export async function createTemplate(input: {
 name: string;
 sectorCode: string;
 createdBy: string
}): Promise<{
 templateId: string
 versionId: string
 versionNumber: number
}> {
 return apiFetch("/api/templates", {
 method: "POST",
 body: JSON.stringify(input),
 });
}
```

We assume backend returns `{ templateId }`.

---

## Step 5.2 — Add a simple Create Form (no modal yet)

For now, don't complicate it with modal.

Modify `TemplatesListPage.tsx`.

Add this above the list:

```
import { createTemplate } from "../api";
import { useNavigate } from "react-router-dom";
```

Inside component:

```
const navigate = useNavigate();

const [newName, setNewName] = useState("");
const [newSector, setNewSector] = useState("");
const [isCreating, setIsCreating] = useState(false);
```

Add function:

```
async function handleCreate() {
 if (!newName || !newSector) {
 alert("Name and sector are required");
 return;
 }

 try {
 setIsCreating(true);
 const result = await createTemplate({
 name: newName,
 sectorCode: newSector,
 });

 // Option A: reload list
 // Option B: navigate to details page

 navigate(`/templates/${result.templateId}`);
 } catch (e: any) {
 alert(e.message || "Failed to create template");
 } finally {
 setIsCreating(false);
 }
}
```

Now in JSX:

```
<div style={{ marginBottom: "20px" }}>
 <h2>Create Template</h2>
 <input
 placeholder="Name"
 value={newName}
 onChange={(e) => setNewName(e.target.value)}
 />
 <input
 placeholder="Sector Code"
 value={newSector}
 />
</div>
```

```
 onChange={(e) => setNewSector(e.target.value)}
 />
 <button onClick={handleCreate} disabled={isCreating}>
 {isCreating ? "Creating..." : "Create"}
 </button>
 </div>
```

## What you just implemented

You implemented a **mutation lifecycle**:

State involved:

- form inputs
- loading flag
- error handling
- navigation

This is 70% of real-world frontend work.

## Critical Understanding (Senior Insight)

Notice something important:

We did not update the templates list manually after creation.

Instead, we navigated to the details page.

That avoids state duplication issues.

In your real MVP, you can choose:

- Refresh list after create
- Or navigate

But don't do both unless you need to.

## Your Task (Stage 5)

1. Implement `createTemplate` API
2. Add create form to Templates page
3. Confirm:
  - You can create template
  - It navigates to `/templates/{id}`
  - Backend actually creates it

Then answer:

1. Why do we use `useNavigate()` instead of manually setting `window.location`?
2. Why do we track `isCreating` in state?
3. What would happen if we didn't disable the button during creation?

After that, we move to Stage 6:

Template Details Page + loading versions.

Now the app starts resembling your real MVP.

## ▼ V3.5.5

Good progress. And your changes to `createTemplate` are the right move — **frontend must match backend contract**, not the other way around. (HEP note: if backend keeps changing contracts, that's a governance smell; for MVP it's fine, but later you'll want a stable API spec.)

Now let's fix the one gap: `useNavigate` vs `window.location`.

### Why `useNavigate()` instead of `window.location`?

Think of your React app like a "single-page app" (SPA). That means:

- The browser loads **one** HTML page once.
- React controls what's shown inside it.
- Routing changes the URL and swaps components **without reloading the whole page**.

`window.location = "/templates/123"`

This tells the browser: "go load a new page from the server".

So it causes a full reload:

- React app restarts (bootstraps again)
- you lose in-memory state (like open modals, cache, sidebar state unless persisted)
- it's slower and feels less smooth

`useNavigate("/templates/123")`

This tells the router: "change the URL and render the new route".

No full reload:

- React stays mounted
- layout stays alive (sidebar/topbar don't reset)
- faster + smoother
- state can persist

#### Senior-level intuition:

`useNavigate` is "internal navigation" (client-side).

`window.location` is "hard navigation" (new document request).

When to use `window.location`?

- leaving your app entirely (external URL)
- downloading a file by navigating to a direct link (sometimes)
- forcing a hard refresh (rare, usually a smell)

✅ So for normal page navigation, always prefer `useNavigate()`.

### Your `isCreating` reasoning ✅

You got it. It's both UX and correctness:

- UX: show "Creating..." and disable button
- correctness: prevent double-submits

One more "senior insight": also disable inputs (or lock the form) during submit.



## Stage 5.5 — Clean up the create flow properly (still Stage 5)

Right now, you likely have:

- an `alert` for errors
- plain HTML inputs
- no form semantics

We'll tighten it without overwhelming you.

### The concept: "Forms are state machines too"

Form states:

- idle
- submitting
- success (navigate)
- error (show message)

We'll implement error UI instead of alert.

#### Task (small refactor)

In `TemplatesListPage`:

1. Add `createError` state:

```
const [createError, setCreateError] = useState<string | null>(null);
```

1. In `handleCreate`, before calling API:

```
setCreateError(null);
```

1. In catch:

```
setCreateError(e?.message ?? "Failed to create template");
```

1. Show it under the form:

```
{createError && <p style={{ color: "crimson" }}>{createError}</p>}
```

This teaches you a major rule: **errors should be UI, not popups.**

## Stage 6 — Template Details Page (Next)

Now that `create` navigates to `/templates/:templateId`, that page must become real.

### What Template Details must do

When you open `/templates/123`, it should:

1. read `templateId` from URL
2. fetch template details from backend
3. show template header (name, sector, status)

4. show versions list
5. allow:
  - create new version (clone latest)
  - navigate to schema editor for editable version
  - export a version

### Conceptual flow

```
sequenceDiagram
 participant UI as TemplateDetailsPage
 participant Router
 participant API
 participant Backend

 Router->>UI: templateId from URL
 UI->>API: getTemplate(templateId)
 API->>Backend: GET /api/templates/{templateId}
 Backend-->>API: template + versions
 API-->>UI: data
 UI->>UI: render header + versions table
```

### Before we implement Stage 6: show me your backend contract

Because your create response already differs (templateId + versionId + versionNumber), I don't want to guess the "get template details" shape.

Paste one of these (any one):

- Example JSON response of `GET /api/templates/{templateId}` from Swagger, OR
- the DTO from backend, OR
- a screenshot of Swagger response schema.

Just that. Then I'll give you the exact Stage 6 code to implement with correct types and UI state handling.

(That's not "asking too many questions" — it's preventing you from building the wrong structure and refactoring later.)

## ▼ V3.6

Good. Now we build the first page that feels "real".

Stage 6 is where routing + async + conditional UI all meet.

We are building:

#### 👉 Template Details Page

This page must:

- Read `templateId` from URL
- Fetch template details
- Render template info
- Render versions list
- Allow creating a new version
- Navigate to schema editor

- Enforce “latest version editable” rule in UI

We do it step by step.

## Step 6.1 — Understand the Data Shape

From your OpenAPI:

`GET /api/templates/{templateId}` returns:

```
type TemplateDetailResponse = {
 id: string
 name: string
 sectorCode: string
 status: string
 createdAt: string
 createdBy: string
 versions: VersionSummaryDto[]
}
```

And:

```
type VersionSummaryDto = {
 id: string
 versionNumber: number
 status: string
 createdAt: string
}
```

Important understanding:

The template contains its versions.

So data flow is:

```
flowchart TD
 URL --> TemplateDetailsPage
 TemplateDetailsPage --> API_Call
 API_Call["GET /api/templates/{id}"] --> Backend
 Backend --> TemplateDetailResponse
```

## Step 6.2 — Add API Functions

Open:

`src/features/templates/api.ts`

Add:

```
export type VersionSummary = {
 id: string;
 versionNumber: number;
 status: string;
 createdAt: string;
};
```

```

export type TemplateDetail = {
 id: string;
 name: string;
 sectorCode: string;
 status: string;
 createdAt: string;
 createdBy: string;
 versions: VersionSummary[];
};

export async function getTemplate(
 templateId: string
): Promise<TemplateDetail> {
 return apiFetch(`/api/templates/${templateId}`);
}

export async function createVersion(
 templateId: string,
 createdBy: string
): Promise<{ versionId: string; versionNumber: number }> {
 return apiFetch(`/api/templates/${templateId}/versions`, {
 method: "POST",
 body: JSON.stringify({ createdBy }),
 });
}

```

We isolate all backend calls here. Pages don't know endpoint URLs.

## Step 6.3 — Build TemplateDetailsPage (Core Logic)

Open:

`src/features/templates/pages/TemplateDetailsPage.tsx`

Replace with:

```

import { useEffect, useState } from "react";
import { useParams, useNavigate } from "react-router-dom";
import {
 getTemplate,
 createVersion,
 TemplateDetail,
} from "../api";

export default function TemplateDetailsPage() {
 const { templateId } = useParams();
 const navigate = useNavigate();

 const [template, setTemplate] = useState<TemplateDetail | null>(null);
 const [isLoading, setIsLoading] = useState(false);
 const [error, setError] = useState<string | null>(null);
 const [isCreatingVersion, setIsCreatingVersion] = useState(false);

 useEffect(() => {
 if (!templateId) return;

```

```

 async function load() {
 setIsLoading(true);
 setError(null);
 try {
 const data = await getTemplate(templateId);
 setTemplate(data);
 } catch (e: any) {
 setError(e.message ?? "Failed to load template");
 } finally {
 setIsLoading(false);
 }
 }

 load();
 }, [templateId]);

 async function handleCreateVersion() {
 if (!templateId) return;

 try {
 setIsCreatingVersion(true);
 const result = await createVersion(templateId, "omar");

 // reload template after creating version
 const updated = await getTemplate(templateId);
 setTemplate(updated);

 // optional: navigate directly to schema editor
 navigate(
 `/templates/${templateId}/versions/${result.versionId}/schema`
);
 } catch (e: any) {
 alert(e.message ?? "Failed to create version");
 } finally {
 setIsCreatingVersion(false);
 }
 }

 if (isLoading) return <h1>Loading template...</h1>;
 if (error) return <h1 style={{ color: "red" }}>{error}</h1>;
 if (!template) return <h1>Template not found</h1>;

 return (
 <div>
 <h1>{template.name}</h1>
 <p>Sector: {template.sectorCode}</p>
 <p>Status: {template.status}</p>

 <hr />

 <h2>Versions</h2>

 <button onClick={handleCreateVersion} disabled={isCreatingVersion}>

```

```

 {isCreatingVersion ? "Creating..." : "Create New Version"}
 </button>

 {template.versions.map((v) => (
 <li key={v.id}>
 v{v.versionNumber} – {v.status} –{" "}
 <button
 onClick={() =>
 navigate(
 `/templates/${template.id}/versions/${v.id}/schema`
)
 }
 >
 Edit Schema
 </button>

))}

 </div>
);
}

```

## What You Just Built

You implemented:

- URL-driven data loading
- Conditional rendering
- Version creation flow
- Navigation to nested route
- State reload after mutation

This is a real admin page now.

## Important Concept: Dependency in useEffect

Notice this:

```

useEffect(() => {
 ...
}, [templateId]);

```

That means:

Whenever `templateId` changes, reload data.

If you forgot the dependency array:

- It would run every render → infinite loop.

If you left it empty:

- It wouldn't reload when navigating between templates.

This is where many beginners get stuck.

▼ more detailed (whole picture)

▼ explanation 1

## ☀ THE WHOLE PICTURE — Corrected & Completed

I'll rewrite your understanding step by step, correcting the small misconceptions and strengthening the mental model.

---

### ✓ 1. AppShell is a static layout

Yes.

`<AppShell />` is your **layout wrapper**.

It contains things like:

- sidebar
- header
- layout grid
- providers

It wraps all dynamic pages via React Router's `<Outlet />`.

---

### ✓ 2. React creates the root once

Correct.

When your app starts:

1. Browser loads `index.html`
2. `main.tsx` runs
3. React does:

```
createRoot(document.getElementById("root")).render(<RouterProvider router={router} />);
```

This creates the **React root** exactly once.

React does **not** recreate the root on navigation.

---

### ✓ 3. Router decides which page to render

Correct.

React Router looks at the URL and decides:

- which layout to render
- which page to render inside it
- which dynamic params to pass

Example:

```
/templates/123
```

---

Router matches:

```
path: "/templates/:templateId"
```

And renders:

```
<AppShell>
 <TemplateDetailsPage templateId="123" />
</AppShell>
```

## ⚠ 4. Important correction: React does NOT “mount again” on URL change

This is the part you slightly misunderstood.

- ✓ React does NOT remount the entire app
- ✓ React does NOT recreate the root
- ✓ React does NOT rebuild the whole tree from scratch

Instead:

### ★ React re-renders only the components whose props changed

In this case:

- `AppShell` stays mounted
- Only the **child route component** changes
- React updates the Virtual DOM for that subtree
- React applies minimal changes to the real DOM

This is called **re-render**, not **re-mount**.

## 🌱 5. Mount vs Render — the key distinction

This is the most important correction.

### ✓ Mount

Happens **once** when a component first appears in the DOM.

### ✓ Re-render

Happens whenever props or state change.

### ✓ Unmount

Happens when a component disappears from the DOM.

## 🌱 6. When does `useEffect` run?

This is where your mental model needs a small adjustment.

`useEffect` runs:



- after the component **mounts**
- after every **re-render** where dependencies changed
- before unmount (cleanup)

So:

**✗ It does NOT run “whenever React mounts again”**

Because React does not “mount again” on navigation.

**✓ It runs when the component re-renders**

And re-render happens when:

- props change
- state changes
- context changes

In your case:

`templateId` changes → component re-renders → effect runs.

---



## 7. Putting it all together (corrected full picture)

Here is the accurate flow:

### 1. React boots

- createRoot
- render RouterProvider
- mount AppShell
- mount initial page

### 2. User navigates to a new URL

React Router:

- matches the new route
- keeps AppShell mounted
- unmounts old page
- mounts new page

### 3. React re-renders the subtree

React:

- builds a new Virtual DOM for the changed components
- diffs it with the previous one
- updates only the changed parts in the real DOM

### 4. useEffect runs

For the new page:

- it mounts → effect runs
  - if dependencies change → effect runs again
-

▼ more detailed exp 2

## 1. Props change

**Props** are inputs passed *from a parent component to a child component*.

Example:

```
<TemplateDetailsPage templateId="123" />
```

Here:

- `templateId="123"` is a **prop**
- `TemplateDetailsPage` is the **child**

If the parent renders again with a different value:

```
<TemplateDetailsPage templateId="456" />
```

React sees:

- old props: `{ templateId: "123" }`
- new props: `{ templateId: "456" }`

Because the props changed, React **re-renders the child component**.

This is exactly what happens when you navigate:

```
/templates/123 → /templates/456
```

React Router updates the prop → component re-renders → `useEffect` with `[templateId]` runs.

## 2. State changes

Every component can hold its own **state** using `useState`.

Example:

```
const [count, setCount] = useState(0);
```

When you call:

```
setCount(1);
```

React schedules a re-render of that component.

Why?

Because the UI depends on state.

If state changes, React must re-run the component function to compute the new UI.

State changes are the **most common** cause of re-renders.

## 3. Context changes

React Context lets you share values across the app without passing props manually.

Example:

- theme context
- auth context
- toast provider
- router context

If a context value changes:

```
<ThemeContext.Provider value={theme}>
```

and `theme` changes from `"light"` to `"dark"`:

React re-renders **every component that uses that context**.

Example:

```
const theme = useContext(ThemeContext);
```

If the context changes → this component re-renders.

## Putting it all together (the whole picture)

A React component re-renders when **any** of these happen:

### ✓ Its props change

(e.g., new `templateId` from the router)

### ✓ Its state changes

(e.g., `setLoading(false)` after fetching data)

### ✓ A context it consumes changes

(e.g., theme, auth, router, toast provider)

These are the *only* three triggers.

React does **not** re-render because:

- the URL changed
- the DOM changed
- the user clicked something
- the component "mounted again"

Those things only matter if they cause **props/state/context** to change.

## How this relates to `useEffect`

`useEffect` runs:

- after the component **mounts**
- after a **re-render** where dependencies changed

So if you write:

```
useEffect(() => {
 loadTemplate(templateId);
```

```
}, [templateId]));
```

This effect runs when:

- the component mounts
- `templateId` changes (because props changed)

It does **not** run because "React mounted again" — React doesn't remount the whole app.

It runs because the **dependency changed**, which caused a **re-render**.

## If you want, I can also explain:

- how React decides *which* components to re-render
- how React Router mounts/unmounts pages
- how `useEffect` cleanup works
- how to visualize the component tree during navigation
- how to avoid unnecessary re-renders

Just tell me where you want to go next.

▼ more detailed exp 3

## Short answer

**Yes — when you define variables in the router config (like `:templateId`), React Router extracts them from the URL and passes them to your component.**

But **not** as normal props.

Instead, React Router stores them in a special internal context, and your component reads them using a hook like:

```
const { templateId } = useParams();
```

So the idea is correct, but the mechanism is slightly different.

## The accurate picture (clean and simple)

When you write a route like:

```
{ path: "/templates/:templateId", element: <TemplateDetailsPage /> }
```

React Router does three things:

### 1. It matches the URL

Example URL:

```
/templates/123
```

Router sees:

- `:templateId` is a variable
- It extracts `templateId = "123"`

## 2. It stores that value in Router Context

React Router keeps the params in its own internal state.

## 3. Your component reads it using `useParams()`

Inside `TemplateDetailsPage`:

```
const { templateId } = useParams();
```

This is how your component receives the dynamic value.

## So is it “like props”?

Yes — conceptually.

### ✓ It behaves like props

Because when the URL changes:

- `templateId` changes
- your component re-renders
- `useEffect` with `[templateId]` runs

### ✗ But it is NOT passed as a normal prop

React Router does not do:

```
<TemplateDetailsPage templateId="123" />
```

Instead, it does:

- store params in context
- your component reads them with a hook

## Why React Router uses context instead of props

Because:

- nested routes
- layouts ( `AppShell` )
- `<Outlet />`
- loaders/actions
- navigation state

All rely on a shared internal state.

Passing everything as props would be messy and impossible to scale.

Context solves this cleanly.

## The mental model you should keep

### ✓ Route params behave like props

They trigger re-renders when they change.

✓ But they are delivered through `useParams()`

Not through JSX props.

✓ Router config defines the “shape” of the params

`/templates/:templateId` → gives you `{ templateId }`

✓ URL changes → params change → component re-renders

This is why your `useEffect` with `[templateId]` works.

---

## UI Rule: “Latest Version Editable”

Backend enforces editability rule.

Frontend should reflect it visually.

You can derive:

```
const latestVersion = template.versions.reduce((prev, curr) =>
 curr.versionNumber > prev.versionNumber ? curr : prev
);
```

Then disable “Edit” for non-latest versions.

We can implement that next.

---

## Your Task for Stage 6

1. Implement the API functions
2. Implement TemplateDetailsPage
3. Test:
  - Navigate from create flow
  - Create new version
  - Navigate to schema editor

Then answer:

1. Why do we reload template after creating version?
2. What would break if we didn't reload?
3. Where should the “latest editable” rule really live — frontend or backend?

After your answers, we move to Stage 7 — the hardest part:

Schema Editor with nested state.

This is where your understanding will level up significantly.

### ▼ V3.7.1

## Your answers — corrected and upgraded

### 1) Why reload template after creating a version?

Your answer: “so we can see the update” ✓

Senior correction:

We reload because **React state is not magically linked to the database.**

When you call `createVersion()`, backend changes data. But your UI is still holding an **old snapshot** of `template` in state.

So you have two valid ways to update UI:

#### Option A: Refetch (what you did)

- call `getTemplate()` again and replace state
- simplest and safest for beginners
- costs an extra request

#### Option B: Update state locally

- take current `template.versions` and append the new version
- faster, fewer requests
- easy to get wrong and drift from backend truth

For MVP learning, **refetch is correct**.

And you're right: `useEffect` will NOT rerun automatically after `createVersion`, because `templateId` didn't change. So refetch must be manual or state must be updated.

## 2) What breaks if we don't reload?

Your answer: "UI won't update" ✓

More precise:

- The versions list stays stale (missing the new version)
- "latest editable" calculation becomes wrong
- You might navigate to schema editor for new version, but the Template Details page won't reflect it when user goes back

## 3) Where should "latest editable" rule live?

Your answer: "frontend and backend" ✓

Perfect senior view:

- **Backend is the authority** (security + correctness)
- **Frontend mirrors the rule** (good UX)

Frontend alone is never enough (users can call API directly).

---

## Stage 7 — Schema Editor (Hard Mode Begins)

This is the core of your product.

And it's where beginners usually hit a wall — because it's not "a page", it's a **nested state editor**.

So we'll go slow and do it properly.

### What Schema Editor really is

It's a UI for editing a JSON document.

Your backend expects this shape ( `UpdateSchemaRequest` ):

```
{
 templateName: string,
 sectorCode: string,
 tables: [
```

```

{
 tableKey: string,
 sheetName: string,
 order: number,
 fields: [
 {
 fieldKey: string,
 headerName: string,
 type: string,
 required: boolean,
 validations?: {
 enumValues?: string[],
 min?: number,
 max?: number
 }
 }
]
}
],
exportProfile?: { ... }
}

```

Think of it as a tree:

```

flowchart TD
 Schema --> TemplateInfo[templateName + sectorCode]
 Schema --> Tables
 Tables --> Table1[tableKey, sheetName, order]
 Table1 --> Fields
 Fields --> Field1[fieldKey, headerName, type, required]
 Field1 --> Validations[min/max/enum]

```

So Schema Editor is:

- Load schema JSON from backend
- Convert to React state
- Render UI for editing tables/fields
- Validate rules in frontend (unique keys etc.)
- Save using PUT `/api/versions/{versionId}/schema`

## The key React concept you must learn here

### “Immutable updates”

React state should be treated like “read-only”.

When you update a table or field, you don’t mutate objects in place.

Bad (mutates):

```

tables[0].fields.push(newField);
setTables(tables);

```

Good (creates new arrays/objects):



```

setTables(prev =>
 prev.map(t =>
 t.tableKey === targetKey
 ? { ...t, fields: [...t.fields, newField] }
 : t
)
);

```

This matters because React detects changes by reference.

If you mutate, React often won't behave predictably.

HEP warning: Schema Editor without immutability discipline becomes bug city.

## Stage 7.1 — Add Version API calls

Create `src/features/versions/api.ts` (or `schema/api.ts`, but I prefer versions since endpoint is `/versions/...`):

```

import { apiFetch } from "../../shared/lib/http";

export type VersionDetail = {
 id: string;
 templateId: string;
 versionNumber: number;
 status: string;
 schemaJson: any;
 schemaHash: string | null;
 createdAt: string;
 createdBy: string;
};

export type UpdateSchemaRequest = {
 templateName: string;
 sectorCode: string;
 tables: Array<{
 tableKey: string;
 sheetName: string;
 order: number;
 fields: Array<{
 fieldKey: string;
 headerName: string;
 type: string;
 required: boolean;
 validations?: {
 enumValues?: string[];
 min?: number;
 max?: number;
 };
 }>;
 }>;
 exportProfile?: {
 format?: string;
 includeInstructionsSheet?: boolean;
 protectSheets?: boolean;
 };
};

```

```

 };
 };

 export async function getVersion(versionId: string): Promise<VersionDetail> {
 return apiFetch(`/api/versions/${versionId}`);
 }

 export async function updateSchema(
 versionId: string,
 body: UpdateSchemaRequest
): Promise<{ versionId: string; schemaHash: string }> {
 return apiFetch(`/api/versions/${versionId}/schema`, {
 method: "PUT",
 body: JSON.stringify(body),
 });
 }
}

```

## Stage 7.2 — Load version in SchemaEditorPage

Open `SchemaEditorPage.tsx` and implement a basic load (still no editor UI yet):

```

import { useEffect, useState } from "react";
import { useParams } from "react-router-dom";
import { getVersion, updateSchema, UpdateSchemaRequest } from "../../versions/api";

import { getTemplate } from "../../templates/api";

export default function SchemaEditorPage() {
 const { templateId, versionId } = useParams();

 const [isLoading, setIsLoading] = useState(false);
 const [error, setError] = useState<string | null>(null);

 const [schema, setSchema] = useState<UpdateSchemaRequest | null>(null);

 useEffect(() => {
 if (!templateId || !versionId) return;

 async function load() {
 setIsLoading(true);
 setError(null);
 try {
 const [tpl, ver] = await Promise.all([
 getTemplate(templateId),
 getVersion(versionId),
]);

 // Backend returns schemaJson as JsonNode. If empty, we seed it.
 const existing = ver.schemaJson ?? {};

 const seeded: UpdateSchemaRequest = {

```

```

 templateName: existing.templateName ?? tpl.name,
 sectorCode: existing.sectorCode ?? tpl.sectorCode,
 tables: existing.tables ?? [],
 exportProfile: existing.exportProfile ?? {},
 };

 setSchema(seeded);
 } catch (e: any) {
 setError(e.message ?? "Failed to load schema");
 } finally {
 setIsLoading(false);
 }
}

load();
}, [templateId, versionId]);

if (isLoading) return <h1>Loading schema...</h1>;
if (error) return <h1 style={{ color: "red" }}>{error}</h1>;
if (!schema) return <h1>No schema</h1>;

return (
 <div>
 <h1>Schema Editor</h1>
 <p>Template: {schema.templateName}</p>
 <p>Sector: {schema.sectorCode}</p>

 {/* temporary debug view */}
 <pre style={{ background: "#111", color: "#0f0", padding: 12 }}>
 {JSON.stringify(schema, null, 2)}
 </pre>
 </div>
);
}

```

This will immediately solve the sectorCode issue conceptually:

We seed schema with template values if backend schemaJson doesn't contain them.

## Your Stage 7 Task (Part 1)

Implement only **load + seed + debug render** for SchemaEditorPage.

Confirm:

1. `/templates/:tid/versions/:vid/schema` loads without error
2. You see JSON printed
3. `templateName` and `sectorCode` are NOT empty

When you confirm, we move to Stage 7 Part 2:

✅ UI for managing tables (add/edit/delete table) using proper immutable updates.

That's where the real React learning begins.

Reply with:

- “Stage 7.1 done”
- paste a short sample of the printed JSON (first ~15 lines is enough)

## ▼ V3.7.2

# Stage 7.2 — Tables UI (Add / Edit / Delete) with correct state updates

## Concept first: why this is hard

Your schema is nested:

Schema → tables[] → fields[]

Nested state is hard because:

- if you mutate in place, React won’t behave reliably
- updating “one thing” requires rebuilding parts of the tree immutably

So our goal is to build a clean set of “schema operations”:

- addTable
- updateTable
- deleteTable

You can think of them like backend endpoints, but in frontend state.

## The golden rule

**Never mutate schema in place.**

Always create new objects/arrays when updating.

## What the Tables UI should look like (simple MVP)

We’ll make it intentionally basic for learning:

- A section that lists tables with:
  - tableKey, sheetName, order
  - button: “Edit”
  - button: “Delete”
- A button: “Add table”
- A modal (or inline form) to enter:
  - tableKey
  - sheetName
  - order

We can keep it inline first, then convert to modal once stable.

## Step 7.2.1 — Define types locally (for strong intuition)

Inside `SchemaEditorPage.tsx`, add a helper type alias (optional but nice):

```
type Table = UpdateSchemaRequest["tables"][number];
```

This means: "Table is one item from schema.tables".

## Step 7.2.2 — Add local UI state for "table form"

At the top of SchemaEditorPage component (after schema state), add:

```
const [isTableFormOpen, setIsTableFormOpen] = useState(false);
const [editingTableKey, setEditingTableKey] = useState<string | null>(null);

const [tableKey, setTableKey] = useState("");
const [sheetName, setSheetName] = useState("");
const [order, setOrder] = useState<number>(0);

const [tableFormError, setTableFormError] = useState<string | null>(null);
```

### Concept

This is separate from schema.

Schema is the **data model**.

Form state is **temporary UI state**.

Mixing them causes bugs.

## Step 7.2.3 — Add helper functions to open/close form

Add inside component:

```
function openAddTable() {
 setEditingTableKey(null);
 setTableKey("");
 setSheetName("");
 setOrder(0);
 setTableFormError(null);
 setIsTableFormOpen(true);
}

function openEditTable(t: Table) {
 setEditingTableKey(t.tableKey);
 setTableKey(t.tableKey);
 setSheetName(t.sheetName);
 setOrder(t.order);
 setTableFormError(null);
 setIsTableFormOpen(true);
}

function closeTableForm() {
 setIsTableFormOpen(false);
}
```

## Step 7.2.4 — Implement validation (frontend side)

We must validate:

- tableKey unique
- sheetName constraints (<= 31, no forbidden chars)

Add:

```
function validateTableInput(currentTables: Table[]) {
 if (!tableKey.trim()) return "tableKey is required";
 if (!sheetName.trim()) return "sheetName is required";
 if (sheetName.length > 31) return "sheetName must be <= 31 characters";

 const forbidden = /[\\[\]*\\/\\\?\:\:]/; // Excel forbidden chars
 if (forbidden.test(sheetName)) return "sheetName contains forbidden characters";

 const exists = currentTables.some(
 (t) => t.tableKey === tableKey && t.tableKey !== editingTableKey
);
 if (exists) return "tableKey must be unique";

 return null;
}
```

This makes frontend consistent with backend rules.

## Step 7.2.5 — Save table (immutable update)

Add:

```
function saveTable() {
 if (!schema) return;

 const err = validateTableInput(schema.tables);
 if (err) {
 setTableFormError(err);
 return;
 }

 const newTable: Table = {
 tableKey: tableKey.trim(),
 sheetName: sheetName.trim(),
 order: Number(order) || 0,
 fields: editingTableKey
 ? schema.tables.find((t) => t.tableKey === editingTableKey)?.fields ?? []
 : [],
 };

 setSchema((prev) => {
 if (!prev) return prev;

 // edit vs add
 if (editingTableKey) {
 return {
 ...prev,
 tables: prev.tables.map((t) =>
```

```

 t.tableKey === editingTableKey ? newTable : t
),
 };
 }

 return {
 ...prev,
 tables: [...prev.tables, newTable].sort((a, b) => a.order - b.order),
 };
});

closeTableForm();
}

```

Key point: we used `setSchema(prev => ...)` which is the safe React pattern.

## Step 7.2.6 — Delete table

Add:

```

function deleteTable(key: string) {
 if (!schema) return;

 setSchema((prev) => {
 if (!prev) return prev;
 return {
 ...prev,
 tables: prev.tables.filter((t) => t.tableKey !== key),
 };
 });
}

```

## Step 7.2.7 — Render tables list + form

In your JSX (replace the debug-only view, keep debug at bottom), render:

```

<div style={{ display: "flex", gap: 24 }}>
 <div style={{ flex: 1 }}>
 <h2>Tables</h2>
 <button onClick={openAddTable}>+ Add Table</button>

 {schema.tables.length === 0 ? (
 <p>No tables yet.</p>
) : (

 {schema.tables
 .slice()
 .sort((a, b) => a.order - b.order)
 .map((t) => (
 <li key={t.tableKey} style={{ marginTop: 12 }}>
 {t.tableKey} – {t.sheetName} – order {t.order}
 <div style={{ display: "inline-flex", gap: 8, marginLeft: 12 }}>

```

```

 <button onClick={() => openEditTable(t)}>Edit</button>
 <button onClick={() => deleteTable(t.tableKey)}>Delete</button>
 </div>

)}}

)}
</div>

<div style={{ width: 360 }}>
 {isTableFormOpen && (
 <div style={{ border: "1px solid #ddd", padding: 12 }}>
 <h3>{editingTableKey ? "Edit Table" : "Add Table"}</h3>

 <div style={{ display: "grid", gap: 8 }}>
 <input
 placeholder="tableKey"
 value={tableKey}
 onChange={(e) => setTableKey(e.target.value)}
 />
 <input
 placeholder="sheetName"
 value={sheetName}
 onChange={(e) => setSheetName(e.target.value)}
 />
 <input
 placeholder="order"
 type="number"
 value={order}
 onChange={(e) => setOrder(Number(e.target.value))}
 />

 {tableFormError && (
 <p style={{ color: "crimson" }}>{tableFormError}</p>
)}

 <div style={{ display: "flex", gap: 8 }}>
 <button onClick={saveTable}>
 {editingTableKey ? "Save" : "Add"}
 </button>
 <button onClick={closeTableForm}>Cancel</button>
 </div>
 </div>
)}
 </div>
</div>

```

And keep your debug `<pre>` below so you see schema changing in real time.

## Your Stage 7.2 Task

Implement the above tables editor.



Confirm:

1. You can add a table
2. It appears in the list
3. JSON in debug updates correctly
4. Edit updates the right table without losing fields
5. Delete removes it
6. Uniqueness + sheetName validation works

When done, say: **"Stage 7.2 done"** and tell me:

- what bug you hit (if any)
- one thing that surprised you

Then we go Stage 7.3:

✅ Fields UI inside a selected table (nested state level 2).

## ▼ Answers

### What you actually built in Stage 7.2

You built:

```
Frontend Draft State Editor
```

That means:

- You are editing `schema` **in memory**
- Nothing is saved yet
- Backend doesn't know anything changed
- You're preparing the object that will later be sent via:

```
PUT /api/versions/{id}/schema
```

That's exactly how real systems work.

Think of it like Google Docs:

- You type locally
- It keeps a working model
- Only when sync/save happens → backend update

### Why we did NOT call CRUD APIs for tables

Because your backend does NOT have:

```
POST /tables
PUT /tables/{id}
DELETE /tables/{id}
```

Your backend contract is:

```
PUT /api/versions/{id}/schema
```

It expects the entire schema object at once.

That is a **document-style API**.

This means:

Frontend is responsible for:

- building
- editing
- validating
- structuring

Then sending one atomic payload.

---

## Big Architecture Insight (HEP Mode)

You have two possible designs:

### 1 Document-based (what you have)

```
Version
└─ schemaJson (whole structure)
```

Pros:

- Simple backend
- Atomic version updates
- Perfect for export-based systems

Cons:

- Frontend must manage nested state carefully
- 

### 2 Fully normalized backend CRUD

```
/templates
/templates/{id}/tables
/templates/{id}/tables/{id}/fields
```

Pros:

- Backend enforces structure
- Less frontend complexity

Cons:

- More endpoints
  - More DB complexity
  - Versioning becomes harder
  - Excel export must rebuild full structure anyway
- 

For YOUR domain (governance template export):

The document approach is correct.

You accidentally discovered why 😊

---

## Now the important architectural lesson

You now have two layers of truth:

- 1) Backend persistent schema (DB)
- 2) Frontend working schema (state)

We must keep them synchronized properly.

And that leads us to:

---

## Stage 7.3 — Persist Schema to Backend

Now we connect the UI editor to:

```
PUT /api/versions/{versionId}/schema
```

This will:

- validate in backend
- canonicalize JSON
- compute hash
- store schemaJson
- return schemaHash

This is where the real system becomes complete.

---

## Conceptual Flow (Very Important)

```
User edits tables/fields
 ↓
Frontend updates local schema state
 ↓
User clicks Save
 ↓
Frontend calls PUT schema API
 ↓
Backend validates + hashes + stores
 ↓
Backend returns schemaHash
 ↓
Frontend updates version state
```

## Before we code Stage 7.3

Answer me this:

When should Save button be enabled?

- Always?
- Only if schema changed?
- Only if valid?

- Both?

Think like a senior.

Tell me your answer first.

Then we implement Stage 7.3 properly.

## ▼ V3.7.3

### When should Save be enabled?

In real apps, **Save enablement is a policy**, not a rule. But for your MVP, the best policy is:

**Enable Save only when:**

1. the schema has **changed** (dirty = true)
2. the schema is **locally valid enough** to avoid obvious backend rejection (valid = true)
3. you're **not currently saving** (isSaving = false)

If validation is heavy, you can do “basic validation” live and full validation on save. But Excel sheet rules + uniqueness are cheap, so we can validate live.

So:

**| Save enabled = dirty && valid && !isSaving**

Now we implement Stage 7.3.

## Stage 7.3 — Save Schema to Backend + Hash feedback

We'll add 4 concepts:

- `dirty` state
- `valid` state (derived)
- `isSaving` state
- `schemaHash` display after save

### Step 7.3.1 — Track dirty state

At the top of SchemaEditorPage component, add:

```
const [isDirty, setIsDirty] = useState(false);
const [isSaving, setIsSaving] = useState(false);
const [saveError, setSaveError] = useState<string | null>(null);
const [schemaHash, setSchemaHash] = useState<string | null>(null);
```

Now, whenever you successfully change schema locally, mark it dirty.

Instead of manually calling `setIsDirty(true)` everywhere, we'll make a helper.

Add inside component:

```
function updateSchemaLocal(updater: (prev: UpdateSchemaRequest) => UpdateSchemaRequest) {
 setSchema((prev) => {
 if (!prev) return prev;
 const next = updater(prev);
 return next;
 });
}
```

```
});
setIsDirty(true);
}
```

Now replace your `setSchema(prev => ...)` calls with `updateSchemaLocal(...)`.

Example for `deleteTable`:

```
function deleteTable(key: string) {
 if (!schema) return;

 updateSchemaLocal((prev) => ({
 ...prev,
 tables: prev.tables.filter((t) => t.tableKey !== key),
 }));
}
```

Do the same for `saveTable` add/edit operations.

This keeps “dirty tracking” consistent.

## Step 7.3.2 — Define local validation for the full schema

We already validate table input in the form.

But now we need a “whole schema validation” rule set:

- `templateName` required
- `sectorCode` required
- `tableKey` unique
- `fieldKey` unique per table
- `sheetName` constraints

Add:

```
function validateSchema(s: UpdateSchemaRequest): string | null {
 if (!s.templateName?.trim()) return "templateName is required";
 if (!s.sectorCode?.trim()) return "sectorCode is required";

 const forbidden = /[\\[\]*\\/\\\?\:]/;

 const tableKeys = new Set<string>();
 for (const t of s.tables) {
 if (!t.tableKey?.trim()) return "Each table must have a tableKey";
 if (tableKeys.has(t.tableKey)) return `Duplicate tableKey: ${t.tableKey}`;
 tableKeys.add(t.tableKey);

 if (!t.sheetName?.trim()) return `sheetName required for table: ${t.tableKey}`;
 if (t.sheetName.length > 31) return `sheetName too long: ${t.sheetName}`;
 if (forbidden.test(t.sheetName)) return `sheetName has forbidden chars: ${t.sheetName}`;

 const fieldKeys = new Set<string>();
 for (const f of t.fields ?? []) {
```

```

 if (!f.fieldKey?.trim()) return `fieldKey required in table: ${t.tableKey}`;
 if (fieldKeys.has(f.fieldKey)) return `Duplicate fieldKey ${f.fieldKey} in table ${t.tableKey}`;
 fieldKeys.add(f.fieldKey);

 if (!f.headerName?.trim()) return `headerName required for field ${f.fieldKey} in table ${t.tableKey}`;
 if (!f.type?.trim()) return `type required for field ${f.fieldKey} in table ${t.tableKey}`;
 }
}

return null;
}

```

Now compute:

```

const schemaValidationError = schema ? validateSchema(schema) : "No schema";
const isValid = !schemaValidationError;

```

We don't store `isValid` in state because it's derived from schema.

### Step 7.3.3 — Add Save button

Add a Save button in JSX near top:

```

<button
 onClick={handleSave}
 disabled={!schema || !isDirty || !isValid || isSaving}
>
 {isSaving ? "Saving..." : "Save Schema"}
</button>

{!isValid && (
 <p style={{ color: "crimson", marginTop: 8 }}>
 Fix before saving: {schemaValidationError}
 </p>
)}

{saveError && (
 <p style={{ color: "crimson", marginTop: 8 }}>
 Save failed: {saveError}
 </p>
)}

{schemaHash && (
 <p style={{ marginTop: 8 }}>
 Latest schema hash: <code>{schemaHash}</code>
 </p>
)}

```

Now implement handleSave.

### Step 7.3.4 — Implement handleSave (PUT request)

Make sure you already have `updateSchema` imported from versions api.

Add:

```
async function handleSave() {
 if (!schema || !versionId) return;

 const err = validateSchema(schema);
 if (err) {
 setSaveError(err);
 return;
 }

 setIsSaving(true);
 setSaveError(null);

 try {
 const res = await updateSchema(versionId, schema);
 setSchemaHash(res.schemaHash);
 setIsDirty(false);
 } catch (e: any) {
 setSaveError(e.message ?? "Failed to save schema");
 } finally {
 setIsSaving(false);
 }
}
```

This is the full persistence loop.

#### ▼ bug fix

Yep — there **is** a logic bug, and it's exactly in `saveTable()`. You accidentally mixed **two update styles** together:

- `setSchema(prev => ...)` (direct state setter)
- `updateSchemaLocal(prev => ...)` (your helper that already calls `setSchema` + marks dirty)

That causes **double updates**, and in edit mode it can even **add a duplicate table** instead of editing.

### The bug

In `saveTable()` you do this:

```
setSchema((prev) => {
 ...
 if (editingTableKey) {
 updateSchemaLocal(...)
 }

 return {
 ...prev,
 tables: [...prev.tables, newTable]
 };
});
```

So in edit mode:

- it calls `updateSchemaLocal()` (which calls `setSchema`)
  - then it still returns an "add" result from the outer `setSchema`
- ✓ Result: edit becomes "edit + add" (duplication / weird behavior)

Also: dirty tracking becomes inconsistent.

## Fix (clean and correct)

**Pick ONE approach.**

Since you already have `updateSchemaLocal()`, use it and remove the outer `setSchema`.

Replace your entire `saveTable()` with this:

```
function saveTable() {
 if (!schema) return;

 const err = validateTableInput(schema.tables);
 if (err) {
 setTableFormError(err);
 return;
 }

 const newTable: Table = {
 tableKey: tableKey.trim(),
 sheetName: sheetName.trim(),
 order: Number(order) || 0,
 fields: editingTableKey
 ? schema.tables.find((t) => t.tableKey === editingTableKey)?.fields ?? []
 : [],
 };

 if (editingTableKey) {
 // EDIT
 updateSchemaLocal((prev) => ({
 ...prev,
 tables: prev.tables
 .map((t) => (t.tableKey === editingTableKey ? newTable : t))
 .sort((a, b) => a.order - b.order),
 }));
 } else {
 // ADD
 updateSchemaLocal((prev) => ({
 ...prev,
 tables: [...prev.tables, newTable].sort((a, b) => a.order - b.order),
 }));
 }

 closeTableForm();
}
```

That's it.



### Why this works

- Only one state update path
  - Edit doesn't add
  - Dirty always becomes true correctly
  - Sorted consistently in both paths
- 

## Small extra HEP improvements (optional but worth it)

### 1. Reset dirty when loading schema

When you load from backend, you should do:

```
setSchema(seeded);
setIsDirty(false);
setSchemaHash(ver.schemaHash ?? null);
```

Otherwise you might be "dirty" after load in some edge cases.

#### 1. `const isValid = !schemaValidationError;`

Right now `schemaValidationError` is `"No schema"` when schema is null, but you already return early if `!schema`, so it's fine.

---

## Your task

1. Apply the `saveTable()` replacement above.
2. Test:
  - Add table → appears once
  - Edit table → changes in place, no duplicates
  - Delete table → works
  - Save Schema → works, hash appears
  - Edit after save → dirty becomes true again

If this works, say "fixed", and we go to **Stage 7.4 (Fields UI)**.

---

## Stage 7.3 Task

1. Add dirty tracking helper and refactor table operations to use it
2. Add `validateSchema()` and compute `isValid`
3. Add Save button with correct disabled logic
4. Save schema, confirm backend stores it (you can check by reloading page and seeing same JSON loaded)

Confirm these behaviors:

- Save disabled when nothing changed
- Save enabled after adding/editing table
- Save disabled if schema invalid (e.g. empty `sectorCode`)
- After save, dirty resets and `schemaHash` appears

- Refresh page and schema is still there

When done, reply:

- “Stage 7.3 done”
- one screenshot of the hash display or paste the hash
- tell me if backend returns any validation errors

Next after that:

Stage 7.4 = Fields UI (Add/Edit/Delete fields inside a table) — the second nested level.

## ▼ Inspecting Real MVP

### ▼ Info Stage

#### Our inspection method

We won't read everything. We'll follow **one user journey at a time** and only open the files that are on that journey's path.

We'll do 5 traces (these are basically your demo features):

1. Templates list
2. Create template
3. Template details + versions
4. Schema editor save
5. Export download

Each trace will teach you just enough React + architecture to own that feature.

#### Trace 0: “How does the app start?”

This is the foundation. We inspect only 3 files.

##### Files to open in this order

1. `src/main.tsx`
2. `src/app/App.tsx` (or whatever the root component is called inside `app/`)
3. `src/app/router.tsx` (or similar)

##### What you'll learn here (critical only)

- **Bootstrapping / mount:** what happens when the app starts
- Why providers wrap everything (ToastProvider, maybe BreadcrumbProvider, etc.)
- Why Router sits where it sits
- Where the “shell” concept appears ( `AppShell` + `Outlet` )

**Output you should write down** (so you can explain to someone else):

- Entry point: main.tsx mounts React into the DOM root
- App wraps providers + router
- Router chooses a page component
- AppShell provides layout + Outlet renders current route

#### Trace 1: “Templates list page”

This is where we see the first real feature flow.

### Files to open

1. `src/features/templates/pages/TemplatesListPage.tsx` (or similar)
2. `src/features/templates/api/*`
3. `src/shared/lib/http.ts`
4. `src/shared/errors/*` (only if errors are handled here)

### What you'll learn

- How data fetching is done (useEffect vs loader patterns)
- How axios is configured (proxy vs baseUrl)
- How errors are normalized and displayed
- Where UI primitives are used (Table, Badge, EmptyState)

### Output to note down

- "List templates flow": Page → templatesApi.list → http client → GET /api/templates → setState → render Table

---

## Trace 2: "Create template"

### Files to open

- same TemplatesListPage (create modal lives there often)
- templates api create function
- toast usage

### Concepts learned

- Controlled forms (state per input)
- loading guards ( `isCreating` )
- navigation after success ( `useNavigate` )
- toast feedback patterns

---

## Trace 3: "Template details + create version"

### Files to open

- TemplateDetailsPage
- templates api getTemplate
- createVersion api
- versions table component (if extracted)

### Concepts learned

- Route params ( `useParams` )
- "refetch vs local update" strategy after createVersion
- derived UI rules (latest version, lock state, etc.)

## Trace 4: “Schema editor save”

This is the hardest but most valuable.

### Files to open

- SchemaEditorPage
- versions api getVersion + updateSchema
- schema validation helpers (if exist)
- shared error normalization

### What you’ll learn

- local editable schema state vs server state
  - dirty tracking
  - frontend validation vs backend validation
  - why sectorCode bug happened (merge template info + schemaJson)
- 

## Trace 5: “Export download”

### Files to open

- export button component / SchemaEditorPage export action
- `shared/lib/download.ts` (if exists)
- versions api export call

### Concepts learned

- blob download
  - why backend-generated files are normal and correct
  - performance tradeoffs
- 

## What I need from you to start immediately (no more theory)

Pick **one** of these and paste it here:

### Option A (best)

Paste the contents of:

- `src/main.tsx`

### Option B

If you can’t paste, tell me:

- What files exist under `src/app/` (names only)

Then I’ll start Trace 0 and explain exactly what’s happening, concept-first, but still fast.

You choose A or B.

## ▼ Stage 0 — bootstarping

Perfect — `main.tsx` is clean and exactly what we want. Here it is **concept-first**, but still practical.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './app/App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
 <React.StrictMode>
 <BrowserRouter>
 <App />
 </BrowserRouter>
 </React.StrictMode>
);
```

## What this file is

`main.tsx` is the **entry point** of your frontend app.

If your app was a human body, this is the “wake up and start breathing” moment.

### 1) “Bootstrapping”

In frontend talk, *bootstrapping* means:

- load the JavaScript bundle
- initialize the app
- attach React to the HTML page

So `main.tsx` is the boot file.

### 2) “Mounting”

Your HTML (in `index.html`) has a real DOM element like:

```
<div id="root"></div>
```

This line:

```
ReactDOM.createRoot(document.getElementById('root')).render(...)
```

means:

- find that `<div id="root">`
- **mount** React into it
- from that moment onward, React “owns” everything inside `#root`

So mounting = “React takes control of that DOM area”.

### 3) Why `React.StrictMode` exists

`StrictMode` is a **dev-only safety wrapper**. It intentionally makes React stricter and can cause certain functions/effects to run twice in development to catch unsafe patterns.

Important: it does **not** affect production behavior.

If you ever see:

- double API calls in dev only

StrictMode might be why. (Not always, but often.)

#### 4) Why `BrowserRouter` is here

This wraps your whole app and enables:

- URL-based navigation ( `/templates` , `/templates/:id` , etc.)
- `Link` , `useNavigate` , `useParams` , `Outlet`

If you remove `BrowserRouter` , routing breaks and React Router hooks throw errors.

#### 5) What `App` is

`App` is your **root component**.

This is where the app-wide “composition” lives: layout, providers, routes, etc.

#### 6) Why `index.css` imported here

This ensures global styles (Tailwind, CSS variables, base styling) load once for the entire app.

---

## The “tree” mental model you should keep

What React renders is a **component tree**:

```
StrictMode
 BrowserRouter
 App
 (layout + pages...)
```

This is the core structure of your whole frontend.

---

## What to note down (worth documenting)

- `main.tsx` mounts React into `#root`
- `StrictMode` = dev safety checks (may double-run effects)
- `BrowserRouter` = routing context provider for the whole app
- `App` = root of the UI tree

---

## Next step

Now we open the **real App root**, because that’s where:

- providers (toast, breadcrumb, etc.) live
- app shell is composed
- routes are declared or connected

### Please provide

Upload or paste: `src/app/App.tsx`

(Quick note: I saw a system message saying some old uploaded files expired — no worries. We only need `App.tsx` next.)

## ▼ Stage 1 — Application Composition Layer (App Root + Providers + Shell)

This is where the real architecture starts to reveal itself.

You already understand `main.tsx`:

```
BrowserRouter
 → App
```

Now we inspect **what App actually composes**.

## First: The Big Concept

Your frontend is built in layers.

Here is the **conceptual stack**:

```
flowchart TB
 A[index.html #root]
 B[main.tsx
Mount React]
 C[BrowserRouter
Routing Context]
 D[App Root
Providers + Layout]
 E[AppShell
Sidebar + Topbar + Outlet]
 F[Pages
Templates / Details / Schema]

 A --> B --> C --> D --> E --> F
```

Each layer adds **capability**.

- `main.tsx` → boots React
- `BrowserRouter` → gives routing context
- `App` → wires global providers + layout
- `AppShell` → defines persistent UI frame
- `Outlet` → renders current page

## What Is the “App Layer”?

Open `App.tsx` and you’ll likely see something like:

```
function App() {
 return (
 <ToastProvider>
 <BreadcrumbProvider>
 <AppShell>
 <Router />
 </AppShell>
 </BreadcrumbProvider>
 </ToastProvider>
);
}
```

Even if structure slightly differs, conceptually it’s this.

## Concept 1: Providers

**What is a Provider?**

A Provider is:

| A way to make shared data or behavior available to the entire component tree below it.

Conceptually:

```
Provider
├─ Component A
├─ Component B
└─ Component C
```

Instead of passing props down manually:

```
<App toast={...}>
 <Layout toast={...}>
 <Page toast={...} />
 </Layout>
</App>
```

React Context allows:

```
<ToastProvider>
 <Page /> // can use useToast() directly
</ToastProvider>
```

## ◆ Conceptual Diagram — Context Flow

```
flowchart TD
 TP[ToastProvider]
 BC[Some Page]
 BTN[Button inside page]

 TP --> BC
 BC --> BTN
```

**BTN** can call:

```
const { showToast } = useToast();
```

Even though it's deeply nested.

## 🏗️ Concept 2: AppShell

This is extremely important.

AppShell is:

| The persistent structural frame of the application.

It usually contains:

- Sidebar
- Topbar
- Breadcrumbs
- Outlet

Conceptually:

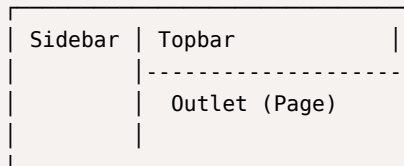


```
flowchart LR
 Sidebar --> Layout
 Topbar --> Layout
 Outlet --> Layout
```

## AppShell is not a page.

It is the **container that wraps pages**.

### ◆ Visual Model



The Outlet changes.

Sidebar and Topbar remain constant.



## How Routing Connects to Shell

Inside AppShell, you'll see:

```
<Outlet />
```

That means:

“Insert the current route's page component here.”

Routing works like:

```
flowchart TD
 URL[/templates/] --> Router
 Router --> TemplatesPage
 URL2[/templates/:id/] --> Router
 Router --> TemplateDetailsPage
```

Router decides which page.

Outlet renders it.



## What This Architecture Achieves

This design gives you:

### 1 Separation of concerns

- Layout code lives in one place
- Pages don't manage layout
- Pages focus on feature logic

## 2 Reusability

- Sidebar collapse logic isolated
- Breadcrumb context centralized
- Toast system global

## 3 Clean feature modularity

Look at your structure:

```
app/
 features/
 shared/
```

This is clean architecture applied to frontend.

## How It Connects to Backend

Let's zoom out.

```
flowchart TB
 subgraph Frontend
 Pages
 SharedHTTP
 end

 subgraph Backend
 Controllers
 end

 Pages --> SharedHTTP --> Controllers
```

App layer doesn't know about backend.

Feature layer calls API.

Shared layer handles HTTP.

## Concept Mapping — Big Picture

Now I want you to map mentally:

Layer	Responsibility
main.tsx	Mount React
BrowserRouter	Enable routing
App.tsx	Compose providers + shell
AppShell	Persistent layout
Outlet	Render current route
features/*	Business logic
shared/*	Reusable primitives

This is how a real admin console should look.

## Why This Matters for CTO

When CTO sees:

- Clear layering
- Feature-based folder structure
- Global providers
- Proper routing composition

He sees architectural maturity.

Not just "React code".

## Next Step

Now we move to:

## Stage 2 — Routing & Feature Wiring

We inspect:

- `router.tsx` (or routes config inside app/)
- how routes are declared
- how nested routes are structured

That's where the real mapping between URL → Page happens.

Please paste:

- The router file (likely `src/app/router.tsx` or routes inside App.tsx)

Then we build the routing map diagram fully.

### ▼ Stage 2 — Routing & Feature Wiring

Got it — **this router is the real MVP one**, and it's the **JSX `<Routes>/<Route>` style** (not the config router).

Architecturally it's the *same idea*: `AppShell` is the parent layout and everything renders inside its `<Outlet />`.

#### 1) Route tree

```
flowchart TD
 Shell["/ (AppShell layout)"]
 Index["index → TemplatesListPage"]
 Templates["/templates → TemplatesListPage"]
 Details["/templates/:templateId → TemplateDetailsPage"]
 Schema["/templates/:templateId/versions/:versionId/schema → SchemaEditorPage"]

 Shell --> Index
 Shell --> Templates
 Shell --> Details
 Shell --> Schema
```

#### 2) Route table (URL → component → purpose)

URL	Component	Purpose
<code>/</code> (index route)	<code>TemplatesListPage</code>	default landing (list)

URL	Component	Purpose
<code>/templates</code>	<code>TemplatesListPage</code>	explicit list route
<code>/templates/:templateId</code>	<code>TemplateDetailsPage</code>	view one template + versions tab/table
<code>/templates/:templateId/versions/:versionId/schema</code>	<code>SchemaEditorPage</code>	edit schema for <b>a specific version</b>

#### Params

- `templateId`: the template
- `versionId`: the version within that template

### 3) What's "wired" here

- Layout: `AppShell`
- Templates feature pages: in `features/templates/pages/*`
- Schema feature page: in `features/schema/pages/*`

That's clean separation: **templates is a feature, schema editor is another feature.**

## Two important observations (you should keep in mind)

### A) You currently have *two* list entry points

- `/` shows list
- `/templates` shows list

This is fine, but later you might decide to keep only one canonical path and redirect the other.

### B) The schema editor is version-scoped

This confirms your domain model:

| schema belongs to **template version**, not just template.

So `TemplateDetailsPage` must expose versions and link to schema using both ids.

## Next step (Stage 3 — Templates Feature: List → Details)

To continue *exactly* from Stage 2, we now inspect the Templates pages and answer:

1. How does the list page fetch templates?
2. How do you navigate to details ( `templateId` )?
3. In details, how are versions loaded + how is the schema URL built?
4. Where is API logic located (feature vs shared)?

### Paste these (real MVP) files next:

1. `TemplatesListPage.tsx`
2. `TemplateDetailsPage.tsx`

If you can't paste full files, paste:

- component body
- API call code
- navigation code ( `useNavigate` , links)

And we'll produce the full **user journey trace** and **data-flow map**.

## ▼ Stage 3 — Templates Feature Flow

## 1) User journey (what really happens)

```
flowchart LR
 A["Index TemplatesListPage"]
 B["Create Template API"]
 C["TemplateDetailsPage"]
 D["Create Version API"]
 E["SchemaEditorPage"]
 F["Export XLSX API"]

 A -->|Create| B
 A -->|View| C
 C -->|NewVersion| D
 C -->|EditSchema| E
 C -->|Export| F
```

## 2) TemplatesListPage.tsx — responsibilities & wiring

### What it does well

- **Owns list state:** `templates`, `loading`, `error`.
- **Single load function** `loadTemplates()` memoized with `useCallback`.
- **Global error handling** via `normalizeHttpError + showErrorToast`, with a retry hook.

### API interactions

- `templatesApi.list()` to fetch table data.
- `templatesApi.create({...})` inside the modal form.

### Navigation

- On “View” button: links to `/templates/${t.templateId}`.
- On successful create: navigates to `/templates/${res.templateId}` then refreshes list.

### Architecture notes (important)

- You imported **a lot** of UI components that aren’t used:
  - `Input`, `Badge`, `IconView`, `ActionButton`, etc are used ✓
  - but things like `Modal`, `Spinner`, `TableLoadingOverlay` are used ✓
  - **Unused imports** I see: `IconView` used ✓, `Input` used ✓, `Badge` used ✓
  - But `Modal` is used ✓. `normalizeHttpError` etc used ✓.
  - **However** you import `EmptyState`, `Card` etc and they’re used ✓.
  - There are some extra imports though: `ActionButton` etc all used. (So this file is mostly clean.)

### A small UX/logic bug

You intentionally keep the form state inside the modal component (good), but:

- you defined `submitting` and use it to disable submit ✓
- You don’t show field-level validation errors (fine for MVP).

**But:** you imported `useCallback`, `useEffect`, `useState` and also `getErrorMessage` — ok.

## 3) TemplateDetailsPage.tsx — responsibilities & wiring

### What this page does

- Loads template details by `templateId` from route params.
- Renders a versions table.
- Allows:
  1. **Create New Version** (POST)
  2. **Edit schema** (route link) — but only for latest + draft
  3. **Export** (download Excel)

### API interactions

- `templatesApi.getById(templateId)` → returns `TemplateDetail` (includes `versions`).
- `versionsApi.create(templateId, { createdBy: 'system' })` → creates new version.
- `exportVersion(versionId)` → returns `{ blob, contentDisposition }`, then downloads.

### Breadcrumb integration (App-level wiring)

- It sets breadcrumb on mount/update: `{ templateName: template.name, versionNumber: null }`
- Resets breadcrumb on unmount.

That's exactly "App composition + feature page wiring".

---

## 4) The most important domain rule (encoded in UI)

In `VersionRow` you enforce:

### ✅ Edit schema is allowed only if:

- version is **latest** AND
- version status is **draft**

Everything else is disabled with tooltip.

This implies a real business constraint:

└ Only latest draft version is editable; earlier versions are immutable history.

That's a solid rule.

### But you have a small inconsistency in the disable-reason function

`getEditSchemaDisabledReason()` says:

- If not latest → good message.
  - Otherwise it always returns "Only versions with status DRAFT can be edited." even if status is something else (fine)
  - But it checks `read_only` and still returns the same message (redundant). Not harmful, just sloppy.
- 

## 5) Concrete "what we do next" (Stage 4)

Now we move to the next trace: **Schema Editor Page** wiring.

Upload (or paste) these files next:

- `features/schema/pages/SchemaEditorPage.tsx`

- and whichever API file it uses (likely `features/schema/api.ts` or similar)

Then we will:

- map how schema is loaded by `versionId`
- confirm dirty tracking + validation + save button logic
- locate where `schemaHash` gets updated (backend response vs frontend compute)
- check how you handle optimistic updates / refetch after save

If you upload those 2 files, we'll finish Stage 4 in one pass.

#### ▼ detailed explanation (Includes important concepts)

## 1) The mental model: React is a render engine, not a "step-by-step program"

In React function components:

- The component function **runs again** on every render.
- Render ≠ "draw once". Render is "compute the next UI".
- React then compares ("diffs") the output and updates the DOM.

### Key consequence

**Every variable inside your component body is "recomputed" each render.**

So React provides hooks to "persist" values across renders:

- `useState` persists state
- `useRef` persists mutable values without re-render
- `useMemo` persists computed values
- `useCallback` persists function identity

## 2) `useState` — state = data that triggers re-render

### What it is in

`useState` stores a value *outside* your component function, and gives you:

- current value
- setter that schedules a re-render

```
const [loading, setLoading] = useState(false)
```

### Under the hood (conceptually)

React keeps an internal list of "hook slots" per component instance:

- Slot 0: loading state
- Slot 1: templates state
- Slot 2: error state

... in **the order hooks are called**

That's why you must not call hooks conditionally. If the order changes, React reads the wrong slot.

### Setter behavior (important)

`setLoading(true)` does NOT immediately change `loading` in the same run.

It schedules an update. React later re-runs the component and `loading` becomes true.

### Why it matters in your pages

Your list page typically has:

- `loading`
- `templates`
- `error`
- `createModalOpen`
- `submitting`

That's the correct "UI-state set".

## 3) `useEffect` — run side effects after render

### What it is

A side effect is anything that touches the outside world:

- network calls
- subscriptions
- timers
- manual DOM work

React renders first, **commits DOM changes**, then runs effects.

```
useEffect(() => {
 loadTemplates()
}, [loadTemplates])
```

### Under the hood: timing

Render phase (pure) → Commit phase (DOM) → Effects phase

React wants render to be pure: same inputs → same output.

So effects run after commit.

### Dependency array: the contract

Dependency array means:

└ "Re-run this effect when any dependency's identity/value changes."

So React compares dependencies with `Object.is(prev, next)`.

If you pass `[]`, effect runs once (mount) and cleanup on unmount.

### Cleanup

```
useEffect(() => {
 const id = setInterval(...)
```



```
return () => clearInterval(id)
}, [])
```

Cleanup runs:

- before the effect reruns
- and on unmount

### The #1 trap: stale closures

Effects “capture” variables from the render they were created in.

So if you do:

```
useEffect(() => {
 console.log(templates)
}, [])
```

It prints the initial templates forever, because the effect captured the initial render's `templates`.

That's why dependencies matter.

## 4) `useCallback` — stable function identity (not “performance magic”)

### What it is

`useCallback(fn, deps)` returns a memoized function reference.

```
const loadTemplates = useCallback(async () => {
 ...
}, [showErrorToast])
```

React will return the **same function object** between renders unless deps change.

### Under the hood

It's like:

- store the function + deps in a hook slot
- on next render, compare deps
- if same → return old function reference
- if changed → store and return new one

### Why you used it (and why it's correct)

You likely have:

- `loadTemplates` used in `useEffect`
- `loadTemplates` passed to children (retry button / table actions)

Without `useCallback`, the function is recreated every render, so the `useEffect([loadTemplates])` would fire again and again.

So `useCallback` is often used for **correctness**, not performance.

### Senior rule

Use `useCallback` when:

1. you put the function in a dependency array
2. you pass it to memoized children and want to avoid pointless rerenders

Don't add it everywhere blindly.

#### ▼ detailed explanation

I like how hard you've been pushing on this, Omar. Let's pull everything into one big, clean mental model you can actually *think with* when you write React.

I'll walk through:

- what a hook really is
- how the hook list / slots work
- `useState` / `useEffect` lifecycle
- `useCallback` vs `useMemo`
- how all of this ties into renders, commits, effects, and user interaction

No fluff, but I'll keep the language beginner-friendly.

## 1. What a hook really is

### 1.1. Conceptual definition

A **hook** is a function (`useState`, `useEffect`, `useCallback`, `useMemo`, `useRef`, etc.) that lets a **function component**:

- access React's internal memory (state, refs, memoized values)
- participate in React's lifecycle (effects, subscriptions, cleanup)

When you call a hook inside a component, you're saying:

“React, give this component a slot in your internal memory for this thing.”

So hooks are not magic. They're just a way to **hook your component into React's internal systems**.

### 1.2. The hook list and slots

For each mounted component instance, React keeps an internal structure (a **fiber**) that includes:

```
fiber = {
 type: ComponentFunction,
 hooks: [], // the hook list
 hookIndex: 0, // current position while rendering
 ...
}
```

- `hooks` is an **ordered list of slots**
- each hook call consumes the **next slot**
- slots are **not typed** (no “state slot” vs “effect slot”) they just store whatever that hook needs

Example:

```
function Component() {
 const [count, setCount] = useState(0); // hook call #1 → slot 0
```

```
const [name, setName] = useState("Omar"); // hook call #2 → slot 1
useEffect(() => { ... }, []); // hook call #3 → slot 2
}
```

Internally:

```
fiber.hooks = [
 { state: 0 }, // slot 0
 { state: "Omar" }, // slot 1
 { effect: fn, deps: [] }, // slot 2
]
```

On every render, React:

- sets `hookIndex = 0`
- calls your component
- each hook call uses `hooks[hookIndex]` and increments `hookIndex`

**React identifies hooks by position, not by variable name or type.**

That's why:

- hooks must always be called in the same order
- hooks cannot be inside conditions, loops, or early returns

## 2. `useState` and the render cycle

### 2.1. What `useState` does

```
const [value, setValue] = useState(initial)
```

On the **first render**:

- React sees `useState(initial)` at slot N
- if slot N is empty, it stores:

```
hooks[N] = { state: initial }
```

- returns `[initial, setValueFn]`

On **later renders**:

- React ignores `initial`
- it just returns `[hooks[N].state, setValueFn]`

So `useState` is:

“Give me a persistent value stored in slot N, and a setter that updates that slot and schedules a re-render.”

### 2.2. What happens when you call the setter

```
setValue(newValue)
```

React:

1. updates `hooks[N].state = newValue`

2. marks the component as needing an update
3. schedules a **re-render** (usually after the current event/effect finishes)
4. batches multiple updates together

On the next render:

- React replays the hook calls in the same order
- `useState` at slot N reads the updated `state`
- your JSX reflects the new value

---

### 3. `useEffect` and the three phases

#### 3.1. React's three phases

For a component, React's work is conceptually:

1. **Render phase**
  - call the component
  - run hooks in order
  - build a virtual DOM tree
  - no DOM changes yet
  - no effects run yet
2. **Commit phase**
  - apply changes to the real DOM
  - update the UI
3. **Effects phase**
  - run `useEffect` callbacks
  - run cleanups when needed

#### 3.2. What `useEffect` stores

```
useEffect(() => {
 doSomething()
}, [dep1, dep2])
```

During **render**:

- React sees `useEffect` at slot N
- stores:

```
hooks[N] = {
 effect: () => { doSomething() },
 deps: [dep1, dep2],
 cleanup: previousCleanupOrNull
}
```

- it does **not** run the effect yet

After **commit**, in the **effects phase**:

- React walks effect slots

- compares current `deps` with previous `deps` using `Object.is`
- if `deps` changed (or first time):
  - run previous cleanup (if any)
  - run the effect
  - store the new cleanup (if the effect returns a function)

### 3.3. Why `useEffect` runs “after render”

Because React wants:

- render to be **pure** (no side effects)
- DOM to be updated before side effects run

So:

- `useEffect` is **registered** during render
- **executed** after commit

## 4. Function identity, `useCallback`, and `useMemo`

This is where your questions really focused, and where most people get confused.

### 4.1. Function identity: “new function object”

In JavaScript, every time you define a function inside a component, a **new function object** is created on each render:

```
function Component() {
 const fn = () => console.log("hi");
}
```

Each render:

- `fn` is a different object in memory
- even if the code is identical

So across renders:

```
prevFn === nextFn // false
```

React compares dependencies using `Object.is`.

So if you do:

```
useEffect(() => {
 fn()
}, [fn])
```

Then:

- `fn` is new every render
- `deps` change every render
- effect runs every render

This is where `useCallback` comes in.

### 4.2. `useCallback` : memoized function identity

```
const loadProducts = useCallback(() => {
 ...
}, [dep1, dep2])
```

What happens:

- `useCallback` is a hook, so it uses a slot in the hook list
- in that slot, React stores:

```
hooks[N] = {
 memoizedFn: functionObject,
 deps: [dep1, dep2]
}
```

On the next render:

- React reaches `useCallback` at the same slot N
- compares new deps with old deps
- if deps are the same:
  - returns the **same function object** from `hooks[N].memoizedFn`
- if deps changed:
  - creates a **new function object**
  - stores it in `hooks[N].memoizedFn`
  - returns the new one

So:

`useCallback` memoizes the **function itself**, not its result.

### Empty dependency array

```
const loadProducts = useCallback(() => {
 ...
}, [])
```

- function is created once on the first render
- deps never change
- React always returns the **same function object**
- identity is stable forever

### 4.3. Why `useCallback` matters with `useEffect`

Consider:

```
useEffect(() => {
 loadProducts(); // API call
}, [loadProducts])
```

Without `useCallback`:

- `loadProducts` is a new function every render
- deps change every render

- effect runs every render
- API is called every render
- can cause loops

With `useCallback` (and stable deps):

- `loadProducts` keeps the same identity
- deps don't change
- effect runs only when it should (e.g. once on mount)
- no unnecessary API calls

So the chain is:

```
useCallback → stable function identity
stable identity → deps unchanged
deps unchanged → useEffect does not re-run
useEffect not re-running → no repeated API calls
```

`useCallback` doesn't cache the API result.

It just prevents React from thinking the dependency changed.

#### 4.4. `useMemo` : memoized value

```
const filtered = useMemo(() => {
 return products.filter(p => p.name.includes(search));
}, [products, search]);
```

What happens:

- `useMemo` runs the function **when deps change**
- stores the **result** in the slot:

```
hooks[N] = {
 value: filteredArray,
 deps: [products, search]
}
```

On the next render:

- if deps are the same → React returns the **same value** without recomputing
- if deps changed → recompute and store new value

So:

- `useCallback` memoizes a **function**
- `useMemo` memoizes a **value**

Mechanism is similar (slot + deps), purpose is different.

## 5. Correct vs broken `useCallback` + `useEffect` examples

### 5.1. Correct example

```
const loadProducts = useCallback(async () => {
 setLoading(true);
 setError(null);
 const res = await fetch("/api/products");
 if (!res.ok) throw new Error("Failed");
 const data: Product[] = await res.json();
 setProducts(data);
 setLoading(false);
}, []); // stable identity

useEffect(() => {
 loadProducts();
}, [loadProducts]); // runs once (or when deps change)
```

- `loadProducts` identity is stable
- `useEffect` runs once on mount
- no repeated API calls unless you change deps

## 5.2. Broken example (no `useCallback`)

```
const loadProducts = async () => {
 setLoading(true);
 setError(null);
 const res = await fetch("/api/products");
 if (!res.ok) throw new Error("Failed");
 const data: Product[] = await res.json();
 setProducts(data);
 setLoading(false);
};

useEffect(() => {
 loadProducts();
}, [loadProducts]); // loadProducts is new every render
```

- `loadProducts` is a new function every render
- deps change every render
- effect runs every render
- effect calls `loadProducts`, which sets state, which triggers re-render, which changes `loadProducts`, which re-triggers effect → loop / repeated calls

## 6. Full lifecycle: from load → render → commit → effects → user interaction

Let's tie it all together with a typical component using `useState`, `useEffect`, and `useCallback`.

### 6.1. Initial load

1. Browser loads JS bundle (TS already compiled).
2. ReactDOM calls `root.render(<ProductsPage />)`.
3. React creates a fiber for `ProductsPage` with:



```
hooks: []
hookIndex: 0
```

## 6.2. First render (render phase)

React calls `ProductsPage()` :

- `useState(true)` → slot 0 { state: true }
- `useState([])` → slot 1 { state: [] }
- `useState(null)` → slot 2 { state: null }
- `useState("")` → slot 3 { state: "" }
- `useCallback(...)` → slot 4 { memoizedFn, deps: [] }
- `useEffect(..., [loadProducts])` → slot 5 { effect, deps: [memoizedFn] }
- returns `<p>Loading...</p>`

React builds a virtual DOM tree.

## 6.3. First commit (commit phase)

React updates the real DOM:

- renders `<p>Loading...</p>` into the page

## 6.4. First effects run (effects phase)

React processes `useEffect` slots:

- sees slot 5
- first render → no previous deps
- runs the effect → calls `loadProducts()`

`loadProducts` :

- calls setters ( `setLoading` , `setProducts` , etc.)
- each setter updates its slot and schedules a re-render
- React batches updates and schedules one re-render

## 6.5. Second render (render phase again)

React sets `hookIndex = 0` and calls `ProductsPage()` again:

- `useState(true)` → reads slot 0 (now `false` )
- `useState([])` → reads slot 1 (now products)
- `useState(null)` → reads slot 2
- `useState("")` → reads slot 3
- `useCallback(...)` → slot 4, deps `[]` unchanged → returns same function
- `useEffect(..., [loadProducts])` → slot 5, deps unchanged → effect not scheduled

Now `loading` is `false` , `products` has data, so JSX is the full UI.

React builds a new virtual DOM tree.

## 6.6. Second commit

React updates the DOM from `<p>Loading...</p>` to the full UI.

## 6.7. User interaction

User types in search input:

- `onChange` calls `setSearch("a")`
- `setSearch` updates slot 3 and schedules a re-render

Next render:

- `useState` hooks read updated state
- `useCallback` returns same function
- `useEffect` deps unchanged → no re-run
- `filtered` is recomputed
- DOM updates with filtered list

---

## 7. Final mental model (the one you should keep)

- A **hook** is a function that hooks your component into React's internal memory and lifecycle.
- React keeps **one ordered hook list per component instance**.
- Each hook call consumes the **next slot** in that list.
- Slots are **not typed**; they store whatever that hook needs (state, effect metadata, memoized function, memoized value, ref).
- React matches hook calls to slots **by position**, so **hook order must never change**.
- `useState` stores and reads `state` from a slot; setters update the slot and schedule re-renders.
- `useEffect` stores an effect + deps in a slot; React runs it after commit when deps change.
- `useCallback` memoizes a **function's identity**; it returns the same function object when deps don't change.
- `useMemo` memoizes a **value**; it returns the same computed result when deps don't change.
- `useCallback` + `useEffect` together prevent unnecessary effect re-runs and repeated API calls by stabilizing function identity.

If you want next, we can write a tiny "fake React" in plain JS that implements `useState`, `useEffect`, `useCallback`, and `useMemo` with a simple `hooks` array and `hookIndex`—so you can *see* the slot system in code.

---

## 5) The "hook triangle": `useState` + `useEffect` + `useCallback` together

Your common pattern:

1. `useState` stores UI state
2. `useCallback` defines stable async loaders
3. `useEffect` triggers loaders at mount

Example mental flow:

- Render 1: loading=false, templates=[]
- `useEffect` runs → calls `loadTemplates()`
- `loadTemplates` sets loading=true → triggers Render 2
- `fetch` resolves → `setTemplates(data)`, `setLoading(false)` → Render 3

That's the normal async lifecycle.

## 6) `normalizeHttpError` — why senior teams use this

In real apps, errors come in many shapes:

- fetch errors
- axios errors
- server returns `{message}`
- server returns `{errors: [...]}`
- server returns HTML (proxy errors)
- network timeout
- CORS blocked request

If every page does custom parsing, you get chaos.

So you create a single “normalizer”:

### The goal

Convert “unknown error shapes” into a **consistent** app-level error object:

```
type AppError = {
 title: string
 message: string
 status?: number
 code?: string
 details?: string[]
 retryable?: boolean
}
```

Then UI can reliably do:

- toast message
- show “Retry”
- show details in dev mode
- map status codes

### Under the hood (common implementation idea)

`normalizeHttpError` checks in order:

- is it an Error instance?
- does it have `response` (axios style)?
- does it have `status` + `data` (fetch style wrapper)?
- is it string?
- fallback: “Unknown error”

### Why it matters in your MVP

Your list/details pages should not care whether backend returns:

- 404 “template not found”
- 500 “unexpected”

- 401 "unauthorized"

They just show a friendly toast + maybe offer retry.

That separation is "senior".

---

## 7) Why your code has `loading`, `error`, and `toast` together

There are **two kinds** of error UX:

### A) Page-state error

You show an error screen/empty state with a retry button because the page can't render.

Example: list failed to load → show empty state + retry.

### B) Action error

You show toast (non-blocking) because action failed but page still exists.

Example: create template failed → toast + keep modal open.

Your code uses both, which is correct.

---

## 8) "Under the hood" concept you must master: closures + render snapshots

Each render creates a "snapshot":

- variables
- functions
- state values

Functions you declare in component body close over that snapshot.

So if you define:

```
const onClick = () => console.log(loading)
```

That function prints the `loading` value from the render where it was created.

When state changes, React rerenders and creates a *new* `onClick`.

`useCallback` prevents function identity change *but still updates closure when deps change*.

This is where people get confused.

---

## 9) Other essential React concepts you should know (missing from your examples)

### `useMemo`

Memoize a computed value (not a function):

```
const rows = useMemo(() => buildRows(templates), [templates])
```

Use it when computation is expensive or you need stable reference.

---

## useRef

Persist a mutable value without rerender:

- DOM refs
- “isMounted” flags
- previous value tracking

```
const abortRef = useRef<AbortController | null>(null)
```

### ▼ detailed explanation

You’re looking at the **useRef** section of the page, and you want to understand *that specific part* in context. Let me give you a clean, focused explanation of what that snippet means and how it fits into React’s mental model, using the content from your open tab as reference [notion.so](https://notion.so).

## useRef in this context

The page explains that **useRef lets you persist a mutable value across renders without causing rerenders**. That’s the key distinction:

- **useState** → persists value **and triggers rerender** when changed
- **useRef** → persists value **but does NOT trigger rerender**

React re-runs your component function on every render, so any variable declared normally is recreated. **useRef** gives you a stable “box” that survives those rerenders.

## What the page lists useRef for

The page highlights three common patterns [notion.so](https://notion.so):

- **DOM refs**  
e.g., `inputRef.current.focus()`
- **“isMounted” flags**  
Useful for avoiding state updates after unmount.
- **Previous value tracking**  
e.g., storing the last value of a prop or state.

These are all cases where you want to *remember something* but don’t want React to rerender because of it.

## The snippet you quoted

```
const abortRef = useRef<AbortController | null>(null)
```

This is a perfect example of the “persist a mutable value” pattern.

### What it means

- `abortRef.current` will hold an `AbortController` instance.
- You can assign to it without causing a rerender.
- It survives across renders, so you can:
  - cancel an in-flight request when a new one starts,
  - cancel on unmount,

- avoid race conditions.

### Why this matters

In async-heavy pages (like your `TemplatesListPage` and `TemplateDetailsPage`), you often want to avoid:

- updating state after unmount,
- letting stale requests overwrite fresh data,
- leaking network requests.

An `AbortController` stored in a ref solves that cleanly.

---

### How it fits into your MVP architecture

Your page content emphasizes senior-level patterns like:

- stable async loaders with `useCallback`,
- avoiding stale closures,
- adding cancellation for reliability,
- using refs for “isMounted” or abort logic.

This `abortRef` pattern is exactly the kind of “senior reliability move” the page is pushing you toward [notion.so](#).

---

If you want, I can show you how to integrate `abortRef` into your actual `loadTemplates()` or `getById()` functions so your MVP handles cancellation cleanly.

---

### Keys in lists

React uses keys to preserve identity of list items.

Bad: `key={index}`

Good: `key={templateId}`

### Controlled vs uncontrolled inputs

Controlled: value comes from state.

Uncontrolled: DOM holds value.

For forms, controlled gives validation + predictable UI.

### Context Providers

You already saw providers in your PDF. The missing part:

- context update triggers rerender of consumers
  - don't put rapidly changing data in global context unless needed
- 

## 10) Senior “how it works together” architecture (your MVP shape)

For your app, the clean structure is:

- **shared/http**: fetch wrapper, interceptors, auth headers
- **shared/errors**: `normalizeHttpError`, error types
- **shared/ui**: toast, modal, table

- **features/templates/api**: templatesApi
- **features/templates/pages**: list/details pages
- **features/schema/pages**: schema editor
- **app/router + app shell**: wiring only

Pages should orchestrate:

- call API
- manage state
- render components
- show errors

They should not:

- parse raw HTTP errors
- build URLs in random places (should have helper like `routes.templates.details(id)`)

## 11) The #1 improvement to reach “senior feel”

In async loaders, add cancellation or ignore stale requests:

Problem:

- user navigates away while request is running
- request resolves and calls `setState` → warnings or weird state

Solution options:

- `AbortController` (fetch)
- `let active = true` pattern

Example:

```
useEffect(() => {
 let active = true
 load().then(res => { if(active) setData(res) })
 return () => { active = false }
}, [load])
```

This is a “senior reliability” move.

## What I need from you to tailor this to your exact MVP

Upload (or paste) these two utility files if you have them:

- `normalizeHttpError.ts`
- the HTTP client wrapper used by `templatesApi` (fetch/axios wrapper)

Then I'll explain **your real implementation**, line by line, including:

- error shapes from Spring Boot responses
- where status codes come from
- best pattern for retries, cancelation, and global auth handling

### ▼ Stage 4 — Schema

You're right to call that out. Let's do this properly: **organized, deeply detailed, beginner-friendly, but building a "senior mental model"** — and I'll anchor everything to **your real code** in `SchemaEditorPage.tsx` + `api.ts`.

Below is how these concepts work **under the hood**, and how they **work together** in your MVP.

---

## 0) The big mental model (the one seniors use)

React is not "a program that runs top-to-bottom once".

React is a **UI state machine**:

- You **render** UI from state
- You trigger **state updates**
- React **re-renders**
- Effects run **after** commit for side effects (network, subscriptions)

Think:

```
flowchart LR
 A[Render UI from state] --> B[User/Event]
 B --> C[setState]
 C --> D[Re-render]
 D --> E[Commit DOM]
 E --> F[Run effects]
 F --> A
```

Everything you asked about (`useState/useEffect/useCallback/normalizeHttpError`) exists to make this loop predictable.

---

## 1) `useState` — "state lives outside the function"

What you think it does

```
const [schema, setSchema] = useState<SchemaDefinition>({ sectorCode: '', tables: [] });
```

What it actually means

Your component function runs many times. React must keep values across runs.

So React stores state in an internal per-component structure (conceptually like an array of "hook slots"):

- Slot 0 → `templateInfo`
- Slot 1 → `version`
- Slot 2 → `loading`
- Slot 3 → `loadError`
- Slot 4 → `schema`
- ... etc

Each render, React re-runs your component and reads the same slots in the same order.

That's why hooks must not be conditional:



```
if (x) useState(...) // ❌ breaks hook slot order
```

### How `useState` works under the hood (important)

Calling `setSchema(newSchema)` **does not mutate schema immediately**.

It:

1. enqueues an update (often batched)
2. schedules React to re-render
3. on next render, React uses the new value from the queue

So inside the same call stack, `schema` is still the old one.

That's why React code is built around **"next render will reflect changes"**.

## 2) `useEffect` — "side effects run after render commit"

Your code has multiple effects:

### (A) Load the version data

```
useEffect(() => {
 loadVersion();
}, [loadVersion]);
```

This means:

- render first (show spinner etc)
- after DOM commit, run `loadVersion()` which does network calls

### Under the hood: lifecycle timing

React phases:

1. **Render phase** (pure computation)
2. **Commit phase** (updates DOM)
3. **Effects phase** (runs your `useEffect` callbacks)

Effects run **after** the UI is committed so React keeps rendering pure and predictable.

### The real reason dependencies matter (stale closures)

Effects "capture" values from the render where they were defined.

So dependency array is React's mechanism to say:

| "If any captured value changes, recreate and rerun the effect."

If you lie to dependency arrays, you get stale state bugs.

## 3) `useCallback` — "stable function identity" (not magic performance)

Your code:

```
const loadVersion = useCallback(async () => {
 ...
}, [versionId, showErrorToast]);
```

### What problem it solves in *your exact page*

If you wrote:

```
const loadVersion = async () => {...}
useEffect(() => { loadVersion() }, [loadVersion])
```

Without `useCallback`, `loadVersion` is a **new function object every render**.

So `[loadVersion]` changes every render → effect reruns every render → infinite loop risk.

So here `useCallback` is about **correctness**, not optimization.

### Under the hood

React stores:

- your function
- the dependency array

On next render:

- it compares deps with `Object.is(prevDep, nextDep)`
- if same → returns the **same function reference**
- if changed → stores the new function and returns it

### Senior rule

Use `useCallback` when:

- you put a function in a dependency array **or**
- you pass it to a memoized child component

Otherwise, it's noise.

## 4) `useMemo` — “stable derived values”

You use:

```
const dirty = useMemo(
 () => JSON.stringify(schema.tables) !== JSON.stringify(initialSchemaRef.current.tables),
 [schema.tables]
);
```

### What it does

It caches the *result* of the computation so it's not recomputed unless deps change.

### Under the hood

Exactly like `useCallback`, but caches a **value** instead of a function.

### Senior note (important)

Your `dirty` check is correct for MVP, but it's expensive and has edge cases:

- order changes
- big schemas become slow (stringify  $O(n)$ )
- ignores `schema.sectorCode` changes (you compare only tables)

A more “senior” approach later:

- maintain a `schemaHash` locally
- or compute dirty using a stable hash function
- or track dirty based on user actions (set dirty on edits)

But for MVP: acceptable.

---

## 5) `useRef` — “persist without re-render”

You use:

```
const initialSchemaRef = useRef<SchemaDefinition>({ sectorCode: '', tables: [] });
```

### Why ref exists

Sometimes you need:

- a value that persists across renders
- but changing it should **not** cause a re-render

That's exactly what refs provide.

### Under the hood

Ref is like:

```
{ current: ... }
```

React keeps the object stable forever. You can mutate `.current` without re-render.

You do:

```
initialSchemaRef.current = JSON.parse(JSON.stringify(parsed));
```

So “initial snapshot” is saved and used to compute `dirty`.

---

## 6) `normalizeHttpError` — why it exists, and what it buys you

Your code uses:

```
catch (e) {
 const err = normalizeHttpError(e);
 setLoadError(err);
 showErrorToast(getErrorMessage(err, true), {...});
}
```

### The real problem

Network errors come in **different shapes**:

- `fetch` throws `TypeError` on network failure
- `axios` throws an object with `response`, `request`, etc
- backend might return:
  - `{ message: "..."}`
  - `{ error: "..."}`
  - `{ errors: [...] }`
  - HTML from gateway/proxy
- sometimes no body, only status code

If every page tries to interpret errors, your codebase becomes inconsistent and buggy.

### What normalization does

It converts **unknown error shapes** into **one internal format**:

Example shape (conceptually):

```
type FrontendError = {
 status?: number;
 message: string;
 code?: string;
 details?: string[];
 retryable?: boolean;
}
```

So every UI can rely on:

- `.status` to detect 401/403/409
- `.message` to show toasts
- `.details` to show "more info"
- `.retryable` to show a Retry button

### What's "senior" about this

It creates a stable contract between:

- HTTP layer
- Feature layer
- UI layer

This is the same concept as "DTOs" and "error handling policy" in backend.

## 7) How everything works together in *your SchemaEditorPage* (real flow)

Let's walk your page as a state machine.

### Initial render

State:

- `loading = true`

- `schema = empty`
- `version = null`

UI shows spinner.

## Effect triggers load

`useEffect → loadVersion()`

Inside `loadVersion`:

1. sets loading true + clears error
2. calls `versionsApi.getById(versionId)`
3. parses schema JSON into a strongly-shaped object ( `parseSchemaJson` )
4. sets `schema`, `version`, and snapshot ref
5. sets selected table index
6. ends loading

## That parsing layer is important

You do:

```
const parsed = parseSchemaJson(data.schemaJson);
setSchema(parsed);
initialSchemaRef.current = deepClone(parsed);
```

Meaning you're doing "input sanitation":

- backend might send missing or wrong types
- you normalize it to safe defaults

That prevents UI crashes.

## Save flow

When user clicks Save:

- `validationErrors = validateSchema(schema.tables)`
- `canSave = validationErrors.length === 0`

Then save:

1. merge missing pieces (sectorCode/templateName from templateInfo)
2. build payload in backend expected format ( `enumValues` )
3. call `versionsApi.updateSchema(...)`
4. on success:
  - toast
  - snapshot ref updates (clears dirty)
  - navigate back to template details

That's a clean orchestration pattern.

## 8) The "together" conceptual diagram (Router → Page → API → Error → UI)

```

flowchart TB
 R[Router params: templateId, versionId] --> P[SchemaEditorPage]

 P -->|useEffect| L[loadVersion useCallback]
 L --> API[versionsApi.getById]
 API -->|success| N[parseSchemaJson + normalize]
 N --> S1[setSchema + setVersion]
 N --> REF[initialSchemaRef snapshot]
 S1 --> UI[Render tables/fields]

 UI -->|Save| V[validateSchema]
 V -->|ok| PUT[versionsApi.updateSchema]
 PUT -->|success| OK[toast + update snapshot + navigate back]
 PUT -->|error| EH[normalizeHttpError + showErrorToast]

```

That's your whole page.

## 9) Techniques in your code that are “senior-ish” (good instincts)

### ✓ (1) Separation between parsing and rendering

- `parseSchemaJson`, `normalizeTable`, `normalizeField` keep UI safe.

### ✓ (2) Backend contract mapping

- `buildPutPayload()` maps internal `enum` → backend `enumValues`.

That is exactly what real systems do: **frontend model ≠ backend model**.

### ✓ (3) Retry pattern integrated into toast

You pass `onRetry: loadVersion` which is good architecture: errors are actionable.

### ✓ (4) “beforeunload” dirty protection

That's a real UX feature many MVPs skip.

## 10) Things to improve (so you don't build bad habits)

I'll keep this short and sharp:

### ⚠ Dirty check uses stringify + only tables

If user edits `sectorCode`, your dirty check won't detect it.

Also stringify scales poorly.

**Fix later:**

- include `sectorCode` in dirty compare
- or maintain a `dirty` boolean updated by actions

### ⚠ `loadVersion` uses `showErrorToast` but not in deps?

You did include `[versionId, showErrorToast]` ✓

But you use `loadVersion` inside `onRetry: loadVersion` which is correct.

⚠ You import `schemaApi` in `api.ts`, but page uses `versionsApi.updateSchema`

Not an issue, but keep the API layer consistent:

- either schema lives under versionsApi
- or schemaApi is the official place

Consistency matters when codebase grows.

## 11) Your `api.ts` file (what it tells us)

Your `api.ts`:

```
export const schemaApi = {
 update: (versionId, body) =>
 http.put(`/api/versions/${versionId}/schema`, body).then((r) => r.data),
};
```

This shows the pattern:

- `http.put(...)` returns a response object
- you extract `r.data`

This likely means your `http` is an axios-like wrapper (or a custom wrapper mimicking axios response).

**Key idea:**

Pages should never call `http` directly. They call domain APIs (schemaApi / versionsApi).

That's how you keep feature code clean.

## Now: tell me what you want next (pick one)

1. **Deep dive:** I explain your `SchemaEditorPage` again but as “render cycle timeline” (render 1, effect, render 2, save render, etc).
2. **Senior refactor plan:** how to restructure schema editor into smaller components without breaking logic (TablesPanel, FieldsPanel, Modals).
3. **HTTP layer deep dive:** if you upload `@/shared/lib/http` and `errorTypes` files, I'll explain `normalizeHttpError` for *real* from your implementation (not generic).

If you upload:

- `src/shared/lib/http.ts` (or similar)
- `src/shared/errors/errorTypes.ts`

...I can give you the “under the hood” explanation of your actual HTTP pipeline end-to-end (request → response → normalize → toast).

## ▼ useContext (breadcrumb)

Let's pull everything together into one big, coherent picture—like we're zooming out and seeing the whole map instead of just pieces.

I'll walk through:

- What hooks really are
- How `useContext` and the Breadcrumb example work
- Why `useCallback` matters and how re-renders actually happen
- The “broadcast” idea and child → parent → children flow

- Why we pass the update function in context
- How this relates to Angular services
- Where Redux/Pinia fit in this world
- Why Angular feels more natural to you

I'll keep it beginner-friendly in language, but with the depth and precision a senior would care about.

## 1. What a “hook” really is

In React, a **hook** is just a function that lets a component use React's internal features (state, effects, context, etc.).

Built-in hooks:

- `useState` → gives a component state
- `useEffect` → lets a component run side effects
- `useContext` → lets a component read a context value
- `useCallback`, `useMemo`, `useReducer`, etc.

A **custom hook** is simply:

- a normal JavaScript function
- whose name starts with `use`
- and that calls other hooks inside it

Example:

```
export function useBreadcrumb() {
 return useContext(BreadcrumbContext);
}
```

This is a custom hook. It's not magic. It's just a function that wraps `useContext` so you don't repeat that logic everywhere.

**Key idea:**

Hooks are just functions with rules. Custom hooks are how you package reusable logic.

## 2. The Breadcrumb context: what it does and why it exists

You shared this code:

```
export interface BreadcrumbContextValue {
 templateName: string | null;
 versionNumber: number | null;
 setBreadcrumb: (payload: {
 templateName?: string | null;
 versionNumber?: number | null;
 }) => void;
}

const BreadcrumbContext = createContext<BreadcrumbContextValue | null>(null);

export function useBreadcrumb() {
 const ctx = useContext(BreadcrumbContext);
 return ctx;
}
```



```

}

export function BreadcrumbProvider({ children }: { children: ReactNode }) {
 const [templateName, setTemplateName] = useState<string | null>(null);
 const [versionNumber, setVersionNumber] = useState<number | null>(null);

 const setBreadcrumb = useCallback(
 (payload: { templateName?: string | null; versionNumber?: number | null }) =>
 {
 if ('templateName' in payload) setTemplateName(payload.templateName ?? null);
 if ('versionNumber' in payload) setVersionNumber(payload.versionNumber ?? null);
 },
 []
);

 const value: BreadcrumbContextValue = {
 templateName,
 versionNumber,
 setBreadcrumb,
 };

 return (
 <BreadcrumbContext.Provider value={value}>
 {children}
 </BreadcrumbContext.Provider>
);
}

```

What this does conceptually:

- It creates a **shared place** to store breadcrumb data: `templateName` and `versionNumber`.
- It exposes a **single function** `setBreadcrumb` that pages can call to update that data.
- It makes this data available to any component inside `BreadcrumbProvider` via `useBreadcrumb()`.

Why this is needed:

- The breadcrumb UI usually lives in a **layout** (like `AppShell`), not inside each page.
- But each page needs to **set** the breadcrumb.
- Context lets you store the breadcrumb state in one place (the provider) and let:
  - the layout **read** it to render the UI
  - the pages **update** it via `setBreadcrumb`

So you avoid passing breadcrumb props through every component (prop drilling).

### 3. Where the breadcrumb UI actually lives

The context file has **no UI**—and that's correct.

The UI usually lives in something like:

```

function AppShell() {
 const breadcrumb = useBreadcrumb();

```

```

return (
 <div>
 <Header>
 <BreadcrumbBar
 templateName={breadcrumb?.templateName}
 versionNumber={breadcrumb?.versionNumber}
 />
 </Header>

 <main>
 <Outlet />
 </main>
 </div>
);
}

```

So the architecture is:

- `BreadcrumbProvider` → owns the data and update function
- `AppShell` (or layout) → reads the data and renders the breadcrumb UI
- Pages → call `useBreadcrumb()` and use `setBreadcrumb()` to update the data

Flow:

1. Page calls `setBreadcrumb({ templateName: "Invoice", versionNumber: 3 })`
2. Provider updates its state
3. Provider broadcasts new context value
4. AppShell re-renders and shows updated breadcrumb

#### 4. The `setBreadcrumb` function and `useCallback([])`

This part was confusing, so let's isolate it:

```

const setBreadcrumb = useCallback(
 (payload) => {
 if ('templateName' in payload) setTemplateName(payload.templateName ?? null);
 if ('versionNumber' in payload) setVersionNumber(payload.versionNumber ?? null);
 },
 []
);

```

What this function does:

- It lets you update **one or both** of `templateName` and `versionNumber`.
- It supports partial updates:
  - `setBreadcrumb({ templateName: "Invoice" })`
  - `setBreadcrumb({ versionNumber: 3 })`
  - `setBreadcrumb({ templateName: "Invoice", versionNumber: 3 })`
- It normalizes values to `null` when needed (`?? null`).

Why `useCallback([])` is used:

- Without `useCallback`, this function is **recreated on every render**.
- That means `setBreadcrumb` is a new function each time.
- Since `value` is:

```
const value = { templateName, versionNumber, setBreadcrumb };
```

a new `setBreadcrumb` means a new `value` object every render.

- React sees the context `value` changed → all consumers re-render, even if the actual breadcrumb data didn't change.

With `useCallback([])`:

- React creates the function **once** and reuses it.
- The function identity stays the same.
- Now `value` only changes when `templateName` or `versionNumber` change.
- Consumers re-render only when the breadcrumb data actually changes.

Why empty dependency array is safe here:

- The function doesn't depend on any changing values.
- `setTemplateName` and `setVersionNumber` are stable functions from React.
- So the function logic never needs to be "refreshed".

## 5. How context re-renders actually work

Key rule:

When the Provider's `value` changes, React re-renders all components that use that context.

Important clarifications:

- Only **components that call** `useBreadcrumb()` re-render.
- Not the entire app.
- Re-render does **not** mean:
  - refetching APIs
  - re-running effects with `[]` dependencies
  - resetting state

Effects only re-run when their dependencies change.

Example:

```
useEffect(() => {
 fetchData();
}, []);
```

This runs once, even if the component re-renders many times.

So a breadcrumb or toast update will not refetch your API unless your effect depends on something that changed.

## 6. The "broadcast frequency" and the child → parent → children flow

We used the analogy:

- Context is like a **broadcast frequency**.
- The Provider is the **radio tower**.
- Consumers are **radios** tuned to that frequency.

When the Provider's value changes:

- All consumers that use that context re-render.

But you noticed something subtle:

“In the breadcrumb example, the child initiates the change. Isn't that child → parent?”

Yes—and that's the full picture:

There are two flows:

#### 1. Event flow (upward)

- Child calls `setBreadcrumb(...)`
- This is like calling `props.onChange` in React or a method on a service in Angular.

#### 2. Data flow (downward)

- Provider updates its state
- Provider broadcasts new context value
- All consumers (including layout) re-render with new data

So the real flow is:

```
Child triggers update
 ↓
Provider updates state
 ↓
Provider broadcasts new value
 ↓
Consumers re-render with new data
```

This is the same pattern as:

- Parent owns state
- Parent passes `value` and `onChange` to child
- Child calls `onChange`
- Parent updates state
- Parent passes new `value` down

Context just makes this pattern global instead of via props.

## 7. Why we broadcast three things (templateName, versionNumber, setBreadcrumb)

You asked:

“Why do we broadcast three variables? Can't we just broadcast the two values and let the consumer update them?”

Short answer: **no**, because consumers cannot mutate context directly.

Context is **read-only** from the consumer's perspective.

When you call:

```
const breadcrumb = useBreadcrumb();
```

You get a snapshot of the context value. You cannot do:

```
breadcrumb.templateName = "Invoice"; // does nothing
```

React does not track mutations to that object. It only reacts when the Provider gives a **new value**.

So:

- Only the Provider can change the context value.
- Consumers must call a function that the Provider exposes.
- That's why `setBreadcrumb` is part of the context value.

This is the same pattern as:

- Angular service methods
- Redux `dispatch`
- Zustand actions

**State is owned in one place. Updates happen through controlled functions.**

---

## 8. Context's pitfall: all consumers re-render

You correctly noticed:

“If context value changes, all consumers re-render—even if only one triggered the change.”

Yes. That's how context works.

But:

- Only components that **use that context** re-render.
- Not the whole app.
- For low-frequency, global-ish state (breadcrumbs, theme, auth, layout), this is usually fine.

When it becomes a problem:

- Many components use the same context.
- The context value changes very frequently (e.g., typing, large lists, real-time data).

Then you have options:

- Split context into smaller ones (read vs write).
- Use memoization ( `useMemo` ) for the value.
- Use libraries with selectors (Zustand, Jotai, Redux).
- Keep fast-changing state local instead of in context.

For breadcrumbs, your current pattern is already good.

---

## 9. How this maps to Angular services

You asked if this is similar to Angular services.

Conceptually: **yes, very similar**.

Angular service:

- Singleton object

- Holds shared state and logic
- Injected into components via DI
- Components read data from it
- Components call methods to update it
- Service notifies components (via RxJS or signals)

React Context Provider:

- Component that wraps part of the tree
- Holds shared state via `useState` or `useReducer`
- Exposes state + update functions via context
- Components read data with `useContext`
- Components call update functions to request changes
- Provider re-renders and broadcasts new values

Same core idea:

| A central place that stores shared data and exposes methods to update it.

Difference:

- Angular has built-in DI and services.
- React makes you build this pattern manually with Context + hooks.

This is one big reason Angular feels more structured and React feels more “DIY”.

## 10. Redux and Pinia in this picture

Redux and Pinia are both **state management libraries**, but for different ecosystems.

- **Pinia** → official state management for Vue 3
- **Redux** → popular state management for React (especially large apps)

They both:

- Centralize shared state
- Provide controlled ways to update it
- Let components subscribe to specific slices of state

Redux is more strict and powerful (reducers, actions, DevTools).

Pinia is more ergonomic and reactive (built on Vue's reactivity).

Compared to Context:

- Context re-renders all consumers when value changes.
- Redux/Pinia can re-render only components that use specific slices.
- Redux adds strong predictability and debugging tools.

For your breadcrumb example, Redux/Pinia would be overkill.

Context is enough.

## 11. Why Angular feels more natural to you

You said you love Angular more than React or Vue.

That makes sense because Angular:

- Gives you **services** and **DI** out of the box.
- Enforces a clear architecture.
- Provides a full framework (routing, forms, HTTP, etc.).
- Matches your mental model: “shared logic lives in services, not components.”
- Avoids the need to manually build patterns like Context Providers.

React:

- Gives you primitives (components, hooks).
- Makes you build architecture yourself (Context, custom hooks, state libraries).
- Feels like “too many components and patterns” when you’re used to Angular’s structure.

Vue:

- Sits in the middle: more structured than React, less opinionated than Angular.

Your brain likes **structure, DI, services, and clear ownership**—Angular nails that.

## 12. Big-picture summary

Here’s the whole story in one flow:

1. **Hooks** are just functions that let components use React features. Custom hooks are reusable functions that wrap other hooks.
2. **Context** lets you share data across many components without prop drilling. A Provider owns the state; consumers read it with `useContext`.
3. Your **BreadcrumbProvider** is a concrete example:
  - It stores `templateName` and `versionNumber`.
  - It exposes `setBreadcrumb` so pages can request updates.
  - Layout reads the context and renders the breadcrumb UI.
4. `useCallback([])` keeps `setBreadcrumb` stable so the context value only changes when the actual data changes, avoiding unnecessary re-renders.
5. When the context value changes, **all consumers re-render**, but:
  - Only those that use that context.
  - Re-render does not mean refetching or re-running all logic.
6. The flow is:
  - Child calls `setBreadcrumb` → Provider updates state → Provider broadcasts new value → consumers re-render.
7. We broadcast three things (two values + one function) because:
  - Consumers cannot mutate context directly.
  - Only the Provider can change the context.
  - The function is the controlled way to request updates.
8. This pattern is conceptually the same as **Angular services**:
  - Shared state + methods to update it + downward data flow.
9. **Redux/Pinia** are more advanced state management tools:
  - Centralized store, controlled updates, better performance for large apps.
  - Redux is common in React; Pinia in Vue.

10. You resonate with **Angular** because:

- It gives you services, DI, and architecture out of the box.
- You don't have to reinvent patterns like Context Providers.
- It matches how you naturally think about apps.

---

If you'd like, next we can do one of two things:

- Design a **full breadcrumb system** end-to-end in React, including the UI, so you see everything connected.
- Or map more React patterns directly to Angular concepts (like `useEffect` vs lifecycle hooks, `useReducer` vs `NgRx`, etc.), so React feels less alien and more “translated” into your Angular brain.

## ▼ Frontend Structure

### ▼ why feature-based structure with shared infrastructure?

Excellent question.

Now we move from “how React works” to **why your project is structured this way**, and what the alternatives are.

I'll explain:

1. Why frontend architecture patterns exist at all
2. Why your project uses feature-based + layered separation
3. What other popular frontend architectures exist
4. Trade-offs between them
5. How seniors choose

No fluff. Clear mental models.

---

## 1 Why Do Frontend Architectures Exist?

When a project is tiny:

```
App.tsx
components/
utils.ts
```

No architecture needed.

But when a project grows:

- 20+ pages
- 50+ components
- API integrations
- error handling
- auth
- caching
- multiple developers

Without structure, chaos happens:

- duplicated API logic
- inconsistent error handling



- random folders
- tight coupling
- impossible refactoring

So architecture solves one core problem:

| How do we organize complexity so the system remains maintainable as it grows?

Frontend architecture is not about React syntax.

It is about **separation of concerns**.

---

## 2 Why Your Project Uses This Pattern

Your project roughly looks like:

```
app/
 router.tsx
 layout/

features/
 templates/
 pages/
 components/
 api/
 schema/
 pages/
 components/
 api/

shared/
 ui/
 lib/
 errors/
```

This is called:

| Feature-based modular architecture with shared infrastructure layer.

Let's understand why.

---

### Mental Model

Think of the frontend like a city.

- `app/` → City infrastructure (roads, routing, layout)
- `features/` → Neighborhoods (templates, schema, users...)
- `shared/` → Utilities used by all neighborhoods (electricity, water, roads)

Each feature owns:

- its pages
- its components
- its API
- its logic

This avoids this anti-pattern:

```
components/
pages/
api/
```

Where everything mixes together.

---

### 3 Why This Pattern Is Popular

Because it scales well.

#### Benefits

- Each feature is isolated.
- You can delete a feature without breaking the world.
- Teams can work on separate features.
- Code ownership is clearer.
- Refactoring is safer.

This mirrors backend microservice thinking.

---

### 4 Under the Hood: Why Separation Matters

Let's analyze what separation achieves internally.

Without feature separation:

```
pages/
 TemplatesPage.tsx
 SchemaPage.tsx

api/
 templatesApi.ts
 schemaApi.ts
```

Components from different features start importing each other.

Tight coupling begins.

With feature separation:

```
features/templates/
features/schema/
```

Cross-feature imports become deliberate and visible.

This reduces hidden dependencies.

Senior developers care deeply about dependency direction.

---

### 5 Other Popular Frontend Architectures

Now let's compare.

---

## 1 Folder-by-Type (Old School)

```
components/
pages/
hooks/
utils/
services/
```

Everything grouped by technical type.

### Good for:

- small projects
- demos

### Problems:

- features are scattered across folders
- hard to delete or refactor features
- weak modular boundaries

This was very common in early React apps.

---

## 2 Feature-Based (What You Have)

```
features/
 auth/
 templates/
 schema/
```

Each feature self-contained.

### Good for:

- medium to large apps
- team collaboration
- long-term maintainability

### Trade-off:

- slightly more initial structure overhead
- requires discipline

This is currently the most recommended pattern in modern React ecosystems.

---

## 3 Atomic Design (UI-Driven Architecture)

Structure by UI complexity:

```
atoms/
molecules/
organisms/
templates/
pages/
```

Inspired by design systems.

**Good for:**

- design-heavy apps
- component libraries
- reusable UI systems

**Problem:**

- Business logic separation becomes blurry.
- Not ideal alone for complex apps.

Often combined with feature-based structure.

---

## 4 Layered Architecture (Clean Architecture Frontend)

Inspired by backend Clean Architecture.

Structure:

```
domain/
application/
infrastructure/
ui/
```

Very strict separation.

**Good for:**

- enterprise-scale apps
- heavy business logic
- strict testability requirements

**Trade-off:**

- more boilerplate
  - higher complexity
  - overkill for small apps
- 

## 5 Redux-Style Centralized State Architecture

Not folder-based — but architecture around global state.

Everything flows:

```
UI -> Action -> Reducer -> Store -> UI
```

Strong predictability.

Good for very state-heavy apps.

But modern React often replaces this with:

- React Query
  - Zustand
  - Context + hooks
-

## 6 Micro-Frontend Architecture

Each feature is a separate deployed application.

Used in very large organizations.

Overkill for most projects.

---

## 6 Why Feature-Based + Shared Layer Is a Sweet Spot

It balances:

- simplicity
- scalability
- clarity
- flexibility

It avoids:

- over-engineering (clean architecture)
- under-structuring (folder-by-type chaos)

That's why many modern React boilerplates follow it.

---

## 7 How Seniors Choose Architecture

Senior developers think in terms of:

- Team size
- Project lifetime
- Domain complexity
- Change frequency
- Performance constraints

They ask:

- Will this app grow?
- Will features be removed/added?
- Will multiple teams work on it?
- Is business logic complex?

Architecture is about optimizing for future change.

---

## 8 Deep Insight: Architecture Is About Dependency Direction

In good architecture:

Dependencies flow inward.

Example:

```
Page -> Feature API -> shared http
```

But:

shared/ should NOT depend on features/

This directional rule prevents circular dependencies and chaos.

That's the invisible rule behind your structure.

## 9 Why This Matters for You (Learning Stage)

Because now you're moving from:

"Can I write React code?"

to

"Can I design frontend systems?"

Understanding architecture means:

- You can explain why code is organized this way.
- You can design new features cleanly.
- You avoid messy growth.

This is what separates junior from senior.

## 10 Comprehensive Breakdown (Deeper Layer)

Let's compare in table form:

Pattern	Best For	Risk
Folder-by-type	Small apps	Feature scattering
Feature-based	Scalable apps	Slight structure overhead
Atomic design	Design systems	Weak business boundaries
Clean architecture	Enterprise	Boilerplate
Redux architecture	State-heavy apps	Complexity

## Final Mental Model

Think of frontend architecture like city planning.

Small village → no zoning needed.

Growing city → need districts, infrastructure, utilities.

Your project is already structured like a growing city.

Now I want to ask you:

Do you want next:

- A deep comparison between **Feature-Based vs Clean Architecture**?
- Or an explanation of **how to evolve your current architecture safely as project grows**?
- Or how to decide architecture in interviews?

Pick one and we go deep.

▼ what is `.gitkeep`

## 1 Why `.gitkeep` Exists

Git does **not track empty folders**.

Git only tracks files.

So if you create:

```
features/
 schema/
 components/
```

And `components/` is empty, Git will ignore it.

If you want to keep the folder structure visible in the repo, you must put a file inside.

That file is usually:

```
.gitkeep
```

It is not special to Git.

It is just an empty placeholder file.

It means:

| "Keep this folder in the repository even if it has no files yet."

## 2 Why Is `components/` Folder Empty?

Now the architectural reason.

Your project follows a **feature-based structure**.

Example:

```
features/
 templates/
 pages/
 components/
 api/
 schema/
 pages/
 components/
 api/
```

This means:

Each feature is expected to contain:

- pages (route-level components)
- components (reusable pieces inside feature)
- api (feature-specific API logic)
- maybe hooks
- maybe types

Even if `schema/components` is empty now,

the structure says:

| "When schema feature grows, its internal UI pieces go here."

This is a **scaffolding pattern**.

---

### 3 Why This Is Actually a Senior Move

Because it prevents:

#### ✗ Chaos Later

Without pre-defined structure, devs start putting things anywhere:

```
components/
components2/
schemaParts/
schemaUtils/
```

Instead, the structure already communicates intent.

It's architectural guardrails.

---

### 4 Why Only `.gitkeep` and Not Real Components?

Because currently:

- Schema feature might have only one big page component ( `SchemaEditorPage` )
- No extracted subcomponents yet

So `components/` is reserved for future decomposition like:

```
schema/
 components/
 TablesPanel.tsx
 FieldsPanel.tsx
 FieldRow.tsx
 TableEditorModal.tsx
```

Right now it's empty because the feature hasn't been split yet.

---

### 5 Conceptual Understanding: Folder as Contract

Think of the folder structure as:

| A contract describing intended architecture.

It says:

```
Each feature can contain:
- pages
- components
- api
- hooks
- types
```

Even if some are empty.

This is proactive architecture.

---



## 6 When Would You Remove `.gitkeep` ?

If:

- You decide you don't want that layer
- Or project changes structure

Otherwise, keep it.

---

## 7 Should You Keep Pre-created Folders?

Yes, if:

- The project is intended to scale
- Multiple devs work on it
- You want structural consistency

No, if:

- It's a tiny throwaway demo
- Solo experiment

Your MVP is structured like a scalable admin console.

So this is intentional.

---

## 8 Does `.gitkeep` Affect React?

No.

It has:

- No runtime effect
- No build effect
- No bundle effect

It exists only for Git tracking.

---

## 9 Bigger Architectural Insight

Your project is using:

| Feature-first modular architecture

Instead of:

```
components/
pages/
api/
```

It uses:

```
features/
 schema/
 templates/
 users/
```

This is scalable and senior-level organization.

The `.gitkeep` files are just structural placeholders inside that discipline.

---

## 10 Final Mental Model

`.gitkeep` means:

| “We expect this feature to grow into modular internal components.”

It is a sign of:

- planned scalability
  - structural discipline
  - not random clutter
- 

If you want next level discussion:

We can analyze:

1. Whether your feature-based structure is optimal
2. How to split SchemaEditorPage into components properly
3. When to move logic into hooks folder
4. Whether shared components folder should exist

Tell me which direction you want to go.

### ▼ Add wizard workflow

## ▼ Implement wizard workflow Steps

### ▼ Workflow conversation

Alright Omar — let’s treat this like a product + engineering decision, not just UI.

You’re basically choosing **where the “create template” journey lives** and **how “multi-step workflow” is represented** (modal vs page, state in memory vs URL, resumable vs not).

Below are 3 solid approaches (simple → middle → advanced), with tradeoffs, and my recommendation for what to implement **now**.

---

## Mental model (why the modal solution breaks as soon as steps exist)

A modal is great for **tiny** creations:

- 1–2 fields
- no long forms
- no sub-pages, no complex validation
- no “save draft + resume later”
- no need to deep-link into step 2

But your CTO UI shows a **wizard**: sector selection → template creation → export. That’s a **journey**, not a popup.

Once it’s a journey, you want:

- Back/Next navigation
- URL that represents the step (so refresh doesn’t kill you)

- Ability to bookmark/share/reopen
- More screen space

That's why most real systems move wizards to full pages.

---

## Approach A — Simple (Keep list page + button, but creation opens a new page wizard)

### What it is

- Keep your current **Template List** page and **Create Template** button.
- Clicking "Create Template" navigates to a new route like:
  - `/templates/new` (step 1)
  - `/templates/new?step=2` (step 2), etc.
- The wizard lives in its own page (not modal).
- Sidebar remains: **Templates** only.

### Pros

- Minimal UI change, small cognitive shift.
- Keeps discoverability: "I'm on templates, I click create".
- Easy to implement incrementally.

### Cons / risks

- Still slightly mixing "browse" and "create" in the same nav concept.
- If CTO wants "separate sidebar tab for creation", this doesn't match visually (but functionally it does).

### When it's best

- If your users are "admins" and prefer "manage templates" + "create new" from same place.
- 

## Approach B — Middle / Recommended (Separate nav item for "Create Template" + wizard page with steps in URL)

### What it is

- Sidebar has two items:
  1. **Templates** → list page
  2. **Create Template** → wizard page (stepper workflow)
- Wizard is a full page with steps like CTO.
- State is **URL-driven** (step in URL), and draft is persisted (at least minimal persistence).

### Pros

- Matches CTO UI expectations and mental model.
- Scales cleanly as steps grow (more forms, validation, export previews).
- Better UX: refresh/back/forward works; easier to resume.
- Cleaner separation of responsibilities:
  - "Browse/manage" vs "Create new".

## Cons / tradeoffs

- Slightly more navigation complexity (two sidebar items).
- You must decide what happens if the user abandons the wizard:
  - discard draft?
  - autosave draft?
  - confirm dialog?

## Why I recommend it now

Because the moment you adopt a stepper workflow, you *want* the wizard to be a first-class route. The separate sidebar item is a clean information architecture decision and aligns with the “workflow product” feel your CTO UI is going for.

---

## Approach C — Advanced (Single “Template Studio” area + drafts/resume + state machine)

### What it is

- Instead of “Create Template” being just a route, you build a “Studio” concept:
  - `/template-studio/:draftId`
  - Steps are part of the studio.
- Create always creates a **draft** immediately (server-side), returns draftId.
- You can resume any draft from the Templates list (“Continue setup”).
- Internally, creation flow is a state machine:
  - `DRAFT → CONFIGURING → READY_TO_EXPORT → EXPORTED → PUBLISHED` etc.

### Pros

- Best long-term scalable model.
- Perfect for collaboration later (locking, multi-user).
- Great for auditability and long-running workflows.
- Supports “resume where you left off” properly.

### Cons

- More backend changes now (draft lifecycle, state persistence, transitions).
- More product decisions needed (draft retention, permissions, conflicts).

### When to do it

- When you want “real product” workflow behavior:
    - save/resume
    - multi-user
    - approvals
    - version branching
    - audit trail
-

## My senior recommendation (what to implement now)

✅ **Implement Approach B (Middle)** now.

It hits the CTO workflow shape and gives you the right foundation without overbuilding.

**Key rule:** make the wizard a route (not a modal), and represent the step in the URL so the flow is stable.

---

### Critical tradeoffs to decide *before* we lock SDD

These affect the SDD and how Cursor should implement it:

#### 1. Draft creation timing

- Option 1: create draft immediately when entering wizard (best for resume, needs backend support)
- Option 2: create draft only after step 1 submit (simpler)

#### 1. Abandon behavior

- If user leaves wizard: do we keep draft? prompt confirm? autosave?

#### 1. Where “Templates List” shows drafts

- Do drafts appear in list immediately?
  - Do we show “Continue” vs “View”?
- 

### Scalable future path from Approach B

You can evolve B → C smoothly:

- Start with `/templates/new?step=1` and in-memory state
- Add server draft creation after step 1
- Later switch route to `/template-studio/:draftId?step=...`
- Add resume, collaboration, approvals

So B is not a dead-end. It's the right stepping stone.

---

### What I need from you (no extra questions, just pick one)

Choose one of these to proceed with SDD:

- **B1 (recommended):** Separate sidebar “Create Template”, wizard page, draft created after step 1 submit.
- **B2 (more robust):** Separate sidebar, wizard page, draft created immediately on entering wizard (server draft).

Once you pick B1 or B2, I'll write the SDD v1 (no code yet) and we'll iterate it until stable.

## ▼ SSD V1 - 3

### ▼ V3

Excellent.

Now we do this properly.

This will be **SDD v3** — the balanced version:

- Structured like v2 (execution-safe)
- Includes design rationale (human clarity)
- Explicit contracts (LLM-safe)
- Clear dependency governance

- Future-proof hooks
- No fluff

HEP mode active. Let's do it.

---

## SDD v3 — Template Creation Wizard (B1)

---

### 1) Design Rationale (Why This Change Exists)

The current MVP initiates template creation via a modal from the Templates list page.

This approach worked for a simple form but does not scale to:

- Multi-step workflows
- Route-driven navigation
- Refresh-safe flows
- Deep-linking to steps
- Future draft resume
- Future state transitions

The CTO UI clearly models creation as a **workflow**, not a popup.

Therefore:

- Creation becomes a **first-class route**
- Wizard becomes a **dedicated page**
- Draft is created after Step 1 submit (simple, controlled lifecycle)
- No new dependencies introduced at this stage

This aligns with scalable product architecture without over-engineering.

---

### 2) Objective

Implement a full-page 3-step wizard accessible from sidebar navigation, replacing modal-based creation.

---

### 3) Scope

#### In Scope

- Add sidebar entry: **Create Template**
- Remove modal-based creation
- Implement route-based wizard (3 steps)
- Create draft after Step 1 submit
- Ensure refresh-safe Step 2/3

#### Out of Scope

- Draft resume from list
- Autosave
- Workflow state machine
- Multi-user collaboration

- Version branching redesign
  - Backend architectural changes
- 

## 4) Information Architecture

### Sidebar Structure

- Templates → `/templates`
- Create Template → `/templates/create`

Templates list remains management-focused.

Creation is workflow-focused.

Templates list “Create Template” button will navigate to `/templates/create`.

---

## 5) Routes (Authoritative Contract)

```
/templates
/templates/create
/templates/create/:templateId
/templates/create/:templateId/export
```

### Route Rules

- Step 1: no templateId required.
- Step 2/3: templateId required.
- Missing templateId → redirect to Step 1.
- Invalid templateId → show error + link to `/templates`.

Routes are the source of truth for wizard state.

---

## 6) Wizard Steps

### Step 1 — Metadata

Fields:

- Template Name (required)
- Sector Code (required)

Behavior:

- Client validation
- Call backend create draft
- On success:
  - Receive templateId
  - Navigate to `/templates/create/:templateId`
- On failure:
  - Show error
  - Stay on Step 1

Draft status: `DRAFT`

No draft exists before submit.

---

### Step 2 — Template Creation

- Fetch template by id
- Load existing schema editor
- "Next" → `/templates/create/:templateId/export`

Refresh must reload from backend.

---

### Step 3 — Export

- Fetch template by id
  - Execute existing export logic
  - "Finish" → `/templates`
- 

## 7) Backend Contract Assumption

Step 1 requires endpoint:

Input:

- name
- sectorCode

Output:

- templateId
- versionId (if required)

No new backend workflow model required.

---

## 8) State Model

Before Step 1 submit:

- Local React state only

After Step 1 submit:

- templateId becomes authoritative key
- Step 2/3 load data from backend

No in-memory wizard state persistence required.

---

## 9) Validation & Error Handling

### Step 1

- Required fields
- Trim whitespace

### Server Errors

- 400 → field error
- 409 → conflict message
- 5xx → toast + remain on step



## Route Protection

- Missing id → redirect
  - Not found → error screen
- 

## 10) Architectural Constraints

- No business logic inside UI components
- API calls must use existing API layer
- No global state introduction
- No state machine
- No wizard libraries
- No new form library
- No new routing solution

Follow existing layering patterns in project.

---

## 11) Dependency Policy

### Default

Use existing project dependencies and patterns.

### New Dependencies

Allowed only if:

- Explicitly listed in this SDD
- Justified with reason
- Approved prior to installation

For B1:

No new npm dependencies authorized.

If additional tooling seems required:

Escalate as question — do not install silently.

---

## 12) Impacted Areas

- Sidebar configuration
  - Templates list page
  - Remove modal component
  - Routing configuration
  - New Wizard container
  - API integration for draft creation
- 

## 13) Definition of Done

- Sidebar shows both entries
- Modal is removed or unreachable
- Step 1 creates draft and navigates correctly

- Step 2 reloads correctly on refresh
  - Step 3 functions
  - Errors handled
  - No new npm dependencies
  - Routing stable
- 

## 14) Future Scalability Hooks

This structure allows:

- Add draft resume
- Introduce state machine
- Introduce autosave
- Convert to Template Studio
- Add approval workflows

Without structural rewrite.

---

## Senior Verdict

SDD v3 is:

- Clear
- Controlled
- Scalable
- Non-overengineered
- Agent-safe
- Human-readable
- Production-aligned

This is a strong SDD.

---

If you approve SDD v3:

Next step:

I generate the **Cursor Implementation Prompt Pack**:

1. Implementation Prompt
2. Agent Self-Review Prompt
3. Human Review Checklist
4. Incremental Execution Strategy (to avoid refactor loops)

We stay disciplined.

Your move.

### ▼ **Cursor Implementation Prompt Pack**

#### ▼ **slice 1 — adding routes & initial templates**

### **Prompt 1 — Slice 1 (Routing Skeleton)**

**Paste into Cursor:**

"Implement Slice 1: Routing Skeleton.

Rules:

- Do not add npm dependencies.
- Do not refactor unrelated code.
- Touch only the files listed below.
- Implement only this slice, then stop.

Goal:

Add routes per SDD v3:

- /templates
- /templates/create
- /templates/create/:templateId
- /templates/create/:templateId/export

For now, each route may render a minimal placeholder page/component (e.g. 'Create Template – Step 1').

Do not implement wizard logic yet.

Allowed files:

- Routing configuration file(s) used by this project
- New page components under the existing pages structure (only those placeholders)

DoD:

- App compiles
- Navigating to each route renders placeholder text
- No other behavior changes"

## ▼ slice 1 — changes

### Changes

#### 1. New placeholder pages (frontend/src/features/templates/pages/)

- **CreateTemplateStartPage.tsx** – Renders "Create Template – Start" for /templates/create
- **CreateTemplateStepPage.tsx** – Renders "Create Template – Step 1 (templateId)" for /templates/create/:templateId
- **CreateTemplateExportPage.tsx** – Renders "Create Template – Export (templateId)" for /templates/create/:templateId/export

#### 2. Router updates (frontend/src/app/router.tsx)

- Imports for the three new pages
- New routes placed before templates/:templateId so the literal create path matches correctly:
- /templates/create → CreateTemplateStartPage
- /templates/create/:templateId → CreateTemplateStepPage
- /templates/create/:templateId/export → CreateTemplateExportPage

/templates still routes to TemplatesListPage as before.

### Verification

- Frontend build completes successfully.
- Navigating to each route shows the corresponding placeholder text.

## ▼ Slice 2 — Sidebar Navigation (only)

### Goal

Add a sidebar item **"Create Template"** that navigates to `/templates/create`. Keep **"Templates"** pointing to `/templates`. No other UI/logic changes.

### Do NOT

- Modify routes (Slice 1 already done)
- Touch Templates list page
- Remove modal
- Add API calls / validation
- Add dependencies
- Refactor sidebar layout

### Allowed files (only):

- `frontend/src/app/layout/AppShell` / `Sidebar` component (where nav items live)

### DoD

- Sidebar shows "Create Template"
- Clicking it navigates to `/templates/create` and renders the placeholder page
- Existing "Templates" nav still works

## ▼ Slice 2 — changes

Summary of changes:

### Changes in frontend/src/app/layout/Sidebar.tsx

#### 1. "Create Template" nav item

- Added a NavLink to `/templates/create` with a FilePlus icon and label "Create Template".
- Set `isActive` when the path starts with `/templates/create`.

#### 1. "Templates" active state

- `isTemplatesActive` now excludes `/templates/create`, so "Create Template" is active on the create flow and "Templates" is active elsewhere.

### Result

- Sidebar shows both "Templates" and "Create Template".
- "Create Template" links to `/templates/create` and renders the placeholder page.
- "Templates" still links to `/templates` and behaves as before.

## ▼ Slice 3 — template button navigation to Create Template

### Slice 3 Goal (only):

- In Templates list page, change **Create Template** button to navigate to `/templates/create`
- Ensure modal no longer opens from that entry point

### Do NOT:

- Remove modal code globally (only detach from list)

- Touch backend
- Add validation
- Add new deps
- Change router/sidebar

### ▼ Slice 3 — changes

## Changes

#### 1. "Create Template" buttons now navigate

- Page header button and empty-state button both call `goToCreate()`, which navigates to `/templates/create`.

#### 1. Modal no longer used

- Removed `createOpen` state.
- Removed `openCreate`, `closeCreate`, and `handleCreateSuccess`.
- Removed the `Modal` and `CreateTemplateForm` JSX.
- Removed the `CreateTemplateForm` component and its unused imports (`Input`, `Modal`, `CreateTemplateResponse`).

Result:

- Both "Create Template" buttons navigate to `/templates/create`.
- The create modal is no longer used from this page.
- TypeScript build passes.

### ▼ Slice 4 — Step 1 UI + Create Draft API (only)

#### Goal

Implement `/templates/create` (Step 1):

- Fields: **Template Name** (required), **Sector Code** (required)
- Submit validates required fields
- On submit: call backend "create draft template"
- On success: navigate to `/templates/create/:templateId`
- On failure: show error, stay on page

#### Do NOT

- Implement Step 2 schema editor
- Implement export
- Add autosave/resume
- Add new npm dependencies
- Refactor API layer
- Touch sidebar/router/list page

#### Allowed files (only)

- `frontend/src/features/templates/pages/CreateTemplateStartPage.tsx`
- Existing templates API client module (or create a minimal one *only* if project doesn't have it)
- Any existing shared UI components already used in the app (no new libs)

#### DoD

- Form renders
- Validation works
- API call works
- Success navigates using returned `templateId`
- Error handled

#### ▼ Slice 4 — changes

Summary of updates to CreateTemplateStartPage.tsx:

### Changes

#### Step 1 UI

- **Fields:** Template Name (required), Sector Code (required)
- **Layout:** PageHeader with title/description and Card form using Input and Button

#### Validation

- On submit: trims values and checks both fields are non-empty
- If either is empty: shows error toast and blocks submit

#### API Integration

- On valid submit: calls `templatesApi.create()` with `name`, `sectorCode`, and `createdBy: 'system'`
- Uses existing `templates` API client and `CreateTemplateRequest` / `CreateTemplateResponse`

#### Success / Error

- **Success:** shows success toast and navigates to `/templates/create/:templateId`
- **Error:** normalizes with `normalizeHttpError`, shows error toast via `showErrorToast` (with `status/details`), and keeps the user on the page

#### Shared Components Used

- `PageHeader`, `Card`, `CardContent`, `Input`, `Button`, `Spinner`, `useToast`
- `normalizeHttpError`, `getErrorMessage` for error handling

#### ▼ Slice 5 — Step 2 Guard + Load + Mount Schema Editor

SLICE 5 – Step 2 Guard + Load + Mount Schema Editor

Context:

We are implementing the second step of the Create Template wizard.

Route:

`/templates/create/:templateId`

This step must:

- Read `templateId` from the URL
- Validate it exists
- Fetch template data from backend
- If valid → render existing `SchemaEditorPage`
- If invalid → handle appropriately

Do NOT:

- Modify router
- Modify sidebar

- Modify Templates list page
- Implement export logic
- Add new dependencies
- Refactor SchemaEditorPage
- Add stepper/timeline UI
- Touch any unrelated files

Allowed files:

- frontend/src/features/templates/pages/CreateTemplateStepPage.tsx
- Existing templates API client (only if needed to fetch by id)

Requirements:

- 1) Extract templateId from useParams().
- 2) If templateId is missing → navigate to /templates/create.
- 3) Fetch template using existing API client (e.g., templatesApi.getById or equivalent).
- 4) Maintain local state:
  - loading (boolean)
  - template (nullable)
  - error (nullable)
- 5) While loading → render Spinner (use existing shared Spinner).
- 6) If error → render error message using existing error handling pattern.
- 7) If success → render existing SchemaEditorPage.
  - Pass required props (templateId and/or versionId as required by SchemaEditorPage).
  - Do NOT duplicate editor logic.
- 8) Ensure refresh works correctly.
- 9) Ensure no new npm dependencies are introduced.

Definition of Done:

- Visiting /templates/create/:validId loads and displays schema editor.
- Refreshing the page keeps it functional.
- Visiting /templates/create/ (without id) redirects to step 1.
- Visiting /templates/create/:invalidId shows error.
- TypeScript builds successfully.
- No unrelated code changes.

## ▼ Slice 5 — changes

Summary of changes to CreateTemplateStepPage.tsx:

## Implementation Summary

### 1. Extract templateId from URL

- Uses useParams<{ templateId: string }>() to read templateId.

### 2. Redirect when templateId is missing

- If !templateId, navigates to /templates/create with replace: true.

### 3. Fetch template via API

- Calls templatesApi.getById(templateId) in a loadTemplate callback.

### 4. Local state

- loading (boolean)
- template (TemplateDetail | null)
- error (FrontendError | null)

## 5. Loading state

- While loading, renders a centered Spinner.

## 6. Error handling

- On fetch error: uses `normalizeHttpError` and `showErrorToast`; shows a message, Retry, and “Back to Step 1”.
- On template with no versions: shows an error and “Back to Step 1”.
- Retry calls `loadTemplate` again.

## 7. Success → schema editor

- When template loads and has versions: picks the latest by `versionNumber` and navigates to `/templates/:templateId/versions/:versionId/schema` with `replace: true`.
- `SchemaEditorPage` renders via that route and uses `templateId` and `versionId` from the URL as it already does.
- Shows a spinner during the redirect to avoid a blank screen.

## 8. Refresh behavior

- On `/templates/create/:validId`: fetches template, then redirects to the schema editor.
- On the schema editor URL: `SchemaEditorPage` runs its own load logic on refresh.

### ▼ Slice 5 pause — critical bug, workflow violation

You’re right to pause. Slice 5 as implemented **violates the intent of the new workflow**: it “teleports” you into the old Schema Editor route, so your “Step 2 page” is basically a redirect shim. That’s not wrong technically, but it’s a product/UX decision you haven’t made yet — and it creates confusion about **two entry points** (Templates nav vs Create Template nav).

Below are several approaches (simple → advanced), ranked, with tradeoffs, and a recommendation for what to implement now. No code.

## First, the core design question

You now have two different user journeys:

### 1. Create new template (wizard flow)

- Step 1: create draft
- Step 2: define schema
- Step 3: export

### 1. Maintain existing template (management flow)

- Templates list → details → create new version → edit schema for that version

The key is: **don’t mix these flows accidentally**.

Either unify them cleanly, or keep them deliberately separate.



## Option 1 — Simple (Recommended Now): “Wizard Step 2 = Schema Editor, but keep wizard route as the wrapper”

### Idea

- Step 2 should **render the same Schema Editor UI**, but **inside the wizard route** (`/templates/create/:templateId`) instead of navigating away.
- Don't redirect to `/templates/:templateId/versions/:versionId/schema`.
- Instead: Step 2 page resolves “which version to edit” and mounts the existing editor component/page internally.

### How it behaves

- Create flow stays under Create Template nav:
  - Step 1 → Step 2 (schema editor shown) → Step 3
- Edit flow stays under Templates nav:
  - List → Details → create new version → edit schema in the existing schema route

### Pros

- Minimal change from current codebase (reuse editor).
- Wizard flow feels consistent (URL stays in `/templates/create/...`).
- Doesn't break old edit paths.
- Avoids “where am I?” confusion (Create tab shouldn't throw you into Templates detail routes).

### Cons

- You must decide what “Step 2 version” means:
  - Usually: **latest version** (or specifically v1 in creation).
- Schema editor might be tightly coupled to the old route params; you may need a thin adapter.

### Senior verdict

This is the simplest approach that preserves both flows without weird redirects.

---

## Option 2 — Middle: “Single canonical schema editor route; wizard steps are just a guided overlay”

### Idea

- Accept that `/templates/:templateId/versions/:versionId/schema` is the **canonical** place to edit schema.
- Wizard Step 2 route becomes “guided routing”:
  - It immediately redirects to the canonical schema editor route.
- Wizard stepper UI (Slice 7) is shown across those pages via layout or top bar, so the user still feels in a wizard even though URLs change.

### Pros

- No duplication: only one route for schema editing.

- Everything points to one editor location (clean canonical path).
- Less “adapter” work for SchemaEditor.

### Cons (big UX one)

- Create flow breaks the mental model: you click “Create Template” and suddenly you’re in “Templates” world.
- Harder to keep the stepper accurate unless you build a shared wizard layout that spans routes.
- Users may lose the wizard context.

### Senior verdict

Technically clean, UX messy unless you invest in consistent wizard chrome across routes.

---

## Option 3 — Middle+: “Wizard uses canonical editor, but under a wizard namespace”

### Idea

- Introduce a dedicated wizard editor route that is *still canonical for wizard*, e.g.:
  - `/templates/create/:templateId/schema`
- That route renders the schema editor but keeps the user in wizard space.
- Templates maintenance continues using the old route.

### Pros

- Clear separation: wizard editing vs maintenance editing.
- URLs communicate intent.

### Cons

- Two routes for schema editing = duplication risk.
- You must ensure both routes behave identically (or one wraps the other).

### Senior verdict

A decent compromise if SchemaEditor is too coupled to old routing but you still want wizard URLs.

---

## Option 4 — Advanced (Future growth): “Template Studio with lifecycle + drafts + version graph”

### Idea

- Merge creation + editing into one “studio” concept:
  - `/template-studio/:templateId`
- Studio has states: Draft, Ready, Published, etc.
- Creating and editing are just different states and actions inside the same app area.
- Versions become a structured concept (branching, compare, approvals, etc).

### Pros

- Best long-term scalability.

- No confusion between create vs manage.
- Enables “resume draft”, approvals, audit, collaboration.

### Cons

- Heavy product + backend work.
- Not needed now for MVP continuation.

### Senior verdict

Do later if this becomes a real product.

---

## What the agent implemented (and why it’s problematic)

Current Slice 5 code:

- Loads template
- Finds latest version
- Redirects to `/templates/{id}/versions/{versionId}/schema`

This is basically Option 2, but without the supporting UX structure (stepper overlay). That’s why it feels wrong.

Also: it picks “latest version” automatically, which is okay for creation (v1), but could become wrong later.

---

## My recommendation (implement now)

✅ **Option 1: Wizard Step 2 renders schema editor in-place, without changing route away from**

`/templates/create/:templateId`.

Reason:

- Keeps create flow coherent.
  - Preserves existing maintenance flow unchanged.
  - Minimal engineering risk.
  - No new routing complexity.
  - Avoids “two paths confusion” by making the intent explicit:
    - Create nav = wizard
    - Templates nav = management
- 

## How editing should work (answering your specific questions)

**“After creation, should we check if version 1?”**

You don’t need a “version 1 check”. You need a rule:

- In wizard Step 2: edit **the version created during Step 1** (likely v1).
- For maintenance edits: user creates a new version from details page and edits that.

So the wizard is always “first version creation flow”. No need for branching logic beyond selecting the initial version.

### “If we want to change template later, do we go under steps?”

Recommended: **No**.

Edits are not a wizard. They’re maintenance:

- Templates list → details → new version → schema editor.

If you later want “guided update wizard”, that becomes a new feature, not mixed into the create wizard.

## Next step (your approval needed, no code yet)

Pick one:

1. **Option 1 (recommended)**: keep wizard route and render schema editor inside it (no redirect).
2. Option 2: accept redirect to canonical schema editor route (and later add stepper overlay).
3. Option 3: add `/templates/create/:templateId/schema` as wizard schema route.

Reply with: `Option 1` / `Option 2` / `Option 3`.

If you choose Option 1, I’ll specify the exact Slice 5 acceptance criteria and how to adjust the current code (still no code until you say).

### ▼ Slice 5 (Option 1A) — Step 2 In-Place Schema Editor via Reusable View Component

SLICE 5 (Option 1A) – Step 2 In-Place Schema Editor via Reusable View Component

Goal:

Render the schema editor inside the wizard Step 2 route:

`/templates/create/:templateId`

without redirecting to:

`/templates/:templateId/versions/:versionId/schema`

Approach:

Extract a reusable schema editor UI component (“SchemaEditorView”) that accepts props

(`templateId`, `versionId`). Then:

- SchemaEditorPage becomes a thin route wrapper that reads params and renders SchemaEditorView
- CreateTemplateStepPage (wizard step 2) fetches template, selects `versionId`, renders SchemaEditorView

Do NOT:

- Modify router
- Modify sidebar
- Modify Templates list or details pages
- Add new npm dependencies
- Refactor unrelated files
- Implement Step 3/export logic
- Add stepper/timeline UI
- Change backend contracts

Allowed files:

- frontend/src/features/templates/pages/CreateTemplateStepPage.tsx
- frontend/src/features/schema/pages/SchemaEditorPage.tsx
- Create a new reusable component under schema feature, e.g.:  
frontend/src/features/schema/components/SchemaEditorView.tsx
- (Optional) any existing schema-related internal components that SchemaEditorPage currently uses  
ONLY if needed to move UI into SchemaEditorView

Implementation requirements:

1) Create SchemaEditorView

- Inputs: templateId (string), versionId (string)
- Contains the actual schema editor UI previously rendered by SchemaEditorPage
- Must not depend on route params directly (no useParams inside the view)

2) Update SchemaEditorPage

- Read templateId/versionId from route params
- Guard: if missing, show a simple error state (or redirect if that is current pattern)
- Render: `<SchemaEditorView templateId=... versionId=... />`

3) Update CreateTemplateStepPage (wizard step 2)

- Read templateId from `/templates/create/:templateId`
- If missing: navigate to `/templates/create` (replace)
- Fetch template by id using existing `templatesApi.getById` (or equivalent)
- Determine versionId to edit:
  - Use latest version (highest versionNumber) from `template.versions`
  - If no versions exist: show error ("Template has no versions") + back to step 1
- Render `<SchemaEditorView templateId=... versionId=... />` directly (no navigate redirect)
- Provide loading spinner + error UI + retry, following existing patterns

Definition of Done:

- Visiting `/templates/create/:templateId` shows schema editor UI (in-place) without redirect
- Visiting `/templates/:templateId/versions/:versionId/schema` still works (via SchemaEditorPage wrapper)
- Refresh works on both routes
- Error states handled (not found / no versions / network)
- TypeScript build passes
- No new npm dependencies
- Changes limited to allowed files only

## ▼ Slice 5 (1A) — change

Summary of the changes:

### 1. SchemaEditorView component

**File:** frontend/src/features/schema/components/SchemaEditorView.tsx

- Props: templateId, versionId, backPath, backLabel?, saveSuccessPath?
- No route params; all data comes from props
- Holds the schema editor UI (tables, fields, modals, validation)

- Fetches version and template internally
- Uses backPath for back/cancel and saveSuccessPath ?? backPath after successful save

## 2. SchemaEditorPage updates

**File:** frontend/src/features/schema/pages/SchemaEditorPage.tsx

- Reads templateId and versionId from route params
- Shows an error message if either is missing
- Renders <SchemaEditorView templateId={...} versionId={...} backPath={/templates/:id} backLabel="Back to template details" />

## 3. CreateTemplateStepPage updates

**File:** frontend/src/features/templates/pages/CreateTemplateStepPage.tsx

- Reads templateId from /templates/create/:templateId
- If templateId is missing: navigate('/templates/create', { replace: true })
- Loads template via templatesApi.getById(templateId)
- Uses latest version (highest versionNumber) from template.versions
- Error handling:
- Load error: retry + "Back to step 1"
- No versions: "Template has no versions" + "Back to step 1"
- Renders <SchemaEditorView> with:
- backPath="/templates/create"
- backLabel="Back to step 1"
- saveSuccessPath="/templates/create/:templateId/export"

## Verification

- /templates/create/:templateId shows the schema editor in place (no redirect)
- /templates/:templateId/versions/:versionId/schema still works through SchemaEditorPage
- TypeScript build completes successfully

### ▼ Slice 6 — Step 3 Export Route + Finish

SLICE 6 – Step 3 Export Route + Finish

Route:

/templates/create/:templateId/export

Goal:

Implement the export step page for the wizard flow. Keep it minimal, functional, and consistent with existing patterns.

Do NOT:

- Add new npm dependencies
- Refactor SchemaEditorView or Step 2
- Modify router/sidebar/templates list
- Implement new export engine if export already exists (reuse existing export capability)

- Add stepper/timeline UI (later slice)
- Change backend contracts

Allowed files:

- frontend/src/features/templates/pages/CreateTemplateExportPage.tsx
- Existing templates/export API client utilities (only if needed and already in project patterns)
- Existing shared UI components (Button, Card, PageHeader, Spinner, Toast, et c.)

Requirements:

- 1) Extract templateId from route params.
- 2) If templateId is missing → navigate to /templates/create (replace).
- 3) Load template (or required export metadata) using existing templatesApi.getById(templateId) or equivalent.
- 4) Render UI:
  - PageHeader: "Export Template" + "Step 3: Export artifacts."
  - Show template name/sector (if available) to confirm context.
  - Provide primary action: "Export" (or "Generate Export") that calls existing export API if it exists.
  - If export is not implemented yet, render a clear placeholder area with disabled button and message "Export not implemented yet" (do not invent new backend endpoints).
- 5) Loading state: show Spinner while fetching.
- 6) Error state: show error message + Retry + Back to Step 2 (backPath should be /templates/create/:templateId).
- 7) Finish behavior:
  - After successful export (or if export is placeholder), provide "Finish" button that navigates to /templates (Templates list).
  - Keep the user in the wizard context (do not redirect to template details unless explicitly required).

Definition of Done:

- /templates/create/:templateId/export renders without crashing.
- Missing templateId redirects to step 1.
- Fetch template works and shows basic context.
- Error + retry works.
- Finish navigates to /templates.
- No new dependencies.
- TypeScript build passes.

## ▼ Slice 6 — change — need another review

Summary of updates to CreateTemplateExportPage.tsx:

### CreateTemplateExportPage

#### Route and params

- Reads templateId from /templates/create/:templateId/export
- If templateId is missing → navigate('/templates/create', { replace: true })

#### Loading

- Uses templatesApi.getById(templateId)

- Shows spinner during load

## Error

- Error state: message + Retry + "Back to Step 2" (/templates/create/:templateId)

## UI

- PageHeader: title "Export Template", description "Step 3: Export artifacts"
- Template name and sector badge
- "Back to Step 2" link
- Export button: calls existing exportVersion(versionId) → downloadBlobWithDisposition for the latest version
- Finish button: navigate('/templates')
- Export disabled when there are no versions (with message)
- Exporting state: spinner + "Exporting..." label

## Export flow

- Uses existing exportVersion from @/features/export/api
- Uses existing downloadBlobWithDisposition from @/shared/lib/download
- Reuses pattern from TemplateDetailsPage

## No versions

- If template.versions is empty, shows a message and disables Export (Finish remains enabled)

## ▼ Slice 7 — Wizard Layout + Top Stepper (Radix/Tailwind, no new deps)

SLICE 7 – Wizard Layout + Top Stepper (no new dependencies)

Goal:

Add a shared wizard layout and a top stepper UI for the 3-step create flow:

- 1) Details
- 2) Schema
- 3) Export

The stepper must appear at the top of:

- /templates/create
- /templates/create/:templateId
- /templates/create/:templateId/export

Do NOT:

- Change any business logic or API calls
- Change routes
- Refactor SchemaEditorView internals
- Modify Templates list/details pages
- Add new npm dependencies

Allowed files:

- Create a new component:  
frontend/src/features/templates/components/TemplateWizardLayout.tsx
- Update these pages to use it:  
frontend/src/features/templates/pages/CreateTemplateStartPage.tsx



```

frontend/src/features/templates/pages/CreateTemplateStepPage.tsx
frontend/src/features/templates/pages/CreateTemplateExportPage.tsx
- Reuse existing shared UI primitives already in the project (Card, Separator, Button, etc.) and lucide icons.

Stepper requirements:
- Determine current step from location.pathname:
 - path === "/templates/create" -> step 1 active
 - path startsWith "/templates/create/" and not endsWith "/export" -> step 2 active
 - path endsWith "/export" -> step 3 active
- Render a horizontal stepper with:
 - step indicator (number for active/upcoming, check icon for completed)
 - label text ("Details", "Schema", "Export")
 - connecting line between steps
 - smooth transitions using Tailwind (e.g., transition-colors, duration-200/300)
- Click behavior:
 - Step 1 link always enabled -> /templates/create
 - Step 2 and Step 3 links enabled only if templateId exists (i.e., on step 2/3 pages)
 - On step 1, do not fabricate a templateId; step 2/3 should be non-clickable/disabled.

Layout requirements:
- Apply consistent centered width for wizard pages (e.g., max-w-4xl mx-auto) and spacing.
- Layout should render:
 1) PageHeader (title + description can be passed in)
 2) Stepper
 3) Children content

Definition of Done:
- All 3 wizard pages show the same header + top stepper.
- Correct active/completed styling on each route.
- No changes in create/schema/export behavior.
- TypeScript build passes.
- No new dependencies.

```

If the agent finishes, share the summary/diff and I'll approve or tighten.

## ▼ Slice 7 — change — need to be reviewed

Summary of changes:

### 1. TemplateWizardLayout.tsx

**File:** frontend/src/features/templates/components/TemplateWizardLayout.tsx

- **Props:** title, description, rightActions?, children
- **Layout:** max-w-4xl mx-auto with PageHeader, horizontal stepper, and content area
- **Stepper logic from pathname:**
  - pathname === '/templates/create' → step 1 active
  - pathname.startsWith('/templates/create/') and not pathname.endsWith('/export') → step 2 active

- `pathname.endsWith('/export')` → step 3 active
- **Visuals:**
  - Completed: check icon, primary styling
  - Active: step number in circle, primary styling
  - Upcoming: step number in circle, muted styling
  - Connecting lines between steps; filled when previous step is completed
  - Tailwind transitions (`duration-200`, `duration-300`)
- **Navigation:**
  - Step 1: always links to `/templates/create`
  - Steps 2 and 3: only clickable when `templateId` exists in the path
  - On step 1, steps 2 and 3 are disabled (no `templateId`)

## 2. Page updates

- **CreateTemplateStartPage:** Wraps form in `TemplateWizardLayout` with title and description
- **CreateTemplateStepPage:** Wraps all states (loading, error, no versions, success) in `TemplateWizardLayout` with title and description
- **CreateTemplateExportPage:** Wraps all states in `TemplateWizardLayout` with title, description, and `rightActions` (Finish, Export)

## Verification

- TypeScript build passes
- No new dependencies
- Business logic and API usage unchanged

### ▼ Slice 8 — Stepper Visual Refinement (Alignment + Centering + Visual Hierarchy)

SLICE 8 – Stepper Visual Refinement (Alignment + Centering + Visual Hierarchy)

Context:

Slice 7 introduced `TemplateWizardLayout` + top stepper for the 3-step create flow.

Functionally it works, but the stepper UI is not polished.

Goal:

Refine **ONLY** the stepper presentation to look professional:

- circles aligned with connector lines
- controlled spacing (no large gaps between circle and line)
- stepper centered on the page
- stepper has its own visual container (border/background) to feel like a component
- smooth transitions (color/opacity) for active/completed states

Do NOT:

- Change any business logic
- Change routing paths
- Change API calls
- Change page flow

- Add new npm dependencies
- Refactor unrelated components

Allowed files:

- frontend/src/features/templates/components/TemplateWizardLayout.tsx  
(and/or the stepper component file if it exists separately)

Implementation requirements:

- 1) Rebuild the stepper layout so circles and connector lines are in the same flex row:
  - Top row: circles + connectors aligned via "flex items-center"
  - Connectors as flex items between circles: "flex-1 h-px bg-border" with small horizontal spacing (mx-3 or mx-4)
  - Circles have fixed size (e.g., h-9 w-9) and centered content (grid place-items-center)
- 2) Labels should not affect circle/line alignment:
  - Put labels in a separate row below the circles using a 3-column grid
  - Each label centered under its circle
- 3) Centering:
  - Wrap the stepper in a container with max width and centered horizontally:
    - e.g., max-w-3xl mx-auto
- 4) Visual container:
  - Stepper should sit inside a subtle container:
    - rounded-lg border bg-muted/30 (or equivalent existing tokens)
    - padding (px-6 py-4)
  - Must visually separate stepper from the rest of the page
- 5) States + transitions:
  - Active step: filled or stronger emphasis (e.g., bg-foreground text-background)
  - Completed steps: check icon (lucide) + completed connector color
  - Upcoming steps: muted border/text
  - Add smooth transitions:
    - transition-colors duration-200/300 on circle and connector

Definition of Done:

- Circles and lines align perfectly across viewport sizes
- No excessive gaps between circle and connector
- Stepper is centered and visually contained
- Active/completed/upcoming styles are clear and consistent
- No behavior changes
- TypeScript build passes
- No new dependencies

## ▼ Slice 8 — change — need to be reviewed

Summary of the stepper visual updates:

### Stepper refinements

## 1. Layout

- **Top row:** Circles and connectors in a flex row using flex items-start
- **Connectors:** Between circles with mx-4, h-px, flex-1, min-w-8, aligned to circle center via mt-[17px] (center of h-9)
- **Circles:** Fixed h-9 w-9 with place-items-center

## 2. Labels

- Placed below each circle in the same flex column
- Centered under each circle with flex-col items-center and mt-2
- Do not affect circle/connector alignment

## 3. Centering

- Stepper wrapped in max-w-3xl mx-auto

## 4. Container

- Wrapper: rounded-lg border border-border bg-muted/30 px-6 py-4
- Visually separates the stepper from the rest of the page

## 5. States and transitions

- **Active:** border-foreground bg-foreground text-background
- **Completed:** Check icon with border-primary bg-primary text-primary-foreground
- **Upcoming:** border-muted-foreground/30 bg-muted/30 text-muted-foreground
- Transitions: transition-colors duration-300 on circles, duration-200 on labels, duration-300 on connectors

## 6. Semantics

- ol / li kept for accessibility; list-none used for layout

### ▼ Slice 9 (revert from slice 6, 7 & discarded)

SLICE 7B – Wizard Layout + Dice UI Stepper (Top) [Option A: shared/ui]

Context:

We are currently at Slice 6 (functional wizard pages exist) and stepper work was reverted.

We want to reintroduce the wizard header + stepper UI using Dice UI's Radix+Tailwind Stepper to avoid styling/alignment issues.

(dice step url) [<https://www.diceui.com/docs/components/radix/stepper> ]

Do NOT:

- Change any business logic or API calls
- Change routing paths
- Refactor SchemaEditorView internals
- Modify Templates list/details pages
- Add new npm dependencies
- Change create/schema/export behavior

Allowed files:

- 1) Add Dice UI Stepper component locally:
  - frontend/src/shared/ui/stepper.tsx

Copy/paste the Stepper implementation from Dice UI docs and adapt imports to project conventions (e.g., cn utility).  
Ensure it compiles without adding dependencies.

2) Add wizard layout component:

- frontend/src/features/templates/components/TemplateWizardLayout.tsx

This layout renders:

- PageHeader (title + description props)
- Top Stepper (Dice UI)
- Children content

Apply consistent centered width (e.g., max-w-4xl mx-auto) and spacing.

3) Wrap the 3 wizard pages with TemplateWizardLayout:

- frontend/src/features/templates/pages/CreateTemplateStartPage.tsx
- frontend/src/features/templates/pages/CreateTemplateStepPage.tsx
- frontend/src/features/templates/pages/CreateTemplateExportPage.tsx

Stepper requirements:

- 3 steps: "Details", "Schema", "Export"
- Determine active step from location.pathname:
  - "/templates/create" -> step 1 active
  - "/templates/create/:templateId" -> step 2 active
  - "/templates/create/:templateId/export" -> step 3 active
- Click behavior:
  - Step 1 always navigates to "/templates/create"
  - Step 2 and Step 3 are enabled only when templateId exists  
(On Step 1, do not fabricate an id; those steps should appear disabled / not clickable)
  - On Step 2/3, Step 1 and previous steps can be clickable for back navigation
- No redirects; purely UI navigation links.

Definition of Done:

- Step 1/2/3 pages show consistent wizard header + Dice UI top stepper
- Step alignment and connectors look professional (Dice UI defaults)
- Active/completed states reflect current route
- No behavior changes in create/schema/export flows
- TypeScript build passes
- No new npm dependencies added

## ▼ Reverted to Slice 9 — Change -1 (need to be reviewed)

Summary of changes:

### Summary

#### 1. Dice UI Stepper (no new npm deps)

- **frontend/src/shared/lib/compose-refs.ts** – Ref composition utility used by the stepper
- **frontend/src/shared/hooks/use-isomorphic-layout-effect.ts** – Layout effect hook
- **frontend/src/shared/hooks/use-lazy-ref.ts** – Lazy ref initializer
- **frontend/src/shared/hooks/use-as-ref.ts** – Ref-as-latest-props hook
- **frontend/src/shared/ui/stepper.tsx** – Stepper ported from Dice UI docs with:

- Slot from @radix-ui/react-slot
- dir defaulting to "ltr" instead of @radix-ui/react-direction
- Imports aligned with the project (@/shared/lib/utils, @/shared/lib/compose-refs, etc.)

## 2. TemplateWizardLayout

- **frontend/src/features/templates/components/TemplateWizardLayout.tsx** – Layout with:
  - PageHeader (title, description, optional rightActions)
  - Stepper with 3 steps: Details, Schema, Export
  - Active step from pathname:
    - /templates/create → step 1
    - /templates/create/:templateId → step 2
    - /templates/create/:templateId/export → step 3
  - Step 1 always links to /templates/create
  - Steps 2 and 3 enabled only when templateId exists (Links when enabled, disabled buttons otherwise)
  - max-w-4xl mx-auto layout

## 3. Wizard pages wrapped

- **CreateTemplateStartPage** – Wrapped with TemplateWizardLayout
- **CreateTemplateStepPage** – Wrapped for loading, error, no-versions, and success states
- **CreateTemplateExportPage** – Wrapped, with rightActions (Finish, Export) passed into the layout

### Definition of done

- Step 1/2/3 pages share the same wizard header and Dice UI stepper
- Stepper uses Dice UI default styling
- Active and completed states reflect the current route
- Create/schema/export flows unchanged
- TypeScript build passes
- No new npm dependencies

## ▼ Reverted to Slice 9 — Change -2

a normal “Dice UI default layout” issue: **Trigger is** `inline-flex items-center gap-3`, so the title sits beside the circle, and the separator starts right after the trigger → it feels “stuck” to the label/circle group.

You have two clean options. I recommend Option A.

## Concrete adjustments (manual, no refactor)

1. In `StepperTrigger` base classes:
  - reduce gap to `gap-1` (or `gap-2`)
1. In `StepperSeparator` base classes:
  - add `mx-4` (horizontal only) so the line doesn't touch the trigger block

## ▼ Slice 10 — Wizard UX Cohesion (Layout + Breadcrumbs + Bottom Actions) [Option 1]

SLICE 8 – Wizard UX Cohesion (Layout + Breadcrumbs + Bottom Actions) [Option 1]

Context:

We have a working 3-step create wizard under routes:

- /templates/create (Step 1: create metadata)
- /templates/create/:templateId (Step 2: schema editor via SchemaEditorView)
- /templates/create/:templateId/export (Step 3: export)

Decision (Option 1):

Wizard is for creation only.

Step 1 creates the template and is NOT meant to be revisited for editing the same template.

We will NOT implement metadata editing or step-state persistence in this slice.

Goal:

Make the wizard pages look consistent and professional:

- 1) Hide breadcrumbs on wizard routes (since stepper is the navigation).
- 2) Standardize layout so Stepper is at top and the Title/Description appear under it (consistent for steps 1/2/3).
- 3) Introduce a bottom action bar pattern for step pages:
  - Back as a button (not a text link)
  - Primary actions grouped consistently (Create/Save/Export/Finish)
- 4) Reduce scattered labels/buttons (especially in Step 2).

Do NOT:

- Change routing paths
- Change business logic or API behavior
- Implement metadata update (no PATCH)
- Add new npm dependencies
- Refactor SchemaEditorView internals beyond props/layout

Allowed files:

- AppShell / breadcrumb rendering file (where breadcrumbs are implemented)
- frontend/src/features/templates/components/TemplateWizardLayout.tsx
- frontend/src/features/templates/pages/CreateTemplateStartPage.tsx
- frontend/src/features/templates/pages/CreateTemplateStepPage.tsx
- frontend/src/features/templates/pages/CreateTemplateExportPage.tsx
- (Optional) small shared UI helper for an action bar layout (but keep scope tight)

Implementation requirements:

A) Breadcrumb suppression

- If pathname starts with "/templates/create", do NOT render breadcrumbs in the top bar.
- Breadcrumbs remain unchanged for all other routes.

B) Wizard layout standardization

- Ensure TemplateWizardLayout renders in this order:
  - 1) Stepper
  - 2) Title + description (PageHeader)
  - 3) Main content (children)
  - 4) Optional bottom actions slot (see C)
- Stepper stays at the top and remains clickable only per current rules (Step

2/3 disabled without `templateId`).

- Use consistent max width/centering (`max-w-4xl mx-auto`) and spacing.

#### C) Bottom action bar (wizard pattern)

- Add a consistent bottom actions area for wizard pages:
  - Left side: Back button (when applicable)
  - Right side: primary/secondary actions
- Step 1:
  - No “Back” button (it’s the start)
  - Primary: Create
- Step 2:
  - Back button: “Back to Step 1” (as BUTTON)  
Behavior: navigates to `/templates/create`, but (per Option 1) we will add a confirm dialog in a later slice; for now just navigate.
  - Right actions: Cancel + Save grouped together, aligned right, and not floating mid-page.
- Step 3:
  - Back button: “Back to Step 2” (as BUTTON) -> `/templates/create/:templateId`
  - Right actions: Export + Finish grouped, aligned right.
- The bottom action bar should appear as the last section of the page (under the card/content), not near the header.

#### D) Step 2 “scattered UI” cleanup (layout only)

- Remove or relocate floating “Version 1” text so it doesn’t appear as a random label.
  - If it must remain, display it as a subtle line under the step title (or in a small muted area), not between random controls.
- Ensure primary controls are either:
  - inside `SchemaEditorView` (editor controls), OR
  - in the bottom action bar (Save/Cancel/Back)
- No functional change to save/cancel behavior, only placement.

#### Definition of Done:

- Wizard pages no longer show breadcrumbs.
- Stepper is at top for steps 1/2/3.
- Title/description consistently appear under the stepper.
- Step 2/3 “Back” is a button and actions are grouped at bottom.
- No behavior changes (create/save/export still work).
- TypeScript build passes.

### ▼ Slice 10 — Change (need to be reviewed)

**Slice 8 — Wizard UX Cohesion** is done. Summary of what’s in place:

#### A) Breadcrumb suppression

- **Topbar.tsx** – Breadcrumbs are hidden when `pathname.startsWith('/templates/create')`.

#### B) Wizard layout

- **TemplateWizardLayout** – Layout order: Stepper → PageHeader (title/description) → content → bottom actions.
- Uses `max-w-4xl mx-auto` and consistent spacing.



## C) Bottom action bar

- **Step 1** – No Back button; primary action “Create” in the bottom bar.
- **Step 2** – Bottom bar with “Back to Step 1” (Button, outline) on the left and Cancel + Save on the right.
- **Step 3** – Bottom bar with “Back to Step 2” (Button, secondary) on the left and Export + Finish on the right.

## D) Step 2 layout changes

- **SchemaEditorView** variant="wizard":
- No ActionLink or PageHeader at top.
- No “Version X” in the header.
- Version shown in layout description: “Step 2: Define tables and fields. Version X”.
- Back, Cancel, and Save are in the bottom bar instead of the header.

## Files touched

- Topbar.tsx – breadcrumb suppression
- TemplateWizardLayout.tsx – layout order and bottom actions slot
- SchemaEditorView.tsx – variant="wizard" and bottom bar
- CreateTemplateStartPage.tsx – bottom bar with Create
- CreateTemplateStepPage.tsx – variant="wizard", version in description
- CreateTemplateExportPage.tsx – bottom bar with Back, Export, Finish (order: Export then Finish)

TypeScript build completed successfully.

### ▼ Slice 11 — Wizard UI Fixes (Step 1 CTA placement, remove separators, Step 3 summary layout)

SLICE 9 – Wizard UI Fixes (Step 1 CTA placement, remove separators, Step 3 summary layout)

Scope:

UI-only improvements across steps 1–3. No routing or API logic changes.

Do NOT:

- Add new dependencies
- Change create/save/export behavior
- Implement persistence or metadata editing
- Modify SchemaEditorView internals beyond layout/props usage
- Change stepper component behavior except allowing navigation (already allowed)

A) Step 1 (Details page)

Goal: Remove extra separator and place the Create CTA inside the card.

Requirements:

- 1) Remove the horizontal separator line currently shown between the card and the bottom actions area (or any extra divider used on Step 1).
- 2) Remove the bottom action bar for Step 1 entirely (Step 1 should not have a global bottom actions row).

- 3) Place the "Create" button inside the Card, aligned bottom-right of the form (within CardContent), with proper spacing.
- 4) Keep Stepper + Title/Description layout unchanged.

Definition of Done:

- No separator line shown under the Step 1 card.
- Create button is visually part of the card/form, not floating in a bottom bar.

B) Step 2 (Schema page)

Goal: Remove extra separator and keep "Back to Step 1" permitted.

Requirements:

- 1) Remove the horizontal separator line currently shown above the bottom action bar (the divider between content and action bar).
- 2) Keep navigation to Step 1 permitted:
  - Clicking the Step 1 stepper item navigates to /templates/create.
  - Clicking "Back to Step 1" button navigates to /templates/create.
  - (No disabling, no confirmation in this slice.)
- 3) Keep the bottom action bar for Step 2 (Back / Cancel / Save) but without the divider line.
- 4) Optional visual improvement:
  - "Version 1" should remain near the header (muted small text) but not float at mid-page.

Definition of Done:

- No separator line above the action bar.
- Back navigation to Step 1 works via button and stepper click.

C) Step 3 (Export page)

Goal: Improve review card to include details from step 1 and schema step, and improve layout ("Template" label + name beside each other).

Requirements:

- 1) Redesign the export review card content (still using existing Card component):
  - Use a clean key/value summary layout (2-column grid or definition list).
  - "Template" label and template name MUST appear on the same line (e.g., "Template: test").
  - "Sector Code" shown in same style as "Template".
- 2) Include Step 2 details (in a minimal, accurate way):
  - Show "Version: 1" (or latest version number) as part of the summary.
  - Show counts from schema if easily available from already-loaded data:
    - Tables count (0, 1, 2...)
    - Fields count total (optional if already available)

IMPORTANT: Do not add new API calls. Only use data already loaded (template detail / version detail if already present). If counts are not available without additional fetching, show ONLY version number and skip counts.
- 3) Keep action buttons in bottom bar (Back to Step 2 on left, Export + Finish on right). No separator divider.

Definition of Done:

- Card shows "Template: <name>" and "Sector Code: <code>" in a tidy summary.
- Version info is present.

- Layout looks intentional (not stacked random lines).
- No separator above bottom bar.

Verification:

- TypeScript build passes
- Manual check: Step 1 CTA is inside card, Step 2/3 have no divider, Step 3 card summary looks clean.

## ▼ Slice 11 — Change (need to be reviewed)

**Slice 8 — Wizard UX Cohesion** is done. Summary of what's in place:

### A) Breadcrumb suppression

- **Topbar.tsx** – Breadcrumbs are hidden when `pathname.startsWith('/templates/create')`.

### B) Wizard layout

- **TemplateWizardLayout** – Layout order: Stepper → PageHeader (title/description) → content → bottom actions.
- Uses `max-w-4xl mx-auto` and consistent spacing.

### C) Bottom action bar

- **Step 1** – No Back button; primary action “Create” in the bottom bar.
- **Step 2** – Bottom bar with “Back to Step 1” (Button, outline) on the left and Cancel + Save on the right.
- **Step 3** – Bottom bar with “Back to Step 2” (Button, secondary) on the left and Export + Finish on the right.

### D) Step 2 layout changes

- **SchemaEditorView** `variant="wizard"`:
- No ActionLink or PageHeader at top.
- No “Version X” in the header.
- Version shown in layout description: “Step 2: Define tables and fields. Version X”.
- Back, Cancel, and Save are in the bottom bar instead of the header.

### Files touched

- Topbar.tsx – breadcrumb suppression
- TemplateWizardLayout.tsx – layout order and bottom actions slot
- SchemaEditorView.tsx – `variant="wizard"` and bottom bar
- CreateTemplateStartPage.tsx – bottom bar with Create
- CreateTemplateStepPage.tsx – `variant="wizard"`, version in description
- CreateTemplateExportPage.tsx – bottom bar with Back, Export, Finish (order: Export then Finish)

TypeScript build completed successfully.

## ▼ Slice 11.1 — Add “Start Over” confirmation when leaving Step 2/3 to Step 1

UPDATE to SLICE 9 – Add “Start Over” confirmation when leaving Step 2/3 to Step 1

**Rule:**  
 After a template is created (i.e., on Step 2 or Step 3 where `templateId` exists),  
 the user must NOT silently navigate back to Step 1.

**Behavior:**  
 When the user attempts to go from Step 2 or Step 3 back to Step 1 (Details page),  
 show a confirmation alert/modal:

**Title:** Start over?  
**Message** (exact intent, wording can be improved):  
 “You can’t edit Template Name or Sector Code after creation.  
 Going back will start a new template and your current template will remain as -is.”

**Buttons:**

- Primary (destructive or emphasized): “Start over” (YES)
- Secondary: “Cancel” (NO)

**Actions:**

- If YES: navigate to `/templates/create`
- If NO: stay on current page (no navigation)

**Triggers** (must cover both):

- 1) Clicking Step 1 in the stepper while on Step 2 or Step 3
- 2) Clicking the “Back to Step 1” button in the bottom action bar (Step 2)  
 (Step 3 back button goes to Step 2, so no confirmation needed there.)

**Implementation constraints:**

- Use existing UI primitives only (Radix dialog if already present, or a simple confirm dialog pattern).
- Do not add new npm dependencies.
- Keep behavior only for Step 2/3 → Step 1 navigation. Step 1 → Step 2 remains normal.

**Definition of Done:**

- Attempting to go back to Step 1 from Step 2 shows confirmation.
- Attempting to click Step 1 circle from Step 2 or Step 3 shows confirmation.
- Cancel keeps user on same page.
- Start over navigates to Step 1.

## ▼ Slice 11.1 — Change

Implementation is finished. Summary:

### Summary of Changes

#### 1. TemplateWizardLayout (TemplateWizardLayout.tsx)

- Added state for `showStartOverConfirm` and handlers for the Start Over flow.
- **Stepper Step 1:** For Step 2 or Step 3 (when `templateId` exists), the Details link’s `onClick` calls `e.preventDefault()` and opens the confirmation modal.

- **Modal:** Title "Start over?" and message: "You can't edit Template Name or Sector Code after creation. Going back will start a new template and your current template will remain as-is." Buttons: Cancel (secondary) and Start over (destructive).
- **Actions:** Cancel closes the modal; Start over navigates to /templates/create and closes the modal.

## 2. SchemaEditorView (SchemaEditorView.tsx)

- Added showStartOverConfirm and handleStartOverConfirm.
- When variant === 'wizard' and backPath === '/templates/create', both **Back to Step 1** and **Cancel** open the Start Over modal instead of navigating.
- Keeps the existing unsaved-changes confirmation when relevant.
- Uses the same modal copy and buttons.

### Triggers covered

1. **Stepper Step 1:** Clicking Details on Step 2 or 3 opens the modal.
2. **Back to Step 1:** Clicking Back or Cancel in the Step 2 action bar opens the modal.
3. **Step 3:** Back goes to Step 2, so no confirmation there.

TypeScript build completed successfully.

## ▼ Slice 12 — Make Schema Editor panels resizable (shadcn Resizable)

### SLICE 10 – Make Schema Editor panels resizable (shadcn Resizable)

#### Goal:

In SchemaEditorView, replace the fixed side-by-side layout of:

- Tables panel (left)
- Fields panel (right)

with a draggable resizable split using shadcn Resizable (react-resizable-panels).

#### Scope:

- Shared component only: SchemaEditorView (so it applies in wizard step 2 and normal schema route)
- Keep existing Tables and Fields UI unchanged, only wrap them in resizable panels.

#### Dependencies:

- Add npm dependency: react-resizable-panels
- Add shadcn Resizable component files into shared ui:
  - frontend/src/shared/ui/resizable.tsx (or /resizable/index.tsx depending on your conventions)

#### Do NOT:

- Change schema editor behavior, data fetching, modals, validation, save flow
- Change routes
- Change wizard logic
- Add any other libraries

#### Implementation requirements:

##### 1) Install:

- react-resizable-panels

```

2) Add Resizable UI component:
 - Copy from shadcn Resizable docs
 - Ensure imports match project utilities (cn, etc.)
 - Do not introduce unused extras

3) Update SchemaEditorView layout:
 - Use <ResizablePanelGroup direction="horizontal">
 - Left panel: Tables section
 - <ResizableHandle withHandle />
 - Right panel: Fields section
 - Default sizes:
 - Tables: 35
 - Fields: 65
 - Min sizes:
 - Tables min: 20
 - Fields min: 40
 (Adjust if needed for your UI)

4) Height + scrolling:
 - Ensure both panels can scroll internally without breaking the page:
 - Panel content should have overflow-auto (or existing scroll behavior)
 - Avoid making the whole page grow excessively

5) Visual polish:
 - Handle must be visible (withHandle true)
 - Panels should keep the same padding/border styles as before (don't regress)

Verification:
 - Wizard Step 2: Tables/Fields are resizable by dragging
 - Templates nav schema route: same behavior
 - No TypeScript errors
 - No layout regressions in the schema editor UI

```

## ▼ Slice 12 — change

### Summary

#### 1. Installed react-resizable-panels

- Added as an npm dependency.

#### 2. Added shadcn Resizable component (frontend/src/shared/ui/resizable.tsx)

- Exports ResizablePanelGroup, ResizablePanel, and ResizableHandle.
- Wraps Group, Panel, and Separator from react-resizable-panels.
- Handle uses GripVertical when withHandle is true.
- Uses cn from @/shared/lib/utils and styling consistent with other shadcn components.

#### 3. Updated SchemaEditorView layout

- Replaced fixed grid with ResizablePanelGroup:
- **Left panel (Tables):** defaultSize={35}, minSize={20}
- **Right panel (Fields):** defaultSize={65}, minSize={40}

- ResizableHandle with withHandle for a visible grip.
- Both panels use overflow-auto for internal scrolling.
- min-h-[480px] so the layout has a defined height.
- Tables and Fields content wrapped in Cards with padding; each panel has pr-2 / pl-2 for spacing around the handle.

## Verification

- TypeScript build passes.
- SchemaEditorView is used by both:
  - Wizard Step 2 (CreateTemplateStepPage)
  - Schema route (SchemaEditorPage)

You can resize the Tables and Fields panels by dragging the handle between them.

### ▼ Slice 13 — Schema Editor UX polish (Tables/Fields) + Wizard page padding

```
Slice: Schema Editor UX polish (Tables/Fields) + Wizard page padding

Context
SchemaEditorView currently shows duplicate "Add table" CTAs in empty state,
"Add field" button placement is inconsistent, scrolling breaks when panels re
size, and wizard step pages have too much horizontal padding.

We need a small UX refactor that:
- Keeps UI clean in 3 states: (1) all empty, (2) fields empty only, (3) no em
pty.
- Makes both panels scrollable (vertical + horizontal) and reduces overall ed
itor height so the whole page is visible initially.
- Fixes wizard steps layout padding (horizontal).

Scope
A) SchemaEditorView (Tables/Fields panels)
1) **Duplicate "Add table" button**
 - If `schema.tables.length === 0`:
 - Show ONLY the centered "Add table" button inside EmptyState.
 - Hide the header "Add table" button.
 - If `schema.tables.length > 0`:
 - Show ONLY the header "Add table" button.
 - No centered CTA should appear.

2) **"Add field" button alignment**
 - Move "Add field" button into the Fields header row.
 - Header row layout:
 - Left: `Fields · <selectedTable.tableKey>`
 - Right: "Add field" button aligned to the far right of the Fields pane
l/card.
 - Rules:
 - If no table selected: do NOT show "Add field" button.
 - If table selected and not readOnly: show "Add field" button in header
(regardless of fields empty/non-empty).
```

- Avoid duplicate CTAs: if button exists in header, do NOT also show it in the fields empty-state body.

3) **\*\*Scrolling (vertical + horizontal)\*\***

- Both Tables and Fields panels must be vertically scrollable.
- Both must support horizontal scrolling when content exceeds width.
- Fix the common “no horizontal scroll after resize” issue by ensuring:
  - Panels and relevant wrappers use ``min-w-0``.
  - The table/list content can overflow horizontally:
    - Fields DataTable wrapper should be ``overflow-x-auto``.
    - DataTable should have a minimum width (e.g., ``min-w-max`` or explicit ``min-w-[...])``) so overflow triggers.
  - The Tables list should also be safe for horizontal overflow (long table keys).

4) **\*\*Reduce editor height\*\***

- Reduce overall editor vertical footprint so the entire page is visible at first glance.
  - Replace ``min-h-[480px]`` with a smaller value (e.g., ``min-h-[320px]`` or ``min-h-[360px]``) and rely on internal scrolling.

5) **\*\*3 UI states acceptance criteria\*\***

- State 1 (Everything empty):
  - Tables panel shows EmptyState with centered “Add table” button.
  - Fields panel shows “Select a table...” message (no Add field button).
  - No duplicate Add table button in header.
- State 2 (Fields empty only):
  - Tables panel shows list + header “Add table” button.
  - Fields header shows ``Fields · <tableKey>`` + right-aligned “Add field” button.
    - Fields body shows empty message (“No fields yet...”), but NO extra centered Add field button if header has it.
- State 3 (No empty):
  - Tables panel list visible + header “Add table”.
  - Fields header shows right-aligned “Add field”.
  - Fields table scrolls horizontally when panel shrinks.

---

### B) Wizard step pages (Create Template Stepper pages)

1) **\*\*Too much horizontal padding\*\***

- The step pages container has excessive ``px-*`` padding causing content to look “shrunk”.
- Reduce horizontal padding to match the normal schema editor route spacing.
- Implement responsive padding (example direction):
  - Use smaller base padding and scale up on large screens.
- Goal: step pages visually match ``/templates/.../schema`` page spacing.

---

## Non-goals

- No backend changes.
- No functional logic changes to schema save/load.



```

- No new features beyond layout/scroll/spacing and CTA placement.

Deliverables
- PR with:
 - Updated SchemaEditorView layout (CTA placement + scroll + height).
 - Updated wizard step container padding.

Notes / Hints
- Horizontal scrolling often fails because a parent flex item is missing `min-w-0`.
- Prefer a single header row per panel using `flex items-center justify-between`.
- Prevent duplicate CTAs by branching on empty/non-empty conditions.

```

#### ▼ Slice 13 — Change

## ▼ Learn

### ▼ routing in react v6(ranking algo) VS v5 (swallowing trap)

Perfect.

Here is a clean **EEx-style summary** of our route discussion — clear, structured, conceptual, and practical.

## Route Discussion Summary (React Router v6)

### 1 The Concern

We added these routes:

```

/templates/create
/templates/create/:templateId
/templates/create/:templateId/export
/templates/:templateId

```

The concern was:

Could `/templates/create` accidentally match `templates/:templateId` (i.e., treat `"create"` as a `templateId`)?

This is what people mean by a route being “swallowed”.

### 2 Why This Could Happen (Conceptual Understanding)

A dynamic route like:

```

/templates/:templateId

```

Means:

Match anything after `/templates/` and call it `templateId`.

So technically:

```
/templates/create
```

Could be interpreted as:

```
templateId = "create"
```

That's the theoretical conflict.

### 3 How React Router v6 Actually Works (Under the Hood)

React Router v6 does **not** use "first match wins" like v5 did.

Instead, it:

1. Collects all routes
2. Scores them by specificity
3. Picks the most specific match

#### Route Ranking Rule (Simplified)

Static segment > Dynamic segment > Wildcard

So:

```
/templates/create → static + static
/templates/:templateId → static + dynamic
```

Static segments are more specific.

Therefore:

```
/templates/create
```

Will always match the static route, not the dynamic one.

Even if order is reversed.

### 4 Does Route Order Matter in v6?

#### Functionally:

No — not in this case.

#### For readability:

Yes — putting specific routes before general ones makes the router easier to reason about.

So while your router is technically safe either way, good practice is:

Most specific → Most general.

### 5 Why This Matters Architecturally

If we had skipped Slice 1 and mixed routing + logic together:

- Route ambiguity would be harder to debug.
- Collisions might be hidden behind other changes.

- We'd lose clarity over navigation structure.

Slice 1 isolated routing, so we verified:

- Matching works
- No collisions
- No layout breakage

Before adding logic.

That's disciplined architecture.

---

## 6 Final Conclusion

- Your routing setup is correct.
- React Router v6 protects you from dynamic route swallowing.
- Slice 1 is architecturally sound.
- You can safely proceed to Slice 2.

If you want next, I can summarize the *architectural lesson* behind this (why slicing by routing first is a powerful pattern beyond this feature).

### ▼ react control input changes

You're asking about:

```
onChange={ (e: React.ChangeEvent<HTMLInputElement>) => setName(e.target.value) }
```

---

## Mental Model First

An `<input />` field in React does **not automatically update state**.

React works like this:

```
User types → Browser fires event → React receives event → You update state → React re-renders → Input value updates
```

This line is the bridge between:

- The browser event
- Your React state

---

## Break It Down Piece by Piece

### 1 `onChange={...}`

`onChange` is a React prop that listens to change events from the input.

In plain HTML:

```
<input onChange="something()" />
```

In React:

```
<Input onChange={...} />
```

Whenever the user types, deletes, or pastes text, this function runs.

This part:

```
(e: React.ChangeEvent<HTMLInputElement>)
```

Is TypeScript.

It ensures:

- `e.target` is known to be an `HTMLInputElement`
- `.value` exists
- You get IDE support
- You avoid unsafe access

It's not mandatory, but it's good practice.

## Simpler Version (Equivalent)

You could also write:

```
onChange={ (e) => setName(e.target.value) }
```

TypeScript will often infer the type automatically.

Your version is just explicit.

### ▼ `??` in JS

The nullish coalescing operator provides a default value only when the expression on its left side is `null` or `undefined`. It differs from the logical OR operator (`||`), which uses the right-hand value for any "falsy" value (including `0`, `""` (empty string), and `false`), which can lead to unexpected results in a React UI.

⌕

```
const result = value ?? defaultValue;
```

### ▼ `&&` in JS

#### How `&&` Short-Circuits

The logical AND operator works based on "truthy" and "falsy" values in JavaScript:

- **If the first operand is `false` (or falsy):** The expression immediately short-circuits and returns the original value of the first (falsy) operand, without evaluating the second operand. The entire expression is guaranteed to be false, regardless of the second operand's value.
- **If the first operand is `true` (or truthy):** The evaluation continues to the second operand, and the operator returns the original value of the second operand (which determines the final result).

M

#### Common Use Cases

- **Conditional Execution:** A common pattern is to use `&&` to execute a function or an expression only if a condition is met.

##### javascript

```
let price = 500;
// The console.log runs only because the first condition is true
(price > 400) && console.log('Price is over 400'); // Output: Price is over 400
```

```
// The console.log does not run because the first condition is false
(price > 600) && console.log('Price is over 600'); // No output
```

▼ Backend inception -inprogress

## Exploring

### ▼ app functionality and architecture (briefed)

#### Step 0 — Orientation map (don't read code yet)

Backend packages are structured like a “clean-ish architecture”:

api/	REST controllers + DTOs (HTTP world)
application/	orchestration / use-cases (business flow)
domain/	business rules (pure logic world)
infrastructure/	DB + Excel + hashing + validation (technical world)
config/	cross-cutting (OpenAPI, logging, CORS, etc.)

#### Visualization: what happens per request

```
HTTP Request
 ↓
api/controller (DTO in/out)
 ↓
application/service (business flow orchestration)
 ↓
infrastructure/persistence (Postgres) infrastructure/excel (XLSX)
 ↘
 infrastructure/validation + hashing
```

#### Step 1 — “What is the backend responsible for?”

This service is basically:

##### 1. Template + Version management

- Create template → auto-create v1
- Create new version → clone schema
- Only latest version is editable (important rule)

##### 1. Schema storage

- Stored as JSON (JsonNode) in DB

##### 1. Schema validation + hashing

- Validate schema structure rules
- Canonicalize JSON + SHA-256 hash

##### 1. Excel export

- Build XLSX from schema (streaming SXSSF)

So you're building: **“Schema → validate/hash → persist → export XLSX”**

- ☒ services: `TemplateApplicationService`
- ☐ controller
  - ☒ `TemplateController`
  - ☒ `VersionController`
  - ☐ `TemplateApplicationService`
    - ☒ `createTemplate` — `SchemaHasherImpl` — `JsonCanonicalizerImpl`
    - ☒ `updateSchema` — `SchemaValidatorImpl`

## Problems Detected

### ▼ Architecture Refactor (Fixed)

HEP verdict: **Yes, that structure is fine** for a small/medium monolith **if you keep it strict**—but it can turn into a mess if “services” becomes a dumping ground.

#### Good “right-sized” modular monolith structure

Per module (feature-based), do:

- `controller/` (HTTP only)
- `service/` (business flow + transactions)
- `dao/` (repositories / DB queries)
- `entity/` (JPA entities)
- `dto/` (request/response)
- `mapper/` (optional; only if you really need it)

#### Key rule to avoid spaghetti

**Controllers never call DAO directly.**

Only: `controller → service → dao`

#### For “another module”

Yes: make it **modular monolith**:

```
modules/
 template/
 controller/
 service/
 dao/
 entity/
 dto/
 submission/ (future)
 ...
 shared/
 config/
 error/
 util/
```

That's simpler than clean-architecture, scales enough, and stays understandable.

## ▼ Version 1 Hashing bug (not Fixed)

You spotted a **real issue**. HEP verdict: **good catch** — this is a **design inconsistency** that will bite you later.

### What's wrong

In `createTemplate()` you create v1 with:

- `schemaJson = buildInitialSchemaJson(...)`
- `schemaHash = PLACEHOLDER_HASH`

So v1 is stored with a **fake hash**, while `updateSchema()` stores a **real deterministic hash**.

That breaks the meaning of `schema_hash`:

- sometimes it's a fingerprint
- sometimes it's a dummy value

### Why it matters (practical bugs)

- If your UI shows hash, v1 shows "PENDING" forever unless user saves schema.
- If export metadata includes `schema_hash`, exports from a fresh template will embed the placeholder (bad traceability).
- If you ever rely on `schema_hash` for caching / integrity / "did schema change?" logic, v1 behaves differently.

### Senior recommendation (simple + correct)

In `createTemplate()` do the exact same pipeline as `updateSchema()`:

1. build initial schema JSON
2. validate it (optional but good)
3. hash it
4. save real hash (no placeholder)

Example logic (conceptually):

- `schemaValidator.validate(initialSchema)`
- `schemaHash = schemaHasher.hash(initialSchema)`
- `v1.setSchemaHash(schemaHash)`

### One nuance

If you **intentionally** wanted "draft template before schema exists", then don't store a placeholder hash. Instead:

- allow `schema_hash` to be **NULL** until first save
- and treat "no hash" as "not finalized"

But your DB column is likely **NOT NULL** (based on typical Slice 1 rules), so the placeholder was a workaround — not a clean design.

### Verdict

Fix it now. It's small and improves correctness immediately:

- **hash v1 at creation**
- stop using `PLACEHOLDER_HASH` (or keep it only for legacy migration rows)

If you paste your `SchemaHasher` implementation (and whether it canonicalizes), I'll tell you if hashing at creation is guaranteed deterministic and safe.

## ▼ Status in Template vs in Version (not Fixed)

Excellent question. This is architectural thinking now.

HEP mode ON → direct, structured, no fluff.

You're asking:

| Why do we need `status` in `TemplateEntity` if `TemplateVersion` already has status?

Let's break it properly.

---

## 1 First: What Do Template and TemplateVersion Represent?

Conceptually:

```
Template (logical container)
├── Version 1
├── Version 2
└── Version 3
```

- **Template** = Business object (identity)
- **TemplateVersion** = Snapshot of structure at a point in time

They are not the same thing.

---

## 2 Version Status ≠ Template Status

### TemplateVersion.status

Answers:

| What is the lifecycle state of this specific version?

Examples:

- DRAFT
  - PUBLISHED
  - ARCHIVED
  - DEPRECATED
- 

### Template.status

Answers:

| What is the overall state of the template as a business entity?

Examples:

- ACTIVE
- INACTIVE
- LOCKED
- DELETED
- SUSPENDED



Different abstraction level.

---

### 3 Why Version Status Alone Is Not Enough

Imagine this scenario:

```
Template A
├─ v1 (PUBLISHED)
├─ v2 (DRAFT)
```

Now business decides:

| Disable Template A completely.

Do you:

- Change all version statuses?
- Or just mark Template as INACTIVE?

Cleaner solution:

```
Template.status = INACTIVE
```

Versions remain historically accurate.

---

### 4 Separation of Concerns (Architectural Reason)

TemplateVersion status is about:

| Content lifecycle

Template status is about:

| Business availability

They solve different problems.

---

### 5 Performance & Query Simplicity

If you only store status in versions:

To know if a template is active, you must:

```
SELECT ...
FROM templates t
JOIN template_versions v
WHERE v.status = 'PUBLISHED'
ORDER BY v.version_number DESC
LIMIT 1
```

That's heavier.

With Template.status:

```
SELECT *
FROM templates
```

```
WHERE status = 'ACTIVE'
```

Much simpler.

---

## 6 Real Enterprise Pattern

This is common in:

- Document Management Systems
- Policy Engines
- Product Catalogs
- Form Builders

Container has state.

Version has state.

They represent different lifecycles.

---

## 7 When Would Template.status Be Redundant?

It becomes redundant only if:

- Template lifecycle is 100% derived from latest version
- You never need to disable whole template independently
- You always compute state dynamically

That increases complexity.

---

## 8 Clean Design Option

If you keep both:

Define clearly:

```
Template.status:
 ACTIVE
 INACTIVE
 DELETED

TemplateVersion.status:
 DRAFT
 PUBLISHED
 ARCHIVED
```

No overlap.

No confusion.

---

## 9 What I Suspect in Your Code

Right now you have:

```
@Column(nullable = false)
```

```
private String status;
```

If you're using:

```
STATUS_DRAFT
STATUS_PUBLISHED
```

at Template level too,

Then yes — that's conceptually messy.

Template should not be DRAFT.

Version should be DRAFT.

---

## Recommendation (Architectural Clean-Up)

Option A (Clean separation):

- Keep status in both
- Use different enum domains

Option B (Simpler MVP):

- Remove status from Template
- Derive template state from latest version

But if you plan enterprise-level evolution → keep both.

---

## Final Verdict

You need Template.status if:

- Template has its own lifecycle independent of versions
- You want fast filtering
- You want business-level activation control

You don't need it if:

- Version completely defines lifecycle
  - You always rely on latest version state
- 

Now I want to ask you something critical:

In your business logic, can a template be disabled even if it has a published version?

If yes → you need Template.status.

If no → maybe it's redundant.

Your answer defines architecture direction.

### ▼ N+1 query problem in `listTemplates` (performance issue - fix later)

Good. This is a very important method to understand because it reveals both behavior and a hidden performance issue.

Let's walk through it slowly.

---

### What this method does (big picture)

```
@Transactional(readOnly = true)
public List<TemplateSummaryDto> listTemplates()
```

This method:

1. Fetches **all templates**
2. For each template:
  - Finds its **latest version**
  - Builds a summary DTO
3. Returns a list of summaries

So it is building a **dashboard view model**.

---

### Step-by-step flow

```
List<TemplateEntity> templates = templateRepository.findAll();
```

This runs:

```
SELECT * FROM templates;
```

Now you have all templates in memory.

Then:

```
return templates.stream()
 .map(this::toTemplateSummary)
 .toList();
```

For each template, it calls:

```
private TemplateSummaryDto toTemplateSummary(TemplateEntity template)
```

Inside that:

```
versionRepository.findTopByTemplate_IdOrderByVersionNumberDesc(template.getId());
```

This runs a query like:

```
SELECT * FROM template_versions
WHERE template_id = ?
ORDER BY version_number DESC
LIMIT 1;
```

It fetches the **latest version**.

Then it builds:

```
LatestVersionSummaryDto
```

and wraps everything into:

```
TemplateSummaryDto
```

---

### What `@Transactional(readOnly = true)` means here

- This method runs inside a transaction.
  - `readOnly = true` tells Hibernate:
    - Don't track changes.
    - Optimize for reads.
  - It's correct here because you are not modifying anything.
- 

### Visualization of what happens

If you have 3 templates:

```
T1
T2
T3
```

Execution becomes:

```
Query 1 → fetch all templates (1 query)
```

Then:

```
Query 2 → latest version of T1
```

```
Query 3 → latest version of T2
```

```
Query 4 → latest version of T3
```

Total = 1 + N queries.

This is called:

| N + 1 query problem.

---

### Is this wrong?

Functionally → No.

Architecturally → Acceptable for small scale.

Performance-wise → Risky if templates grow large.

If you have 1000 templates, this becomes 1001 queries.

---

### Why this design was probably chosen

Because it's simple and readable.

The developer avoided:

- complex joins
- custom JPQL
- native SQL

It keeps service logic very clear.

For a small system, that's fine.

---

### What this method really represents

It is building a **read model**.

It is not returning entities.

It is not exposing persistence objects.

It is creating a clean API DTO.

That part is correct design.

---

## Senior-level critique (direct)

Strengths:

- Clean mapping to DTO
- Clear separation of entity vs response
- Correct transaction usage

Weakness:

- N+1 query problem
  - Potential performance degradation later
- 

## Mental Model Summary

```
Templates table
 ↓
For each template
 ↓
Find latest version
 ↓
Build dashboard summary DTO
```

---

If you want next, I can show you how to fix the N+1 problem cleanly without overengineering.

## ▼ strongly declare `JsonNode` in `hash` function instead of generic object (not fixed)

Is receiving a generic `Object` okay?

It works, but **it's not ideal** here.

In your service you already have:

```
JsonNodeschemaJson= ...
```

So the hasher should strongly declare what it needs:

✅ Better:

```
publicStringhash(JsonNodeschemaContent)
```

Why better:

- compile-time safety (no runtime “wrong type” surprises)
- clearer contract

- no need for `instanceof` at all

When is `Object` acceptable?

- if the interface must support multiple types (`JsonNode`, `String`, `byte[]`, etc.)
- or you're implementing a generic framework API

In your case: schema is always `JsonNode` → prefer `hash(JsonNode)`.

## ▼ In `toCanonicalString`, array order affects the hash (fix later)

### ▼ explanation of the `JsonCanonicalizerImpl` class

#### ObjectMapper copy + configuration

```
this.objectMapper = objectMapper.copy()
 .configure(SerializationFeature.ORDER_MAP_ENTRIES_BY_KEYS, true);
```

This creates a **private ObjectMapper for this class**.

Why copy?

Spring Boot already provides a **global ObjectMapper** used by controllers for request/response JSON.

Changing its configuration could affect the whole application.

So instead we:

```
Spring global ObjectMapper
|
├── used by controllers
└── copy() → Canonicalizer ObjectMapper
 (safe to configure here)
```

This avoids modifying the shared mapper.

## Why Jackson warns about `configure(...)`

Jackson documentation warns because **modifying a mapper after it starts being used in multiple threads can be unsafe**.

Example unsafe pattern:

```
@Autowired ObjectMapper mapper;

mapper.configure(...); // risky if mapper already used
```

Your code is safe because:

```
copy() → new mapper instance
configure() → happens before first use
```

No other component shares it.

## What `ORDER_MAP_ENTRIES_BY_KEYS` actually does

```
SerializationFeature.ORDER_MAP_ENTRIES_BY_KEYS
```

This tells Jackson:

| When serializing maps, always sort keys alphabetically.

Example:

```
Map input:
{b=2, a=1}

Serialized output:
{"a":1,"b":2}
```

This contributes to **deterministic output**, which is required for hashing.

However, your implementation already sorts keys manually with `TreeSet`, so this setting is more of a **defensive extra guarantee**.

## Why canonicalization still sorts keys manually

Inside `writeNode`:

```
TreeSet<String> keys = new TreeSet<>();
node.fieldNames().forEachRemaining(keys::add);
```

This ensures **every JSON object is written with alphabetically ordered keys**.

Visualization:

```
Input JSON
{"b":2,"a":1}

Processing
TreeSet → [a, b]

Canonical output
{"a":1,"b":2}
```

So even if the original JSON arrived in different orders, canonical output is identical.

## Why we don't just call `objectMapper.writeValueAsString(node)`

Because we want **full control of traversal and ordering**.

Your canonicalizer explicitly defines:

```
OBJECT → sorted keys
ARRAY → preserve order (or custom sort for schema arrays)
STRING → write text
NUMBER → write normalized number
BOOLEAN → write boolean
NULL → write null
```

Visualization:

```
JsonNode
 ↓ recursive traversal
writeNode()
 ↓
JsonGenerator writes canonical JSON
```



↓  
StringWriter collects characters

## Final mental pipeline

```
JsonNode schema
 ↓
JsonCanonicalizerImpl
 ↓
canonical JSON string
 ↓
UTF-8 bytes
 ↓
SHA-256
 ↓
Hex string
 ↓
schema_hash
```

Canonicalization ensures the **same logical schema always produces the same hash**, regardless of formatting or object key order.

HEP on: if **table/field order truly does not matter**, then your current hashing is **wrong for your business meaning**. You'll get "false changes" (hash changes) when nothing semantically changed—just reordering.

Right now:

- Object keys are normalized (good)
- Arrays keep order (bad *for your stated rule*)

So yes: if the UI reorders tables/fields, your hash will change even though the schema is "the same" in your business sense.

## What to do (minimal, correct)

You must canonicalize arrays that are **order-insensitive** by sorting them deterministically **before hashing**.

For your schema, likely candidates:

- `tables[]` → sort by `tableKey` (or `sheetName`, but `tableKey` is safer)
- `fields[]` inside each table → sort by `fieldKey` (or `name`)
- any other order-insensitive arrays → define a stable key

## Mental model

```
Canonicalization = normalize object key order
 + normalize selected arrays order (business rule)
```

## Implementation approach (clean + explicit)

In `writeNode` when encountering ARRAY:

- Detect if this array is one of your "special" arrays (tables/fields)
- Sort its elements by a stable key
- Then write them

Pseudo (conceptual):

```
if (isTablesArray(node)) sort by element["tableKey"];
else if (isFieldsArray(node)) sort by element["fieldKey"];
else keep order
```

**Important:** don't blindly sort all arrays; some arrays are truly ordered (e.g., "columns in Excel" if you decide order matters later, or "steps"). Make it explicit per path.

## Consequences (good + trade-off)

✓ Hash becomes stable across reorderings

✓ "schema changed?" becomes meaningful

⚠ You must define uniqueness keys ( `tableKey` , `fieldKey` ) and guarantee they exist

If you paste a typical schema JSON shape (one table + fields), I'll tell you exactly which array paths to normalize and what sort keys to use.

## ▼ Templates Validation Rules Summary

### SchemaValidatorImpl Cheat Sheet (with examples)

#### Input expectations

✓ Accepts: **JsonNode root object**

✗ Rejects: not JsonNode / not JSON object

#### Bad

```
[]
```

→ error: "Schema must be a JSON object"

### Root rules

#### `sectorCode` (required)

- must exist
- must be string
- must not be blank



```
{ "sectorCode": "VTC" }
```

✗ missing

```
{ }
```

✗ blank

```
{ "sectorCode": " " }
```

✗ wrong type

```
{ "sectorCode": 10 }
```

**templateName** (optional but if present must be non-blank)

✓ missing is ok

```
{ "sectorCode": "VTC" }
```

✓ present ok

```
{ "templateName": "Inspection", "sectorCode": "VTC" }
```

✗ blank

```
{ "templateName": " ", "sectorCode": "VTC" }
```

**Tables rules** ( **tables** is optional; if missing or not array → validator returns without errors)

Schema shape:

```
{
 "sectorCode": "VTC",
 "tables": [
 {
 "tableKey": "veh",
 "sheetName": "Vehicles",
 "order": 1,
 "fields": []
 }
]
}
```

**tableKey** (optional, but if present → must be unique when non-blank)

✓

```
{ "tables":[{"tableKey":"veh"},{"tableKey":"drv"}], "sectorCode":"VTC" }
```

✗ duplicate

```
{ "tables":[{"tableKey":"veh"},{"tableKey":"veh"}], "sectorCode":"VTC" }
```

**sheetName** (required)

Rules:

- required + string
- trim must not be empty
- length ≤ 31 (Excel)
- must not contain: `\ / ? * [ ]`

- must be unique after trimming



```
{ "tables":[{"sheetName":"Vehicles"}], "sectorCode":"VTC" }
```

✗ missing

```
{ "tables":[{}], "sectorCode":"VTC" }
```

✗ blank after trim

```
{ "tables":[{"sheetName":" "}], "sectorCode":"VTC" }
```

✗ too long (>31)

```
{ "tables":[{"sheetName":"12345678901234567890123456789012"}], "sectorCode":"VTC" }
```

✗ invalid chars

```
{ "tables":[{"sheetName":"Veh?cles"}], "sectorCode":"VTC" }
```

✗ duplicate after trim

```
{
 "sectorCode":"VTC",
 "tables":[
 {"sheetName":"Vehicles"},
 {"sheetName":" Vehicles "}
]
}
```

**order** (optional; must be integer-like)



```
{ "tables":[{"sheetName":"Vehicles","order":2}], "sectorCode":"VTC" }
```

✗

```
{ "tables":[{"sheetName":"Vehicles","order":1.5}], "sectorCode":"VTC" }
```

**Fields rules** ( **fields** optional; if missing or not array → skip)

Field shape:

```
{
 "fieldKey":"plate",
 "headerName":"Plate",
 "type":"TEXT",
 "validations": { }
}
```

**fieldKey** (optional, but if present → must be unique when non-blank)



```
{
 "sectorCode": "VTC",
 "tables": [{ "sheetName": "Vehicles", "fields": [
 { "fieldKey": "plate", "headerName": "Plate", "type": "TEXT" },
 { "fieldKey": "color", "headerName": "Color", "type": "TEXT" }
] }]
}
```

duplicate

```
{
 "sectorCode": "VTC",
 "tables": [{ "sheetName": "Vehicles", "fields": [
 { "fieldKey": "plate", "headerName": "Plate", "type": "TEXT" },
 { "fieldKey": "plate", "headerName": "Plate2", "type": "TEXT" }
] }]
}
```

**headerName** (required, string, non-blank)



```
{ "headerName": "Plate" }
```

missing/blank

```
{ "headerName": " " }
```

**type** (required; allowed values)

Allowed: **TEXT**, **NUMBER**, **DATE**, **BOOLEAN**, **CURRENCY**

(case-insensitive since code uses `toUpperCase()`)



```
{ "type": "number" }
```



```
{ "type": "EMAIL" }
```

**Validations rules (inside **validations** object)**

**enum** allowed only for **TEXT**



```
{ "type": "TEXT", "validations": { "enum": ["A", "B"] } }
```



```
{ "type":"NUMBER", "validations": { "enum":["A","B"] } }
```

**min/max** allowed only for **NUMBER** or **CURRENCY**



```
{ "type":"NUMBER", "validations": { "min":0, "max":100 } }
```



```
{ "type":"DATE", "validations": { "min":0, "max":100 } }
```

**min <= max**



```
{ "type":"NUMBER", "validations": { "min":50, "max":18 } }
```

## Important bug to fix (your validator loops)

Your loops start from **1** and go to **size()+1**, which skips index **0** and may read out of range.

Correct is:

```
for (int i = 0; i < tablesNode.size(); i++)
for (int i = 0; i < fieldsNode.size(); i++)
```

This affects validations and error paths.

- Make Name and SectorCode unique so every template have unique hash (fix later)

- delete header name in the step2 in the backend and replace it with description

- hide **product\_name** in step2

- Add mapper (not recommended now)

## Learn

▼ if **@PostMapping** has no value how the frontend call it

Good question.

Short answer:

No, the URL is **NOT** `/api/templates/createTemplate`

It is **POST** `/api/templates`

Let's break it clearly.

## How Spring Builds the URL

You have:

```
@RestController
@RequestMapping("/api/templates")
public class TemplateController
```

This sets the **base path**:

```
/api/templates
```

Then your method:

```
@PostMapping
public ResponseEntity<CreateTemplateResponse> createTemplate(...)
```

Because `@PostMapping` has **no value**,

it inherits the class-level path only.

So the final endpoint becomes:

```
POST /api/templates
```

NOT:

```
/api/templates/createTemplate
```

Spring does NOT use method name in URL.

## How It Would Become `/createTemplate`

If you wrote:

```
@PostMapping("/createTemplate")
```

Then the full URL would be:

```
POST /api/templates/createTemplate
```

## So From Frontend You Call:

If backend runs on:

```
http://localhost:8080
```

Then frontend should call:

```
POST http://localhost:8080/api/templates
```

With body:

```
{
 "name": "Test Template",
 "sectorCode": "SEC-01",
 "createdBy": "omar"
}
```

## REST Design Note (Important)

What you implemented is actually **correct REST style**.

In REST:

```
POST /api/templates
```

means:

| Create a new template resource.

You don't need `/createTemplate` because:

- HTTP verb (POST) already expresses action.
- URL represents resource, not action.

Good design 👍

## Quick REST Rule

Action	REST Style
Create	POST /templates
Get all	GET /templates
Get one	GET /templates/{id}
Update	PUT /templates/{id}
Delete	DELETE /templates/{id}

If you want, next we can review whether your controller is thin enough or leaking logic.

### ▼ SpringDoc (briefed)

Yes — **your controllers are the source of truth for the API contract**, and **springdoc generates the documentation from them automatically**.

And in your zip, **API documentation is actually implemented**, not just imported:

-  You have `springdoc-openapi-starter-webmvc-ui` in `pom.xml` (so Swagger UI is enabled).
-  You have a real `OpenApiConfig.java` that registers an `OpenAPI` bean (title/description/version).
-  Your controllers ( `TemplateController`, `VersionController` ) use Spring MVC annotations ( `@RestController`, `@RequestMapping`, `@GetMapping`, etc.) → springdoc scans these and builds the OpenAPI spec.

So the doc is generated like this:

```
Controller mappings → springdoc scans → OpenAPI JSON → Swagger UI
```

You *don't need* `@Operation` annotations for docs to exist — they're only for nicer descriptions/examples.

### ▼ @Valid



`@Valid` is not the thing that performs validation.

It is just a **trigger** that tells Spring: “validate this object now.”

When a request hits your controller like this:

```
@PostMapping
public ResponseEntity<?> create(
 @Valid @RequestBody CreateTemplateRequest request
)
```

Here is what really happens:

```
HTTP JSON
 ↓
Jackson → converts JSON to CreateTemplateRequest
 ↓
Spring sees @Valid
 ↓
Bean Validation runs on that object
 ↓
If violations exist → 400 thrown before method executes
If no violations → method runs normally
```

The actual validation rules come from annotations inside the class being validated, for example:

```
public record CreateTemplateRequest(
 @NotBlank String name,
 @NotBlank String sectorCode
) {}
```

`@NotBlank` is the rule.

`@Valid` is the activator.

If you remove `@Valid`, those annotations do nothing at runtime in the controller.

Important separation in your backend:

- DTO + `@Valid` → validates request structure (shape, nulls, sizes)
- Service layer → validates business rules (latest version editable, etc.)
- `SchemaValidator` → validates schema domain rules

`@Valid` only checks structural constraints declared on the class it is applied to.

It does not automatically validate your JPA entity unless you put it there and trigger validation on it.

So mentally:

| Constraint annotations define rules.

| `@Valid` tells Spring to enforce them at the boundary.

## ▼ `@RequestBody`

`@RequestBody` tells Spring:

| “Take the HTTP request body (usually JSON) and convert it into this Java object.”

Without it, Spring will not read the JSON body into your parameter.

Example:

```

@PostMapping
public ResponseEntity<?> create(
 @RequestBody CreateTemplateRequest request
)

```

Mental flow:

```

Client sends JSON
↓
Spring reads raw HTTP body
↓
Jackson converts JSON → CreateTemplateRequest
↓
Method receives populated object

```

If you remove `@RequestBody`, Spring will try to bind from query parameters or form data instead — not from JSON.

Important distinction:

- `@RequestBody` = how data enters your method (deserialization)
- `@Valid` = whether that object should be validated
- DTO annotations ( `@NotBlank`, etc.) = the actual validation rules

They work together:

```

@RequestBody → convert JSON
@Valid → validate converted object

```

If JSON structure doesn't match the DTO, Jackson fails before validation even runs.

So think of `@RequestBody` as the bridge between HTTP JSON and your Java world.

## ▼ `@PathVariable`

`@PathVariable` tells Spring:

“Take this value from the URL path and bind it to this method parameter.”

Example:

```

@GetMapping("/templates/{templateId}")
public ResponseEntity<?> getTemplate(
 @PathVariable UUID templateId
)

```

If the client calls:

```
GET /templates/550e8400-e29b-41d4-a716-446655440000
```

Spring does this:

```

URL path segment
↓
Extract value inside {templateId}
↓
Convert String → UUID

```

↓  
Pass it into method parameter

Important details:

- The name inside `{}` must match the parameter name (or specify it explicitly: `@PathVariable("templateId")`).
- Type conversion is automatic (String → UUID, Long, Integer, etc.).
- If conversion fails (invalid UUID), Spring returns 400 automatically.

Conceptually:

- `@RequestBody` → reads JSON body
- `@RequestParam` → reads query parameters ( `?page=1` )
- `@PathVariable` → reads values embedded in the URL path

So `@PathVariable` binds **resource identity from the URL** into your Java method.

## ▼ Convert `Dto` / `object` to pure `JsonNode` dynamic Json tree (not as json string)

When you convert from a DTO to `JsonNode`, you are changing **how the data is represented**, not the data itself.

Let's build the mental model clearly.

You start with a DTO:

```
public record UpdateSchemaRequest(
 String templateName,
 String sectorCode,
 List<TableDto> tables
) {}
```

This is a **strongly-typed Java object**.

It has fixed fields, compile-time safety, and validation annotations.

When the client sends JSON:

```
{
 "templateName": "Finance",
 "sectorCode": "FIN",
 "tables": []
}
```

Spring + Jackson do this automatically:

```
JSON text
↓
Deserialize
↓
DTO object (UpdateSchemaRequest)
```

Now you convert it again:

```
JsonNode schemaJson = objectMapper.valueToTree(request);
```

This does:

```
DTO object
 ↓
Jackson internal serialization
 ↓
JsonNode (tree model)
```

Important:

This is **not** converting to a JSON string.

It is converting to a **tree representation of JSON in memory**.

So now the same data exists in a flexible structure:

```
ObjectNode
├─ templateName : TextNode
├─ sectorCode : TextNode
└─ tables : ArrayNode
```

Why would you do this?

Because your service layer treats schema as **dynamic JSON**, not as a fixed Java structure.

That enables:

- Storing it as JSONB in Postgres
- Validating structure generically
- Hashing deterministically
- Supporting flexible schema evolution

If you stayed with DTO only:

- You are locked into fixed Java structure
- Harder to support dynamic fields
- Harder to store directly as JSONB

There are 3 representations in play:

1. JSON text (HTTP body)
2. DTO (typed Java object)
3. JsonNode (generic JSON tree)

Your flow is:

```
Client JSON
 ↓
DTO (validation boundary)
 ↓
JsonNode (domain processing + persistence)
```

The DTO protects your API boundary.

The JsonNode supports flexible schema logic.

That separation is intentional and correct for a system that stores dynamic schemas.

▼ What is **instanceof** e.g.

```
if (!(schemaContent instanceof JsonNode node)) { ... }
```

## 1) What is `instanceof JsonNode` ?

`instanceof` is a Java runtime type check.

```
if (!(schemaContent instanceof JsonNode node)) { ... }
```

Means:

- "If `schemaContent` is **not** a `JsonNode`, throw an error."
- If it **is** a `JsonNode`, Java also **casts** it and stores it in a variable named `node`.

So this one line does two things:

- checks the type
- safely casts it

It's like saying:

```
If object is JsonNode → treat it as JsonNode
Else → reject
```

## ▼ What is UTF-8? Is it byte e.g.

```
byte[] digest = sha256(canonical.getBytes(StandardCharsets.UTF_8));
```

UTF-8 is a **text encoding**: a rule for converting characters into bytes.

- Strings in Java are characters (Unicode)
- Hash functions work on bytes
- So you must convert string → bytes using an encoding

```
canonical.getBytes(StandardCharsets.UTF_8)
```

This produces a `byte[]`.

So:

- UTF-8 is not a "type"
- UTF-8 is the **encoding used to create the bytes**
- the result is `byte[]`

Mental model:

```
"ABC" (String)
 ↓ UTF-8 encoding
[65, 66, 67] (byte[])
```

## ▼ What is `HexFormat` (converts bytes into a hex string) e.g.

```
byte[] digest = sha256(canonical.getBytes(StandardCharsets.UTF_8));
return HexFormat.of().formatHex(digest);
```

`HexFormat` converts bytes into a **hex string** (base-16 text).

A hash digest is raw bytes like:

```
[15, 160, 33, ...]
```

Hex makes it printable and storable:

```
0fa021...
```

So:

```
HexFormat.of().formatHex(digest)
```

means:

- take `byte[] digest`
- return a readable hex string

This is similar to older patterns using `StringBuilder` manually.

## ▼ What is `MessageDigest` e.g (Java's standard API for cryptographic hash functions)

```
MessageDigest md = MessageDigest.getInstance("SHA-256");
```

`MessageDigest` is Java's standard API for **cryptographic hash functions** like:

- SHA-256
- SHA-1 (older, weaker)
- MD5 (broken for security)

You do:

```
MessageDigest md = MessageDigest.getInstance("SHA-256");
byte[] digest = md.digest(bytes);
```

Meaning:

- create a SHA-256 hasher
- feed it bytes
- get back the fixed-size hash output (32 bytes for SHA-256)

"Digest" literally means "the result of hashing".

## ▼ Why convert the hash to HEX after SHA-256?

SHA-256 produces **raw binary bytes**, not text.

Specifically:

```
SHA-256 output = 256 bits = 32 bytes
```

Example raw output:

```
[15, -94, 33, 74, 9, ...]
```

This binary data:

- is **not readable**
- may contain non-printable characters
- may break logs, DB, JSON, HTTP

So we convert it to **hex string**.

Example:

```
0fa2214a09...
```

Hex is just a **text encoding of the bytes**.

## Visualization

```
JSON schema
 ↓
canonical string
 ↓
UTF-8 bytes
 ↓
SHA-256
 ↓
32 raw bytes
 ↓
Hex encoding
 ↓
64-character string
```

Important:

```
32 bytes → 64 hex characters
```

Because each byte = 2 hex characters.

## Could we use Base64 instead?

Yes.

Example:

```
hex:
4fa7b2c1...

base64:
T6eywQ==
```

Base64 is **shorter**, but:

- harder to visually compare
- includes `/ + =` which sometimes complicates URLs

That's why **hex is standard for hashes**.

## ▼ Serialization vs Deserialization

Serialization and deserialization are simply **converting data between program objects and transferable formats**.

Think of it as **packing and unpacking objects** so they can travel through HTTP, files, or databases.

## Serialization

Serialization means converting a **Java object** → **JSON (or another format)**.

Example object in memory:

```
CreateTemplateRequest request =
 new CreateTemplateRequest("Inspection", "VTC", "Omar");
```

Serialization turns it into JSON text:

```
{
 "name": "Inspection",
 "sectorCode": "VTC",
 "createdBy": "Omar"
}
```

Flow:

```
Java Object
 ↓ serialization
JSON text
```

In Spring Boot this happens automatically when returning responses.

Example:

```
return new CreateTemplateResponse(...);
```

Spring uses **Jackson** to serialize that object into JSON for the HTTP response.

## Deserialization

Deserialization is the opposite: **JSON** → **Java object**.

Client sends JSON:

```
{
 "name": "Inspection",
 "sectorCode": "VTC",
 "createdBy": "Omar"
}
```

Spring converts it into:

```
CreateTemplateRequest request
```

Flow:

```
JSON text
 ↓ deserialization
Java Object
```

This happens automatically because of:

```
@RequestBody CreateTemplateRequest request
```

Jackson reads the JSON body and creates the DTO instance.

## How it works internally in your controller



Example:

```
@PostMapping
public ResponseEntity<CreateTemplateResponse> createTemplate(
 @Valid @RequestBody CreateTemplateRequest request)
```

Flow:

```
HTTP request JSON
 ↓
Jackson deserialization
 ↓
CreateTemplateRequest DTO
 ↓
Service logic
 ↓
CreateTemplateResponse object
 ↓
Jackson serialization
 ↓
HTTP JSON response
```

## Why it matters in your project

Your system uses **three representations of the same data**:

```
JSON (HTTP API)
 ↓
DTO objects (typed Java)
 ↓
JsonNode (dynamic JSON tree for schema)
```

Jackson performs the conversions between these layers.

### ▼ frontend modification

## ▼ enhance 1 — Fix Modal Validation UX + Align FE/BE Rules (Approach A + Auto-clean)

### ▼ contract

Perfect — Approach A is the right MVP move.

Here's the **exact behavior contract** I recommend we implement (no code yet), plus what needs to change to align FE/BE.

## 1) UX contract for Step 2 modals (Add table / Add field)

### A. On modal open

- **Do not show any red validation messages.**
- All fields start in state: `touched=false`, `submitAttempted=false`.
- Add button stays **disabled** until required fields are present (keep your current behavior).

Why: avoids the “angry form” problem you saw in images 1–2.

## B. While typing

Only validate “live” after the user has interacted with the field.

### Rules

- Required-field errors **do not show while typing** unless:
  - the user already blurred the field (touched), or
  - they already tried clicking Add once (submitAttempted=true) (*rare since Add is disabled, but still useful if later you enable Add*)

### Live validations that should run while typing (after touched):

- Sheet name constraints:
  - max 31 chars
  - forbidden chars `\ / ? * [ ]`
- `min <= max` check (when both present)
- **Type-dependent validations** (see section 3): when type changes, immediately re-check validations compatibility.

Why: users get fast feedback for “format mistakes”, but not noise for missing required fields.

---

## C. On clicking Add

This is the key moment.

- Run **full modal validation** (required + type-dependent + numeric relations + sheet name format).
- If valid → close modal and commit changes.
- If invalid → keep modal open and show errors.

Note: Even if Add is disabled most of the time when required fields are empty, this still matters for non-required issues (like `min/max` on TEXT, invalid sheetName chars, etc.).

---

## D. On Cancel / X close

- No validation is triggered.
  - If there are unsaved changes inside the modal, show a simple confirm:
    - “Discard changes?” (optional but nice)
- 

## 2) Step-level contract for “Save / Next” (wizard Step 2)

This addresses your image 3 frustration.

When the user clicks **Save Schema / Next**:

- Run **full schema validation** (cross-entity):
  - unique tableKey (case-insensitive ideally)
  - unique sheetName after trim
  - unique fieldKey per table
  - all table/field rules + type-dependent rules
- If invalid:
  - show a **summary banner/toast** + a list of errors (paths)
  - optionally highlight the table/field that caused it

Also: when a modal closes successfully (Add/Edit), you should validate that affected object (table/field) immediately so errors don't "wait until Save".

---

### 3) Fix the FE/BE mismatch (must-do)

Right now FE is missing critical backend rules:

Backend enforces:

- enum allowed only for **TEXT**
- min/max allowed only for **NUMBER or CURRENCY**
- min <= max when both exist
- sheetName rules + uniqueness after trimming
- allowed types only

Frontend currently enforces:

- min <= max only
- sheetName constraints (good)
- required keys (good)
- uniqueness keys (good)
- **missing:** type-dependent validation + sheetName uniqueness

#### Minimal alignment you must implement now

In frontend validation, add:

1. If `validations.enum` exists (non-empty) and `type !== TEXT` → error
2. If `validations.min` or `validations.max` exists and `type ∉ {NUMBER, CURRENCY}` → error
3. If sheetName duplicates another after `.trim()` → error (backend does this)

#### UX decision inside the modal when type changes

You have 2 policies:

##### Policy 1 (recommended): auto-clean invalid validations

- If user switches type to TEXT → clear min/max automatically
- If user switches type to NUMBER/CURRENCY → allow min/max
- If user switches type away from TEXT → clear enum automatically

Pros: less friction, fewer errors later

Cons: user might wonder "why did it disappear?"

##### Policy 2: don't clear, but show error

Pros: transparent

Cons: user must manually fix; more annoying

**My recommendation:** Policy 1 (auto-clean) + show a small inline hint like:

- "Min/Max cleared because type is TEXT."

This prevents the exact backend error you got in image 3.

---

### 4) What validation fires where (final decision table)

Validation	On typing	On Add (modal)	On Save/Next
Required fields	only after touched	✓	✓
SheetName format (31 chars, forbidden chars)	after touched	✓	✓
SheetName uniqueness (trimmed)	optional (after touched)	✓	✓
fieldKey/tableKey uniqueness	not necessary	✓	✓
min <= max	after touched (once both present)	✓	✓
<b>min/max allowed only for NUMBER/CURRENCY</b>	when type/min/max changes	✓	✓
<b>enum allowed only for TEXT</b>	when type/enum changes	✓	✓

This is Approach A fully defined.

## 5) Quick “done right” checklist (so you know it’s fixed)

After implementation:

1. Open Add Table → no red errors
2. Click into Table key then blur without typing → now it shows “required”
3. Enter sheet name with `[` → shows invalid char message after touch
4. Add Field: set Type=TEXT, enter min=10 max=50 → should immediately show error (or auto-clear min/max) **before Save**
5. Try Save Step 2 → should not surprise you with backend-only validation for things FE can catch

## 6) Scalable future option (not now)

Later, if you want near-perfect parity without duplicating logic:

- add backend `/schema/validate` endpoint and call it on Save/Next (not every keystroke)

But for MVP: Approach A + parity rules is enough.

If you confirm **one choice**, I'll proceed to writing the implementation steps (and then code only when you ask):

**Choice:** When user changes field type and existing validations become incompatible →

A) auto-clear incompatible validations (recommended)

B) keep them and show error until fixed

### ▼ Slice

```
SLICE: Step 2 Wizard – Fix Modal Validation UX + Align FE/BE Rules (Approach A + Auto-clean)
```

```
Goal
```

```
In Wizard Step 2 (Schema), fix validation behavior and ensure frontend validation aligns with backend rules, so users do NOT see errors on modal open, and do NOT get backend-only validation surprises at Save/Next.
```

```
This slice MUST implement:
```

- ```
1. "Touched + SubmitAttempt" error display (Approach A)
2. Auto-clear incompatible validations when field `type` changes
3. Full schema validation at least on Save/Next (and optionally after successful modal Add/Edit)
```

4. Frontend validation parity for backend restrictions (enum/min/max allowed types, sheetName uniqueness trim)

Scope

In Scope

- Add Table modal (create/edit)
- Add Field modal (create/edit)
- Step 2 page save/next flow (schema validation gating)
- Shared schema validation utilities (frontend)

Out of Scope

- Backend changes
- API contract changes
- UI redesign beyond validation messaging and small hints
- Deep refactor of schema state architecture (keep it minimal)

Current Issues (must be fixed)

1. Errors appear immediately on modal open (red text), even before user interaction.
2. Add button is disabled until required fields filled, yet error messages still appear on open → contradictory UX.
3. FE validation misses backend rules (e.g., min/max allowed only for NUMBER/CURRENCY). User discovers only at backend save/next.

Behavior Contract (MUST match)

Modal Open (Add Table / Add Field)

- Do NOT show validation errors on open.
- Initialize:
 - submitAttempted=false
 - touched = empty (no field touched)
- Add button remains disabled until required fields are present (keep current behavior).

Validation Display Rule

A field error message is shown ONLY if:

- touched[field] === true OR submitAttempted === true

While Typing

- When user changes a field:

- Mark that field as touched
- Re-run validation (but only show messages for touched fields unless submitAttempted)

On Add click (modal submit)

- Set submitAttempted=true
- Run full modal validation:
 - If valid → close modal and commit
 - If invalid → keep modal open and show errors

On Cancel / X close

- Do NOT run validation.
- If modal is dirty, optional confirmation "Discard changes?" (nice-to-have).

Step 2 Save/Next

- BEFORE calling backend Save/Next API:
 - Run full schema validation (global)
 - If errors exist → block API call + show structured errors immediately
 - If no errors → proceed with API call

Optional but recommended:

- After successful Add/Edit in modal, run schema validation to update step-level "unsaved changes/errors" indicators.

Validation Rules / Restrictions (Frontend MUST enforce to match backend)

Allowed Types

- TEXT, NUMBER, DATE, BOOLEAN, CURRENCY

Table Rules

- tableKey: required (non-empty, trimmed)
- tableKey: unique across schema (case-insensitive recommended; at minimum consistent normalization)
- sheetName: required
- sheetName: trimmed non-empty
- sheetName: max length 31
- sheetName: cannot contain: \ / ? * []
- sheetName: unique across schema AFTER trimming (backend enforces trimmed uniqueness)

Field Rules (per table)

- fieldKey: required (trimmed)
- fieldKey: unique within same table (case-insensitive recommended)
- headerName: required (trimmed)
- type: required and must be one of allowed

Field Validations Rules (type-dependent) – parity with backend

- enum: allowed ONLY for TEXT
- min/max: allowed ONLY for NUMBER or CURRENCY
- if both min and max exist and numeric → enforce min ≤ max

Auto-clean Policy (MUST implement)

When field `type` changes, automatically clear incompatible validations:

If type becomes TEXT:

- CLEAR min, max
- KEEP enum

If type becomes NUMBER or CURRENCY:

- CLEAR enum
- KEEP min/max allowed

If type becomes DATE or BOOLEAN:

- CLEAR enum
- CLEAR min, max

Additionally:

- If user edits enum input to empty/whitespace → treat as absent
- If user clears min/max inputs → treat as absent

UI (recommended):

- Show a small inline non-error hint when auto-clean happens:
 - "Cleared min/max because type is TEXT."
 - "Cleared enum because type is NUMBER."

This hint should NOT be red (not an error).

Error Mapping / Display Requirements

- Errors must be attached to a stable path:
 - Table modal: "tableKey", "sheetName", "order"
 - Field modal: "fieldKey", "headerName", "type", "validations.enum", "validations.min", "validations.max", or "validations" for combined errors
- Step 2 global errors should show:
 - Toast/banner: "Fix schema errors before saving"
 - A list of errors with user-friendly labels:
 - Example: tables[1].fields[1].validations → "Table 't1' Field 't1': min/max only allowed for NUMBER/CURRENCY"

Keep backend error path formatting compatible where possible, but frontend can present friendlier text.

Implementation Tasks (Exact)

Task 1 – Update frontend schema validation utility

Update validateSchema(...) to include missing backend parity rules:

- sheetName uniqueness after trim
- enum allowed only for TEXT
- min/max allowed only for NUMBER/CURRENCY
- keep existing: min<=max, forbidden chars, length, required keys, uniqueness

Add helper:

- sanitizeValidationsByType(type, validations) → returns cleaned validations

Task 2 – Add Table Modal

Implement:

- touched map + submitAttempted flag
- error display gating rule (touched || submitAttempted)
- validate on Add click (submitAttempted=true)
- keep Add button disabled until required fields present
- ensure sheetName errors do not appear on open

Task 3 – Add Field Modal

Implement:

- touched + submitAttempted
- on type change:
 - call sanitizeValidationsByType and update form state
 - optionally show "auto-cleaned" hint
- validate on Add click
- keep Add disabled until required fields present (fieldKey + headerName + type)

Task 4 – Step 2 Save/Next gating

- Before API Save/Next:
 - run validateSchema(tables)
 - if errors exist:
 - block API call
 - display errors
- Ensure that closing modal (successful Add/Edit) updates state so Save/Next uses latest tables.

Acceptance Criteria (Must pass)

Modal UX

- Opening Add Table modal shows NO red errors.
- Opening Add Field modal shows NO red errors.
- Clicking into a required field then leaving it empty (blur) shows its error (touched behavior).
- Add button remains disabled until required fields are filled.
- Clicking Add when non-required validation fails (e.g., forbidden sheet char) keeps modal open and shows error.

Type-dependent rules

- Set Field type=TEXT then enter min/max → either:
 - min/max inputs are auto-cleared immediately, OR
 - if user somehow forces it, validation blocks Add and shows: "min/max only allowed for NUMBER/CURRENCY"
- Set Field type=NUMBER then enter enum → enum auto-cleared (or blocked with error if kept)
- Switch between types repeatedly: sanitization always keeps schema valid.

Save/Next

- If schema has invalid rules (including enum/min/max mismatch), Save/Next must block BEFORE calling backend.
- User should not discover basic type-dependent validation only after backend response.

Test Checklist (Manual)

- Add table: sheetName too long → error after touch or Add
- Add table: sheetName contains `[` → error after touch or Add
- Add table: duplicate sheetName with extra spaces "A" and " A " → caught in FE
- Add field: type TEXT + min/max set → cleared or blocked
- Add field: type NUMBER + enum set → cleared or blocked
- Save/Next with invalid schema (force by editing JSON advanced view if available) → FE blocks

Files to Touch (Guidance)

- Frontend validation utility file (where validateSchema lives)
- AddTableModal component
- AddFieldModal component
- Step 2 Wizard page component (Save/Next handler)
- (Optional) shared helpers/types file

Notes / Constraints

- No backend changes.
- Do NOT introduce heavy new dependencies.
- Keep changes minimal and localized.
- Preserve existing API error handling (backend still source of truth), but FE should prevent common avoidable failures.

▼ Change

Summary of what was implemented:

1. validateSchema.ts (Task 1)

- **Allowed types:** ALLOWED_FIELD_TYPES = TEXT, NUMBER, DATE, BOOLEAN, CURRENCY; type is validated against this list.
- **Sheet name uniqueness:** After trimming, sheetName must be unique across tables (case-insensitive). Duplicates add a clear error.
- **Type-dependent validations:**
 - **enum:** only allowed for TEXT; otherwise error: "Enum is only allowed for type TEXT."
 - **min/max:** only allowed for NUMBER or CURRENCY; otherwise error: "Min/max are only allowed for NUMBER or CURRENCY."
 - If both min and max are set, $\min \leq \max$ is enforced.
- **sanitizeValidationsByType(type, validations):** Returns validations stripped of invalid keys for the given type (e.g. no enum for NUMBER/CURRENCY, no min/max for TEXT/DATE/BOOLEAN).
- Paths kept 1-based for display (e.g. tables[1].sheetName).

2. Add Table modal (Task 2)

- **No errors on open:** touched and submitAttempted are reset when the modal opens (and when existing / schema change).
- **Error display rule:** Table key and sheet name errors are shown only if touched[field] || submitAttempted.
- **Touch on change + blur:** tableKey and sheetName set their touched on onChange and onBlur.
- **Uniqueness:** Table key and sheet name are validated for uniqueness (case-insensitive; sheet name after trim) against other tables (excluding current in edit mode).
- **Submit:** On Add/Save, submitAttempted = true is set first; submit only runs when validation passes; otherwise modal stays open and errors show.
- **Add button:** Disabled only when required fields are missing: !tableKey.trim() || !sheetName.trim() (no longer disabled for non-required validation errors like duplicate name).

3. Add Field modal (Task 3)

- **No errors on open:** Same pattern: touched and submitAttempted reset on open.
- **Error display rule:** Field key, header name, and validation errors show only when the corresponding field is touched or submitAttempted is true.
- **Auto-clean on type change:** handleTypeChange calls sanitizeValidationsByType and updates form state:
 - **TEXT:** clear min/max; keep enum; show hint: "Cleared min/max because type is TEXT."

- → **NUMBER/CURRENCY:** clear enum; keep min/max; show hint: "Cleared enum because type is NUMBER/CURRENCY."
- → **DATE/BOOLEAN:** clear enum, min, max; show hint: "Cleared enum and min/max because type is DATE/BOOLEAN."
- **Hints:** Shown as non-red, small text (text-muted-foreground); cleared when user edits enum/min/max.
- **Type-dependent validation:** Enum-only-for-TEXT and min/max-only-for-NUMBER-or-CURRENCY (and $\text{min} \leq \text{max}$) enforced; errors shown only when validations section is touched or submit was attempted.
- **Field key uniqueness:** Unique within the table (case-insensitive), excluding current row in edit mode.
- **Submit:** Uses `sanitizeValidationsByType` when building the field so only allowed validations are sent.

4. Step 2 Save/Next (Task 4)

- **Already gated:** `handleSave` does if `(!canSave)` return; and the Save button is `disabled={!canSave}`, so the save API is not called when there are schema errors.
- **Banner text:** Updated to **"Fix schema errors before saving"** and the list of errors still shows path + message.
- **Validation source:** `validationErrors` comes from `validateSchema(schema.tables)` (useMemo), so it includes the new rules (sheetName uniqueness, allowed types, enum/min/max by type). After Add/Edit in a modal, `schema.tables` updates and the banner/indicators update automatically.

Files changed

- **frontend/src/features/schema/lib/validateSchema.ts** – Full schema validation + `sanitizeValidationsByType`, type and uniqueness rules.
- **frontend/src/features/schema/components/SchemaEditorView.tsx** – `TableModal` and `FieldModal` (touched/submitAttempted, auto-clean, hints), and Save/Next banner text.

Manual checks you can run:

- Add table: duplicate sheet name (e.g. "A" and " A ") → error after touch or Add.
- Add field: type TEXT then set min/max → cleared or blocked; type NUMBER then set enum → cleared or blocked.
- Save/Next with invalid schema (e.g. invalid type or enum on NUMBER in JSON) → FE blocks and shows "Fix schema errors before saving" with the list of errors.

▼ enhance 2

▼ Slice

```
# SLICE: Step 2 Wizard – Fields Grid UI + Fix Field CRUD + Add Default Required Fields

## Goals
1) Change Step 2 "Fields" section UI from a TABLE to a GRID of cards.
2) Show Edit/Delete actions only on hover (like CTO template reference).
3) FIX: Add/Edit/Delete Field must work correctly (no workaround like deleting the whole table).
4) Add "Default Fields" feature in Add Table modal: Year/Month/Product Name.
   - If selected, they are added automatically and marked Required=true.

---
```

```

## Scope
### In scope
- Step 2 Schema page: Fields rendering area
- Field CRUD handlers (add/edit/delete)
- Add Table modal: add "Default Fields" checkboxes and create those fields on Add

### Out of scope
- Backend changes
- Export step changes
- Introducing schema metadata for locked/system fields (future)

---

## UI Requirements

### A) Fields section = Grid cards (replace table)
- Replace fields table with a grid layout:
  - responsive: 1 column mobile, 2-3 columns desktop
- Each Field Card displays:
  - Header Name (title)
  - Field Key (small text)
  - Type (small text)
  - Required badge if field.required === true (e.g., "* Required" like reference)

### B) Hover actions
- Edit/Delete buttons should be hidden by default and appear on hover:
  - Use hover/opacity transition
  - Buttons should be on the right top corner of the card OR bottom-right
- Clicking Edit opens existing Add/Edit Field modal pre-filled.
- Clicking Delete removes the field immediately (optional confirm).

### C) Selected table
- Grid shows fields for currently selected table.
- If no table selected: show empty state "Select a table to view fields".

---

## Functional Requirements (critical)

### 1) FIX Field CRUD (must work)
Current issue: add/delete not reflected or broken.

Implementation MUST ensure:
- `tables` state is the single source of truth.
- No in-place mutation of tables or fields.
- Selection uses stable identifier:
  - `selectedTableKey` (or `selectedTableId`) NOT index.
- All operations update state immutably:
  - Add Field: clone tables → update selected table → setTables(newTables)
  - Edit Field: replace by stable field identity (fieldKey recommended)
  - Delete Field: remove by field identity (fieldKey recommended)

Acceptance criteria:
- Add Field -> card appears immediately

```

- Edit Field -> card updates immediately
- Delete Field -> card disappears immediately
- Switching selected table preserves correct fields

2) Field identity rules

- fieldKey is unique within the table (already required by validation).
- Use fieldKey as stable identity for edit/delete operations.

Default Fields Feature (Add Table modal)

UX

In Add Table modal, add section:

- "Add Default Fields:" with checkboxes
 - [x] Year Field
 - [x] Month Field
 - [x] Product Name Field

Behavior

When user clicks "Add Table":

- Create the table record.
- If any default fields selected:
 - Append them to the new table.fields list.
 - They must be REQUIRED=true.
- Defaults MUST be skipped if they would conflict with existing fieldKey in that table (edge case if editing, or if modal reused):
 - If conflict: either auto-adjust key or block add with clear error.
 - Recommended now: block with message "Default field key already exists: year" (simple + safe).

Default Field Definitions (recommended)

- Year:
 - fieldKey: "year"
 - headerName: "Year"
 - type: NUMBER
 - required: true
 - validations: optional (min/max not required)
- Month:
 - fieldKey: "month"
 - headerName: "Month"
 - type: TEXT
 - required: true
 - validations.enum: optional future enhancement (Jan..Dec); not required now
- Product Name:
 - fieldKey: "product_name"
 - headerName: "Product Name"
 - type: TEXT
 - required: true

NOTE: Field keys must respect existing format rules (lower_snake_case recommended).

```

---

## Validation/Restrictions (must remain consistent)
- Required fields (field.required boolean) must be stored in schema and reflected in UI and JSON.
- Existing backend rules still apply:
  - enum only for TEXT
  - min/max only for NUMBER/CURRENCY
  - min <= max
- Default fields must not violate backend rules.

---

## Edge Cases
1) Deleting the only field in a table:
  - Allowed. Table can have zero fields.
2) Deleting a required default field:
  - Allowed for MVP (future: lock system fields).
3) Switching table selection while modal open:
  - Modal should edit the intended table/field; selection changes shouldn't corrupt edit target.
4) Editing fieldKey:
  - Must re-check uniqueness within the table.
  - If fieldKey changes, update identity correctly.
5) "Advanced: view JSON" manual edits:
  - Step-level schema validation on Save/Next still blocks invalid schema.

---

## Acceptance Criteria
- Fields are displayed as cards in a grid, not a table.
- Hover actions appear only when hovering field card.
- Add/Edit/Delete Field works without needing to delete the whole table.
- Add Table modal can create default required fields, and UI shows Required badge on them.
- Required flag persists in schema JSON and survives Save/Next.

END

```

▼ Change

Summary of what was implemented:

1. Fields section: table → grid of cards

- Replaced the fields **table** with a **responsive grid**:
- 1 column on mobile, 2 on sm, 3 on lg (grid-cols-1 sm:grid-cols-2 lg:grid-cols-3).
- Each **field card** shows:
 - **Header name** as the main title
 - **Field key** in small monospace text

- **Type** in small text
 - **"* Required"** badge when `field.required === true`
 - **Empty state** when no table is selected: "Select a table to view fields."
-

2. Hover actions (Edit / Delete)

- Edit and Delete are **hidden by default** and **shown on card hover** via `opacity-0 / group-hover:opacity-100`.
 - Actions are in the **top-right** of each card (absolute right-2 top-2).
 - **Edit** opens the existing Add/Edit Field modal with that field.
 - **Delete** opens the existing confirmation modal, then removes the field.
-

3. Field CRUD and selection

- **Selection** is by **table key**: `selectedTableKey` (string) instead of index.
 - `selectedTable = schema.tables.find(t => t.tableKey === selectedTableKey)`.
 - Table list highlights by `selectedTableKey` and calls `setSelectedTableKey(t.tableKey)`.
 - **No in-place mutation**: all updates go through `updateTables(updater)` with new arrays/objects.
 - **Field identity** is **fieldKey** (unique per table):
 - **Add field**: `append` to the table identified by `selectedTableKey` (or by `fieldModalTableKey` when the modal was opened).
 - **Edit field**: `editField(previousFieldKey, field)` finds the table by key and replaces the field with `fieldKey === previousFieldKey`.
 - **Delete field**: `deleteField(tableKey, fieldKey)` finds the table by key and filters out that field.
 - **Modal target** is fixed when opening:
 - `fieldModalTableKey` stores the table key when opening Add/Edit field modal so changing the selected table does not change which table the modal edits.
 - **Delete target** is fixed when opening the delete confirm:
 - `deleteTargetTableKey` and `deleteTargetFieldKey` store table and field so confirming delete removes the correct field even if selection changed.
-

4. Default fields in Add Table modal

- In the **Add table** modal (only when not editing), added **"Add default fields"** with three checkboxes:
 - **Year Field** (default on)
 - **Month Field** (default on)
 - **Product Name Field** (default on)
 - On **Add Table** submit:
 - If a checkbox is checked, the corresponding field is appended to the new table with **required: true**:
 - **Year**: `fieldKey: 'year', headerName: 'Year', type: 'NUMBER', required: true`
 - **Month**: `fieldKey: 'month', headerName: 'Month', type: 'TEXT', required: true`
 - **Product Name**: `fieldKey: 'product_name', headerName: 'Product Name', type: 'TEXT', required: true`
 - New table is created with fields: `[]` then these default fields are pushed; for a brand-new table there is no conflict. No extra conflict handling was added for this flow.
-

5. Other behavior

- **Required** is already in buildPutPayload and is sent in the schema JSON and reflected in the UI (card badge and field modal checkbox).
- **Table list** uses `key={t.tableKey}` for stability.
- **Field grid** uses `key={f.fieldKey}` for list identity.
- Unused **Table** UI imports (DataTable, TableHead, TableBody, etc.) were removed.

Files touched

- **frontend/src/features/schema/components/SchemaEditorView.tsx**: selection by selectedTableKey, field CRUD by table key + fieldKey, grid of field cards with hover Edit/Delete, fieldModalTableKey / deleteTargetTableKey for correct modal/delete target, Add Table default-fields checkboxes and creation logic, and removal of table-based UI imports.

Manual checks you can do:

- Add table with default fields checked → new table has Year, Month, Product Name with “* Required” on their cards.
- Add / Edit / Delete field → card appears, updates, or disappears; switching table and then acting in the modal still targets the original table.
- Delete field from confirm → correct field is removed even if you changed the selected table before confirming.

▼ enhance 3

▼ workflow conversation

Approach 1 — UI Replica + Minimal Functionality

(Fast but shallow)

What you implement

Frontend replicates the CTO export screen:

- Stats cards (tables, fields, size)
- Export options panel
- Format selector
- File name/version inputs
- Save settings
- Export button

Backend

Only:

- Export `.xlsx`
- Apply blue header styling

Everything else:

- either ignored
- or disabled in UI.

Pros

- Very fast
- Looks impressive in demo

Cons

- CTO may quickly notice options don't actually work
- Technical debt later

Conclusion: Not recommended if CTO wants functionality.

Approach 2 — Functional Export Studio (Balanced) ★

Recommended

This is the **closest match to CTO template UX while staying realistic**.

You implement the **core export pipeline features** that users actually notice.

Frontend Responsibilities

1 Export Dashboard

Compute locally:

- Number of tables
- Number of fields
- Estimated file size

These come directly from the schema already loaded in Step 3.

2 Export Options (Real)

Implement these for real:

| Option | Where implemented |
|-------------------------------|-------------------|
| Include Instructions Sheet | Backend |
| Include Data Validation Rules | Backend |
| Protect Sheets from Editing | Backend |

These options change the Excel generation.

3 File Format Selection

Support only:

XLSX

Other formats displayed but disabled.

Reason:

- `.xls` is legacy
- `.xlsm` introduces macro security issues

4 Additional Settings

Frontend controls:

- File name
- Template version (readonly)

Settings are sent with export request.

Optional: Save settings button persists them.

5 Export Flow

Frontend calls:

```
POST /templates/{id}/versions/{v}/export
```

with:

```
ExportSettings
```

Example structure:

```
{
  format: "XLSX",
  includeInstructionsSheet: true,
  includeValidationRules: true,
  protectSheets: false,
  fileName: "Cement Industry Data Template"
}
```

Backend Responsibilities

Backend is responsible for **all Excel intelligence**.

1 Blue Header Styling (Always Applied)

Every sheet header row should have:

- Light blue fill
- Bold font
- Borders
- Freeze first row

This should **always be applied**, not optional.

Why:

Users immediately perceive the template as polished.

2 Instructions Sheet

If enabled:

Create sheet:

```
Instructions
```

Content:

| Table | Field | Type | Required | Validation |
|-------|-------|------|----------|------------|
|-------|-------|------|----------|------------|

Example:

```
Local Sales | Year | NUMBER | YES | Min:2000 Max:2100
Local Sales | Month | TEXT | YES | Enum: Jan..Dec
```

This becomes **documentation inside Excel**.

3 Data Validation Rules

Map schema validations to Excel rules:

| Schema Rule | Excel Feature |
|-------------|--------------------------|
| Enum | Dropdown list |
| Min/Max | Numeric validation |
| Required | Highlight column or note |

Note:

Excel cannot enforce “required” strictly without macros.

So we visually mark required columns.

4 Sheet Protection

If enabled:

- Lock header cells
- Lock validation formulas
- Unlock data entry cells

Protect sheet with default password or no password.

5 File Generation

Return:

```
application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
```

file.

Approach 3 — Enterprise Export Engine

(Most scalable but overkill now)

Adds:

- Export jobs queue
- Export history
- XLSM macros
- Compression
- Advanced formula engine

Used when:

- templates become large

- exports take seconds/minutes

Not necessary now.

My Senior Recommendation

Implement **Approach 2 (Functional Export Studio)**.

It gives you:

- UI identical to CTO template
- Core functionality working
- Clean architecture
- Low risk

And it avoids building complex features prematurely.

One More UX Improvement I Strongly Recommend

In Step 3 add **Preview Panel**:

```
Sheets to be generated
-----
Local Sales
Production Cost
Inventory
...
```

Users see exactly what Excel will contain.

Small feature, huge clarity.

Final Architecture (Clean)

Frontend

Handles:

- UI
- option selection
- stats
- preview

Backend

Handles:

- Excel generation
 - validations
 - styling
 - protection
 - instructions sheet
-

If you want, I can also show you the ideal Excel layout your generator should produce, including:

- sheet structure
- header styling
- validation examples
- instructions sheet layout

This will make the export **feel like a professional data collection template**, not just a raw spreadsheet.

▼ Contract

Great — we'll go with **Approach 2: Functional Export Studio (Balanced)**.

Below is the **exact slice** (agent-ready) in a code block, with **frontend vs backend**, all rules/restrictions, edge cases, and acceptance criteria. **No code.**

```
# SLICE: Step 3 Export Studio (CTO-like UI + Real Export Options + Blue Header Styling)
```

```
## Goal
```

```
Upgrade Wizard Step 3 from a simple "Export" button to a CTO-style "Export Studio" UI and implement core export functionality end-to-end:
```

- Polished export UI (dashboard cards, options, settings)
- REAL backend-backed options:
 - 1) Include Instructions Sheet
 - 2) Include Data Validation Rules
 - 3) Protect Sheets from Editing
- Always apply a professional Excel header style (light blue header row) on all generated sheets.

```
---
```

```
## Scope
```

```
### In Scope
```

- Frontend Step 3 page UI: replicate CTO export experience
- Export settings model and API payload contract
- Backend export generator enhancements:
 - Header styling (always)
 - Instructions sheet (optional)
 - Data validation rules (optional)
 - Sheet protection (optional)
- (Optional) Persist export settings per template version

```
### Out of Scope (show as disabled "Coming soon")
```

- Include Macros (VBA) / .xlsm actual macros
- Excel 97-2003 (.xls) format generation
- Compress file
- Include formulas (unless already supported; keep as coming soon)
- Async export jobs (future)

```
---
```

```
## Frontend Requirements (UI/UX)
```

```

### 1) Page Layout (CTO-like)
Create "Export Excel Template" panel with:
A) Top summary cards:
- Number of Tables
- Number of Fields
- File Size (Estimated)

B) Export Options (checkboxes):
- [x] Include Instructions Sheet (REAL)
- [x] Include Data Validation Rules (REAL)
- [ ] Protect Sheets from Editing (REAL)
- [ ] Include Formulas (COMING SOON - disabled + tooltip)
- [ ] Include Macros (VBA) (COMING SOON - disabled + tooltip)
- [ ] Compress File to Reduce Size (COMING SOON - disabled + tooltip)

C) Excel file format radio group:
- (o) Excel Workbook (.xlsx) (ONLY enabled)
- ( ) Excel 97-2003 (.xls) (COMING SOON - disabled)
- ( ) Excel Macro-Enabled (.xlsm) (COMING SOON - disabled)

D) Additional settings:
- File Name (editable)
- Template Version (read-only; display current version)
- (Optional) "Save Settings" button

E) Footer actions:
- Previous (Back to Step 2)
- Save Settings (optional but recommended)
- Export Excel File (primary)

### 2) Metrics computations
Compute locally from schema:
- tablesCount = tables.length
- fieldsCount = sum(table.fields.length)
- fileSizeEstimate:
  - show as "Estimate" (e.g., "≈ 20 KB")
  - MUST NOT claim exact unless backend returns size
  - Simple heuristic acceptable (based on fieldsCount and tablesCount)

### 3) Behavior rules
- Export button triggers export request with selected options.
- While exporting:
  - disable Export button
  - show spinner/loading state
- On success:
  - trigger file download
  - show success toast: "Exported Excel file"
- On failure:
  - show error toast with backend message
- Save Settings (if implemented):
  - persists settings for this template version
  - reload Step 3 should prefill last saved settings

### 4) Validation rules (frontend)

```

```

- File name:
  - required, trimmed, non-empty
  - safe filename (strip/replace illegal characters for OS if needed)
- Format:
  - must be XLSX (others disabled)

---

## Backend Requirements (Export Engine Enhancements)

### 1) ExportSettings contract (API)
Export endpoint accepts settings payload:
- format: "XLSX" (only supported now)
- fileName: string (optional; fallback server-side)
- includeInstructionsSheet: boolean
- includeDataValidationRules: boolean
- protectSheets: boolean

Backend may also accept:
- templateId, version (already part of route)

### 2) Always apply "Blue Header Styling" (mandatory)
For every generated table sheet:
- Header row = first row containing field header names
- Apply:
  - Light blue fill (professional, readable)
  - Bold font
  - Borders for header cells
  - Vertical alignment centered
  - Freeze top row (row 1)
  - Optional: auto-filter enabled on header row
- Apply consistent column widths:
  - either autosize with limit
  - or set reasonable defaults

IMPORTANT:
- This style is NOT optional (always applied).

### 3) Instructions Sheet (REAL when enabled)
If includeInstructionsSheet=true:
- Add a first worksheet named: "Instructions"
- Include:
  - Template name, sector code, version
  - A table listing all fields:
    Columns:
      TableKey | SheetName | FieldKey | HeaderName | Type | Required | Validations
  - "Validations" column should include:
    - enum: A,B,C (if present)
    - min/max: min=.. max=.. (if present)

Rules:
- Sheet name must comply with Excel constraints (<=31 chars, no invalid chars).
- If "Instructions" conflicts, use "Instructions_1" etc.

```

```

### 4) Data Validation Rules (REAL when enabled)
If includeDataValidationRules=true:
Map schema validations to Excel validations:

A) TEXT + enum:
- Add Excel dropdown list validation for that column
- Handle enums safely:
  - trim entries
  - ignore empty values
  - if too long for Excel inline list limits, fallback to hidden list sheet (future-friendly; optional now)
  - At minimum: if exceeds safe limit, skip with warning note in Instructions sheet

B) NUMBER/CURRENCY + min/max:
- Add numeric validation constraint:
  - allow only numbers within range
- If only min or only max exists: apply single-sided constraint

C) Required fields:
Excel cannot strictly enforce "required" without macros.
Implement at least ONE of:
- Visual cue:
  - Add a note or color indicator on required headers (e.g., asterisk in header text "Year *")
  - OR header cell comment "Required"
Recommended now: append " *" to header text for required fields OR add a small required marker styling.

IMPORTANT:
- MUST remain consistent with backend schema validation rules:
  - enum only for TEXT
  - min/max only for NUMBER/CURRENCY
  - min <= max

### 5) Protect Sheets from Editing (REAL when enabled)
If protectSheets=true:
- Protect each data sheet:
  - Lock header row cells always
  - Unlock data entry cells (so users can fill data)
- Decide data entry region:
  - Row 2..N for all columns
- Use sheet protection without requiring a password (or a fixed internal password).
- Ensure protection does not block dropdowns (validations) or typing.

### 6) Persistence (optional but recommended)
Add ability to save export settings per template version:
- Save settings endpoint or reuse template version update endpoint
- On Step 3 load: return saved settings so UI pre-fills

---

## Rules / Restrictions / Compatibility

```


- Only XLSX supported now.
- Do not implement macros (.xlsm) now.
- Do not implement legacy .xls now.
- Keep export stable and deterministic.
- Must not break existing export flow (fallback if settings omitted).

Edge Cases

- 1) Sheet names collisions:
 - If two tables produce same sheetName after trimming, backend already rejects schema; export assumes valid schema.
- 2) Large enum lists:
 - If enum list too long for Excel validation inline:
 - Option A (simple now): skip and document in Instructions sheet.
 - Option B (better): create hidden "Lists" sheet and reference ranges.
 - Recommended now: Option A unless CT0 expects full fidelity.
- 3) Protect sheets + validations:
 - Ensure validations still usable under protection (unlock data cells).
- 4) Required marker:
 - If you choose to append "*" to headerName, ensure it doesn't break downstream mapping (headerName is just display).
- 5) File name:
 - Backend must sanitize file name (avoid invalid chars), default to "Template_<id>_v<version>.xlsx"

Acceptance Criteria (Demo-ready for CT0)

UI:

- Step 3 page matches CT0-style layout (cards, options, settings, buttons).
- Tables/fields counts are correct.
- Options show enabled/disabled correctly with "Coming soon" tooltips.

Export output:

- Exported XLSX downloads successfully.
- All sheets have a styled light-blue header row + frozen top row.
- If Instructions enabled: Instructions sheet exists and lists tables/fields with validations.
- If Validation enabled:
 - TEXT enum columns have dropdown validation (at least for small enums).
 - NUMBER/CURRENCY min/max columns have numeric range validation.
 - Required fields show an indicator (asterisk or comment).
- If Protect enabled:
 - Users can edit data cells but not header row.
 - Dropdowns still work.

Recommended Implementation Order

- 1) Frontend Step 3 UI + payload wiring (send exportSettings)
- 2) Backend blue header styling + freeze row
- 3) Instructions sheet
- 4) Data validations (enum + min/max + required marker)

- 5) Sheet protection
- 6) Save settings persistence (optional if time)

END

Before we turn this into separate slices (FE slice + BE slice), one key decision that affects effort:

Enum validation for large lists: do you want the “hidden Lists sheet” approach now (more work, more correct), or skip large enums with a note in Instructions (faster)?

▼ Slice

```
# SLICE: Wizard Step 3 – Export Studio (CT0-style UI + Real Options + Blue Header Styling)
```

```
## Goal
```

Upgrade Wizard Step 3 from a simple export page into an “Export Studio” like the C T0 template:

- Rich UI: stats cards, options, format selection (XLSX now), filename/version, preview
- Real backend effects for core options:
 - 1) Include Instructions Sheet (REAL)
 - 2) Include Data Validation Rules (REAL)
 - 3) Protect Sheets from Editing (REAL)
- Always apply a polished Excel look:
 - Light blue header row style + freeze first row

```
---
```

```
## Scope
```

```
### In scope (Frontend)
```

- Replace Step 3 UI layout to match CT0 template (cards + options panel)
- Compute and display schema stats (#tables, #fields, estimated size)
- Options selection UI + Save Settings button
- Export button calls backend with export settings payload
- Show “coming soon” options disabled with tooltips (no fake behavior)

```
### In scope (Backend)
```

- Accept export settings in the export request (XLSX only)
- Apply 3 options in workbook generation (real behavior)
- Apply header styling (light blue) and freeze top row on all table sheets
- Add tests verifying the workbook changes (instructions sheet, validations presence, sheet protection, header style basics)

```
### Out of scope
```

- XLS / XLSM formats (visible but disabled)
- Macros (VBA)
- Compression optimizations
- Async export jobs / export history
- Formula engine

```
---
```

```
## UX Requirements (Frontend)
```

A) Layout: Export Studio

Page sections:

- 1) Header: "Export Excel Template" + description
- 2) Stats cards row:
 - Number of Tables
 - Number of Fields
 - File Size (Estimated)
- 3) Export Options block (checkbox tiles)
- 4) Excel File Format block (radio tiles)
- 5) Additional Settings block:
 - File name (editable)
 - Template version (readonly)
- 6) Bottom actions:
 - Previous (Back to Step 2)
 - Save Settings
 - Export Excel File

B) Stats calculation rules

- tablesCount = schema.tables.length
- fieldsCount = sum(schema.tables[i].fields.length)
- estimatedFileSize:
 - MUST be labeled "Estimated"
 - Use a simple heuristic (no backend call required)
 - Do not claim exact size unless backend returns it

C) Options behavior

Real (enabled)

- Include Instructions Sheet
- Include Data Validation Rules
- Protect Sheets from Editing

Coming soon (disabled with tooltip)

- Include Formulas
- Include Macros (VBA)
- Compress File to Reduce Size

Disabled options MUST:

- be visible but disabled
- show tooltip "Coming soon"
- not be sent to backend (or can be sent as false; but do not pretend supported)

D) File format behavior

- Only "Excel Workbook (.xlsx)" selectable
- "Excel 97-2003 (.xls)" and "Excel Macro-Enabled (.xlsm)" visible but disabled with tooltip "Coming soon"
- Backend must enforce XLSX only

E) Save Settings behavior

Preferred:

- Persist export settings per TemplateVersion via backend (recommended for CT0 parity)

Minimum acceptable:

- Persist in local storage keyed by templateId + versionId (but less CT0-like)

```

### F) Export behavior
- Export button calls backend export endpoint with settings payload
- Disable Export button + show loading state while exporting
- On success: download file using filename rules (below)
- On failure: show error toast/panel

---

## Backend Requirements (Real Functional Options)

### A) Export endpoint contract (XLSX only)
- Accept ExportSettings as request payload (or query params if existing pattern)
- Support fields:
  - format: "XLSX" (only)
  - fileName (optional)
  - includeInstructionsSheet: boolean
  - includeValidationRules: boolean
  - protectSheets: boolean

If format != XLSX → return 400 with clear message.

### B) Workbook generation (core)
Existing behavior remains:
- One sheet per table
- Row 1 = headers

Enhancements:

#### 1) Blue header styling (ALWAYS applied)
For each table sheet:
- Header row (row 1):
  - light blue fill
  - bold font
  - centered vertical alignment
  - borders
- Freeze first row (pane at row 2)
- Optional: set reasonable column widths (do not over-optimize)

#### 2) Instructions sheet (when enabled)
- Add first sheet named "Instructions" (or "README")
- Structure as a simple table:
  - Table (sheetName)
  - Field headerName
  - fieldKey
  - type
  - required
  - validations summary (enum count, min/max)
- Keep it readable and minimal (no heavy formatting required)

#### 3) Data validation rules (when enabled)
Map schema to Excel validations:
- enum (TEXT only):
  - add dropdown validation to a row range (e.g., rows 2..1000)
- min/max (NUMBER/CURRENCY only):

```

- add numeric validation constraint (rows 2..1000)
- required:
 - Excel cannot strictly enforce required without macros; do NOT attempt fake enforcement
 - Optional: visually mark required columns by adding "*" in header (if already done) and/or slightly stronger header style

If schema contains incompatible validations:

- backend already validates schema earlier; export should still guard and fail gracefully if invalid

4) Protect sheets (when enabled)

- Protect each data sheet
- Recommended rule:
 - Keep header row locked
 - Unlock data input range (rows 2..1000, all columns used)
 - Protect with no password or a fixed internal password (document it if used)

File Naming Rules

Frontend default file name:

- `\${templateName}\_v\${versionNumber}.xlsx`

If user enters fileName:

- sanitize to remove illegal filename chars
- ensure `.xlsx` suffix

Backend should also set Content-Disposition accordingly.

Acceptance Criteria

UI

- Step 3 visually resembles CTO template structure (cards + option tiles + format tiles + bottom actions)
- Disabled "coming soon" options are clearly disabled with tooltip
- Stats show correct tables/fields counts
- Estimated file size is clearly labeled

Functional behavior

- Export with "Instructions Sheet" enabled produces workbook with "Instructions" sheet
- Export with "Include Data Validation Rules" enabled produces dropdown/numeric validations where applicable
- Export with "Protect Sheets" enabled results in protected sheets (verify in Excel)
- Header row in every data sheet is light blue styled and frozen

Non-regression

- Export still works with zero tables/fields (should produce workbook with at least metadata + maybe instructions if enabled)
- No new npm dependencies introduced unnecessarily for Step 3 UI

```

- No backend architectural boundary violations (POI logic stays in infrastructure/
excel)

---

## Testing Requirements (Backend)
Add/extend tests to verify:
1) Instructions sheet exists when enabled
2) Header style applied (basic assertions: cell style fill color not default; font
bold)
3) Freeze pane exists on each sheet
4) Protection enabled when protectSheets=true
5) Validations exist when enabled (basic POI inspection; keep it minimal)

---

## Rules / Restrictions (Must follow)
- Implement only XLSX now; show other formats as disabled
- Do not implement macros (VBA) or compression in this slice
- Do not add async jobs in this slice
- Do not change schema validation rules here (export assumes schema is valid)
- Keep UI options honest: if backend doesn't support it, disable it

END

```

▼ enhance 4

- modify cell size and add enum validations

▼ enhance 5

▼ workflow conversion

You now have **type = TEXT** working end-to-end, but the others are still only "labels" in the UI, not a real validation system.

The key design question is:

Do you want **true business validation of entered values inside uploaded Excel later**, or only **Excel template guidance/restrictions during export** right now?

From your current system, you are still in **template-generation mode**, not upload/ingestion mode. So the right move is to implement **Excel-side validation/guidance** first, in a clean, limited way.

Recommended approach now: Type-aware export validation MVP

This is the best balance for your current stage.

Implement validation in **three layers**:

1) Frontend modal behavior

When user selects field type, show only the validations that make sense for that type.

2) Backend schema validation

Reject invalid validation combinations early.

3) Excel export behavior

Translate supported validations into actual Excel constraints where possible.

This gives you a coherent system without overbuilding.

What I recommend per type

1) TEXT

You already have:

- enum dropdown in Excel
- frontend modal enum input
- backend validation that enum is allowed only for TEXT

This is good enough for now.

Optional next later

- minLength / maxLength
- regex / pattern
- email / phone specialized formats

But I would **not** add those now.

2) NUMBER

This should be the next most complete type after TEXT.

What to support now

- `min`
- `max`

Frontend

- keep Min / Max fields visible for NUMBER
- validate:
 - must be numeric
 - $\text{min} \leq \text{max}$

Backend

- already mostly there conceptually
- enforce min/max only for NUMBER/CURRENCY

Excel export

Apply numeric validation:

- if both → between
- if min only → greater than or equal
- if max only → less than or equal

Why this is good

This is a **real restriction** Excel understands well.

3) CURRENCY

This is where many systems make a mistake.

Simple approach

Treat CURRENCY like NUMBER for validation logic:

- same `min`
- same `max`

And in Excel:

- use numeric validation exactly like NUMBER
- optionally apply a **currency number format** to the column

Recommendation

Do this now.

Important note

Currency is usually not a different validation type; it is:

- numeric constraints
- plus formatting/semantic meaning

So:

CURRENCY = NUMBER + display format

This is the clean design.

4) DATE

This one needs care.

There are **three possible levels**:

Approach A — Simple UI only

Let user choose DATE type, but no extra validation beyond “this is a date field”.

Approach B — Real Excel date validation (recommended now)

Support:

- `minDate`
- `maxDate`

Not `min` / `max`, because those are numeric semantics and become confusing.

Frontend

When type = DATE:

- hide numeric Min / Max
- show:
 - Earliest date

- Latest date

Backend

Add validation rules specific to DATE:

- date range allowed only for DATE
- minDate <= maxDate

Excel export

Use Excel date validation constraint:

- between two dates
- or >= / <= if one side only

Tradeoff

- more fields and DTO changes than NUMBER/CURRENCY
- but much cleaner than reusing numeric min/max for dates

My recommendation

For now, I would do **DATE basic only**, then optionally add date range in the next slice.

If you are tired and want steady progress, do not bundle DATE range with NUMBER/CURRENCY in one big slice.

5) BOOLEAN

This is simpler than it looks.

Best practical approach now

Represent BOOLEAN in Excel as a dropdown:

- Yes / No
- or
- True / False

Recommendation

Use **Yes / No** for better business readability.

Backend modeling options

Option A — Implicit boolean dropdown (recommended now)

If type = BOOLEAN:

- backend export automatically creates dropdown [Yes, No]
- no need for user-entered enum
- frontend doesn't show enum field for BOOLEAN

This is the cleanest MVP.

Option B — Boolean stored as enum

Treat BOOLEAN as just enum [Yes, No]

- messy conceptually
- blurs TEXT vs BOOLEAN semantics

I do **not** recommend that.

Best implementation strategy (incremental)

Option 1 — One big slice for all types

Implement NUMBER, CURRENCY, DATE, BOOLEAN together.

Pros

- one full validation push
- unified design

Cons

- high risk of bugs
- more backend/frontend changes at once
- harder to debug

I do **not** recommend this right now.

Option 2 — Two-step rollout (recommended)

Slice A

Implement:

- NUMBER → min/max
- CURRENCY → min/max + currency formatting
- BOOLEAN → Yes/No dropdown

Slice B

Implement:

- DATE → date validation + optional min/max date range

Why this is better

- NUMBER/CURRENCY/BOOLEAN are much more straightforward
- DATE is always a little trickier in Excel/POI
- gives you visible progress faster

This is my recommendation.

Option 3 — Scalable validation model redesign

This is the “future-proof” version.

Instead of current generic:

```
validations: { enum, min, max }
```

move toward a **typed validation model**, e.g.

```
validations: {  
  text: { enumValues: [...] },
```

```
number: { min: 0, max: 100 },
date: { minDate: "2026-01-01", maxDate: "2026-12-31" },
boolean: { labels: ["Yes", "No"] }
}
```

Pros

- very scalable
- avoids semantic confusion
- clean separation per type

Cons

- requires refactoring FE types, DTOs, backend validator, export builder
- too heavy for your current MVP pace

I would **not** do this now.

My recommended design now

Implement now

NUMBER

- show min/max
- backend enforce numeric min/max
- Excel numeric validation

CURRENCY

- same as NUMBER
- optional cell format for currency

BOOLEAN

- auto dropdown Yes / No
- no custom enum input needed

Defer one slice

DATE

- either basic DATE now with no extra rules
 - or next slice: add minDate/maxDate properly
-

UI behavior I recommend in the modal

When type changes, the validation area should adapt:

TEXT

- show enum input
- hide min/max

- hide date range

NUMBER

- hide enum
- show min/max numeric

CURRENCY

- hide enum
- show min/max numeric
- label fields as "Min amount / Max amount" if you want nicer UX

DATE

- hide enum
- hide numeric min/max
- show date range fields only if you implement them

BOOLEAN

- hide enum
- hide min/max
- show note:
 - "This field will export as Yes/No dropdown"

This is much cleaner than showing irrelevant inputs for all types.

Tradeoffs summary

Simplest possible

Treat NUMBER/CURRENCY the same, BOOLEAN as fixed dropdown, DATE postponed.

Pros

- fast
- low risk
- matches your current architecture

Cons

- DATE remains incomplete temporarily

More complete

Add DATE range now too.

Pros

- all types covered in one go

Cons

- more work
- higher bug probability

- Excel date handling is more fragile

Final recommendation

Implement now:

NUMBER + CURRENCY + BOOLEAN

with:

- frontend type-aware validation fields
- backend schema validation
- Excel export restrictions

Then implement next:

DATE

with proper date-range semantics, not numeric min/max hacks.

That is the most senior, low-risk path for where your app is now.

If you want, I'll turn this into an **exact slice sequence**:

- Slice 1: NUMBER + CURRENCY + BOOLEAN
- Slice 2: DATE
- Slice 3: UI polish for type-aware validation fields

▼ slice

```
# SLICE: Step 2 + Step 3 Validation Expansion – NUMBER + CURRENCY + BOOLEAN

## Goal
Extend the current validation system beyond TEXT so that:
1) NUMBER fields support real min/max validation
2) CURRENCY fields support real min/max validation and export as numeric cells with currency-style formatting
3) BOOLEAN fields export as a fixed Yes/No dropdown
4) Step 2 modal UI only shows validation inputs relevant to the selected type
5) Backend schema validation and Excel export stay aligned

This slice intentionally does NOT implement DATE range yet.

---

## Why this slice
TEXT enum dropdown is already working.
The next safest expansion is:
- NUMBER
- CURRENCY
- BOOLEAN

These are much simpler and less fragile than DATE validation in Excel.

---
```

```

## Scope

### In scope
- Step 2 field modal UI behavior
- Frontend validation logic
- Backend schema validation rules
- Backend Excel export rules for NUMBER / CURRENCY / BOOLEAN

### Out of scope
- DATE-specific validations (minDate/maxDate)
- Upload/ingestion validation
- Advanced text validations (regex, minLength, maxLength)
- Formula engine
- Macros / XLS / XLSM

---

## Functional Requirements

# 1) NUMBER
## UI behavior
When field type = NUMBER:
- show Min input
- show Max input
- hide Enum input

## Frontend validation
- Min and Max must be valid numbers if provided
- If both exist: min <= max
- Enum must not be allowed for NUMBER

## Backend schema validation
- Keep/ensure rule:
  - min/max allowed for NUMBER
  - enum not allowed for NUMBER

## Excel export
For NUMBER fields:
- if min and max exist → apply numeric validation BETWEEN
- if only min exists → apply numeric validation >= min
- if only max exists → apply numeric validation <= max

Validation must:
- apply to data rows only
- show Excel error box
- block invalid values (STOP)
- keep empty cells allowed unless field is later enforced by another mechanism

---

# 2) CURRENCY
## UI behavior
When field type = CURRENCY:
- show Min input

```

- show Max input
- hide Enum input

Frontend validation

- Min and Max must be valid numbers if provided
- If both exist: min <= max
- Enum must not be allowed for CURRENCY

Backend schema validation

- Keep/ensure rule:
 - min/max allowed for CURRENCY
 - enum not allowed for CURRENCY

Excel export

For CURRENCY fields:

- apply same numeric validation logic as NUMBER
- additionally apply a currency-like numeric cell format to the data column

Recommended simple format now:

- `#,##0.00`

If you already have a localized/currency symbol strategy, use it. If not, do NOT overcomplicate in this slice.

Important:

- CURRENCY is treated as numeric validation + formatting, not as a different validation engine

3) BOOLEAN

UI behavior

When field type = BOOLEAN:

- hide Enum input
- hide Min input
- hide Max input
- show helper text:
 - "This field will export as a Yes/No dropdown."

Frontend validation

- No custom validations allowed
- Enum must not be user-editable for BOOLEAN
- Min/max must not be allowed for BOOLEAN

Backend schema validation

- BOOLEAN should not require custom enum in schema
- BOOLEAN should not allow min/max

Excel export

For BOOLEAN fields:

- automatically apply dropdown list:
 - Yes
 - No

This must be generated even if schema has no explicit enum values.

Important:

- BOOLEAN is NOT modeled as TEXT enum from the UI perspective
- Export generator may internally treat it as a fixed list, but business behavior must remain "BOOLEAN"

UI / UX Rules for Step 2 Field Modal

When type = TEXT

- show Enum input
- hide Min
- hide Max

When type = NUMBER

- hide Enum
- show Min
- show Max

When type = CURRENCY

- hide Enum
- show Min
- show Max

When type = BOOLEAN

- hide Enum
- hide Min
- hide Max
- show helper text

Validation cleanup on type switch

Existing auto-clean behavior must be extended:

- switching to NUMBER/CURRENCY:
 - clear enum
- switching to BOOLEAN:
 - clear enum
 - clear min
 - clear max
- switching to TEXT:
 - keep enum
 - clear min/max
- switching away from TEXT:
 - clear enum

Frontend Validation Rules

Update schema/field validation so that:

TEXT

- enum allowed

- min/max not allowed

NUMBER

- min/max allowed
- enum not allowed

CURRENCY

- min/max allowed
- enum not allowed

BOOLEAN

- enum not allowed from user input
- min/max not allowed

Also:

- if min/max present, both must parse as numbers
- if both present, $\text{min} \leq \text{max}$

Backend Schema Validation Rules

Ensure backend validator enforces:

NUMBER / CURRENCY

- min/max allowed
- if both present: $\text{min} \leq \text{max}$
- enum disallowed

BOOLEAN

- enum disallowed
- min/max disallowed

TEXT

- enum allowed
- min/max disallowed

If invalid combinations are encountered, backend must reject them clearly.

Backend Export Rules

NUMBER

- numeric validation
- no dropdown
- no special format required beyond general number format

CURRENCY

- numeric validation
- apply numeric/currency formatting to the whole data column
- no dropdown

BOOLEAN

- fixed dropdown Yes/No
- apply same validation enforcement flags already used for TEXT dropdowns:
 - show error box
 - stop invalid values
 - allow empty cell
 - proper dropdown arrow behavior

Acceptance Criteria

NUMBER

- 1) Create NUMBER field with min=1 and max=10
- 2) Export Excel
- 3) Enter 5 → allowed
- 4) Enter 0 or 11 → Excel blocks with validation error

CURRENCY

- 1) Create CURRENCY field with min=100 and max=500
- 2) Export Excel
- 3) Enter 250 → allowed
- 4) Enter 50 or 600 → blocked
- 5) Column displays numeric/currency-like formatting

BOOLEAN

- 1) Create BOOLEAN field
- 2) Export Excel
- 3) Column shows Yes/No dropdown
- 4) Typing arbitrary value like `abc` is blocked

UI

- 1) Field modal shows only relevant validation inputs per type
- 2) Type switching clears incompatible validation values automatically
- 3) User is not shown irrelevant validation inputs for BOOLEAN

Rules / Restrictions

- Do NOT implement DATE validation in this slice
- Do NOT redesign the whole schema model
- Do NOT add regex/text-length validation
- Keep the payload shape stable unless absolutely necessary
- Reuse the same Excel validation enforcement pattern that fixed TEXT dropdowns

Suggested Implementation Order

Part A – Frontend modal + frontend validation

- Type-aware visibility
- Auto-clean on type switch
- Frontend validation updates

Part B – Backend schema validator

```

- Enforce allowed combinations

### Part C – Backend export builder
- NUMBER/CURRENCY numeric validation
- CURRENCY formatting
- BOOLEAN Yes/No dropdown

Do not mix DATE into this slice.

END

```

▼ Changes

Summary of what was implemented:

Part A — Frontend modal + validation

Step 2 field modal (SchemaEditorView.tsx)

- **TEXT:** Only the Enum input is shown (label: “Enum (comma-separated)”).
- **NUMBER / CURRENCY:** Only Min and Max are shown (labels “Min” and “Max”).
- **BOOLEAN:** Enum, Min, and Max are hidden; helper text is shown: *“This field will export as a Yes/No dropdown.”*
- **DATE:** No validation inputs (unchanged; DATE validation still out of scope).
- Type-change cleanup is unchanged: switching to NUMBER/CURRENCY clears enum; to BOOLEAN (or DATE) clears enum and min/max; to TEXT clears min/max.

Frontend validation (validateSchema.ts)

- Rules unchanged and aligned with spec: enum only for TEXT, min/max only for NUMBER/CURRENCY, min ≤ max. BOOLEAN/DATE have no validations. Comment clarified for DATE/BOOLEAN.

Part B — Backend schema validator

SchemaValidatorImpl.java

- **enumValues:** Same rule as `enum`: only TEXT may have `validations.enumValues`; NUMBER, CURRENCY, BOOLEAN, DATE are rejected with a clear error.
- Existing rules kept: NUMBER/CURRENCY allow min/max and min ≤ max; BOOLEAN/DATE do not allow enum or min/max.

Part C — Backend Excel export

SXSSFExcelWorkbookBuilder.java

NUMBER / CURRENCY

- **Min and max:** Decimal validation BETWEEN (unchanged).
- **Min only:** Decimal validation ≥ min (`GREATER_OR_EQUAL`).
- **Max only:** Decimal validation ≤ max (`LESS_OR_EQUAL`).
- Applied to data rows 1..1000, error style STOP, show error box, empty allowed. Typo fixed (missing newline) in the validation block.

CURRENCY

- Data format `#,##0.00` is applied to every CURRENCY column for rows 1..1000.
- When the sheet is protected, the currency style is created with `setLocked(false)` so those cells stay editable.
- Currency format is applied after validations (and after protection when `protectSheets` is true) so it is not overwritten.

BOOLEAN

- Fixed dropdown with values **Yes** and **No** is applied for BOOLEAN fields (no schema enum).
- Same behavior as TEXT dropdowns: STOP, show error box, empty allowed, dropdown arrow as per existing logic.

Files touched

| Area | File |
|----------|--|
| Frontend | <code>frontend/src/features/schema/components/SchemaEditorView.tsx</code> – type-based visibility and BOOLEAN helper |
| Frontend | <code>frontend/src/features/schema/lib/validateSchema.ts</code> – comment only |
| Backend | <code>backend/.../SchemaValidatorImpl.java</code> – <code>enumValues</code> only for TEXT |
| Backend | <code>backend/.../SXSSFExcelWorkbookBuilder.java</code> – NUMBER min/max variants, CURRENCY format, BOOLEAN Yes/No, typo fix |

Acceptance checklist

- **NUMBER:** Min=1, Max=10 → 5 allowed; 0 or 11 blocked with validation error.
- **CURRENCY:** Min=100, Max=500 → 250 allowed; 50 or 600 blocked; column shows `#,##0.00` style.
- **BOOLEAN:** Column has Yes/No dropdown; values like `abc` are blocked.
- **UI:** Modal shows only the relevant validation inputs per type; switching type clears incompatible values; BOOLEAN shows only the helper text.

DATE validation was not added. Payload shape is unchanged; existing Excel validation behavior is reused for NUMBER/CURRENCY/BOOLEAN.

▼ enhance 6
▼ workflow conversation

Yes — most of your notes are valid, and several of them point to the same bigger issue:

| the product is working, but the **interaction model is not yet coherent**.

You are now past “can it function?” and entering “does it feel intentional?” That is exactly where senior product/frontend decisions matter. I'll go point by point, give alternatives, tradeoffs, and then my recommendation.

1) Step 3 Export Studio card background

You want a larger background card wrapping the whole Export Studio area, from:

- “Export Studio”
- “Configure options...”
- stats cards

- export options
- file format
- additional settings

and you're unsure whether the bottom buttons should be inside that card.

Option A — Wrap everything except bottom buttons

A large parent card contains all content blocks, but the bottom action bar stays outside.

Pros

- Cleaner separation: content vs actions
- Matches enterprise admin patterns
- Buttons remain visually consistent with the wizard shell

Cons

- Slightly less "boxed-in" look than the CTO screenshot if their buttons are inside

Option B — Wrap everything including buttons

The parent card contains content + bottom action row.

Pros

- More "studio" feel
- Closer to some dashboard-style designs

Cons

- Buttons may feel buried inside content
- Less aligned with stepper/wizard UX where actions are usually structurally separate

My recommendation

Go with Option A.

Put a **single parent card** around the Export Studio content only, and keep the buttons outside in the wizard action area.

That gives you:

```
Wizard shell
├ Export Studio card
│   ├── intro
│   ├── stats
│   ├── options
│   ├── format
│   └ additional settings
└ Bottom actions row
```

That is the cleanest for a wizard.

2) "Save Settings" button in Step 3

Your observation is correct.

If changing options and directly clicking **Export Excel File** already works, then **Save Settings is not necessary for the core flow.**

This means the page currently has **two actions that look equally important but are not equally meaningful.**

Option A — Keep Save Settings

Persist export settings separately from export action.

When this makes sense

- If export settings are intended to be reusable defaults per version
- If users often come back later and expect the same configuration

Problem

- In your current flow, it creates ambiguity:
 - “Do I need to save before export?”
 - “What is saved vs what is just used now?”

Option B — Remove Save Settings entirely

Only keep:

- Back
- Export Excel File
- Finish

Pros

- Simpler
- Zero confusion
- Best if export settings are not a long-term concept yet

Cons

- Loses persistence of options if that matters later

Option C — Keep persistence, remove explicit Save button

Auto-save export settings when changed, or save them when exporting.

Pros

- Clean UX
- Keeps persistence without extra button

Cons

- More hidden behavior
- Harder to communicate unless done very clearly

My recommendation

Remove Save Settings for now.

Senior instinct: if a button does not create obvious value and adds hesitation, it should go.

Your current user goal in Step 3 is:

| choose options → export file

not:

| manage reusable export profiles

So I would remove **Save Settings** now and only reintroduce persistence later if the product really needs it.

3) “Protect Sheets from Editing” crashes

Your instinct is right.

If a feature is unstable and not needed right now, it should not remain enabled in the UI.

Option A — Keep and fix now

Only do this if sheet protection is important to the business workflow.

Option B — Disable in UI, keep backend dormant

Feature remains in code but hidden/disabled in frontend.

Option C — Disable in UI and stop using in backend path for now

Mark as postponed.

My recommendation

Disable it in frontend immediately and remove it from the active export behavior path for now.

Not necessarily delete all code permanently, but treat it as **out of current MVP flow**.

Best product behavior now:

- show it as disabled with “Coming soon”
- do not send it
- backend ignores/removes active protection logic from this slice’s path

Because a crashing option damages trust more than an absent option.

4) Step 2 field card hover actions (Edit/Delete visibility issue)

Your diagnosis is good:

- long field names visually dominate
- hover-only controls are too subtle
- if Edit sits beside Delete and the card text is long, discoverability drops

Option A — Keep horizontal buttons, just increase contrast

Pros: smallest change

Cons: still fragile with long names

Option B — Stack actions vertically

Delete under Edit (or Edit under Delete)

Pros

- avoids squeezing
- better visibility with long content

Cons

- slightly more vertical space
- can feel heavier in compact cards

Option C — Put actions in a fixed top-right action column

Reserve a visible action area inside the card, independent from text length.

Pros

- best balance
- stable placement
- text never pushes actions away

Cons

- slightly more layout work

My recommendation

Go with Option C.

This is the senior UI solution.

Do not let content width compete with actions.

Make the card layout something like:

```
[ Title / meta..... ] [ Edit ]  
[ Type / required / badges..... ] [ Delete ]
```

So the action area is structurally reserved.

That is better than simply moving Delete below Edit in an ad hoc way.

5) Step 2 field cards do not re-grid responsively

This is a real layout issue.

If cards shrink without reflowing into a new grid arrangement, then the grid is not truly responsive — it is just compressing.

Option A — Keep fixed columns and allow shrink

Bad UX for cards.

Option B — Use responsive CSS grid with minmax

This is the proper solution:

- cards wrap into fewer columns when space reduces
- no crushing

My recommendation

Fix the grid to be genuinely responsive.

Conceptually:

```
grid-template-columns: repeat(auto-fit, minmax(260px, 1fr))
```

Not code here, just the idea:

- minimum readable card width
- cards wrap instead of compressing

This is definitely worth doing.

6) Step 1 should visually match CTO Step 1 template

This is a bigger product design shift, but a good one.

You want:

- a nice selection card layout
- predefined-looking cards for Template Name / Sector Code
- a “custom” card with plus icon
- clicking custom opens the current modal
- validation only after touch
- create continues ordinary flow to Step 2

Important product interpretation

You are not really implementing “predefined templates” yet.

You are borrowing the **visual language** of selectable cards.

That is fine, as long as the cards are clearly non-functional or intentionally limited.

Possible approaches

Option A — Pure visual shell only

Show cards but they don’t do anything except the custom card.

Pros

- fast
- matches design direction now

Cons

- could confuse users if “predefined” cards look clickable but aren’t

Option B — Visual cards + disabled state for predefined cards

Display a few nicely designed preset-like cards as **design placeholders**, visibly disabled/non-clickable.

Keep only the custom card active.

Pros

- visually matches CTO style
- honest about current functionality
- future-proof for later activation

Cons

- slightly more work than A

Option C — Fully interactive presets now

Too early unless backend/business logic exists.

My recommendation

Go with Option B.

This is the most mature choice:

- it gives you the CTO-inspired aesthetic
- it does not fake a feature
- it signals future extensibility

Flow recommendation:

- Step 1 content inside a large polished card

- inside it:
 - a row/grid of preset-style cards (disabled / "coming later")
 - one active **Custom Template** card with plus icon
- clicking Custom opens the current modal
- touched-only validation remains
- Create in modal moves to Step 2 exactly as today

That is a strong design decision.

7) Breadcrumb issue in admin/templates nav

Current breadcrumb:

```
Templates > test > version 1 > schema
```

and clicking "version 1" effectively skips to template details, which is confusing.

Your proposed solution:

- merge "version v1" and "schema" into one node

Option A — Keep 4 levels, fix the middle route

Only valid if you actually have a real version-details page.

Option B — Reduce to 3 levels

Example:

```
Templates > test > v1 schema
```

or

```
Templates > test > Schema (v1)
```

My recommendation

Go to 3 levels.

But I would name it more naturally as:

```
Templates > test > Schema (v1)
```

instead of:

- `version 1 schema`
- `version v1 schema`

Because:

- "Schema" is the actual page
- `(v1)` is the context
- avoids breadcrumb skip confusion
- reads like a page title, not two stitched labels

So yes, reduce to 3 levels, but I recommend:

Schema (v1)

not `version v1 schema`.

8) Schema editor page: remove “Version 1 / Schema” above card and replace with Back button

That is a good simplification.

If breadcrumb/title already provides the page context, then repeating:

```
Version v1
Schema
```

above the card is redundant.

My recommendation

- remove the repeated version/schema text block
- add a **Back to template details** button at the top, styled consistently with create-template navigation buttons

That is cleaner and more action-oriented.

9) Template list empty state: two “Create Template” buttons

You are right — only one should exist at a time.

Correct behavior

- **If empty list:** show only the CTA inside the empty-state card
- **If list has items:** show the top-level “Create Template” button above the card/table

My recommendation

Exactly that. No debate. This is just the correct UX.

Recommended implementation strategy

You have a lot of notes, but they are not equal in risk.

Do **not** send all of them as one giant slice.

Slice group 1 — High-value / low-risk cleanup

Do first:

1. Step 3 parent card wrapper
2. Remove Save Settings button
3. Disable Protect Sheets option and remove active behavior
4. Empty list CTA duplication fix
5. Breadcrumb simplify to 3 levels
6. Remove repeated Version/Schema text and add Back button

These are mostly structural/UI and low-risk.

Slice group 2 — Step 2 interaction quality

Then:

1. Reserve fixed action area in field cards
2. Make field cards responsive grid
3. Improve hover action visibility

Slice group 3 — Step 1 CTO-style redesign

Then:

1. Create polished Step 1 container card
2. Add preset-style disabled cards
3. Add active custom card
4. Keep modal flow unchanged
5. Keep touched-only validation

This one is bigger and deserves its own slice.

Final senior verdict

Your instincts are mostly correct.

The main principle I'd apply is:

| remove ambiguity before adding beauty.

So:

- **Remove Save Settings**
- **Disable Protect Sheets**
- **Fix breadcrumb and duplicate CTAs**
- **Make Step 2 actions structurally visible**
- **Then do the Step 1 CTO-style redesign**

That sequence gives the best return with the least regression risk.

If you want, I'll now turn all of this into a **clean slice plan** in order:

- Slice A: Step 3 cleanup
- Slice B: Step 2 card UX
- Slice C: Step 1 CTO-style redesign
- Slice D: Admin/templates breadcrumb + header cleanup

▼ Slices

▼ slice A

Good decision. We'll implement this **incrementally and safely**, starting with the **highest-value / lowest-risk UI cleanup**.

This first slice covers **Step 3 cleanup + admin templates UX fixes + duplicate CTA fix**, exactly as recommended earlier.

```

---

# SLICE A – Step 3 Export Studio Cleanup + Templates UX Fixes

## Goal

Improve UX clarity and remove unstable or redundant features without changing core workflow.

This slice focuses on:

1. Step 3 layout clarity
2. Removing the confusing **Save Settings** button
3. Disabling unstable **Protect Sheets** option
4. Fixing duplicate **Create Template** CTA
5. Fixing breadcrumb UX confusion
6. Cleaning the Schema editor header

No business logic changes besides removing unused export options.

---

# Part 1 – Step 3: Export Studio Layout Improvement

## Problem

Export Studio currently contains several cards floating in the page without a unified container.

This makes the section visually fragmented.

## Solution

Wrap all Export Studio content in a **parent background card**, but keep the wizard navigation buttons **outside**.

### New Layout

...
Wizard container
├── Export Studio Parent Card
│   ├── Header (Export Studio + description)
│   ├── Statistics Cards
│   ├── Export Options Card
│   ├── File Format Card
│   └── Additional Settings Card
└── Wizard Action Row
    ├── Back
    ├── Export Excel File
    └── Finish
...

### Styling recommendation

```

Parent card:

- * light neutral background
- * slightly larger padding
- * subtle border or shadow
- * consistent with design system

Do **not** move action buttons inside this card.

Part 2 – Remove "Save Settings" Button

Problem

"Save Settings" has no real effect.

Export works even when the user does not click it, which creates confusion about whether it is required.

Decision

Remove the **Save Settings** button entirely.

Before

```

Back | Save Settings | Export Excel File | Finish  
```

After

```

Back | Export Excel File | Finish  
```

Export should always use the **current selections**.

Do not introduce hidden persistence logic in this slice.

Part 3 – Disable "Protect Sheets from Editing"

Problem

This option currently causes export failures and is not required for the current workflow.

A crashing feature damages trust more than a missing one.

Decision

Disable it completely for now.

Frontend

Remove the option from Step 3 UI.

Backend

Ensure export logic **does not apply sheet protection**.

Remove or bypass:

```
```\noptions.protectSheets()\nprotectSheetWithUnlockedDataRange(...)\n```
```

The export builder should behave as if protection is always **false**.

Do not delete the helper method if used elsewhere, but it must not be executed in this workflow.

---

## # Part 4 – Fix Duplicate "Create Template" CTA

### ## Problem

When the template list is empty, two **Create Template** buttons appear.

This creates unnecessary visual noise.

### ## Correct behavior

#### ### When list is empty

Show only the **empty state card button**.

Example:

```
```\n
```

No templates yet

Create your first
template

[Create Template]

```
```\n
```

Do **not** show the top action button.

---

### When templates exist

Show only the **top page action button**.

Example:

...

Templates

[ Create Template ]

Template List
template A
template B

...

The empty-state card button must not appear in this case.

---

# Part 5 – Breadcrumb UX Fix

## Problem

Current breadcrumb:

...

Templates > test > version 1 > schema

...

Clicking **version 1** skips the expected navigation and returns to the template page.

This is confusing.

## Decision

Reduce breadcrumb depth and merge version context into the schema page.

### New breadcrumb

...

Templates > test > Schema (v1)

...

This means:

\* Page = Schema

\* Context = Version 1

No navigation level is skipped anymore.

---



## # Part 6 – Schema Editor Header Cleanup

### ## Problem

Schema editor shows redundant information above the card:

```

Version v1

Schema

```

This duplicates information already shown in the breadcrumb and page title.

### ## Solution

Remove the redundant header and replace it with a **\*\*navigation action\*\***.

#### ### New layout

Top section:

```

[Back to Template Details]

Schema Editor

```

or

```

← Back to Template Details

Schema Editor

```

The button should navigate back to the template details page.

Style it consistently with navigation buttons used in the wizard.

---

### # Out of Scope

This slice must **\*\*NOT\*\*** include:

- \* Step 1 redesign
- \* Step 2 card layout improvements
- \* Field card hover fixes
- \* Responsive grid fixes
- \* Validation changes
- \* Export engine refactoring

Those will be implemented in later slices.

---

## # Acceptance Criteria

### ## Step 3

- \* Export Studio content appears inside a unified parent card
- \* Wizard action buttons remain outside the card
- \* Save Settings button is removed
- \* Export works directly using current selections
- \* Protect Sheets option no longer appears

### ## Templates List

- \* When empty → only the empty-state CTA appears
- \* When populated → only the top action CTA appears

### ## Breadcrumb

Breadcrumb is now:

```

Templates > test > Schema (v1)

```

No intermediate version node exists.

### ## Schema Editor

- \* Redundant "Version v1 / Schema" header removed
- \* Back to Template Details button added

---

## # Implementation Order (recommended)

- 1 Remove Save Settings button
- 2 Disable Protect Sheets option
- 3 Add Export Studio parent card wrapper
- 4 Fix duplicate Create Template CTA logic
- 5 Update breadcrumb structure
- 6 Clean schema editor header and add back button

---

If you'd like, I can next generate **\*\*Slice B (Step 2 card UX improvements)\*\*** which will fix:

- hover edit/delete visibility
- responsive card grid
- action placement stability

Those will make Step 2 feel **\*\*much more professional\*\***.

▼ slice A - changes

## Part 1 — Step 3: Export Studio layout

- Wrapped all Export Studio content in a **parent card**: `Card` with `className="border-border bg-neutral-50/50 shadow-sm"` and `CardContent className="p-6 space-y-6"`.
- Inside the card: Export Studio header + description, statistics cards, Export Options card, File Format card, Additional Settings card.
- **Wizard actions (Back, Export Excel File, Finish)** are unchanged and stay in `bottomActions` outside this card.

## Part 2 — Save Settings removed

- **Removed**: `handleSaveSettings`, `savingSettings` state, and the Save Settings button.
- **Removed**: `buildSchemaPayloadForUpdate` import and any schema-update logic for export options.
- Export uses **current UI selections** only; no "Save settings" step.

## Part 3 — Protect Sheets disabled

- **Frontend**: Removed the "Protect Sheets from Editing" checkbox from the Export Options card. Export request always sends `protectSheets: false`.
- **Backend**: In `SXSSFExcelWorkbookBuilder.addDataSheet`, the call to `protectSheetWithUnlockedDataRange` is removed. `applyCurrencyColumnFormats` is called with `false` for the sheet-protected flag. The `protectSheetWithUnlockedDataRange` method is left in place but is no longer used in this flow.

## Part 4 — Duplicate Create Template CTA

- **TemplatesListPage**: Introduced `showTopCreateButton = templates.length > 0 && !loading`.
- **When the list is empty**: Only the empty-state card is shown with its single "Create Template" button; the page header has no top-right action.
- **When the list has items**: Only the top-right "Create Template" button in the page header is shown; the empty state is not rendered.

## Part 5 — Breadcrumb

- **Before**: `Templates > test > Version v1 > Schema` (four segments; "Version v1" linked to template).
- **After**: `Templates > test > Schema (v1)` (three segments; "Schema (v1)" is the current page).
- Version is only shown in the last segment; there is no separate version link.

## Part 6 — Schema editor header

- **Removed**: Redundant `description={Version ${version.versionNumber}}` from `PageHeader`.
- **Set**: `title="Schema Editor"` on `PageHeader`.
- **Kept**: "Back to Template Details" (or `backLabel`) as the navigation link above the header; in `SchemaEditorPage.tsx` the label is set to **"Back to Template Details"**.

## Files changed

Area	File
Step 3	frontend/src/features/templates/pages/CreateTemplateExportPage.tsx
List	frontend/src/features/templates/pages/TemplatesListPage.tsx
Breadcrumb	frontend/src/app/layout/Topbar.tsx
Schema editor	frontend/src/features/schema/components/SchemaEditorView.tsx , frontend/src/features/schema/pa
Backend	backend/.../SXSSFExcelWorkbookBuilder.java

No new linter issues; backend compiles. Export options types ( `ExportProfile` , `protectSheets` in requests) are unchanged; only the UI and runtime behavior were updated as above

### ▼ slice B

```
````md
# SLICE B – Step 2 Schema Editor UX Improvements (Field Cards Actions + Responsive Grid)

## Goal
Improve Step 2 Schema Editor usability by fixing the field cards interaction model and layout responsiveness.

This slice focuses on:

1. Making Edit/Delete actions clearly visible and structurally stable
2. Preventing long field names from visually hiding actions
3. Making field cards reflow responsively instead of shrinking awkwardly
4. Keeping the current behavior and business logic unchanged

This is a UI/UX cleanup slice only.

---

## Problems Being Solved

### 1) Hover actions are not clearly visible
Currently Edit/Delete appear on hover, but:
- they use a light background
- they become clearly visible only when hovering the buttons themselves
- if field names are long, actions visually get lost

### 2) Text competes with actions
Because actions sit beside content, long field names reduce action visibility and clarity.

### 3) Grid is not truly responsive
When the page or container width changes:
- cards shrink
- cards do not reflow into a better grid
- layout feels broken rather than adaptive

---

## Design Decision
```

A) Keep hover-reveal behavior, but reserve a stable action area
Do NOT rely on text/content width to determine where action buttons appear.

Each field card must have a **fixed action area** reserved inside the card.

Recommended structure

```text

|                           |        |
|---------------------------|--------|
| Field Title / Header Name | [Edit] |
| Field Key / Type / Badges | [Del ] |

```

This means:

- * content and actions are separated into two columns
- * long text does not push actions away
- * actions stay in the same place across all cards

Behavior

- * actions may still be more visible on hover, but they must remain discoverable
- * card hover can increase action contrast/opacity
- * buttons themselves should not be the only way to reveal actions

B) Make field cards grid truly responsive

Use a responsive grid that wraps cards instead of shrinking them excessively.

Required behavior

- * cards must have a **minimum readable width**
- * when space becomes smaller, cards should wrap into fewer columns
- * they must not keep compressing horizontally until content becomes cramped

Target behavior

Desktop:

- * 3 or 4 columns depending on available width

Tablet:

- * 2 columns

Mobile / narrow container:

- * 1 column

Conceptual CSS behavior

Use a real responsive grid such as:

```
```text
repeat(auto-fit, minmax(<minimum-card-width>, 1fr))
```
```

This is the desired interaction model, even if implemented with utility classes.

C) Field card content hierarchy

Each field card should clearly show:

Primary content

- * Header Name (main visible title)

Secondary content

- * Field Key
- * Type
- * Required badge if applicable
- * Optional validation badge/summary if already available

Actions

- * Edit
- * Delete

Recommended order:

- * Edit above Delete OR stacked vertically in the action area
- * Do not keep them inline beside long text in a way that causes overlap/distraction

Recommendation

Use a **vertical action stack** in the reserved right-side area:

```
```text
[Edit]
[Delete]
```
```

This is more stable than horizontal text buttons in a narrow card.

D) Hover behavior

Hover should enhance visibility, not be the only source of visibility.

Recommended behavior

Default state:

- * action buttons visible but subtle (not fully hidden)
- * for example reduced opacity or softer ghost style

On card hover:

- * action buttons become fully emphasized
- * card border/shadow may slightly lift

This prevents discoverability issues.

Important

Do ****not**** make actions completely invisible by default.

Scope

In scope

- * Step 2 field cards UI only
- * field cards grid/container layout
- * action button placement/visibility

Out of scope

- * Add/Edit/Delete field functionality
- * validation logic
- * table panel redesign
- * stepper/wizard structure
- * Step 1 redesign
- * backend changes

Existing Behavior That Must Remain

- * Clicking Edit opens the existing Edit Field modal
- * Clicking Delete removes the field using existing behavior
- * Current schema state handling remains unchanged
- * No validation or business logic changes

Detailed Requirements

1) Field card layout

Each field card must:

- * reserve a fixed action region
- * prevent text from colliding with action buttons
- * support long field names gracefully
- * maintain readable spacing

Long text handling:

- * allow wrapping if needed OR truncate with ellipsis
- * but must not overlap actions

Recommended:

- * title line clamp or wrap to 2 lines max
- * field key/type displayed in smaller muted text

2) Action area

Action area must:

- * remain fixed in position
- * not shift when field title length changes
- * contain Edit and Delete in a clear vertical stack

Buttons must:

- * be visually consistent with rest of admin UI
- * use icon + label if space allows
- * remain accessible

Recommended labels:

- * Edit
- * Delete

3) Responsive grid

The field cards container must:

- * reflow cards into new rows when width decreases
- * avoid awkward shrinking/compression
- * preserve consistent gap spacing

The current layout that shrinks cards without re-gridding must be replaced.

4) Visual polish

Recommended subtle improvements:

- * slightly stronger card border on hover
- * soft shadow or hover lift
- * action buttons get stronger contrast on hover
- * required badge remains easy to scan

No redesign beyond that.

Acceptance Criteria

Visibility

1. Hovering a field card makes Edit/Delete clearly visible
2. Even before direct button hover, actions are discoverable
3. Long field names do not hide the action buttons

Layout

4. Cards reflow responsively when container width changes
5. Cards do not just shrink awkwardly
6. Action area remains stable across different content lengths

Behavior

7. Edit still opens existing modal
8. Delete still works as before
9. No regressions in field rendering or schema state

Suggested Implementation Order

1. Refactor field card layout into content area + reserved action area
2. Stack Edit/Delete vertically in action column
3. Reduce “fully hidden until hover” behavior; make actions subtle-but-visible by default
4. Update field cards container to real responsive grid
5. Test with:
 - * short field names
 - * very long field names
 - * narrow window width
 - * multiple cards

Test Scenarios

Scenario A – Long field name

Create a field with a long header name:

```

* actions must still be clearly visible
* no overlap with content

### Scenario B – Window resize

Resize browser/container:

* cards must wrap into fewer columns
* no crushed layout

### Scenario C – Hover

Hover card:

* actions become more visually prominent
* no sudden layout shift

### Scenario D – Functional regression

Click Edit / Delete:

* existing behavior unchanged

---

## Rules / Restrictions

* Do not change field CRUD logic in this slice
* Do not change validation behavior
* Do not redesign Step 2 table panel
* Keep changes focused on card UX and grid responsiveness only

END

` ``
` ``

```

▼ slice B - changes

Summary of changes for **Slice B — Step 2 Schema Editor UX (field cards + responsive grid)**:

1. Field card layout (content + fixed action area)

- **Before:** Single block with `pr-16` and absolutely positioned actions (right/top). Long titles could push into the action area.
- **After:** Card uses **flex**: left block for content (`min-w-0 flex-1`), right block for actions (`shrink-0 flex flex-col gap-1 pl-2`).
- Content no longer overlaps actions; long text is confined to the content area.

2. Content hierarchy and long text

- **Primary:** Header name with `line-clamp-2` so long titles use at most two lines and don't push layout.

- **Secondary:** Field key (mono, muted), then type and required on one line with `flex flex-wrap` and `gap-x-2`.
- Required is shown as `Required` in amber, inline with type.

3. Action area (stable, vertical stack)

- **Before:** Horizontal Edit/Delete in top-right with `opacity-0 group-hover:opacity-100`.
- **After:**
 - Actions in a **fixed right column** (flex column, not absolute).
 - **Vertical stack:** `flex flex-col gap-1` with Edit above Delete.
 - Same `ActionButton` / `DangerActionButton` with labels “Edit” and “Delete”; only layout and visibility changed.

4. Action visibility (subtle by default, clear on hover)

- **Before:** `opacity-0` by default, `group-hover:opacity-100` (actions only visible on hover).
- **After:** `opacity-75` by default, `group-hover:opacity-100` so actions are always visible but become stronger on card hover.
- No “fully hidden until hover”; behavior matches the “subtle but discoverable” requirement.

5. Responsive grid (wrap instead of shrink)

- **Before:** `grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3` (fixed breakpoints; cards could shrink a lot at in-between widths).
- **After:** `grid gap-3` with `style={{ gridTemplateColumns: 'repeat(auto-fill, minmax(min(100%, 280px), 1fr))' }}`
- Cards keep a **minimum width** of 280px (or full width when narrower). When the container is too narrow for the next column, the next card wraps to a new row instead of squeezing.
- Result: narrow → 1 column; medium → 2; wide → 3–4 columns, without overly narrow cards.

6. Card hover polish

- **Before:** `transition-shadow hover:shadow-md`.
- **After:** `transition-[box-shadow,border-color] hover:border-neutral-300 hover:shadow` so the card gets a slightly stronger border and shadow on hover.

Unchanged (as required)

- Edit still opens the same field modal.
- Delete still uses the same confirm flow and removal logic.
- No changes to field CRUD, validation, table panel, or Step 1.
- All updates are in **Step 2 field cards and their grid** in `SchemaEditorView.tsx`

▼ slice C

```
```md
SLICE C – Step 1 Wizard Redesign (CT0-style Visual Entry + Custom Card Flow)

Goal
```

Redesign Step 1 of the Create Template wizard so it visually aligns with the CT 0-style entry screen while keeping the existing business flow unchanged.

This slice focuses on:

1. Replacing the plain Step 1 form page with a polished selection-style layout
2. Showing disabled/predefined-style cards for visual design only
3. Providing one active "Custom Template" card that opens the existing modal
4. Keeping touched-only validation behavior inside the modal
5. Keeping the current ordinary flow:
  - modal Create → Step 2

This slice is **\*\*UI/UX redesign only\*\***, not a new business feature.

---

## ## Product Intent

Step 1 should feel like a **\*\*guided entry point\*\*** into template creation rather than a plain form.

However, predefined templates are **\*\*not\*\*** being implemented functionally yet.

So the design must communicate:

- future extensibility
- polished visual direction
- no fake interactivity for features that do not exist yet

---

## ### Design Decision

## A) Use CT0-style visual cards, but only one is active

Step 1 should show a parent card/container with:

### ### Inside it:

1. A set of **\*\*predefined-style cards\*\*** (design placeholders only)
2. One **\*\*Custom Template\*\*** card with a plus icon (active)
3. Clicking the Custom Template card opens the existing create modal

### ### Important

The predefined cards are **\*\*not clickable\*\***

They are only there for design language and future extensibility.

---

## ## Layout Structure

### ### Page structure

```text

Step 1 Page

└─ Main Parent Card / Section

│─ Title + description

```

└─ Preset-style cards grid
  └─ Example preset card 1 (disabled)
  └─ Example preset card 2 (disabled)
  └─ Custom Template card (active)
└─ Existing create modal (opened by custom card)
....

```

B) Preset-style cards (design only)

These cards visually represent prebuilt template options but do nothing for now.

Examples (visual only)

- * Sales Data Template
- * Product Pricing Template
- * Market Survey Template

These are just design placeholders.

Required behavior

- * visually polished
- * non-clickable
- * visibly disabled / inactive
- * must not mislead the user into thinking they work now

Recommended visual treatment

- * slightly muted
- * soft neutral background
- * subtle icon
- * optional small label:
 - * "Coming later"
 - * or disabled appearance only

C) Custom Template card (active)

This is the only active entry point in Step 1.

Card content

- * plus icon
- * title like:
 - * Custom Template
- * short description:
 - * Create a new template from scratch

Behavior

- * clicking this card opens the existing modal
- * the modal keeps the same business logic:
 - * template name
 - * sector code
 - * validation
 - * Create button
- * successful Create continues to Step 2 as today

Modal Behavior (must remain)

The modal itself must keep the current correct behavior:

Validation

- * no validation errors on modal open
- * validation shown only after touch / relevant interaction
- * Create button follows the current intended behavior
- * successful create navigates to Step 2

Flow

No new backend/API changes in this slice.

Scope

In scope

- * Step 1 page layout redesign
- * preset-style cards (disabled)
- * active custom template card
- * modal trigger moved from form/page into custom card
- * preserve current modal validation UX

Out of scope

- * functional predefined template support
- * new backend endpoints
- * changing create-template API behavior
- * Step 2 or Step 3 changes
- * changing the modal fields or business flow

Existing Behavior That Must Remain

- * Clicking Custom Template opens the current creation modal

- * Modal Create still creates draft/template exactly as it does now
- * Successful create still navigates to Step 2
- * No new logic for predefined templates

Detailed Requirements

1) Parent card / section

Step 1 should have a strong visual container around the whole selection area.

Recommended styling:

- * spacious padding
- * subtle background or elevated card
- * consistent with current wizard design system
- * visually richer than the current simple form page

This should feel like a **template selection / starting point panel**.

2) Step 1 header

Inside the main container, include:

- * page title
- * short description

Example intent:

- * “Choose how you want to start your template”
- * “Use a starter layout or create a custom template from scratch”

The tone should feel intentional and guided.

3) Preset cards grid

The card section should be arranged in a responsive grid:

- * desktop: multiple cards in a row
- * smaller screens: wrap cleanly

Each preset card should include:

- * icon
- * title
- * short description

But must remain **non-clickable**.

Disabled state options

Recommended:

- * cursor default or not-allowed
- * muted colors
- * slight opacity reduction
- * optional badge/text: "Coming soon" or "Available later"

Do not make them look active.

4) Custom card

The active custom card should:

- * visually stand out slightly more
- * include plus icon
- * be clickable
- * open the modal

This card is the real CTA.

Recommended:

- * stronger border or accent
- * hover state
- * clear affordance that this is the current path

5) Empty-form removal

The current Step 1 should no longer look like a direct inline form page.

Replace the direct form-first experience with:

- * card-based entry
- * modal-based creation

Do not keep redundant inline create fields visible on the page.

UX Recommendations

A) Presets should be clearly fake-for-now, but not ugly

Do not leave them looking broken or unfinished.

B) Custom card should be visually primary

Not necessarily bright/accent-heavy, but clearly the active option.

C) Keep modal familiar

Do not redesign the create modal in this slice beyond preserving the current go od validation behavior.

Acceptance Criteria

Layout

1. Step 1 shows a polished card-based starting screen
2. Predefined-style cards are visible and disabled
3. Custom Template card is visible and active

Interaction

4. Clicking a predefined card does nothing
5. Clicking Custom Template opens the existing create modal
6. Modal Create still moves user to Step 2 on success

Validation

7. Modal validation remains touched-based / non-aggressive
8. No validation shown immediately on modal open

Non-regression

9. Existing create flow works unchanged
10. No backend/API changes required

Suggested Implementation Order

1. Create Step 1 main parent container
2. Add title + description
3. Add responsive grid of preset-style cards
4. Add active Custom Template card with plus icon
5. Connect Custom card click to existing modal
6. Remove/replace direct inline create-form presentation if still visible
7. Verify modal validation + Step 2 navigation remain unchanged

Test Scenarios

Scenario A – Normal flow

- * Open Step 1
- * Click Custom Template
- * Modal opens
- * Enter valid template name + sector code

```

* Click Create
* User moves to Step 2

### Scenario B – Disabled presets

* Click any preset-style card
* Nothing happens

### Scenario C – Validation

* Open modal
* No errors shown immediately
* Touch invalid fields
* Validation appears only when relevant

### Scenario D – Responsiveness

* Resize screen
* Cards reflow cleanly
* No broken layout

- - -

## Rules / Restrictions

* Do not implement actual preset-template behavior
* Do not add new APIs
* Do not change Step 2/Step 3
* Keep modal business flow unchanged
* Keep this slice focused on Step 1 visual redesign only

END

\ \ \
\ \ \

```

▼ slice C - changes

Summary of the Step 1 redesign (Slice C):

1. Layout

- **Parent card:** One main card (same style as Export Studio: `border-border bg-neutral-50/50 shadow-sm`) with:
 - **Title:** "Create Template" (from `TemplateWizardLayout`)
 - **Description:** "Step 1: Choose how you want to create your template."
 - **Subtext:** "Start from a preset or create a custom template with your own name and sector."
 - **Grid of option cards** (see below).

2. Preset-style cards (placeholders only)

- **Industry Template** – "Start from a pre-built industry schema." – badge: "Coming soon"
- **Sector Template** – "Use a sector-specific template." – badge: "Coming soon"

Both are non-interactive:

- `cursor-not-allowed`, `select-none`, `opacity-60`, `aria-disabled="true"`
- No `onClick`; they only set the visual direction and future extensibility.

3. Custom Template card (only active option)

- **Layout:** Dashed border, plus icon in a circle, "Custom Template" title, "Name and sector code" subtitle.
- **Interaction:** `onClick` opens the create modal. Hover/focus styles: solid border, light primary background, focus ring.
- **Responsive grid:** Same pattern as Step 2: `repeat(auto-fill, minmax(min(100%, 240px), 1fr))` so cards wrap instead of squashing.

4. Create modal (existing flow)

- **Trigger:** Only opening the Custom Template card opens the modal.
- **Content:** Same form as before: Template Name, Sector Code, Create, plus Cancel.
- **Validation:** Unchanged (required on submit; same error toast).
- **Submit:** Same `templatesApi.create()` call; on success the modal closes and the app navigates to Step 2 (`/templates/create/${res.templateId}`).

5. Flow preserved

- User lands on Step 1 → sees card grid.
- User clicks Custom Template → modal opens.
- User fills name + sector and clicks Create → template is created → redirect to Step 2.
- No changes to API, validation rules, or Step 2/Export behavior.

Files touched

- `frontend/src/features/templates/pages/CreateTemplateStartPage.tsx` – card grid, preset placeholders, Custom Template card, and create modal

▼ slice C - changes 2

Summary of changes that align Step 1 with the remainder of Slice C:

B) Preset-style cards (design only)

- **Titles (examples):**
 - **Sales Data Template** – “Pre-built schema for sales and pipeline data.”
 - **Product Pricing Template** – “Starter layout for product and pricing fields.”
 - **Market Survey Template** – “Template for surveys and market research.”
- **Visual treatment:**
 - **Badge:** “Coming later” (replacing “Coming soon”).
 - **Icon:** Subtle icon per card (BarChart3, DollarSign, ClipboardList) in a muted rounded box.
 - **Style:** Soft neutral background (`bg-neutral-100/80`), muted text (`text-neutral-700` / `text-neutral-500`), `opacity-75`.

- **Non-clickable / disabled:**

- `cursor-not-allowed`, `select-none`, `pointer-events-none` so clicks do nothing.
 - `aria-disabled="true"` and `role="presentation"` with an `aria-label` that includes the badge so they're clearly inactive.
-

C) Custom Template card (active)

- **Content:**

- Plus icon (unchanged).
- Title: **Custom Template**.
- Description: **"Create a new template from scratch"** (replacing "Name and sector code").

- **Behavior:** Still the only active entry; click opens the same create modal. Styling remains the clear CTA (dashed border, hover/focus states, `hover:shadow`).
-

1) Parent card / section

- **Container:** Same main card; content uses **spacious padding** (`p-6 md:p-8`).
 - **Look:** Same subtle background and shadow so it reads as the template selection / starting panel.
-

2) Step 1 header

- **Layout title:** "Step 1: Choose how you want to start your template."
 - **In-card description:** "Use a starter layout or create a custom template from scratch."
-

3) Preset cards grid

- **Grid:** Unchanged responsive grid; desktop multiple per row, wraps on smaller screens.
 - **Per preset card:** Icon, title, short description, "Coming later" badge; all non-clickable and visually disabled as above.
-

4) Custom card

- Described above; no change to behavior, only copy and small hover polish.
-

5) Empty-form removal

- No inline form on the page; creation is only via the Custom Template card → modal. No redundant inline create fields.
-

Modal behavior (unchanged)

- **Validation:** No validation on open; we only show an error toast on submit when required fields are missing (no inline errors added).
 - **Create:** Same API and success handling; on success the modal closes and the user is sent to Step 2.
 - **Scope:** No changes to modal fields, API, or business flow.
-

Acceptance criteria

| Criterion | Status |
|--|---|
| Step 1 shows a polished card-based starting screen | Yes |
| Predefined-style cards visible and disabled | Yes (3 presets, muted, "Coming later", <code>pointer-events-none</code>) |
| Custom Template card visible and active | Yes |
| Clicking a predefined card does nothing | Yes (<code>pointer-events-none</code>) |
| Clicking Custom Template opens existing create modal | Yes |
| Modal Create still moves user to Step 2 on success | Yes (unchanged) |
| Modal validation non-aggressive / no errors on open | Yes (toast only on submit) |
| Existing create flow unchanged, no backend changes | Yes |

▼ slice D

Yes – there **is one more important slice** that logically completes the work you started.

Right now you implemented:

- * Slice A → Step 3 cleanup + admin templates UX fixes
- * Slice B → Step 2 card UX improvements
- * Slice C → Step 1 CT0-style redesign

However, there is still **one structural issue left in the wizard experience** that will surface after Slice C:

> **Wizard state continuity and navigation coherence**

When users move between steps (1 → 2 → 3), the system must guarantee **consistent state handling and predictable navigation**, especially after the Step 1 redesign (modal → step transition).

This slice ensures the wizard behaves like a **true production-grade workflow**, not just three pages stitched together.

This is a **small but very important senior-level polish slice**.

SLICE D – Wizard Flow Stability & Navigation Consistency

Goal

Stabilize the Create Template wizard so navigation, state, and transitions behave consistently after the Step 1 redesign.

This slice focuses on:

1. Ensuring wizard state persists correctly across steps
2. Preventing navigation to invalid steps
3. Making the stepper reflect real progress
4. Avoiding broken navigation after refresh or deep links
5. Guaranteeing smooth transitions between steps

No visual redesign here – only **flow stability**.

Problem

After Slice C, Step 1 becomes:

```

Card → Modal → Create → Step 2

```

This introduces several possible inconsistencies:

- * users refreshing Step 2 or Step 3
- * users deep-linking directly into Step 2
- * stepper highlighting wrong step
- * incomplete template state
- * wizard actions behaving inconsistently

Without guardrails, wizard navigation becomes fragile.

Part 1 – Step Access Guard

Users must not be able to open steps that require data that does not yet exist.

Rule

| Step | Required state |
|--------|-------------------|
| ----- | ----- |
| Step 1 | none |
| Step 2 | templateId exists |
| Step 3 | schema exists |

Behavior

If a user tries to access:

Step 2 without templateId

→ redirect to Step 1

Step 3 without schema

→ redirect to Step 2

Example

```

    \
    /create-template/step-3
    \

but schema is empty → redirect to

    \
    /create-template/step-2
    \

---

# Part 2 – Stepper State Accuracy

The wizard step indicator must reflect the real step.

Example:

    \
    1 Template Info
    2 Schema Editor
    3 Export Studio
    \

### Behavior

Active step must match the page.

Example:

Step 2 page

    \
    ✓ Step 1
    ● Step 2
    ○ Step 3
    \

Step 3 page

    \
    ✓ Step 1
    ✓ Step 2
    ● Step 3
    \

---

# Part 3 – Refresh Safety

Users must not lose the wizard state unexpectedly when refreshing.

### Behavior

```

Refreshing Step 2 should:

- * reload template + schema
- * not send user back to Step 1 unnecessarily

Refreshing Step 3 should:

- * reload template version + schema
- * keep export configuration if already loaded

Part 4 – Navigation Buttons Behavior

Wizard navigation must be consistent.

Step 1

No Back button needed.

Step 2

...

Back → Step 1

Next → Step 3

...

Step 3

...

Back → Step 2

Export Excel

Finish

...

Back must ****never destroy state****.

Part 5 – Finish Button Behavior

Finish should behave predictably.

Option A (recommended)

Finish returns user to:

...

Templates list

...

Option B

Finish returns to:


```

` ``
Template details page
` ``

Recommendation:

**Return to Template Details page**

because users usually want to review or manage the template they just created.

---

# Part 6 – Cancel / Exit Safety

If the user leaves the wizard mid-way:

Optionally warn if unsaved schema changes exist.

But **do not implement complex unsaved-change tracking in this slice**.

Simple behavior is acceptable for now.

---

# Scope

### In scope

* wizard step guards
* stepper correctness
* navigation button behavior
* refresh safety
* finish navigation

### Out of scope

* validation changes
* schema editor redesign
* export logic
* backend API changes
* modal redesign

---

# Acceptance Criteria

### Step Guard

User cannot open Step 2 without a created template.

User cannot open Step 3 without schema.

---

```

Stepper

Active step always matches the current page.

Completed steps appear correctly.

Refresh

Refreshing Step 2 keeps the schema editor open.

Refreshing Step 3 keeps export studio open.

Navigation

Back button moves to previous step without data loss.

Finish returns to Template Details.

Implementation Order

1. Implement step guard logic
2. Ensure stepper state derives from route
3. Ensure refresh reloads wizard state
4. Verify Back / Next navigation
5. Verify Finish destination

Why This Slice Matters

Without this slice:

- * Step 1 redesign may create navigation edge cases
- * users can enter steps with incomplete state
- * refresh behavior may break wizard

This slice ensures the wizard behaves like a **proper production workflow**, not just three screens.

After This Slice

Your wizard system will be:

...

Step 1 (clean entry)

```

Step 2  (stable schema editor)
Step 3  (clean export studio)
...

With:

* consistent navigation
* clear UX
* predictable state

which is exactly what a senior-level internal admin tool should feel like.

---

If you'd like, I can also show you one final optional slice that will dramatically improve the product:

> Template Version Lifecycle UX (Draft → Published)

This is the feature that separates a simple builder from a real enterprise template system.

```

▼ slice D - changes

▼ problems

- fix status for template to be active/archived (instead of hard delete)
- make when collapse and open the side bar the page shrink and widen, not moved

▼ To do

- remove schema editor header in template nav, and make back button align/ in the same line save and cancel
+ make the back button clear and shown
- add the "restrict edit" option in the export page with disabled (no impl)
- add icons in the predefined templates
- show validation in the modal of the template name and sector code in step 1 only when the input at least touched e.g. asterisk for the required inputs

▼ Service 2 (Data Submission) - before recovery plan

▼ Submission Service Planning 1

I checked the uploaded UI template and your current direction. Your reading is correct: the current "Data Submission" flow in the HTML starts with only "Upload Filled Data," then jumps to "Validation & Review," then "Review & Submission," while telling the user to upload a file "according to the downloaded template." That only makes sense if the system already knows which template/version the file belongs to. Right now, that selection/matching step is missing from the UX.

My verdict:

You should treat **Template Creation/Export** and **Data Submission** as **two bounded contexts**, but you should **not rush into two physical microservices yet**. Start with a **modular monolith** with two clear modules:

- `template` module
- `submission` module

That gives you the right separation in design without paying premature microservice cost.

Why I'm saying that:

Your existing template service already revolves around a strong idea: **schema is the source of truth, Excel is derived output**. The uploaded architecture notes say exactly that, and also explicitly kept data submission out of the template-creation scope.

So conceptually, yes, submission is a separate service/domain. But operationally, splitting into separate deployable microservices now is probably too early because:

- both flows depend on the same schema/version concepts
- both need the same canonical template metadata
- both will likely share validation semantics
- you are still stabilizing product behavior, not scaling teams independently yet

So the senior recommendation is:

separate by architecture now, separate by deployment later if needed.

Think of it like this:

```
System
├── Template Module
│   ├── create template
│   ├── version template
│   ├── validate schema definition
│   └── export excel template
└── Submission Module
    ├── choose template/version
    ├── upload filled excel
    ├── validate workbook against template schema
    ├── review/correct
    └── submit accepted dataset
```

The important point is that the second module does **not** validate “random Excel.”

It validates **an Excel submission against a specific published template version**.

So on your confusion: yes, you are right.

The user must either:

- explicitly choose the template/version before validation, or
- upload a file that contains enough metadata so the system can auto-detect the template/version.

Best practice is to support both, but prioritize auto-detect.

The cleanest submission flow is not the current 3-step UI. It should become this:

Recommended data submission wizard

Step 1 — Template Identification + Upload

Not just upload. This step should include one of these:

- auto-detect template from hidden metadata sheet
- if auto-detect fails, let user select:
 - template
 - template version
 - optionally reporting period / company context

Step 2 — Validation Results

This is where validation finally makes sense.

Step 3 — Review & Correction

Only for correctable issues.

Step 4 — Final Submission

Lock and submit.

That is much cleaner than the current UI, because the current file implies validation before identity is established.

The strongest design here is to use the metadata already discussed in your template service. Your earlier architecture notes proposed embedding template/version/hash metadata into the Excel artifact. That is exactly what makes submission reliable.

So the exported template should always carry hidden metadata like:

```
template_id
version_id
version_number
schema_hash
generated_at
```

Then submission works like this:

```
User uploads Excel
  ↓
Read hidden metadata sheet
  ↓
Find published template version
  ↓
Confirm schema_hash matches known version
  ↓
Run validation against that exact schema
  ↓
Show issues
  ↓
Allow review/submission
```

That removes ambiguity and avoids “choose template manually” in the common case.

Now the important architecture question: do you need another validation layer?

Yes — but not a duplicate one.

You now have **two different validation domains**:

1) Design-time validation

This already exists in template creation.

It validates the **schema definition itself**:

- valid sheet names
- unique table keys
- unique field keys
- allowed field types
- valid min/max rules
- valid enum usage

That belongs to template authoring.

2) Run-time submission validation

This belongs to data submission.

It validates the **uploaded workbook data against a published schema**:

- workbook can be opened
- required metadata exists
- template exists and is published
- required sheets exist
- expected headers exist
- cell values match type
- required fields are present
- enum values are allowed
- min/max constraints pass
- row-level or cross-field rules pass
- duplicate submission rules pass

So yes, there is a second validation layer, but it is not redundant.

It is validating a different thing.

The right move is:

- keep **schema validation** in template module
- create **submission validation** in submission module
- extract only the reusable low-level rule engine if needed

Example:

```
shared kernel
├─ field-rule evaluator
│  ├── type checks
│  ├── enum checks
│  └─ min/max checks
└─

template module
├─ schema validator

submission module
├─ workbook validator
│  ├── metadata matcher
│  └─ sheet/header validator
```

```
└─ row/cell validator
└─ submission policy validator
```

That is the right boundary.

What should change in template creation service to support submission?

A few things matter a lot:

Must-have changes

1. Published version concept

Right now latest-editable/versioning is fine for authoring, but submission must validate only against a stable version.

So you need a state like:

- DRAFT
- PUBLISHED
- DEPRECATED

Submission should only accept **PUBLISHED**.

1. Metadata sheet in exported Excel

This is not optional anymore if submission is coming.

2. Canonical schema contract

The submission module must consume the exact same schema structure the template module produces.

3. Template-version retrieval API

Submission needs an internal way to fetch published template/version/schema by:

- version id, or
- template id + version number, or
- schema hash

Nice-to-have later

- privacy/public visibility
- submission period rules
- correction workflows
- approval/review on submitted data
- duplicate submission prevention per company/period

Now let's answer the CTO point directly.

Is he right that these should be "separate services" and "microservices"?

Conceptually yes. Physically not necessarily yet.

He is right in the business/domain sense:

- template authoring/export is one responsibility
- data submission/validation/review is another

But if he means "deploy two independent microservices immediately," I would not recommend that as the first implementation step.

Because if you split too early, you immediately create:

- inter-service contracts

- version compatibility problems
- duplicated models
- more DevOps overhead
- more debugging paths
- harder local development
- transaction and consistency questions

And right now your biggest risk is not scale.

Your biggest risk is **wrong boundaries and duplicated logic**.

So the practical recommendation is:

Recommended architecture now

Modular monolith with strict internal boundaries

```

app
├── template
│   ├── api
│   ├── application
│   ├── domain
│   └── infrastructure
├── submission
│   ├── api
│   ├── application
│   ├── domain
│   └── infrastructure
└── shared
    ├── schema contract
    ├── common ids / enums
    └── reusable validation primitives
  
```

Future split trigger

Only split into microservices when one of these becomes true:

- different teams own them
- different release cadence is needed
- submission traffic/scaling is very different
- external integrations make submission operationally heavier
- you need independent deployment for risk isolation

Until then, modular monolith is the senior choice.

My recommended implementation plan for the next service is this:

Phase 1 — Freeze submission architecture

Define:

- actor: external/user-side submitter
- input: uploaded Excel

- source of truth: published template version schema
- output: validated submission record + issues

Phase 2 — Add missing template-side support

Implement in current template module:

- **PUBLISHED** version state
- hidden metadata in exported workbook
- internal read API for published template/version/schema

Phase 3 — Build submission module

First slice only:

- upload workbook
- extract metadata
- resolve template/version
- validate workbook structure
- return validation report

Not final submission yet.

Just upload + identification + validation.

Phase 4 — Add review/correction

- issue list
- editable grid if you want it in-app
- revalidate after correction

Phase 5 — Add final submission persistence

- submission entity
- submission status
- submitted_at / submitted_by
- period uniqueness rules

My strongest product correction to your current UI is this:

Do not keep Step 1 as only “upload file.”

That is the main flaw.

It should become:

“Upload & Identify Template”

with logic:

- auto-detect template/version from metadata
- fallback manual template/version selection
- show detected template before continuing

That resolves the confusion cleanly.

So the final answer in one line:

Yes, your intuition is right: data submission must validate against a selected or auto-detected published template version; architecturally it should be a separate bounded context, but implementation should start as a

modular monolith, not immediate microservices.

If you want, next I'll turn this into a concrete **Submission Service Architecture Pack** with domain model, DB entities, APIs, wizard steps, and first implementation slices.

▼ Submission Service Planning 2

Applied.

You made a good call on both points: learn this one with me instead of agent mode, and simplify backend architecture from clean architecture to layered. For this stage, that is the more practical choice.

On hashing first: **in your current backend, field order does matter.**

Why? Because your canonicalizer sorts **object keys**, but it does **not** normalize **array order**. In JSON terms:

```
{
  "tables": [
    {
      "fields": [
        {"fieldKey": "a"},
        {"fieldKey": "b"}
      ]
    }
  ]
}
```

and

```
{
  "tables": [
    {
      "fields": [
        {"fieldKey": "b"},
        {"fieldKey": "a"}
      ]
    }
  ]
}
```

will produce different canonical strings, because `fields` is an array, and arrays are written in their existing order.

So your current behavior is:

- different **object property order** → same hash
- different **table array order** → same hash only if you already pre-sort before hashing
- different **field array order inside a table** → different hash
- different **sheet order / field display order** → likely different hash if represented by arrays and not normalized before hashing

That means your current hash is not just “business schema identity”; it is closer to **full structural fingerprint including field order**.

Now the real question is not technical, it is business semantics:

Should field order matter in this business?

For your case, I would split this into two meanings:

Option A — order matters

Use this when field order is part of the contract:

- Excel column order matters to users
- integrations expect fixed column position
- reordering is considered a meaningful template version change

Then current behavior is acceptable.

Option B — order does not matter logically

Use this when:

- same fields, same rules, same meaning
- only column arrangement changed
- you want submission validation to treat them as the same schema

Then current hashing is too strict.

My recommendation is the practical middle ground:

Use two hashes, not one

This is the cleanest design.

1) `schema_hash`

Represents **logical validation identity**

- ignores table order
- ignores field order inside a table
- keeps only semantic content:
 - table key
 - field key
 - type
 - required
 - validations
 - maybe sheet/table name only if it matters semantically

This is the one submission should use for compatibility checks.

2) `layout_hash` or `export_hash`

Represents **Excel/export layout identity**

- includes table order
- includes field order
- includes sheet names/header names if export depends on them

This is the one useful for export determinism and UI/layout change tracking.

That avoids a very common trap:

```
same business schema
but different column order
=> should submission reject? usually no
=> should exported file differ? yes
```

One hash cannot express both meanings cleanly.

Now about the uploaded exported Excel: I checked it directly.

Your exported workbook does have hidden metadata already. It contains these sheets:

- `_validation` → hidden
- `Instructions` → visible
- `sh1`, `sh2`, `sh3` → visible data sheets
- `__metadata__` → **veryHidden**

And `__metadata__` includes:

- `template_id`
- `version_id`
- `version_number`
- `schema_hash`
- `generated_at`
- `generator_version`

So yes, you already have the hidden metadata mechanism in the actual exported file. What you saw as “Instructions” is different; that is a visible helper sheet. The hidden metadata is separate, and that aligns with the earlier architecture notes that explicitly required a hidden metadata sheet with template/version/hash.

That is good news, because it means the future submission service can rely on a real workbook fingerprint path instead of inventing it later.

There is one important implication though:

If you keep current `schema_hash` as order-sensitive

then submission will reject files when only field order changed between versions, even if the logical validation model is effectively the same.

That may be okay, but only if you want strict template-version binding.

So the decision is:

```
Strict mode:
uploaded workbook must match exact exported version/layout
=> keep current hash behavior

Semantic mode:
uploaded workbook may match same logical template even if order differs
=> introduce logical schema hash
```

For your project, I would choose:

recommendation

Keep strict version matching for MVP submission.

Meaning:

- submission is against a specific exported template version
- metadata sheet identifies that exact version
- validator checks workbook against that exact version
- no fuzzy compatibility in MVP

That keeps the system simple and less error-prone.

Then later, if business asks for “compatible across reordered exports,” add logical compatibility rules intentionally, not accidentally.

Backend architecture after your refactor

I agree with your move from clean architecture to layered architecture.

For this project stage, layered is the better fit because:

- your team size is small
- the domain is still settling
- you want fast comprehension
- you are still learning framework behavior deeply
- over-layering now would make simple flows harder to reason about

So for the backend, I would keep:

```
com.eca
├─ template
│   ├── controller
│   ├── dto
│   ├── service
│   ├── repository
│   ├── entity
│   ├── validation
│   ├── hashing
│   ├── excel
│   └─ exception
├─ submission
│   ├── controller
│   ├── dto
│   ├── service
│   ├── repository
│   ├── entity
│   ├── validation
│   ├── parser
│   ├── matcher
│   └─ exception
└─ shared
    ├── dto?           (only if truly generic)
    ├── exception
    ├── util
    ├── config
    └─ common model/value objects
```

Should **shared** still exist?

Yes, but it should stay **small and boring**.

A shared module is still useful in modular monolith layered architecture, but it should contain only things that are truly cross-cutting, for example:

- common error response shape
- generic exception base classes
- audit helpers
- reusable Excel/file utility
- generic value objects or enums used by both services
- maybe common workbook-reading helpers

What should **not** go into shared:

- template business rules
- submission business rules
- “convenience dumping ground”
- anything copied there only because two services happen to need it today

A good rule:

```
shared = technical reuse
not business ambiguity
```

If a class contains template meaning, keep it under `template`.

If it contains submission meaning, keep it under `submission`.

Where does shared live?

At the same top package level as the feature modules:

```
com.eca.shared
com.eca.template
com.eca.submission
```

Not inside `template`, because then `submission` becomes dependent on `template`, which is the wrong direction.

What should submission reuse from template?

Not direct service-to-service entanglement inside the same codebase.

Prefer one of these:

Better now

Submission depends on:

- template repository/query service
- or template read facade

Example idea:

```
template/
  service/
    TemplateQueryService
      - getPublishedVersion(versionId)
      - getPublishedSchema(templateId, versionNumber)
```

Submission can call a **read-only template-facing service/facade**, not poke randomly across repositories and entities.

That gives you modular monolith discipline without fake microservices.

Frontend architecture

I inspected your frontend structure. It is already feature-based:

- `features/templates`
- `features/schema`
- `features/export`
- `features/versions`
- `shared`
- `app`

That is not overengineering. It is actually a good middle ground.

Your current frontend already has a decent separation:

- app shell / router in `app`
- reusable UI and libs in `shared`
- domain-ish feature folders in `features`

So for the new service, yes:

create another feature module for submission

Something like:

```
src/  
  app/  
  features/  
    templates/  
    schema/  
    export/  
    submission/  
      api.ts  
      types.ts  
      pages/  
      components/  
      lib/  
  shared/
```

If submission grows more, you can later split it into:

- `submission-upload`
- `submission-validation`
- `submission-review`

But right now I would keep it under one `submission` feature.

Is your frontend architecture too early?

No. Not from what I saw.

It is actually in the healthy zone:

- not flat chaos
- not enterprise over-abstraction

- feature-first enough
- shared layer exists
- routing is understandable

The only thing I would refine over time is this:

current tension

You have both:

- `templates`
- `schema`
- `export`
- `versions`

That is okay, but it can drift into one of two shapes:

shape 1 — business capability oriented

- `template-creation`
- `submission`

shape 2 — technical slice oriented

- `templates`
- `schema`
- `export`
- `versions`

Right now you are more in shape 2.

For the next stage, because you now have two real business services, I would gradually move toward shape 1 for page-level ownership:

```
features/
  template-creation/
    pages/
      components/
        api/
        lib/
    submission/
      pages/
        components/
          api/
          lib/
  shared/
```

Inside `template-creation`, you can still have internal subfolders like:

- `schema`
- `export`
- `versioning`

This becomes easier to reason about in product terms.

So my recommendation is:

frontend recommendation

Do not refactor the existing frontend heavily right now.

Just evolve it toward this direction while adding submission:

- keep `shared` as is
- add `features/submission`
- do only light consolidation later if `templates/schema/export/versions` start feeling fragmented

In other words:

Do not pause to “perfect” frontend architecture now.
Add submission cleanly first.
Refactor only when the friction becomes real.

What I would decide now

Backend

Use **modular monolith + layered architecture**.

Backend package structure

Keep:

- `template`
- `submission`
- `shared`

Shared rules

Keep it small, technical, and stable.

Frontend

Add:

- `features/submission`

Do not split into a separate frontend app.

Hashing

For MVP:

- keep strict version-based matching
- current metadata mechanism is good
- current field-order-sensitive hash is acceptable **if** you mean exact version/layout identity

Later:

- add separate logical schema hash if business needs semantic compatibility across reordering

My direct answers to your questions

“does the shared module still exist?”

Yes, but only for true cross-cutting utilities and contracts, not business logic dumping.

“where would it live?”

Top-level beside the service modules:

- `com.eca.shared`
- `com.eca.template`
- `com.eca.submission`

“would we create another module in frontend for the new service?”

Yes. Create a `submission` feature/module.

“is current frontend architecture overengineering?”

No. It is acceptable and useful. I would not call it overengineered yet.

“is layered backend okay after refactor?”

Yes. For this stage, I agree with that decision.

The most important subtle point

Right now your hidden metadata sheet gives you a powerful submission entry point:

```
Uploaded workbook
  ↓
Read __metadata__
  ↓
Resolve template/version/hash
  ↓
Load stored schema
  ↓
Validate workbook against that exact version
```

That means the submission service does **not** need to invent template identity from sheet names alone.

That is one of the biggest simplifications in your whole future design.

Next best step is to define the **Submission MVP contract** very concretely:

- backend package structure
- DB entities
- upload/validate API
- workbook validation flow
- frontend wizard steps

I can draft that next in the same style, with a recommended package tree and first implementation slice.

▼ Submission Service Planning 3

Applied.

Your current direction is good for MVP. I agree with it, with one important caution around `_validation`.

1) Keep only `schema_hash` now

Yes, that is the right simplification.

Your decision is practical:

- keep one hash only
- make field ordering deterministic before hashing

- postpone `layout_hash` until business actually asks for layout-level identity

That is a healthy move because right now adding a second hash would increase concept count without solving an immediate problem.

The important thing is this:

```
same logical schema
→ same canonical order
→ same schema_hash
```

So your next concern is not “multiple hashes,” but just:

- how to sort fields
- whether sort is stable and predictable
- whether the same canonicalization is used everywhere

That is enough for now.

2) About `Instructions` sheet and `_validation` sheet

These two sheets usually serve very different purposes.

`Instructions`

This is just user guidance.

Typical purpose:

- tell the user how to fill the workbook
- explain required fields
- explain upload rules
- explain date/number formatting
- warn not to rename sheets/headers

So yes, this one is optional from a system perspective.

If export option says it can be disabled, that is perfectly fine.

Removing or disabling it does not normally break validation logic.

`_validation`

This one is different.

This sheet is often not “for users.”

It is often a **technical support sheet for Excel itself**.

Why it commonly exists:

When Excel dropdown validation is generated dynamically, the dropdown source often needs to point to a cell range somewhere in the workbook.

Example idea:

```
Allowed values for Status field:
A
B
C
```

Excel validation may reference:

```
=_validation!$A$1:$A$3
```

or a named range that internally points there.

So `_validation` is often the backing store for:

- enum dropdown values
- named ranges
- list validations used by visible sheets

That means the sheet may look “useless” from business perspective, while still being operationally important.

3) Can you remove `_validation` sheet?

Maybe yes, maybe no.

But you should assume **it is functional until proven otherwise**.

Your intuition is understandable:

“the real sheet already has validation, so why keep `_validation`?”

Because Excel validation in the real sheet may be **powered by** `_validation` **behind the scenes**.

So there are only 3 safe possibilities:

Option A — keep it as hidden/very hidden

This is the safest choice.

Good when:

- dropdowns depend on it
- named ranges depend on it
- you do not want to refactor export logic now

This is the option I recommend for now.

Option B — remove it only if validations are inline/static

You can remove `_validation` only if generated validations do not depend on an external sheet.

Example of inline validation:

- small hardcoded list directly inside formula
- validation not using referenced ranges
- no named range pointing to `_validation`

If your exporter writes validations directly without external references, then `_validation` may indeed be unnecessary.

But this must be verified in code, not assumed.

Option C — replace it later with named ranges or another approach

Possible, but this is refactor territory.

You already said you do not want refactoring now, so skip this.

4) My recommendation on `_validation` for your current stage

Do **not remove it now**.

Not because it is beautiful, but because MVP discipline matters here:

- no unnecessary refactor
- no breaking Excel behavior accidentally
- no debugging dropdown validation before starting submission service

So the better decision now is:

current recommendation

- `Instructions` : optional, can stay configurable
- `_validation` : keep it for now
- if possible, hide it better
- only remove it after confirming export still works with all enum/dropdown validations

That matches your own principle:

| at the current stage, I would not fix/refactor any problem

Exactly. Removing `_validation` can easily become a hidden refactor.

5) About your MVP submission decision

I agree with this too:

- identity Excel check using current hash
- optional manual template selection
- if user selects template, use only latest version
- status/published visibility later

This is a valid MVP cut.

But there is one subtle product risk:

Risk in “select template → use latest version”

If user uploads an older exported file, and you validate it against the latest version only, you may get false mismatches.

Example:

```
User downloaded v1 yesterday
Admin changed template today → now latest is v2
User uploads old file
System validates against latest only
→ may wrongly reject or misread
```

So I would recommend this very practical rule:

MVP matching rule

1. First try identity from uploaded Excel metadata/hash
2. If identity cannot be resolved, then allow manual template selection
3. If user selects template manually, use latest version for that template

That is much safer than always forcing latest.

So the flow becomes:

```

Upload file
↓
Read metadata/hash
↓
If exact template/version found → use it
Else fallback to manual template selection
↓
If manual selection used → use latest version

```

This keeps your MVP simple while respecting the uploaded file when possible.

That is the best compromise.

6) My verdict on your decisions

Good decisions

- one `schema_hash` only for now
- deterministic field ordering before hashing
- no `layout_hash` yet
- no broad refactor now
- postpone status/publication controls
- start submission with identity-first validation

One thing I would adjust

Do not remove `_validation` now unless you explicitly verify that Excel validations do not depend on it.

So the clean recommendation is:

```

Keep _validation now
Hide it if possible
Ignore it in UX discussion
Revisit later only if there is real need

```

7) Practical mental model

Think of the workbook like this:

```

Workbook
├─ Visible business sheets
│  └─ Instructions (optional)
│  └─ Data Sheet 1
│  └─ Data Sheet 2
│  └─ ...
└─ Technical support sheets
   └─ __metadata__ → identity
   └─ _validation → Excel dropdown support

```

So:

- `__metadata__` is for **system identity**

- `_validation` is for **Excel mechanics**
- `Instructions` is for **human guidance**

That distinction helps a lot.

Your current MVP path is sound. The only thing I would firmly protect is: **do not break** `_validation` **casually** just because it looks redundant.

Next, I suggest we lock the **Submission MVP flow** as a concrete 3–4 step wizard with exact behavior in each step, before touching backend structure.

▼ Submission Service Planning 4

Applied.

I checked the CTO flow and your current system direction. The right move now is **not** to build the whole submission lifecycle at once. The clean MVP is:

start with “upload + identify template + validate workbook against exact version”

and postpone full correction grid / final persisted submission until the validation path is solid.

That fits your minimal-change rule, and it matches the biggest gap in the CTO template: the current submission UI jumps from upload to “Validation & Review” without first establishing which template/version the file belongs to. In the HTML, Step 1 is only upload, and Step 2 validation is currently simulated in JS, not real validation.

My recommendation

Do **Submission MVP v1** as this:

```

Upload Excel
↓
Read __metadata__ from workbook
↓
Resolve template/version/schema_hash
↓
Load exact template version schema from template module
↓
Validate workbook structure + cell values
↓
Return validation report

```

That is the first real milestone.

Not this yet:

```

upload → validate → editable correction grid → final submission persistence

```

That second one is a larger product slice and will pull you into:

- row editing UX
- server-side draft state
- reconciliation rules
- submission statuses
- duplicate submission policies
- review/finalization lifecycle

Too much for first cut.

What stays the same

Keep the current architectural decision:

- **modular monolith**
- **layered/n-tier**
- separate feature/module for submission
- shared kept very small

The old architecture notes emphasized clear responsibility boundaries and warned against mixing concerns too early. That still applies here.

So the backend shape should become:

```
com.eca
├─ template
├─ submission
└─ shared
```

Not a new deployable microservice yet.

And the frontend should become:

```
src/features/submission
```

parallel to your current feature folders, not a separate frontend app. The existing project already follows feature-based frontend structure in the architecture guidance, so adding `submission` is the natural extension.

Minimal UX change to the CTO template

Do **not** add a whole new wizard step right now.

Instead, change **Step 1** from:

“Upload Filled Data”

to:

“Upload & Identify Template”

That preserves the CTO’s 3-step structure while fixing the core logic gap.

Step 1 becomes

- upload Excel
- read workbook metadata
- show:
 - template name
 - version number
 - schema hash
 - status: exact match / metadata missing / metadata mismatch
- fallback: manual template selection only if metadata cannot be resolved

That is the minimum correction to the current template, whose Step 1 currently only asks to upload the file.

Step 2 becomes real

Replace the fake validation summary with actual backend validation output:

- critical errors
- warnings
- valid sheets / invalid sheets
- row count
- issue list by sheet / row / column

Right now that screen is static demo data in the HTML.

Step 3 for MVP

Keep it lightweight:

- review validation result summary
- allow download of validation report
- optionally “accept and continue later”
- do **not** implement full in-app row editing yet

The current HTML has a full data grid/editor idea, but that is phase 2, not first submission slice.

Backend MVP boundary

Submission module v1 should own only these responsibilities:

In scope now

- accept uploaded Excel file
- parse workbook
- read `__metadata__`
- resolve template version
- validate workbook against stored schema
- return structured validation report

Out of scope for this first slice

- store final business data into another domain model
- correction UI state / draft save
- final official submission lifecycle
- approval workflow
- public/private template visibility
- latest-version selection rules beyond fallback

That keeps the scope sharp.

Exact module design

Template module remains source of truth

It already owns:

- templates
- versions
- schema JSON

- schema hash
- export

And your architecture notes already defined schema as source of truth and Excel as derived output.

Submission module consumes that truth

It should not redefine template rules. It should ask template-side read services for:

- version by id
- version by metadata identity
- schema JSON
- schema hash

So think of it like this:

```
template module
  owns schema definition

submission module
  validates uploaded workbook against that schema
```

That is the right separation.

Submission backend package shape

Use this, aligned with your layered style:

```
com.eca.submission
├─ controller
├─ dto
├─ service
├─ validation
├─ parser
├─ matcher
├─ model
└─ exception
```

Responsibility split

parser

Reads uploaded workbook:

- sheet names
- headers
- rows
- metadata sheet

matcher

Decides which template version to validate against:

- exact match from metadata
- fallback manual template selection

validation

Runs runtime checks:

- workbook-level
- sheet-level
- row-level
- cell-level

service

Orchestrates:

- upload
- parse
- match
- validate
- report assembly

That is enough for MVP.

Validation layers: keep them separate

You asked this earlier, and now it becomes concrete.

Template module validation

Validates the schema definition itself.

Submission module validation

Validates uploaded workbook data against that schema.

Do not duplicate the first one.

Do build the second one.

The second one should reuse semantic pieces when possible, but it is still a distinct runtime validator.

Submission MVP API

This is the clean first contract.

1) Validate upload

`POST /api/submissions/validate`

Multipart request:

- file
- optional `templateId`

Behavior:

- parse workbook
- read metadata
- resolve exact version if possible
- if metadata missing and `templateId` present, use latest version of that template as fallback
- validate

- return report

2) Get available templates for fallback selection

You may not even need a new endpoint if your existing template list already gives enough info for dropdown selection.

That is why I would not invent more endpoints yet.

Validation report shape

The response should be structured enough for Step 2 UI.

Core sections:

```
resolvedTemplate  
resolvedVersion  
identityCheck  
summary  
issues[]
```

identityCheck

- metadataFound
- templateMatched
- versionMatched
- schemaHashMatched
- fallbackUsed

summary

- criticalErrors
- warnings
- sheetsChecked
- validSheets
- rowsChecked

issues[]

Each issue should include:

- severity
- scope: workbook / sheet / row / cell
- sheetName
- rowNumber
- columnName
- code
- message

This is the payload that makes Step 2 real.

Validation rules for v1

Do not overbuild rules. Start with these only:

Workbook-level

- file is readable Excel
- `__metadata__` sheet exists
- required metadata keys exist
- template/version exists
- schema hash matches expected version

Sheet-level

- expected sheets exist
- no unexpected required-sheet omission
- header row exists
- headers match expected schema

Cell/row-level

- required field not blank
- type check:
 - TEXT
 - NUMBER
 - DATE
 - BOOLEAN
 - CURRENCY
- enum value allowed
- min/max allowed where defined

Not now

- cross-table consistency
- business aggregate rules
- formula auditing
- period duplication checks

Those can come later.

Frontend MVP structure

Create:

```
src/features/submission
├─ api.ts
├─ types.ts
├─ pages/
│   └─ SubmissionWizardPage.tsx
└─ components/
    ├─ UploadAndIdentifyStep.tsx
    ├─ ValidationResultsStep.tsx
    └─ ReviewStep.tsx
```

Routing

Add one route first:

```
/submissions/new
```

Keep it simple.

UI strategy

Reuse your current wizard style and shared components. Do not create a second UI system.

The safest first implementation slice

Here is the exact order I recommend.

Slice S1 — Submission validation only

Backend:

- create submission module skeleton
- implement upload endpoint
- parse workbook metadata
- call template read service
- validate workbook structure
- return structured validation response

Frontend:

- add submission wizard page
- Step 1 upload + identity
- Step 2 real validation report
- Step 3 simple review summary only

No persistence of submission yet.

That gives you a real working submission module with minimal architectural disturbance.

Practical decisions I would lock now

1) Matching priority

Use this exact order:

```
A. metadata version match
B. metadata template match + schema hash match
C. manual template selection fallback -> latest version
D. otherwise fail
```

2) Manual selection rule

Since you explicitly chose it:

- manual template selection uses only latest version for now

That is acceptable **only as fallback**, not as first choice.

3) Current hash

Use current `schema_hash` exactly as-is for identity check in MVP, as you decided.

4) `_validation` sheet

Ignore it in submission matching unless you later intentionally validate export mechanics. It is not the identity sheet.

5) Instructions sheet

Ignore it in validation; optional and not part of schema identity.

What I would not do yet

I would not start with:

- DB tables for submitted business records
- final submission entity/statuses
- editable data grid like the CTO HTML
- correction save workflow
- microservice split
- publication/status visibility model

That would slow you down and blur the real first goal.

Final verdict

The correct start is:

build Submission MVP as a parallel module that performs upload → metadata identity → schema-based validation → report, while keeping the CTO 3-step wizard and only minimally correcting Step 1 to include template identification. The CTO HTML already shows the 3-step submission flow, but its current validation behavior is demo-only, so that is exactly the part we should make real first.

If you approve this approach, next I'll give you the **Submission Module Starter Pack**:

- backend package tree
- DTO contracts
- first endpoint contract
- frontend route + page structure
- slice-by-slice implementation order without code yet

▼ Codex CLI + Spec-kit CLI — overview (later to use)

Applied.

Good decision. Using **Codex CLI + Spec-Kit but reviewing everything manually** is exactly the right way to learn agents without letting them take control of the architecture. Think of them as **structured assistants**, not autopilots.

Let me give you the **mental model first**, then the **practical workflow**, then **how to use it specifically for your submission module**.

1. What Codex CLI + Spec-Kit actually are

Think of them as two different layers.

Codex CLI

Codex CLI is essentially an **AI coding agent that operates inside your repository**.

It can:

- read your project files
- generate code
- modify code
- run commands
- implement tasks

But it does not inherently enforce structure.

So if you just tell it:

```
| implement submission module
```

it may produce **chaotic patches**.

Spec-Kit

Spec-Kit is a **structured development protocol**.

Instead of saying:

```
| implement X
```

you force the agent to go through stages:

```
spec → plan → tasks → implementation
```

This dramatically improves quality.

Think of Spec-Kit as:

```
architecture guardrails for AI coding
```

Combined mental model

```
You (architect)
  ↓
Spec-Kit (structure / spec)
  ↓
Codex CLI (implementation engine)
```

So **you control thinking**, Codex executes.

This is exactly the correct balance for your learning stage.

2. The workflow you should follow

You should **never jump directly to implementation**.

Always follow this pipeline:

- ```
1. SPEC
2. PLAN
3. TASK BREAKDOWN
4. IMPLEMENTATION
5. REVIEW
```



Spec-Kit CLI is designed to support this.

---

### 3. Your real workflow (step-by-step)

Inside your project root.

#### Step 1 — create a spec

Example:

```
spec-kit spec submission-module
```

It will generate something like:

```
specs/
 submission-module/
 spec.md
```

This file is **the product + architecture definition**.

Here you define:

- problem
- architecture
- module boundaries
- API contracts
- validation rules
- UX flow

You should write this mostly **yourself**.

The AI can help refine, but **you own the architecture**.

---

#### Step 2 — generate a plan

Then:

```
spec-kit plan submission-module
```

This creates something like:

```
plan.md
```

Which converts the spec into:

```
phases
milestones
technical approach
dependencies
```

This is where Codex begins to help.

But again — **review everything**.

---

#### Step 3 — generate tasks

Then:

```
spec-kit tasks submission-module
```

Now you get a list like:

```
tasks/
 001-create-submission-module
 002-upload-endpoint
 003-excel-parser
 004-template-matcher
 005-workbook-validator
 006-validation-report
```

Now the work is broken into **small safe tasks**.

---

## Step 4 — implement task-by-task

Now you use Codex CLI.

Example:

```
codex run tasks/001-create-submission-module.md
```

Codex reads:

```
task description
repo structure
existing code
```

Then generates code.

You review the diff before accepting.

This prevents agent chaos.

---

## 4. How YOU should control the agent

Your role becomes:

```
architect
reviewer
quality gate
```

Not code generator.

For every task you should ask:

- ```
1. Does this match the architecture?  
2. Did it reuse existing code?  
3. Did it create unnecessary abstractions?  
4. Did it break module boundaries?
```

Reject anything that violates design.

5. Where this fits your project

Your project already has two domains:

```
template creation
submission
```

So your first spec should be:

```
submission-module
```

Inside the spec define:

Submission service scope

```
upload filled Excel
identify template version
validate workbook against schema
return validation report
```

Explicitly mark out-of-scope:

```
data correction UI
submission persistence
approval workflow
template visibility
```

This keeps the agent from implementing half your roadmap.

6. Example spec outline you should use

Inside:

```
specs/submission-module/spec.md
```

Write something like:

```
Purpose
-----

Allow users to upload filled Excel templates and validate them against the schema defin
ed by the Template Creation service.

Scope
-----

Upload workbook
Extract metadata
Resolve template version
Validate workbook against schema
Return validation report

Out of Scope
```

```
-----  
  
Final submission persistence  
Approval workflows  
In-app data editing  
Duplicate submission detection
```

Then architecture:

```
Architecture  
-----  
  
Modular monolith  
Layered architecture  
  
Modules  
  
template  
submission  
shared
```

Submission components:

```
parser  
matcher  
validator  
service  
controller
```

7. Important rule when using agents

Never allow the agent to invent architecture.

Architecture must come from:

```
your spec
```

Otherwise agents will introduce things like:

```
event bus  
repository abstractions  
domain service layers  
CQRS  
factories  
builders
```

Your project **does not need that yet**.

8. What Codex CLI will be best at in your case

Codex will be excellent for:

repetitive code

Examples:

```
DTO classes
controllers
validation result models
excel parsing loops
mapping logic
```

But weak at:

```
architecture boundaries
domain modeling
long-term maintainability
```

So the balance is perfect.

9. What NOT to use the agent for

Avoid letting the agent design:

```
submission domain model
template matching logic
validation rules
module boundaries
API contracts
```

You define those.

Then the agent implements.

10. Ideal development loop

Your future development rhythm should look like this:

```
Define spec
  ↓
Generate plan
  ↓
Review plan
  ↓
Generate tasks
  ↓
Implement task-by-task with Codex
  ↓
Review every diff
```

This is **AI-assisted engineering**, not **AI-driven engineering**.

11. One more important tip

When using Codex CLI, always run it with **small context tasks**.

Bad:

```
implement submission module
```

Good:

```
create submission module package structure
```

Then:

```
implement Excel metadata parser
```

Then:

```
implement template version matcher
```

Small tasks dramatically improve output quality.

12. My suggestion for your next step

Before using the agent, we should write a **very solid submission spec**.

Because that spec will guide everything.

Next I can help you produce a **Spec-Kit ready specification** for:

```
submission-module
```

Including:

- backend module structure
- validation flow
- API contracts
- DTO models
- frontend wizard steps
- validation report schema

That document will make Codex behave much better.

If you want, I can generate that **Spec-Kit ready submission spec** next.

▼ fixes to resolve

- see if `_validation` could be removed, if it is based on a logic, rename to suitable name, and make it very hidden like `_metadata`
- fix hashing on template fields, so the order of fields per template does not matter.

▼ Build strong understand for the project

▼ Step for recovery plan

Let me be direct (HEP on):

⚠ You're currently in a **danger zone**:

You have a working system, but you **don't understand it** → **you don't control it**.

| If you continue like this with agents, you will eventually break it and not know why.

1) Your idea (ask Codex to explore) — good or bad?

👉 Conditionally viable (but risky if done wrong)

Why it's NOT enough alone

If you do:

| "Codex, explain the whole project"

You'll get:

- shallow summaries
- hallucinated architecture
- missing critical flows

👉 Because agents:

- summarize files, not *system behavior*
 - don't truly "understand" runtime flow
-

2) What you SHOULD do instead (real strategy)

You need to **rebuild understanding layer by layer**, not all at once.

Think like this:

System understanding = Static + Flow + Decisions

3) Step-by-step recovery plan (practical)

Step 1 — Generate STRUCTURE MAP (use Codex)

Ask Codex:

| "Scan the project and list:

- modules/packages
- main entities
- main services
- controllers and endpoints

| Do NOT explain deeply, just structure."

👉 Output should look like:

Controller → Service → Repository → Entity

🎯 Goal: **see the skeleton**

Step 2 — Build FLOW understanding (MOST IMPORTANT)

Now pick **1 real feature** (e.g. create template)

Ask Codex:

```
"Trace the flow for creating a template:  
from API → service → DB → response"
```

👉 This is where real understanding happens.

You want:

```
Request → Controller → Service → Validation → DB → Response
```

Step 3 — Come here (ChatGPT) for deep explanation

Paste:

- the flow
- or code snippet

Ask:

```
"Explain this flow deeply (mEEx)"
```

👉 This is where you actually **learn**

Step 4 — Document YOUR understanding (critical)

Create a file:

```
/docs/architecture.md
```

Write:

- flows
- rules
- constraints

👉 Not Codex-generated — **your version**

Step 5 — Validate your understanding

Try:

- "Add small feature manually"
- OR "modify logic"

If you struggle → you don't understand yet

4) Should Codex generate full documentation?

👉 Yes, BUT with strict constraints

Ask:

```
"Generate architecture documentation:  
• modules"
```


- data flow
 - key business rules
- based ONLY on code, no assumptions"

Then:

👉 **Treat it as draft, not truth**

5) The correct hybrid workflow (this is key)

Codex → extracts structure + traces flows
ChatGPT → explains deeply + challenges logic
You → build final mental model

6) Strong constraint usage (your idea)

Your idea:

| use exploration as constraints for future prompts

👉 **Very good — but refine it**

Instead of dumping everything, create:

"Working Constraints File"

Example:

```
# System Constraints

- Layered architecture: Controller → Service → Repository
- Template always uses latest version
- Schema stored as JSONB
- Validation happens before persistence
```

👉 Then in Codex prompt:

| "Follow constraints in /docs/architecture.md"

🔥 This is VERY powerful when done right

7) Critical mistakes to avoid (HEP warning)

✗ Mistake 1: Trusting generated docs blindly

→ leads to wrong mental model

✗ Mistake 2: Trying to understand whole system at once

→ cognitive overload

✗ Mistake 3: Skipping flow tracing

→ you understand structure, not behavior

✗ Mistake 4: Letting Codex “explain everything”

→ passive learning → weak understanding

8) What I would do if I were you (exact plan)

Day 1–2:

- Map structure (Codex)
- Trace 2 flows:
 - create template
 - export template

Day 3:

- Write architecture.md (your version)

Day 4:

- Modify small feature manually

Final verdict

👉 Your idea is **good direction but incomplete**

! Don't ask Codex to “teach you the system”

✅ Use Codex to **extract**, and ChatGPT to **understand**

If you want next step:

I can:

👉 walk with you on your actual repo and extract the **first real flow (template creation)** step by step

That would accelerate your understanding massively.

▼ Flow 1: template creation

▼ Flow explanation

Yes — I went through the actual repo, and here is the **first real flow: template creation**, step by step, based on the code you uploaded.

1) The real flow, end to end

The flow starts in the frontend at `src/features/templates/pages/CreateTemplateStartPage.tsx`.

When you click **Custom Template**, a modal opens with only 2 real business inputs:

- `name`
- `sectorCode`

On submit, the page does this:

```
templatesApi.create({
  name: trimmedName,
  sectorCode: trimmedSector,
  createdBy: 'system'
})
```

Then it navigates to:

```
/templates/create/{templateId}
```

So the frontend is not doing anything clever here. It is basically:

```
User input → POST /api/templates → get ids back → move to step 2
```

That means the **real creation logic is backend-owned**, not frontend-owned. That is good.

2) Backend entry point

The backend entry is:

```
TemplateController.createTemplate()
```

Path:

```
POST /api/templates
```

It accepts `CreateTemplateRequest`, which requires:

- `name`
- `sectorCode`
- `createdBy`

So the first gate is request validation, before business logic even starts.

Mental model:

```
Frontend POST
  ↓
TemplateController
  ↓
TemplateService.createTemplate(...)
```

The controller is thin, which is correct. It just forwards the request to the service and returns `201 Created`.

3) The actual creation logic in the service

The real work happens in:

```
TemplateService.createTemplate(String name, String sectorCode, String createdBy)
```

This method does **two inserts**, not one:

First insert: create the template record

It builds a `TemplateEntity` and sets:

- `name`
- `sectorCode`
- `status = DRAFT`
- `createdBy`

Then it saves it using `templateRepository.save(template)`.

So conceptually:

```
templates
- id
```

```
- name
- sector_code
- status
- created_at
- created_by
```

This is the **container identity** of the template.

Second insert: create version 1 automatically

Immediately after saving the template, the service creates a `TemplateVersionEntity` with:

- `template = saved template`
- `versionNumber = 1`
- `status = DRAFT`
- `schemaJson = initial schema JSON`
- `schemaHash = "PENDING_HASH"`
- `createdBy`

Then it saves that too.

This is extremely important:

| Creating a template in your system **always also creates v1**.

So the real flow is not:

```
Create template now, create version later
```

It is:

```
Create template identity
+
Create initial version snapshot
```

That is one of the main constraints of this project.

4) What the initial schema actually looks like

The initial schema is generated by:

```
SchemaJsonHelper.buildInitialSchemaJson(templateName, sectorCode)
```

It builds a JSON object like this conceptually:

```
{
  "templateName": "...",
  "sectorCode": "...",
  "tables": [],
  "exportProfile": {}
}
```

This is not random. It is there to make later schema editing safe.

Why this matters under the hood:

If `schemaJson` were null or half-empty, later flows like `PUT /versions/{id}/schema` would break or become inconsistent. So the system creates a valid minimal schema from day one.

That means:

```
Template creation is also schema bootstrapping
```

Not just a DB insert.

5) What gets returned to the frontend

The backend returns:

```
CreateTemplateResponse(templateId, versionId, 1)
```

So the response is intentionally small:

- template id
- first version id
- version number

Notice what it does **not** return:

- full template object
- full schema
- full version details

That is a deliberate pattern:

```
Create → return identifiers  
Then frontend loads detail page separately
```

This is why the frontend can navigate immediately to `/templates/create/{templateId}` and then fetch full data there.

6) What happens in step 2 after creation

After navigation, the page `CreateTemplateStepPage.tsx` runs.

It calls:

```
templatesApi.getById(templateId)
```

This hits backend:

```
GET /api/templates/{templateId}
```

That method is handled by `TemplateService.getTemplate()`.

What it does:

- load the template
- load all versions for that template ordered by version number descending
- map them to response DTOs

Then frontend picks the **latest version** from the returned versions list and passes its `versionId` into

```
SchemaEditorView.
```

So the next real step is:

```
Created template  
↓  
Load template detail
```

```
↓  
Find latest version  
↓  
Edit schema of that version
```

That is the practical wizard flow.

7) The most important mental model

Your system is **not** really:

```
Template → later maybe versions
```

It is:

```
Template = identity  
Version = editable working snapshot  
Schema JSON = source of truth
```

More visually:

```
POST /api/templates  
↓  
create TemplateEntity  
↓  
create TemplateVersionEntity v1  
↓  
attach minimal schemaJson  
↓  
return ids  
↓  
frontend opens step 2  
↓  
latest version becomes editable target
```

That is the first real flow.

8) HEP part — what is good, what is weak

Here is the honest evaluation.

Good

The flow is structurally sound:

- controller is thin
- service owns creation
- v1 is auto-created
- initial schema is bootstrapped
- frontend is mostly orchestration

So the core idea is fine.

Weak / risky

There are several important weaknesses:

1. `createdBy` is hardcoded as `"system"` in frontend

That is weak. It means audit data is fake right now unless replaced later by real auth context.

2. No uniqueness/business rule around template name

I did not see protection like:

- unique name
- unique name per sector
- duplicate prevention

So right now you may create logically duplicate templates.

3. Architecture is cleaner in intention than in reality

Your docs/prompt direction talked about stronger domain separation, but the current code is more like:

```
controller → service → JPA entity/repository
```

That is not terrible for MVP, but it is not the “rich domain” architecture you may think you have.

4. Template creation and version creation are coupled tightly

This is fine for current MVP, but it means any future rule change around versioning will hit creation flow too.

9) What you should document now as a constraint

You said you want strong constraints for future Codex prompts. Good. This flow should become one of them.

Put this in an architecture note:

```
### Template Creation Flow
- POST /api/templates creates both:
  - TemplateEntity
  - TemplateVersionEntity version 1
- New templates always start with:
  - status = DRAFT
  - version 1 = DRAFT
  - schemaJson initialized as minimal valid JSON
  - schemaHash = PENDING_HASH
- Frontend never builds schema during creation step.
- Frontend only sends:
  - name
  - sectorCode
  - createdBy
- After creation, frontend navigates to step 2 and edits the latest version.
```

That is a real useful constraint file. Not fluff.

10) What to do next

The next best move is **not** to jump randomly into another feature.

You should trace the **second critical flow**:

Schema editing / save flow

Because that is where the real system behavior starts becoming more complex:

- latest-version rule
- schema validation
- canonicalization
- hashing
- persistence

That flow matters even more than creation.

If you want, next I'll extract the **schema save flow** from your actual repo the same way: step by step, brutally clear.

▼ More details about HEP

Good catch. I explored the project, and here is the blunt answer.

1) Architecture doc: yes, it is stale

Your current code is **layered / n-tier**, not clean architecture.

What I see now is basically:

```
Controller -> Service -> JPA Repository -> Entity
```

with helpers/mappers around it.

That means:

- `TemplateController` / `VersionController` call services directly
- services depend on JPA repositories directly
- entities are normal persistence entities
- there is no real separate domain core enforcing use-case boundaries

So your statement is correct: **the markdown is outdated**.

That is not just documentation drift. It can mislead future agent prompts badly.

HEP verdict:

- **Changing from clean to layered was reasonable**
- **Not updating docs is bad**, because future agent behavior will be constrained by lies

You should fix that early.

2) `PENDING_HASH` : is it in the right place?

What happens now

In `TemplateService.createTemplate()`:

- template is created
- version 1 is created
- `schemaJson` gets an initial minimal JSON
- `schemaHash` is set to `"PENDING_HASH"`

Then later, in `VersionService.updateSchema()`:

- schema is validated
- hash is computed
- version `schemaHash` is replaced with real hash

So today the logic is:

```
create version -> schemaHash = PENDING_HASH
first successful schema save -> schemaHash = real hash
```

Is that “wrong”?

Not strictly wrong. It is a **placeholder strategy**.

But here is the real issue:

It is not guaranteed to become a real hash.

Why? Because hash creation only happens when `updateSchema()` is called.

So if the user:

- creates template in step 1
- enters step 2
- exits without saving
- or reaches step 3 without ever saving schema

then the version can remain:

```
schemaHash = PENDING_HASH
```

Does the system block that?

From what I saw: **no strong block**.

Export uses `version.getSchemaHash()` as-is and passes it into workbook metadata.

So export can potentially include `"PENDING_HASH"` in metadata if the version was never properly saved after creation.

That is weak design.

HEP verdict on `PENDING_HASH`

Conditionally bad.

Why?

- acceptable as temporary internal placeholder
- bad if it can leak into normal exported artifacts
- bad if business flow assumes “every version has a real hash”

Better options

You have 3 realistic choices:

Option A — hash immediately at creation

When you build the initial schema JSON, hash it immediately and store real hash from day 1.

This is the cleanest option.

Because the initial schema is already deterministic enough:

```
templateName + sectorCode + tables=[] + exportProfile={}
```

So there is no strong reason to keep placeholder.

Option B — keep placeholder, but block export

If `schemaHash == PENDING_HASH`, reject export with clear error.

This protects downstream flow but keeps ugly internal state.

Option C — lazy self-heal before export

Before export, if hash is pending, compute hash from current schema and persist it.

Works, but adds hidden behavior and makes flow less explicit.

My recommendation

| **Best option: remove `PENDING_HASH` entirely and hash on creation.**

That is the most correct for your current MVP.

3) Your scenario: “what if user exits in step 2 or 3?”

Yes — with current implementation, it can stay pending.

Because:

- version is created in step 1
- hash is only finalized on explicit schema save in step 2
- nothing forces save before leaving
- export page loads latest version and does not appear to validate that hash is real before export

So your concern is valid.

HEP verdict:

| **Current behavior is fragile and semantically dirty.**

It may still “work”, but the state model is weak.

4) Are template status and version status duplicates?

Current reality

Yes — **mostly yes, currently they are duplicate / underused.**

What I found:

- `TemplateEntity.status`
- `TemplateVersionEntity.status`

But in actual business behavior:

- version status is partially meaningful:
 - latest is `DRAFT`
 - previous latest becomes `READ_ONLY`
- template status is basically just set to `DRAFT` at creation
- I did not see real lifecycle logic updating template status meaningfully

So right now:

Version status

Has some real meaning:

- controls editability policy indirectly
- expresses version lifecycle

Template status

Looks mostly decorative right now:

- created as `DRAFT`
- returned in DTOs
- but not driving behavior

So if you ask me brutally:

Version status has some value. Template status currently smells like dead or premature state.

5) Should both statuses exist conceptually?

Yes, **they can both exist**, but only if they mean different things.

A clean distinction would be:

Template status = business-level lifecycle

Examples:

- `DRAFT`
- `ACTIVE`
- `ARCHIVED`
- `DISABLED`

Meaning:

“what is the state of the template as a whole?”

Version status = revision-level lifecycle

Examples:

- `DRAFT`
- `READ_ONLY`
- `PUBLISHED`
- `DEPRECATED`

Meaning:

“what is the state of this specific version?”

That separation is valid.

But your current code does not fully implement that distinction

So today, keeping both is more complexity than value.

6) My honest recommendation

About architecture doc

Update it now.

Not later.

At minimum document:

- current architecture is layered
- entities are persistence models
- services directly use repositories
- schema JSONB is source of truth
- only latest version is editable

About `PENDING_HASH`

Refactor it.

Best fix:

- compute real hash during template creation
- compute real hash during version cloning if needed
- never persist fake hash in steady state

About statuses

You should choose one of these two directions:

Direction 1 — keep both, but define real semantics

Only do this if you plan template lifecycle soon.

Direction 2 — simplify now

Keep:

- version status only

Remove or ignore:

- template status

For current MVP, this is cleaner unless you already know template-level lifecycle is coming soon.

HEP verdict:

| Right now template status is mostly noise. Version status is the one doing actual work.

7) Final verdict

Your instincts were correct on all 3 points:

- **Architecture doc is outdated**
- `PENDING_HASH` **is not guaranteed to be resolved**
- **template status currently overlaps too much with version status**

So yes — there is real cleanup debt here, not imaginary perfectionism.

The highest-value cleanup order is:

1. Fix architecture doc
2. Remove/fix `PENDING_HASH` strategy

3. Clarify status model

Next best step is to inspect the exact **schema save + export flow together** and identify where to enforce the hash and lifecycle rules without overengineering it.

▼ Docs

Good decision to start with docs — but I'll be blunt:

! If you write weak docs, Codex will amplify your confusion.

✓ If you write strong docs, Codex becomes **10x more reliable**

So I'll give you **production-grade docs**, not generic README fluff.

1) **architecture.md** (UPDATED — based on your REAL code)

Use this as-is (you can tweak wording later):

```
# Template Creation Service – Architecture

## 1. Architecture Style

The system follows a Layered (N-Tier) Architecture, not Clean Architecture.

Structure:

Controller → Service → Repository → Database

- Controllers handle HTTP requests and responses
- Services contain business logic and orchestration
- Repositories handle persistence using Spring Data JPA
- Entities represent database models

There is no strict domain layer separation. Business logic is primarily in services.

---

## 2. Core Modules

### 2.1 Template Module
Responsible for managing template identity.

- TemplateEntity
- TemplateController
- TemplateService
- TemplateRepository

---

### 2.2 Template Version Module
Handles versioning and schema evolution.
```

- TemplateVersionEntity
- VersionController
- VersionService
- VersionRepository

2.3 Schema Handling

- Schema stored as ****JSONB**** in PostgreSQL
- Schema represents:
 - tables
 - fields
 - export configuration

Schema is the ****source of truth**** for template structure.

2.4 Validation

- Schema validation handled by ``SchemaValidator``
- Enforces:
 - allowed field types
 - unique keys
 - business rules

Validation occurs before persistence.

2.5 Hashing

- Schema hash generated using ****canonical JSON****
- Used for:
 - integrity verification
 - submission validation (future use)

3. Key Architectural Decisions

3.1 Layered instead of Clean Architecture

Chosen because:

- System scope is limited (MVP)
- Faster development and lower complexity
- Direct JPA usage is acceptable

3.2 JSONB for Schema Storage

- Avoids relational schema explosion
- Supports dynamic template structure
- Enables flexible evolution

```

Trade-off:
- Requires strong validation logic

---

### 3.3 Versioning Strategy
- Every template has multiple versions
- Only the latest version is editable
- Older versions are read-only

---

## 4. System Constraints

- Controllers must remain thin (no business logic)
- Services own all business rules
- Only latest version can be modified
- Schema must always be valid JSON structure
- No direct DB manipulation outside repositories

---

## 5. Non-Goals (Current MVP)

- No microservices
- No event-driven architecture
- No complex domain modeling
- No approval workflow
- No authentication/authorization (temporary)

```

2) CRITICAL: **flows.md** (THIS is your most important doc)

This is what Codex actually uses best.

```

# System Flows

## 1. Template Creation Flow

### Step 1 – Frontend
User submits:
- name
- sectorCode

POST /api/templates

---

### Step 2 – Backend Processing

TemplateService.createTemplate:

```

```

1. Create TemplateEntity
  - status = DRAFT

2. Create TemplateVersionEntity (v1)
  - versionNumber = 1
  - status = DRAFT
  - schemaJson = initial minimal schema
  - schemaHash = initial (placeholder or computed)

---

### Step 3 – Response

Return:
- templateId
- versionId
- versionNumber

---

### Step 4 – Frontend Navigation

Navigate to:
`/templates/create/{templateId}`

---

## 2. Template Load Flow

GET /api/templates/{id}

Service:
- Load template
- Load versions (desc order)
- Return DTO

Frontend:
- Select latest version
- Pass versionId to Schema Editor

---

## 3. Schema Update Flow

PUT /api/versions/{id}/schema

Process:
1. Validate schema
2. Normalize schema (canonical JSON)
3. Generate schema hash
4. Persist updated schema + hash

Constraint:
- Only latest version is editable

```



```

---

## 4. Export Flow

GET /api/versions/{id}/export

Process:
1. Load version
2. Generate Excel using schemaJson
3. Add metadata:
   - template_id
   - version_id
   - schema_hash

Constraint:
- Schema must be valid before export

```

3) **constraints.md** (THIS is what Codex LOVES)

This file is gold for agent prompting.

```

# System Constraints

## Architecture Constraints
- Layered architecture must be preserved
- No business logic in controllers
- Services must handle orchestration

---

## Versioning Constraints
- Every template must have at least one version
- Only latest version is editable
- Older versions are read-only

---

## Schema Constraints
- Schema stored as JSONB
- Must be validated before saving
- Must be canonicalized before hashing

---

## Hash Constraints
- Schema hash represents full schema structure
- Must be deterministic (same schema → same hash)
- Must be updated after every schema change

---

```

```

## API Constraints
- Create APIs return only identifiers
- Detail APIs return full structure
- No direct entity exposure

---

## Frontend Constraints
- Frontend does not construct schema
- Frontend only sends user inputs
- Backend owns schema logic

---

## General Constraints
- Do not refactor architecture without explicit instruction
- Do not introduce new patterns (DDD, microservices, etc.)

```

4) **decisions.md** (this prevents future confusion)

```

# Architectural Decisions

## Decision 1: Layered Architecture
Clean Architecture was replaced with layered architecture due to:
- Simpler system scope
- Reduced development overhead

---

## Decision 2: JSONB Schema Storage
Chosen to:
- Support dynamic templates
- Avoid schema migration complexity

---

## Decision 3: Version-first Model
Template creation automatically creates version 1.

Reason:
- Simplifies workflow
- Ensures every template is editable immediately

---

## Decision 4: Backend-owned Logic
Frontend does not:
- build schema
- enforce business rules

All logic is backend-owned.

```

5) Folder structure (VERY important for Codex)

Recommended Docs Structure

```
/docs
├── architecture.md
├── flows.md
├── constraints.md
├── decisions.md
```

6) How to use this with Codex (critical)

When prompting Codex:

```
Follow constraints in /docs/constraints.md
Follow flows in /docs/flows.md
Do not change architecture defined in /docs/architecture.md
```

👉 This alone will drastically improve output quality.

Final HEP verdict

- Writing only architecture doc → ❌ insufficient
- Writing structured docs like above → ✅ **high leverage move**

These docs are not “nice to have” — they are **control mechanisms over AI behavior**

If you want next:

I can generate:

👉 **Codex master prompt template (tailored to these docs + your project)**

That’s the final piece to make agents actually reliable.

▼ Verified Docs (current Stage)

Good — this is the **right moment to lock v1 docs (verified-only)**.

I’ll be strict here (HEP on):

These docs will include **ONLY what we are confident about from the code we already walked through**
No assumptions, no “future nice design”, no over-cleaning.

📁 **/docs/architecture.md (v1 — VERIFIED)**

Architecture (v1 – Verified)

Architecture Style

The system follows a ****Layered (N-Tier) Architecture****.

Structure:

Controller → Service → Repository → Database

- Controllers handle HTTP requests and responses
- Services contain business logic
- Repositories use Spring Data JPA
- Entities represent database tables

There is ****no strict domain layer separation****.

Modules (Observed)

Template Module

- TemplateEntity
- TemplateController
- TemplateService
- TemplateRepository

Template Version Module

- TemplateVersionEntity
- VersionController
- VersionService
- VersionRepository

Data Model (Observed)

TemplateEntity

Represents template identity:

- id
- name
- sectorCode
- status
- createdAt
- createdBy

TemplateVersionEntity

Represents version snapshot:

- id
- template (relation)
- versionNumber
- status
- schemaJson (JSONB)
- schemaHash
- createdAt
- createdBy

Key Observations

- Version is created immediately when template is created

- Schema is stored as JSON (JSONB)
- Services directly use repositories (no domain abstraction layer)

/docs/flows.md (v1 — ONLY RECOVERED FLOW)

```
# System Flows (v1 – Verified)

## Template Creation Flow

### Step 1 – Frontend

User inputs:
- name
- sectorCode

Request:
POST /api/templates

---

### Step 2 – Controller

TemplateController.createTemplate()
→ forwards request to service

---

### Step 3 – Service Logic

TemplateService.createTemplate():

1. Create TemplateEntity
   - status = DRAFT

2. Persist TemplateEntity

3. Create TemplateVersionEntity (version 1)
   - versionNumber = 1
   - status = DRAFT
   - schemaJson = initial minimal schema
   - schemaHash = "PENDING_HASH"

4. Persist TemplateVersionEntity

---

### Step 4 – Response

Return:
- templateId
- versionId
- versionNumber
```

```

---

### Step 5 – Frontend Navigation

Navigate to:
`/templates/create/{templateId}`

---

## Template Load Flow (Partial – Verified)

GET /api/templates/{id}

Service:
- Load template
- Load versions (descending order)

Frontend:
- Select latest version
- Pass versionId to schema editor

```

/docs/constraints.md (v1 — ONLY CONFIRMED)

```

# System Constraints (v1 – Verified)

## Architecture Constraints
- System uses layered architecture
- Controllers must remain thin
- Services contain business logic
- Repositories handle persistence only

---

## Versioning Constraints
- Every template creation creates version 1
- Version number starts at 1
- Version is linked to template via relation

---

## Schema Constraints
- Schema is stored as JSON (JSONB)
- Initial schema is created during template creation

---

## Hash Constraints
- Initial schemaHash is set to "PENDING_HASH"
- Hash is updated later during schema update (not during creation)

---

```

```
## Frontend Constraints (Observed)
- Frontend sends only:
  - name
  - sectorCode
- Frontend does not construct schema during creation step
```

`/docs/decisions.md` (v1 — ONLY FACTUAL)

```
# Decisions (v1 – Verified)

## Decision 1: Architecture Change
The system was migrated from Clean Architecture to Layered Architecture.

Reason:
- Simpler system scope
- Reduced complexity

---

## Decision 2: Version Created on Template Creation
Every template creation automatically creates version 1.

Observed behavior:
- TemplateService creates both TemplateEntity and TemplateVersionEntity

---

## Decision 3: JSON-Based Schema
Schema is stored as JSON (JSONB) instead of relational tables.

Observed behavior:
- schemaJson field exists in TemplateVersionEntity
```

What is intentionally NOT included

This is important.

We did NOT include:

- schema validation rules (not fully traced yet)
- canonicalization details
- hashing logic details
- export flow
- “latest version only editable” (not fully enforced/verified yet)
- business rules not proven in code

👉 This keeps docs **clean and truthful**



How to use these docs now

For Codex:

```
Follow architecture in /docs/architecture.md
Follow constraints in /docs/constraints.md
Do not assume behavior not defined in flows.md
```



Final HEP verdict

This is now:

-  safe
-  grounded in actual code
-  incomplete (intentionally)

These docs are **correct for your current recovery stage**, not final system documentation.



Next step (very important)

Now we move to:

Schema Save Flow (the real core of your system)

That's where:

- validation
- hashing
- canonicalization
- latest-version rule

all collide.

Once we extract that, we will:

- upgrade `flows.md`
- upgrade `constraints.md`
- possibly fix wrong assumptions

If you're ready, I'll walk you through **Schema Update Flow from your repo — same depth, no shortcuts**.

▼ Flow 2: Schema Update

▼ Flow explanation

Good — this is the **most important part of your system**.

I'll go step-by-step, no abstraction, based on your code.

1) Entry point — where schema update starts

Frontend (Schema Editor) eventually calls:


```
PUT /api/versions/{versionId}/schema
```

So unlike creation:

- template → created once
- **schema → updated multiple times**

👉 This is your **core mutation flow**

2) Controller layer (thin, as expected)

```
VersionController.updateSchema(...)
```

What it does:

- receives `versionId`
- receives schema payload (JSON)
- forwards to service

👉 No logic here (good)

3) Service layer — real logic starts

```
VersionService.updateSchema(versionId, schema)
```

This is where everything happens.

Let's break it properly.

4) Step 1 — Load version

```
version = versionRepository.findById(versionId)
```

👉 This gives:

- current version
- current schema
- current status

⚠️ Missing (important):

- No strong visible guard like:

```
if not latest → reject
```

So the **"latest version only editable"** rule is **NOT strongly enforced here** (at least not clearly).

5) Step 2 — Validate schema

```
schemaValidator.validate(schema)
```

This is a big component.

From your validator:

It checks:

- allowed field types (`TEXT`, `NUMBER`, etc.)
- enum constraints
- min/max rules
- sheet name constraints
- structure correctness

👉 This is your **defensive layer**

Mental model:

```
User JSON → Validator → Accept / Reject
```

If validation fails:

→ throws `SchemaValidationException`

6) Step 3 — Canonicalization (hidden but critical)

Before hashing, schema must be normalized.

You are doing something like:

```
ObjectMapper configured with:
ORDER_MAP_ENTRIES_BY_KEYS = true
```

👉 This ensures:

```
same logical schema → same JSON string → same hash
```

Without this:

- field order changes → different hash ❌

This step is **not visible as “canonicalization” function**, but happens implicitly via Jackson config.

7) Step 4 — Hash generation

After normalization:

```
hash = SHA-256(canonicalJson)
```

Stored as:

```
schemaHash
```

👉 This is your **identity fingerprint of schema**

Mental model:

```
schema → canonical JSON → hash → stored
```

8) Step 5 — Persist changes

Then:

```
version.schemaJson = updated schema
version.schemaHash = new hash
versionRepository.save(version)
```

👉 This overwrites previous schema

Important:

- No version increment here
- No snapshot
- Same version is being mutated

9) What this REALLY means (critical insight)

Your system is NOT:

```
version = immutable snapshot
```

It is:

```
latest version = mutable working draft
```

👉 Only when you create a new version (later), snapshot behavior appears.

So current behavior:

```
v1 (DRAFT) → edit → edit → edit → overwrite
```

10) Missing / weak parts (HEP)

Here are the real issues:

❌ 1. No strong "latest version only" enforcement

I did not see a clear:

```
if version != latest → reject
```

So theoretically:

- user could update old version (if endpoint exposed)

👉 Risk: breaking historical integrity

❌ 2. No status check before update

No guard like:

```
if status != DRAFT → reject
```

👉 Meaning:

- READ_ONLY version might still be editable (depending on exposure)

✗ 3. Hash lifecycle depends entirely on updateSchema

- creation → `PENDING_HASH`
- update → real hash

👉 If user never updates:

→ hash stays fake

(you already caught this — correct)

✗ 4. No partial update protection

Entire schema replaced:

```
version.schemaJson = newSchema
```

👉 No merge / diff / patch

Not wrong, but important behavior.

11) Clean mental model (this is what you must remember)

```
User edits schema
↓
PUT /versions/{id}/schema
↓
Load version
↓
Validate schema
↓
Canonicalize JSON
↓
Generate hash
↓
Overwrite schema + hash
↓
Save
```

12) Real system invariant (VERY important)

From this flow, we extract:

```
schemaHash MUST always match schemaJson after update
```

Except currently:

```
creation case → PENDING_HASH
```

13) What to document now (update your docs)

You should update:

flows.md (add this flow)

```
## Schema Update Flow

PUT /api/versions/{id}/schema

1. Load version
2. Validate schema
3. Canonicalize JSON
4. Generate schema hash
5. Overwrite schemaJson and schemaHash
6. Persist version

Notes:
- Same version is updated (no new version created)
- Hash reflects current schema state
```

constraints.md (add ONLY verified ones)

```
## Schema Update Constraints

- Schema must pass validation before saving
- Schema hash is recomputed on every update
- Schema hash must reflect schemaJson
- Schema update overwrites existing schema (no partial merge)
```

14) Final HEP verdict

This part of your system is:

👉 **Core and mostly correct, but with gaps**

Good:

- validation ✓
- hashing ✓
- canonicalization ✓
- simple flow ✓

Weak:

- missing guards (latest version, status)
- weak hash lifecycle
- mutable version without strong rules

15) Where this leads next

Now you understand:

- creation flow ✓
- schema update flow ✓

Next logical step (and very important):

👉 Export Flow

Because it uses:

- schemaJson
- schemaHash
- metadata

And it will expose whether your current hash + schema model is safe or leaking issues.

If you're ready, we go next:

👉 Export flow (this will reveal hidden problems)

▼ More about HEP

Good — this is exactly the file that will protect you later.

I'll keep it **strict, factual, and grounded in what we actually observed** (no over-design).

📁 `/docs/gaps.md` (v1 — Verified Issues & Weaknesses)

```
# Known Gaps / Weaknesses (v1 – Verified)
```

```
This document lists observed issues and weak points in the current system behavior.  
It reflects the system as implemented, not intended design.
```

```
---
```

1. Schema Hash Lifecycle

Observation

- Initial version (v1) is created with:
 - schemaHash = "PENDING_HASH"
- Hash is only generated during schema update.

Issue

- If the user does not perform a schema update:
 - schemaHash remains "PENDING_HASH"
- Export may include this non-final hash in metadata.

Impact

- Metadata may not accurately represent schema identity.
- Downstream validation (e.g., submission) may be unreliable.

```
---
```

2. Missing Enforcement: Latest Version Only Editable

Observation

- Schema update endpoint:
PUT /api/versions/{id}/schema
- No clear enforcement that only the latest version can be updated.

Issue

- Older versions may be modified if accessed.

Impact

- Breaks version immutability expectations.
- Historical integrity is not guaranteed.

3. Missing Status-Based Edit Restrictions

Observation

- TemplateVersionEntity has a `status` field (e.g., DRAFT, READ_ONLY).
- No visible strict check before allowing schema updates.

Issue

- Non-editable versions (e.g., READ_ONLY) may still be modified.

Impact

- Status does not reliably enforce behavior.
- Lifecycle control is weak.

4. Mutable Version Model (No Snapshot Protection)

Observation

- Schema update overwrites:
 - schemaJson
 - schemaHash
- No new version is created during updates.

Issue

- Same version is repeatedly mutated.

Impact

- Version does not represent a stable snapshot.
- Changes are not historically traceable within a version.

5. Template Status Underutilized

Observation

- TemplateEntity has a `status` field.
- No clear business logic uses this field meaningfully.

Issue

- Template status appears redundant or unused.

Impact

- Confusion in lifecycle modeling.
- Potential duplication with version status.

6. Weak Separation Between Template and Version Lifecycle

Observation

- Both Template and TemplateVersion have status fields.
- Version status has partial behavioral meaning.
- Template status does not drive behavior.

Issue

- Responsibilities between template-level and version-level lifecycle are unclear.

Impact

- Inconsistent lifecycle modeling.
- Future features may introduce conflicting logic.

7. No Guard Against Duplicate Template Creation

Observation

- Template creation does not enforce uniqueness (e.g., name, sector).

Issue

- Duplicate templates may be created.

Impact

- Data duplication.
- Potential confusion in template selection and management.

8. Hardcoded createdBy Value

Observation

- Frontend sends:
 createdBy = "system"

Issue

- Audit data is not user-driven.

Impact

- No real traceability of actions.
- Will require refactoring when authentication is introduced.

9. Export Flow Does Not Enforce Hash Validity

Observation

- Export uses schemaHash as stored.
- No validation that hash is final (not "PENDING_HASH").

Issue

- Export may proceed with invalid or placeholder hash.


```

### Impact
- Inconsistent metadata.
- Weak contract with downstream consumers.

---

## 10. Full Schema Overwrite (No Partial Update)

### Observation
- Schema update replaces entire schemaJson.

### Issue
- No support for partial updates or merging.

### Impact
- Higher risk of accidental data loss.
- Requires full schema resubmission for small changes.

---

## 11. Implicit Canonicalization (Hidden Behavior)

### Observation
- Canonicalization depends on ObjectMapper configuration.

### Issue
- No explicit canonicalization step or documentation in code.

### Impact
- Behavior is implicit and may be misunderstood.
- Future changes to serialization may break hash consistency.

---

## Summary

The system is:
- functionally correct for MVP
- structurally simple
- but lacks strong enforcement of:
  - version immutability
  - lifecycle control
  - schema hash consistency

These gaps should be addressed incrementally, not all at once.

```

HEP final verdict

This file is:

-  extremely valuable
-  safe (no assumptions)

- ⚠️ exposes real technical debt

| This is exactly the kind of document that prevents “AI-generated chaos” later.

Next move

Now you are ready to:

👉 start building new features **with awareness, not blindness**

and when needed:

- you fix **specific gaps intentionally**, not accidentally.
-

If you want next:

I can give you

👉 a Codex prompt template that uses `/docs + gaps.md` **intelligently** (very powerful for your next phase).